

たくさんの問題が解ける

プログラミング未経験でもOK

最新シラバス対応

基本情報

科目B

技術者

超効率の教科書

TIME PERFORMANCE!

⊕ よく出る問題集

科目Bの

合格をスピーディーに勝ち取ろう

サクッと理解

テキスト

つまづき解消

動画解説

実践力↑

模擬試験
問題

著 石田宏実

難問をやさしく解説
動画解説付き

Udemyやスタディングで大活躍の

人気講師が伝授!! インプレス

インプレス情報処理試験シリーズ 読者限定特典!!

本書の特典は、下記サイトにアクセスすることでご利用いただけます。

<https://book.impress.co.jp/books/1125101034>

※サイトにアクセスし、画面の指示に従って操作してください。

※特典のご利用には、無料の読者会員システム「CLUB Impress」への登録が必要となります。

特典：本書全文の電子版

本書の全文の電子版（PDF ファイル）がダウンロードできます。

※本特典のご利用は、書籍をご購入いただいた方に限ります。

※ダウンロード期間は、いずれも本書発売より1年間です。

※印刷してご利用いただくことはできません。あらかじめご了承ください。

- 本書は、基本情報技術者試験の受験用教材です。著者、株式会社インプレスは、本書の使用による基本情報技術者試験への合格を保証致しません。
- 本書の内容については正確な記述につとめました。著者、株式会社インプレスは、本書の内容にもとづくいかなる結果にも一切責任を負いかねますので、あらかじめご了承ください。
- 本書に記載している試験問題は、独立行政法人 情報処理推進機構の情報処理技術者試験センターが公開している情報をもとに制作しています。
- 本書に記載している会社名や製品名、サービス名は、各社の商標または登録商標です。本文中に、TMおよび®は明記していません。

インプレスの書籍ホームページ

書籍の新刊や正誤表など、最新情報を随時更新しております。

<https://book.impress.co.jp/>

ごあいさつ

学生時代は、毎日のように出される宿題に取り組み、受験勉強に励んでいたことでしょう。ところが社会に出ると、日々の仕事に追われ、勉強を続ける人は一気に少なくなります。総務省統計局の「社会生活基本調査」によれば、日本の社会人のおよそ6割は、仕事以外でまったく学習をしていないとされています。もちろん仕事そのものも学びの場ではありますが、それは周囲の人々にとっても同じことです。殻を破って大きく自己成長するためには、それだけでは限界があります。

プログラマーやシステムエンジニア、あるいはこれからIT業界を目指すあなたにとって、IT系の資格試験の学習は、その殻を破る大きなきっかけになるはずです。とりわけ体系的に整理された「情報処理技術者試験」は、仕事以外の学びの中でも他者との差を生み出すうえで非常に有意義です。IT系資格は民間を含め数多く存在しますが、その中でも情報処理技術者試験は国家試験であり、信頼性と認知度が格段に高い水準にあります。可否にかかわらず、この試験に向けた体系的な学習は、今後のキャリア形成に大きな資産となるでしょう。

本書は、私自身が「こんな参考書があったらよかったのに」という観点で構想し、執筆しました。そのうえで、できるだけわかりやすく、挫折せず最後まで学習を続けられるよう、構成や解説を工夫しています。せっかく勉強の大切さに気づき学習を始めたあなたのモチベーションを下げないことを重視しました。

科目Bで扱う擬似言語は実在のプログラミング言語ではありません。しかしその学習は言語に依存しないアルゴリズム的思考やデータ構造、流れやオブジェクト指向を理解するうえで非常に有効です。擬似言語を攻略することで、JavaやPythonなどの人気言語の習得における土台が築かれ、「自分にプログラミングができるのだろうか」という不安にも自信をもって向き合えるはずです。

この本が、あなたのIT技術者としての人生をさらに高める一歩となれば、これほど嬉しいことはありません。本書を執筆するうえで、筆者の思いに共感していただき、企画や編集面でご尽力いただいた関係者の方々に心から感謝いたします。

石田 宏実

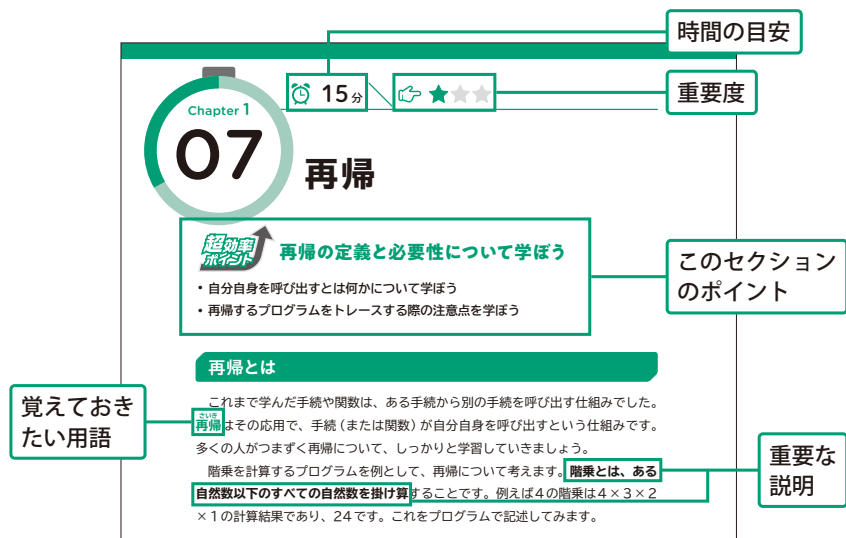
本書を使った学習方法

本書の特徴

本書の特徴は、科目Bの問題が解けるように、擬似言語の構成要素を1つずつ順番に解説している点です。また、節ごとに例題を用意しているため、学習したことの理解を深めやすく、つまづくことなく合格を目指せるように構成しています。

節の冒頭では、その節を読むための時間の目安や、基本情報技術者試験における重要度を星の数で示しています。「超効率ポイント」は、その節のポイントをもとめています。初めて読む前の内容の把握や、復習時の振り返りなど、効率的な学習に役立ててください。

また、覚えておきたい用語は色文字で強調したり、理解するうえで重要な説明は太字にしたりしています。学習したあとに読み返す際など、効率よく確認するのにも有効です。



多数の例題や演習問題を掲載

節末ごとにある例題、5章の演習問題、7章の模擬試験問題と多くの問題を掲載しています。

1～4、6章には、節末ごとに例題があり、その節で学んだ内容について理解を深め、知識を定着させられるようになっています。また5章の演習問題は、1～4章で学んだ要素が組み合わされた問題を掲載しており、知識の総合的な確認ができる構成になっています。

例題にチャレンジ

例題 ①

オリジナル問題

以下のプログラムを実行した結果、出力される値を解答群から選べ。

プログラム

```
1  整数型: price ← 120
2  整数型: quantity ← 3
3  整数型: discount ← 50
4  整数型: total ← price × quantity - discount
5  totalを出力
```

解答群

ア 210

イ 310

ウ 360

エ 510

そして7章の模擬試験問題は、IPAの公式ページで掲載されている公開問題を、実際の試験と同じ出題数と種類にあわせて厳選しています。本書を一通り読んだあと、試験と同じ時間で問題を解けるかを試してみてください。

問題

問1 次の記述中の に入れる正しい答えを、解答群の中から選べ。

プログラムを実行すると、""と出力される。

(プログラム)

```
1  整数型: a ← 5
2  整数型: b ← 7
3  整数型: c ← 9
4  整数型: tmp ← a
5  a ← c
6  c ← b
7  b ← tmp
8  aの値とbの値とcの値をこの順にコンマ区切りで出力する
```

解説動画

問題のによっては動画による解説を用意しているものもあります。解説の右上にQRコードを掲載しており、これを読み込むことで、解説動画を視聴することができます。解説動画ではより細かく説明していますので、解説文を読んでもあまりピンとこなかった方は、ぜひ視聴してみてください。

解説 キューとスタックの問題は頻出です。ただし、単純なキューとスタックはあまり出題されないと考えられます。公開されている問題では優先度付きキューについて出題されていますし、今回のようにisEmptyなどのメソッドが追加されている場合もあります。クラスの説明をよく読んで答えましょう。

キューとスタックの問題は、トレース表を作成すると簡単に正解がわかります。まずは途中経過です。

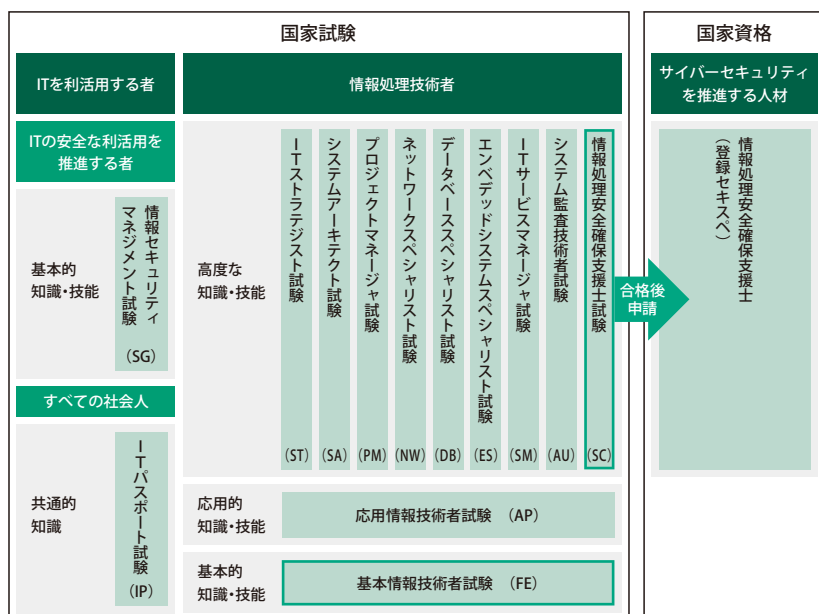


解説動画

	B	C	D	E	F	G	H	I	J	K	L	M	N	O
				text	count	inWord	i							
1		○整数型: countWords(文字列型: text)		a pen		true								
2		整数型: count ← 0												
3		論理型: inWord ← false												
4		整数型: i												
5		for (iを1からtextの文字数まで1ずつ増やす)												
6		if (textのi番目の文字が a)												
7		if (inWordがfalseと等しい)												
8		count ← count + 1												
9		inWord ← true												
10		endif												
11		else												
12		inWord ← false												
13		endif												
14		endfor												
15		return count												

基本情報技術者試験とは

基本情報技術者試験は、情報処理推進機構（IPA）が実施する国家試験です。本試験は、ITに関する基礎的な知識と技術を持ち、情報システムの開発・運用に必要な技能を有する人材を育成することを目的としています。情報処理技術者試験の中では「レベル2」に位置付けられ、初級から中級レベルの技術者向けとされています。



基本情報技術者試験は、CBT（Computer Based Testing）方式で実施されており、受験者は全国の会場から日程と時間帯を選択して受験します。

科目	試験時間	問題数	出題形式
科目 A	90 分	60 問	四択択一式
科目 B	100 分	20 問	多岐選択式

科目Aと科目Bのそれぞれで1,000点満点中600点以上を獲得すると、試験は合格となります。得点はIRT（項目応答理論）に基づいて算出されるため、1問あたりの配点は公表されていません。

●対象者像

IPAによると、本試験の対象者は「ITを活用したサービス、製品、システム及びソフトウェアを作る人材に必要な基本的知識・技能をもち、実践的な活用能力を身に付けた者」とされています。情報システムの開発や運用に携わるエンジニアを目指す社会人・学生に適しています。

上位資格である「レベル3」の応用情報技術者試験では、プロジェクトマネージャやシステムエンジニアなどの職種を想定しており、プログラミング知識は必須ではありません。一方で基本情報技術者試験では、科目B試験で擬似言語を用いたプログラム読解問題が出題されます。

擬似言語は、PythonやJavaのような実際のプログラミング言語とは異なり、文法や構造が試験用に簡略化されています。そのため、普段からプログラムを書いている人でも、まずは擬似言語の記法に慣れる必要があります。

このように、基本情報技術者試験は「プログラムを正確に読み取る力」を前提とした構成になっており、プログラミングに不慣れな受験者にとっては、入念な対策が必要とされます。一方で、業務で日常的にプログラミングを行っている受験者にとっては、多少の慣れは必要ですが、全体としてそれほど難しい試験だとは思いません。

●科目B試験

基本情報技術者試験は、かつては「午前」「午後」に分かれて実施されていましたが、現在は「科目A」「科目B」という形式に改められています。

科目Aでは情報セキュリティやネットワーク、アルゴリズムなど幅広い分野から出題され、四肢択一式の問題に解答します。

科目Bでは「擬似言語を用いたプログラム読解問題」と「セキュリティに関する問題」が、いずれも選択式で出題されます。

プログラム読解問題では、与えられたプログラムの動きを把握し、出力結果を予測する力や、条件分岐・繰り返し処理の流れを理解する力が問われます。これらの力は、単なる暗記では対応できない実践的な読解力であり、演習を通じた学習が不可欠です。プログラム読解問題は16問出題されます。

残りの4問はセキュリティに関する問題です。これらは、情報セキュリティマネジメント試験の科目Bに出題される内容と類似しており、リスク対策、セキュリティポリシー策定、マルウェア対策などの実務的な知識が問われます。

擬似言語とは

科目Bでは、「擬似言語」と呼ばれる独自の記法で記述されたプログラムが出題されます。これは、特定のプログラミング言語（例：CやJavaなど）ではなく試験専用のプログラミング言語です。

実際のプログラミング言語ではないため、試験問題でしか目にするのではなく、現実の開発業務では使われることはありません。しかしプログラミングの能力を測るという点では、十分に実践的な試験です。擬似言語の特徴については、第1章以降で詳しく説明していますので、しっかりと学んでください。ここでは擬似言語の特徴を少しだけ見ていきましょう。

●一部は日本語で記述

擬似言語も多くのプログラミング言語と同様に、多くが英語で記述されています。しかし一部で日本語が使われることがあります。

プログラム

```
1  ○整数型の配列: makeNewArray(整数型の配列: in)
2  整数型の配列: out ← {} // 要素数0の配列
3  整数型: i, tail
4  outの末尾にin[1]の値を追加する
5  for (iを2からinの要素数まで1ずつ増やす)
6    tail ← out[outの要素数]
7    outの末尾に(tail + in[i])の結果を追加する
8  endfor
9  return out
```

この記述方法は、プログラミングの知識がある人にとっては大きな違和感があるかと思います。

なお本書では上のように、**説明用の行番号を先頭に記述していますが、実際の試験では記述はありません。**

● コメントの記述

コメントは「//」を使って記述します。一般的にコメントはプログラムの実行には影響せず、処理の意図や役割をメモするために使います。次の例では、コメントを使って処理の説明をしています（現時点でこの処理の意味はわからなくても大丈夫です）。

```
// 配列の合計を求める
sum ← 0
for (iを1からnまで1ずつ増やす)
    sum ← sum + A[i]
endfor
```

コメントを読み解くことで、処理の目的や動作を把握しやすくなります。また次のように、処理そのものがコメントで示されていることもあります。

プログラム

```
1  大域: 文字列型配列の配列: orders ← {"A", "B", "D"}, {"A", "D"},
2                                     {"A"}, {"A", "B", "E"},
3                                     {"B"}, {"C", "E"}}
4
5  OputRelatedItem(文字列型: item)
6    文字列型の配列: allItems ← ordersに含まれる文字列を
7                                重複なく辞書順に格納した配列
8                                // 表の例では {"A", "B", "C", "D", "E"}
```

本来であれば「ordersに含まれる文字列を重複なく辞書順に格納」も、プログラミングする必要がありますが、この例のように省略されている場合があります。

● クラスは記述できない

現在公開されているサンプル問題などを見る限りは、クラスを記述するための方法はありません。クラスはすでに定義されており、それをどのようにして使うのかだけが示されています。そのためオブジェクト指向についての知識は必要になります。

クラスListElementのメンバ変数の説明

メンバ変数	型	説明
val	文字型	要素の値。
next	ListElement	次の要素の参照。次の要素がないときの状態は未定義。

効率的な勉強法

● トレース表の重要性

科目Bではプログラムの読解力が問われます。

文章の読解は、最初から最後まで1文字ずつ読んでいけばよいのですが、プログラムはそうはいきません。あっちに行ったり、こっちに行ったりします。そこで処理の流れを追いかける方法として有効なのが「トレース表」です。

トレース表とは、変数の値や配列の状態を表形式で整理し、プログラムの実行を追っていくための自作の表です。例えば、次のような擬似言語のプログラムがあったとします。

```
1 x ← 3
2 y ← x + 2
3 x ← y * 2
```

処理の流れを以下のように整理することで、計算の順序や変数の変化を明確に把握できます。

行	x	y
1	3	
2		5
3	10	

トレース表の作成方法については、1章で詳しく解説しています。

● トレース表は本番で必要なのか？

トレース表は学習の初期段階では非常に有効です。しかし**試験本番では時間と体力の勝負になりますので、毎回丁寧に作成するのは現実的ではありません。**

本書では、まずはトレース表を使って理解を深め、最終的には頭の中である程度処理を追えるようになることを目指します。省略しても読解できる力を少しずつ養っていきましょう。

● セキュリティ分野の勉強法

科目Bでは全20問のうち4問が情報セキュリティ分野からの出題です。プログラム問題が大半を占めるため、セキュリティ分野の対策をそれほど重視する必要はありません。とはいえ4問すべてを落としてしまうと、その分プログラム分野で高い正解率が必要となってきますので、ある程度の対策はしておきましょう。

本書では6章をセキュリティ分野に充てており、出題傾向や基本用語、頻出テーマに対応できるように構成しています。科目B対策としてはこの6章をしっかりと学習すれば十分です。

さらに深く学習したい場合は、情報セキュリティマネジメント試験の学習を行ってください。情報処理技術者試験全体を通じて、情報セキュリティは最重要視されています。この機会にしっかりと学習しておくのも、将来を見据えたすばらしい戦略といえます。

● 解説を読むときの姿勢

「答えを見て、なんとなくわかった気になる」。これは学習においてもっとも避けたいことです。

たしかに正解の選択肢やその理由を読めば、一時的には理解したように感じるかもしれません。しかし、それは「理解」ではなく「わかったつもり」に過ぎないことが多いと思います。解説を読むときには、必ず「なぜ自分はその選択肢を選べなかったのか」「問題文のどの部分を読み落としていたのか」「どのような知識が不足していたのか」など、原因を具体的に振り返る姿勢が必要です。そして、「次に似たような問題が出たとき、どうすれば正解できるか」という視点で考えることで、実力の向上につながります。

本書では、読者がそうした振り返りをしやすいように、「つまづきやすいポイント」や「間違いやすい選択肢」についても解説しています。

試験当日の心構え

● 休憩は必ず取る

基本情報技術者試験は、「科目A」と「科目B」の2つのパートに分かれています。時間帯は区切られておらず、受験者は両方を1度に受験します。

科目Aと科目Bの間には休憩時間（最大10分）を設けることができます。休憩は任意であり、休憩を取らずに続けることもできますが、**集中力を維持するためには必ず休憩をとるようにしましょう。**

● 白紙のメモ用紙をもらう

試験中には白紙のメモ用紙をもらうことができます。これは計算やトレース表の作成に役立ちます。

試験中でも試験官に申し出ればその都度もらうことができます。申し出たときにもらえるのは1枚だけですが、何度でも申し出ることが可能です。

特に科目Bのプログラム読解問題では、トレース表や値の変化を紙に書き出すことで読解を助けることになります。何度も試験官に申し出るのは気が引けるかもしれませんが、**遠慮せずにどんどん書いて、どんどん白紙をもらいましょう。**合格することが最優先です。

● 時間配分を考える

科目Bは100分で20問解く必要があります。つまり1問あたり平均5分の計算です。もちろん、すべての問題が同じ難易度ではないため、内容に応じてメリハリをつけた時間配分が必要です。特に、セキュリティ分野の問題はおそらく2分程度で解けると思います。これらを先に解いてしまうのも十分戦略的な判断です。本書やIPAのWebページに掲載されている公開問題を活用しながら、自分なりのペース感を事前に確認しておきましょう。

難しい問題や見慣れない形式の問題が出てきても、無理にその場で解こうとせず、いったん飛ばしてしまっても構いません。CBT試験では、画面の上部にあるボタンを使って問題を前後自由に移動することができます。解けそうな問題から先に取り組み、時間がかかりそうな問題や自信のない問題は後回しにすることで、時間を有効に使うことができます。

●間違いを恐れずに解く

科目Bは選択式です。記述式ではありません。そのため、たとえ正解に自信がなくても、必ず選ぶようにしてください。「なんとなくこれかな？」というものを選んでおけば、確率的には正解の可能性が生まれます。未回答にしてしまうと確実に0点です。

実際には選択肢のうち、いくつかは「絶対はない」という理由で削ることができるはず。そうなると思ったよりも正解率が高くなります。

とにかく忘れてほしくないのは、1000点中600点で合格ということです。1問あたりの点数の算出方法は公開されていないものの、6割程度正解すれば合格できる試験なのです。ある程度のミスは想定内であると考え、恐れずどんどん解いていきましょう。

CONTENTS

ごあいさつ	3
本書を使った学習方法	4
基本情報技術者試験とは	7

Chapter

1

擬似言語の基礎

17

01	変数	18
02	トレース表	26
03	条件分岐	30
04	繰り返し	44
05	ビット演算	57
06	手続きと関数	71
07	再帰	80

Chapter

2

配列

93

01	配列	94
02	二次元配列	114

Chapter

3

オブジェクト指向

127

01	オブジェクト指向	128
----	----------	-----

Chapter 4 データ構造 153

- 01 連結リスト..... 154
- 02 キューとスタック..... 170
- 03 木構造..... 182

Chapter 5 演習問題 203

Chapter 6 情報セキュリティ 279

- 01 情報セキュリティの要求事項..... 280
- 02 マルウェアからの保護..... 286
- 03 バックアップとログ..... 290
- 04 情報の転送における情報セキュリティの維持..... 297
- 05 脆弱性管理とアクセス管理..... 302

Chapter 7 模擬試験問題 313

索引 380

Chapter

1

擬似言語の基礎

本章の学習ポイント

- 変数は値を格納する入れ物であり、必ず型を指定する
- 変数の変化はトレース表を作ると正確に追うことができる
- 条件分岐により、変数の値に応じた処理を行うことができる
- 条件を満たさなくなるまで、一定区間の処理を繰り返すことができる
- 情報の最小単位であるビットの操作は、演算子を使って行える
- 手続や関数は、処理をひとまとまりにしたものである



01

変数



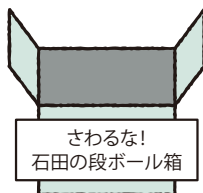
主記憶装置に値を入れる方法を学ぼう

- どんな種類の値を入れるのかを示す「型」について学ぼう
- 変数を使った四則演算や余りを得る方法を学ぼう
- 変数名が問題読解の大きなヒントになることを知ろう

変数とは

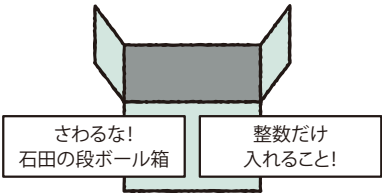
プログラムが扱う値には、数値（1、2、3など）や、日本語文字（「あ」「い」「う」など）、アルファベット（「a」「b」「c」など）といったさまざまな種類があります。私たち人間は、これらの値を脳で記憶しますが、プログラムの場合は主にコンピュータの主記憶装置に記憶します※1。

プログラム内で扱う値を主記憶装置に格納するとき、**変数**を用意します。変数は例えるなら段ボール箱のようなものです。段ボール箱を組み立てて、そこに1や「あ」といった値を詰め込むイメージです。そして、その段ボール箱には名前を付けます。



※1 コンピュータで扱う情報の最小単位はビットです。すべての情報はビットの組み合わせで構成され、ビットの連なりをビット列といいます。ビットの扱いについてはP.57で学習します。

擬似言語では、多くのプログラミング言語と同じように**データ型**（または**型**）を指定する必要があります。データ型には「整数」「文字」などの種類があります。つまり段ボール箱を用意するときに、「これは整数だけを入れる箱」「これは文字だけを入れる箱」などと、入れる値の種類を決めてから組み立てるようなものです。



整数だけを格納する変数を用意するときは、次のように記述します。

1 整数型：年齢

整数を入れるための段ボール箱を用意し、それに「年齢」という名前を付けました。この変数には、整数以外の値を入れることはできません。実際のプログラミング言語の場合でも、データ型と異なる値を変数に入れようとするとエラーになります^{※2}。

変数宣言

変数を用意することを**変数宣言**といいます。変数宣言では、必ずデータ型を指定します。擬似言語で用いられるデータ型の種類は次の通りです。

データ型の種類

型名	扱うデータ	例
整数型	整数	-1、0、1、99999
実数型	小数	-1.5、1.0、3.14
文字型	1つの文字	"A"、"B"
文字列型	複数の文字	"ABCDE"、"石田"
論理型	true（真）またはfalse（偽）のどちらか	true、false

※2 整数型の変数に小数を入れようとした際、プログラミング言語によっては自動的に小数点以下が切り捨てられて、エラーにならずに値が入る場合があります。

難しそうに感じるかもしれませんが、学習を進めることで、自然と理解できるようになるので安心してください。特に「論理型」はイメージが難しいかもしれませんが、多くの例題を解いていくうちに自然とマスターできます※³。

変数宣言しただけでは、中身が入っていない空^{から}の段ボール箱が用意されただけです。この状態を**未定義**と表現します。未定義の変数に値を入れるには、「←」を使って次のように記述します。

2 年齢 ← 20

また変数宣言と同時に値を格納することもできます。

1 整数型: 年齢 ← 20

このように変数宣言と値の格納は、次のように2つの方法があります。

- 宣言と値の格納を別々に行う方法
- 宣言と同時に値を格納する方法

また、複数の変数を一度に宣言することもできます。その場合、変数名をコンマ(,)で区切ります。複数の変数を一度に宣言する場合は、すべて同じデータ型にする必要があります。次の例は、整数型の変数として「年齢」「身長」「体重」を一度に宣言しています。

1 整数型: 年齢, 身長, 体重

複数の変数を一度に宣言するときも、宣言と同時に値を入れることが可能です。

1 整数型: 国語 ← 85, 数学 ← 79, 理科 ← 80

なお、擬似言語における変数名はアルファベットで表記されます。次の例は、実際に出題されるようなプログラムの記述です。

1 整数型: age, tall, weight

変数名は、そのプログラムがどのような処理を行うのかを理解するためのヒント

※³ 今後、変数以外でも「宣言」というキーワードが出てきますが、それらすべて「使えるように用意すること」を意味します。あまり日常で使う言葉ではありませんが、慣れておきましょう。

になります。また、公開されている基本情報技術者試験の問題を見ると、変数名が単なるヒントではなく、とても重要なヒントになっているケースもあります。とはいえ、一般的な英単語がわかっていれば大丈夫です。念のため、次の英単語とその意味を覚えておくとよいでしょう。

変数名によく使われる英単語

英単語	意味	英単語	意味
val、value	値	first、head	最初の要素
data	データ	middle	中央の要素
item	項目、要素	last、tail	最後の要素
words	単語	high	高い値、上限
list	リスト	low	低い値、下限
length	長さ	next	次の要素
count	数	prev、previous	前の要素
num、number	数値	left	左
temp、temporary	一時的な値	right	右
work	一時的な値	in	入力
pivot	基準値	out	出力
flag	フラグ (状態を示す)	array	配列
result	結果	matrix	行列
target	対象	binary	2進数
order	並び	tree	木構造
max	最大値	queue	キュー
min	最小値	stack	スタック

「意味」にある「配列」「木構造」「キュー」「スタック」などは、後の章で学習します。

代入

変数に値を入れることを**代入**といいます。代入する際にはいくつかの注意点があります。

● 文字型や文字列型への代入

文字型や文字列型の変数に値を入れる場合には、**値をダブルクォーテーション(")で囲む**必要があります。

```
1 文字型: class ← "A"  
2 文字列型: name ← "ISHIDA"
```

● データ型が合わないと代入できない

例えば整数型の変数に文字列型の値を代入することはできません。次のプログラムは誤った代入の例です。

```
1 整数型: age ← "ISHIDA" //誤り  
2 文字列型: name ← 20 //誤り
```

このように、変数のデータ型と値のデータ型が一致しない場合、代入することができません。

次のプログラムは変数に値を代入できます。

```
3 文字列型: post ← "0620041"
```

「0620041」は一見すると整数ですが、ダブルクォーテーションで囲まれているので、文字列として扱われます。

● 論理型への代入

論理型の変数に代入できるのは「true」または「false」のみです。次の例では、「isStudent」(生徒か?)という変数を宣言しています。

```
4 論理型: isStudent
```

変数isStudentに、「はい」を意味する「true」を代入します。

```
5 isStudent ← true
```

論理型は「true」や「false」のようにダブルクォーテーションで囲んではいけません。「true」「false」とそのまま記述します。

演算

変数に格納した数値を使って、計算を行うことができます。

```
1 整数型: a ← 10
2 整数型: b ← 20
3 整数型: c ← a + b
```

3行目に注目してください。「a + b」の計算を行い、その結果を変数cに代入しています。

また、小学校で学ぶ算数の四則演算と同じように、プログラムで行う計算にも優先順位があります。足し算や引き算よりも掛け算や割り算が優先され、さらに()で囲った部分の演算がもっとも優先されます。次のプログラムを実行した結果、各変数はどのようなになるかを考えてみてください。

```
1 整数型: a ← 2 + 3 × 4
2 整数型: b ← (2 + 3) × 4
3 整数型: c ← 10 - 2 × 3
4 整数型: d ← (10 - 2) × 3
```

変数aは14、変数bは20、変数cは4、変数dは24となります。

```
1 result ← 100 mod 3
```

「mod」は「^{モジュロ}modulo」の略で、割り算の余りを意味する演算子です。上のプログラムでは $100 \div 3$ の計算を行い、その余りである「1」が、変数resultに代入されます。

CBT方式の場合、modなどの演算子一覧は、画面上の参考資料から確認できませんが、覚えておいたほうがよいでしょう^{※4}。

※4 筆記方式の場合も試験用紙に説明が記載されているはずですが。

例題にチャレンジ

例題①

オリジナル問題

以下のプログラムを実行した結果、出力される値を解答群から選べ。

プログラム

```
1  整数型: price ← 120
2  整数型: quantity ← 3
3  整数型: discount ← 50
4  整数型: total ← price × quantity - discount
5  totalを出力
```

解答群

- ア 210
- イ 310
- ウ 360
- エ 510

解説 3行目までを実行した時点では、各変数は次のような状態です。

- price: 120
- quantity: 3
- discount: 50



解説動画

4行目の変数部分をそれぞれの値で置き換えます。

```
整数型: total ← 120 × 3 - 50
```

「 $120 \times 3 - 50$ 」の計算結果は310となります。これが変数totalに代入され、5行目で変数totalの値が出力されるため、正解は310のイです。

正解 イ

例題②

オリジナル問題

以下は請求額を3人で割り勘した際に余る金額を計算するプログラムである。このプログラムを実行した結果、出力される値を解答群から選べ。なお、 $x \bmod y$ は x を y で割った余りを返す。

プログラム

```
1  整数型: total ← 1000
2  整数型: people ← 3
3  整数型: share ← total ÷ people
4  整数型: remainder ← total mod people
5  shareとremainderをコンマ区切りで出力
```

解答群

- ア 333,0
- イ 333,1
- ウ 334,1
- エ 333,2
- オ 332,1

解説 CBT方式の場合、画面の「参考資料」から演算子の一覧を確認できますが、この例題のように問題文にmodの説明が記載されていることもあります。2行目までを実行した時点では、各変数は以下のようになっています。



解説動画

- total: 1000
- people: 3

3行目では、 $1000 \div 3$ の結果である「333」がshareに代入されます。4行目では $1000 \div 3$ の余りである「1」がremainderに代入されます。それらが5行目でコンマ区切りで出力されます。答えはイです。

正解 イ



02

トレース表



変数の値の変化を追跡する表の作り方を学ぼう

- 変数の変化が伴う場合に特に有効な表であるトレース表の作成方法を学ぼう

トレース表とは

問題によっては、プログラムの中で変数がどのように変化していくかを正確に追跡する必要があります。一般的なプログラムの開発環境では、開発に使用するツールにより、変数の値を容易に追跡することができます。しかし、科目B試験では自分で変数の変化を記録していく必要があります。その際に便利なのがトレース表です。トレース表の1行には、プログラムの1行と、そのプログラムの実行によって変数がどのように変化したかを記します。

プログラムを1行ずつ書く

プログラムを実行した結果、
変数がどう変化したかを書く

プログラム	price	quantity	discount	total
1: 整数型: price ← 120	120			
2: 整数型: quantity ← 3		3		
3: 整数型: discount ← 50			50	
4: 整数型: total ← price × quantity - discount				310

科目B試験ではA4のメモ用紙が配られます※5。メモ用紙に手書きでトレース表を書くとき、このように整ったトレース表にする必要はありません。目的は、選択肢から正解を1つだけ選ぶことなので、うまく省略しながら書いていきましょう。前述のトレース表であれば、次のように手書きすることをおすすめします。

※5 試験中にメモ用紙は何枚でももらうことができます。遠慮せずに、どんどんもらいましょう。

	p	q	d	total
p ← 120	120			
q ← 3		3		
d ← 50			50	
total ← p × q - d				310

変数名は頭文字をとって、priceであれば「p」、quantityであれば「q」などと省略してもよいでしょう。最適なトレース表は、プログラムや出題で問われることによって異なります。本書ではたくさんの例題を用意していますので、その都度、必要であればトレース表を記します。

なお、あまり変数に動きがないプログラムの場合には、無理をしてトレース表を作る必要はありません。トレース表の作成には時間がかかるため、作らなくても解けるのであれば、作らないに越したことはありません。

例題にチャレンジ

例題①

R4/12月サンプル問題-問1

次の記述中の に入れる正しい答えを、解答群の中から選べ。

プログラムを実行すると、と出力される。

プログラム

```
1  整数型: x ← 1
2  整数型: y ← 2
3  整数型: z ← 3
4  x ← y
5  y ← z
6  z ← x
7  yの値とzの値をこの順にコンマ区切りで出力する
```

解答群

- ア 1,2
- イ 1,3
- ウ 2,1
- エ 2,3
- オ 3,1
- カ 3,2

解説 1行目から3行目は変数の宣言と値の代入です。慣れるまでは細かくトレース表に書いたほうがよいのですが、慣れてきたら省略できる処理はできるだけ省略しましょう。今回は細かくトレースしていきます。

まずは1行目から3行目です。



解説動画

プログラム	x	y	z
1: 整数型: x ← 1	1		
2: 整数型: y ← 2		2	
3: 整数型: z ← 3			3

4行目でyの値をxに入れます。このときトレース表を参照し、yの値が何かを確認します。そして同じ値をxの欄に記述します。

プログラム	x	y	z
1: 整数型: $x \leftarrow 1$	1		
2: 整数型: $y \leftarrow 2$		2	
3: 整数型: $z \leftarrow 3$			3
4: $x \leftarrow y$	2		

5行目でzの値をyに入れます。また6行目ではxの値をzに入れます。

プログラム	x	y	z
1: 整数型: $x \leftarrow 1$	1		
2: 整数型: $y \leftarrow 2$		2	
3: 整数型: $z \leftarrow 3$			3
4: $x \leftarrow y$	2		
5: $y \leftarrow z$		3	
6: $z \leftarrow x$			2

7行目で、yの値とzの値をコンマ区切りで出力します。yの値とzの値はトレース表を見ると一目瞭然です。正解は「3,2」の**力**であることがわかります。

今回はプログラム欄にプログラムそのものを記述しました。試験本番ではそのような時間はないため、次のような手書きトレース表がよいと思います。

	x	y	z
$x \leftarrow 1$	1		
$y \leftarrow 2$		2	
$z \leftarrow 3$			3
$x \leftarrow y$	2		
$y \leftarrow z$		3	
$z \leftarrow x$			2

正解 **力**



03

条件分岐



変数の値に応じて処理を柔軟に変える方法を学ぼう

- 変数の値がある値と同じ場合の処理の記述方法を学ぼう
- 変数の値がある値と同じではない場合の処理の記述方法を学ぼう
- できるだけネストしないような記述方法を学ぼう

条件分岐とは

プログラムでは、主に変数の値を使って条件分岐を行います。このときに使われるのが**if文**と**比較演算子**です。比較演算子で値を比較し、結果に応じてどのルートを通るかが決まります。比較演算子には次のような種類があります。

比較演算子の種類

比較演算子	意味	例	結果
=	(左辺と右辺が) 等しい	20 = 20	真 (true)
≠	(左辺と右辺が) 等しくない	20 ≠ 10	真 (true)
<	(左辺が右辺より) 小さい	5 < 10	真 (true)
>	(左辺が右辺より) 大きい	5 > 10	偽 (false)
≤	(左辺が右辺) 以下	20 ≤ 20	真 (true)
≥	(左辺が右辺) 以上	20 ≥ 30	偽 (false)

if文と比較演算子を使った条件分岐は、次のように記述します。

```

1  整数型: age ← 20
2  if (age ≥ 18)
3    "成人"と出力
4  endif

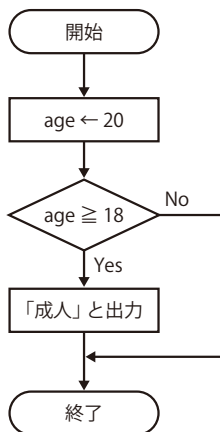
```

1 行目では整数型の変数 `age` に 20 という数値を代入しています。続く 2 行目の `if` 文では、**() 内に、判定する条件を記述します**。ここでは「`age` は 18 以上か？」を判定します。

もし結果が真（正しい）であれば、次の行（3 行目）に移動します。3 行目には「成人」と出力とあるので、その通り「成人」と画面に表示します。

もし 2 行目の結果が偽（誤り）であれば、「`endif`」の次の行に移動します。今回は「`endif`」が最終行なので、そこで処理は終了します。

このプログラムは次のような流れ図で表せます。



なお、**`if` 文の条件は比較演算子を使わずに、文章で「○が○以上」などと表現されている場合もあります**。先ほどのプログラムの条件（2 行目）は、次のように書き換えられます。

2 `if (ageが18以上)`

他にも「`age` が 18 より大きい」や「`age` が 18 と等しい」のように記述されることがあります。比較演算子を使った記述と、文章による記述の両方に慣れておきましょう※6。

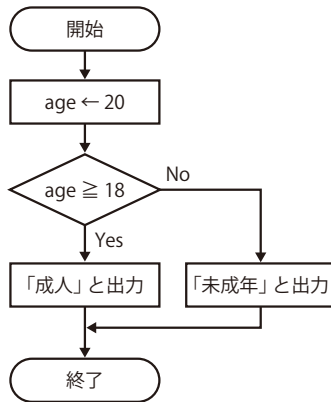
※6 一般的なプログラミング言語の場合、「`=`」は「`==`」、「`≠`」は「`!=`」、「`≤`」は「`<=`」、「`≥`」は「`>=`」などと、表記が異なります。

else による分岐処理

else は、「それ以外」「その他に」といった意味を持ちます。擬似言語では、条件が偽だった場合に実行したい処理を記述するために使用します。

```
1  整数型: age ← 20
2  if (age ≥ 18)
3      "成人"と出力
4  else
5      "未成年"と出力
6  endif
```

このプログラムは次のような流れ図で表せます。



5行目は、2行目のif文で条件を満たさない場合にのみ実行されます。逆に条件を満たす場合には、3行目を実行したあと、5行目の処理はスキップされ、6行目以降に進みます。

また、プログラムをよく見てみると、3行目と5行目は行頭が半角スペース2つ分空いていることがわかります。条件に一致した場合や、一致しなかった場合の処理は、次の図のように何文字がインデント（字下げ）するのが一般的です。試験で出題される擬似言語では、半角スペース2つ分だけインデントされています^{※7}。

※7 多くのプログラミングの現場では、インデントは半角スペース4つ分が主流です。

半角スペース2つ分だけ
インデントされている

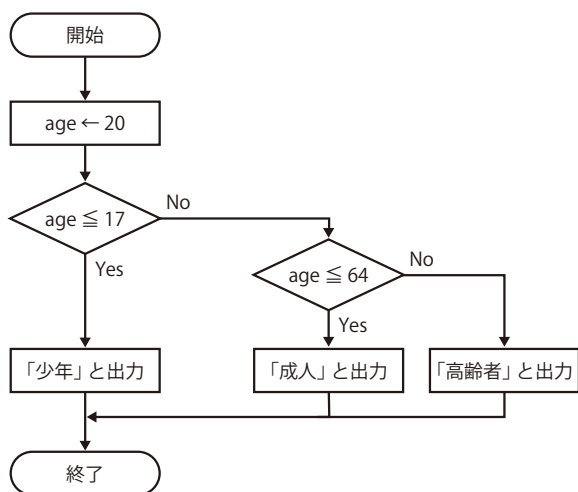
```
if (age ≥ 18)
    "成人"と出力
else
    "未成年"と出力
endif
```

elseif による分岐処理

先ほどの例では、年齢によって「成人」「未成年」のどちらか一方が出力されます。では年齢によって「少年」「成人」「高齢者」のいずれかに該当するかを判断するプログラムを書いてみます。

```
1  整数型: age ← 20
2  if (age ≤ 17)
3      "少年"と出力
4  else
5      if (age ≤ 64)
6          "成人"と出力
7      else
8          "高齢者"と出力
9      endif
10 endif
```

このプログラムは、「年齢が17以下」であれば「少年」と出力されますが、それ以外であれば5行目に移動します。そこで「64歳以下であるか？」を判断します。もし結果が真であれば「成人」と、それ以外であれば「高齢者」と出力されます。



では年齢によって「乳幼児」「少年」「青年」「壮年」「高齢者」に分けたい場合、次のようにするしかないのでしょうか。

```

1  整数型: age ← 20
2  if (age ≤ 6)
3    "乳幼児"と出力
4  else
5    if (age ≤ 17)
6      "少年"と出力
7    else
8      if (age ≤ 29)
9        "青年"と出力
10     else
11       if (age ≤ 64)
12         "壮年"と出力
13       else
14         "高齢者"と出力
15       endif
16     endif
17   endif
18 endif
  
```

このように、if文の中にさらにif文を書く構造を**ネスト構造（入れ子構造）**といいます。**ネスト構造はプログラムが読みにくく、可能であれば避けたほうがよい構造**です。前述のプログラムは、**elseif**を使うと読みやすくなります。

```

1  整数型: age ← 20
2  if (age ≤ 6)
3    "乳幼児"と出力
4  elseif (age ≤ 17)
5    "少年"と出力
6  elseif (age ≤ 29)
7    "青年"と出力
8  elseif (age ≤ 64)
9    "壮年"と出力
10 else
11   "高齢者"と出力
12 endif

```

elseifにも条件を記述できます。条件に一致した場合には、その内部の処理を行ってからendifの次の行に移動します。変数ageが6の場合と64の場合とで、プログラムが進む流れは次のように異なります。

```

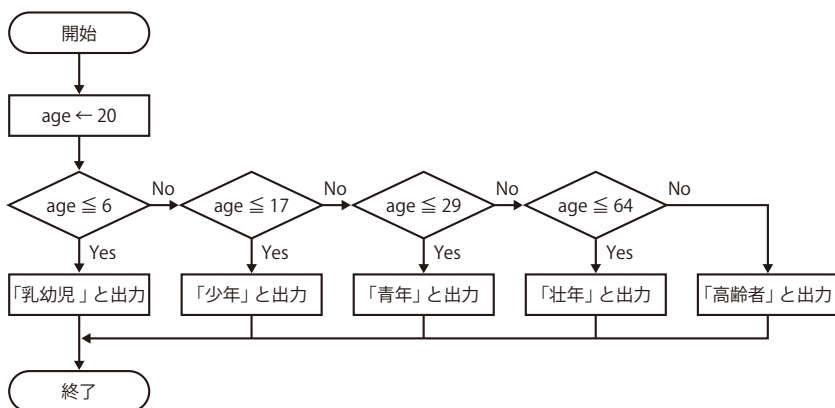
1  整数型: age ← 6
2  if (age ≤ 6)
3    "乳幼児"と出力
4  elseif (age ≤ 17)
5    "少年"と出力
6  elseif (age ≤ 29)
7    "青年"と出力
8  elseif (age ≤ 64)
9    "壮年"と出力
10 else
11   "高齢者"と出力
12 endif

```

```

1  整数型: age ← 64
2  if (age ≤ 6)
3    "乳幼児"と出力
4  elseif (age ≤ 17)
5    "少年"と出力
6  elseif (age ≤ 29)
7    "青年"と出力
8  elseif (age ≤ 64)
9    "壮年"と出力
10 else
11   "高齢者"と出力
12 endif

```



このプログラムは、年齢によって次のように結果が変わります。

- 年齢が6歳以下であれば「乳幼児」と出力
- 年齢が7歳以上17歳以下であれば「少年」と出力
- 年齢が18歳以上29歳以下であれば「青年」と出力
- 年齢が30歳以上64歳以下であれば「壮年」と出力
- それ以外の年齢であれば「高齢者」と出力

論理演算子

複数の条件を組み合わせて判断する場合には**論理演算子**を使います。

論理演算子

論理演算子	意味	例	結果
and	かつ	20 = 20 and 10 > 30	偽 (false)
or	または	20 = 20 or 10 > 30	真 (true)

「20 = 20 and 10 > 30」は「20は20、かつ、10は30より大きい」という判断条件です。当然答えは間違いなので、結果は偽 (false) となります。

もう一方の「20 = 20 or 10 > 30」は「20は20、または、10は30より大きい」という判断条件です。答えは正しいので、結果は真 (true) となります。

P.35のプログラムを論理演算子を使って書き換えてみました。「年齢が0以上、

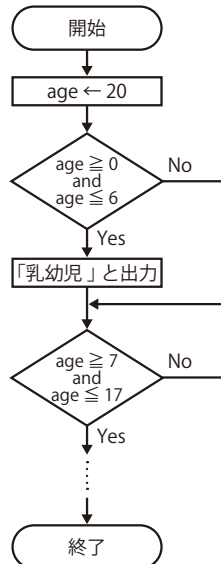
かつ、6以下」は「年齢が0から6の間」という意味になります。

```

1  整数型: age ← 20
2  if (age ≥ 0 and age ≤ 6)
3    "乳幼児"と出力
4  endif
5  if (age ≥ 7 and age ≤ 17)
6    "少年"と出力
7  endif
8  if (age ≥ 18 and age ≤ 29)
9    "青年"と出力
10 endif
11 if (age ≥ 30 and age ≤ 64)
12  "壮年"と出力
13 endif
14 if (age ≥ 65)
15  "高齢者"と出力
16 endif

```

流れ図にすると次のように表せます。



例題にチャレンジ

例題①

R4/12月サンプル問題 - 問2

次のプログラム中の **a** ～ **c** に入れる正しい答えの組合せを、解答群の中から選べ。

以下のプログラムは、numの値が、3で割り切れて5で割り切れない場合は"3で割り切れる"を、5で割り切れて3で割り切れない場合は"5で割り切れる"を、3と5で割り切れる場合は"3と5で割り切れる"を出力する。それ以外の場合は"3でも5でも割り切れない"と出力する。

プログラム

```
1  整数型: num // 整数が代入される
2  文字列型: result
3  if (numが a で割り切れる)
4    result ← "a で割り切れる"
5  elseif (numが b で割り切れる)
6    result ← "b で割り切れる"
7  elseif (numが c で割り切れる)
8    result ← "c で割り切れる"
9  else
10   result ← "3でも5でも割り切れない"
11 endif
12 resultを出力
```

解答群

	a	b	c
ア	3	3と5	5
イ	3	5	3と5
ウ	3と5	3	5
エ	5	3	3と5
オ	5	3と5	3

解説 仮に **a** が「3と5」であれば**ウ**で確定するため、最初に試してみましょう。この問題の場合、変数の値の変化を追う必要はないため、流れ図で処理の流れを追っていきます。

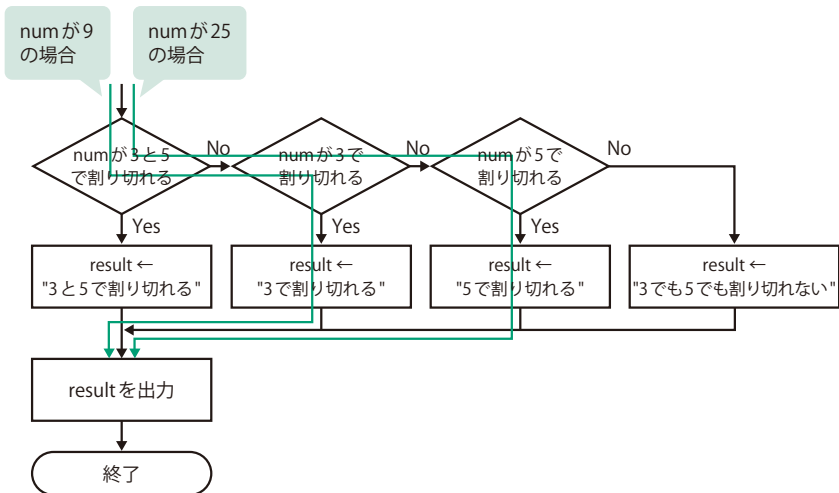


解説動画

numの値は示されていないため、適当な値を想定します。このプログラムでは3や5で割って、割り切れるかどうかを判断しています。そのため、まずは3だけで割り切れる値である9をnumに入れてみます。

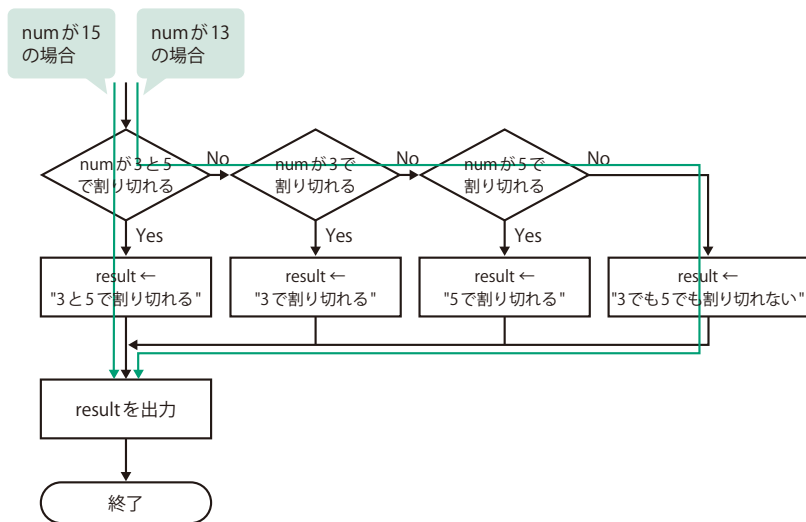
ウを選んだ場合、3行目は「if (9が3と5で割り切れる)」となります。結果は偽なので5行目に移動します。5行目は「if (9が3で割り切れる)」となります。結果は真なので6行目に移動し、「3で割り切れる」がresultに代入されます。resultは12行目でそのまま出力されるため、「3で割り切れる」が出力されます。9は3で割り切れるため、この時点では正しい動作のようです。

次にnumを25にしてみます。3行目は「if (25が3と5で割り切れる)」となります。結果は偽なので5行目に移動します。5行目は「if (25が3で割り切れる)」となり、結果は偽なので7行目に移動します。さらに7行目で「if (25が5で割り切れる)」となります。結果は真なので「5で割り切れる」と出力され、正しい動作であることが確認できました。

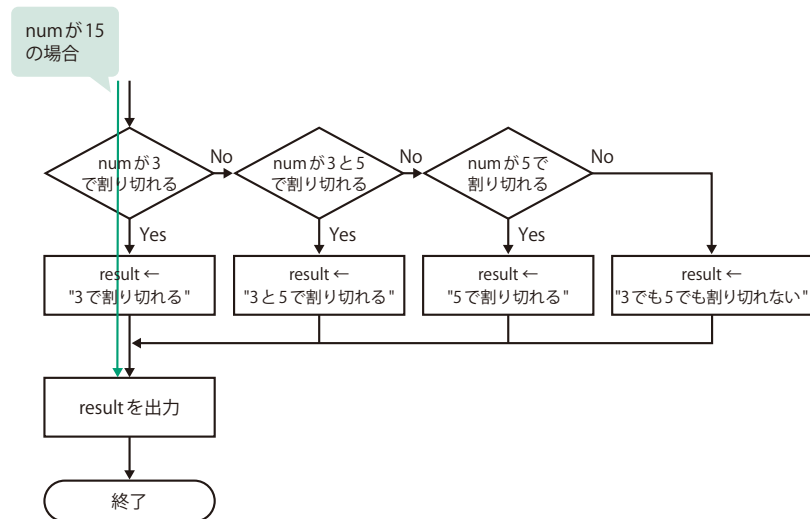


numを15にすると、3行目の判定で真となり、「3と5で割り切れる」と出力され、これも正しい出力です。

3と5のどちらでも割り切れないパターンを確認しておきたいので、numを13にしてみます。3行目、5行目、7行目の判定条件はいずれも偽であるため、10行目に移動し、最後に「3でも5でも割り切れない」と出力されます。これも正しい動作であることが確認できました。したがって、正解は**ウ**です。



念のため、誤りである解答も試してみましょう。例えば**ア**を選んだとします。numを15としてプログラムを実行すると、3行目の判定で真となります。「3で割り切れる」と出力されますが、正しい出力は「3と5で割り切れる」なので、**ア**は誤りです。



他の選択肢も同様におかしな出力となります。

正解 **ウ**

例題②

オリジナル問題

次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

以下のプログラムは、異なる三つの整数のうちの最大値を出力する。

プログラム

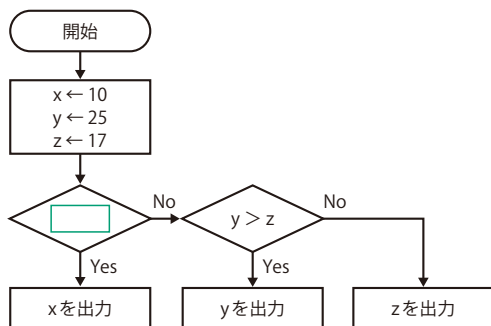
```
1  整数型: x ← 10
2  整数型: y ← 25
3  整数型: z ← 17
4  if (  )
5      xを出力
6  elseif (y > z)
7      yを出力
8  else
9      zを出力
10 endif
```

解答群

- ア $x > y$
- イ $x > y$ and $x > z$
- ウ $x > y$ and $y > z$
- エ $x > z$
- オ $x > z$ and $z > y$
- カ $z > y$



解説動画

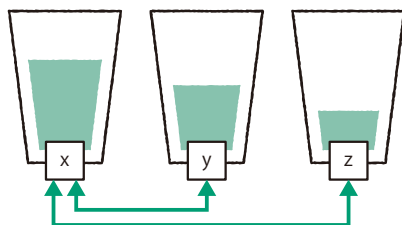


このようにこの問題では、流れ図を書くだけで全体像が把握しやすくなります。試験本番でも、条件分岐が複数あるプログラムの場合には、メモ用紙に手書きで流れ図を書いてみるとよいでしょう。

□の条件の結果が真の場合にxが出力されるわけですから、xが最大値であると判断されます。仮に□が、力の「 $z > y$ 」だとすると、「zがyより大きい」という条件だけで「xが最大」と判断されてしまいますので誤りです。

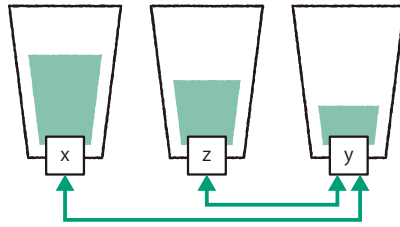
次に論理演算子が入っていないアとエについて検討してみます。アは「 $x > y$ 」です。「xはyより大きいのなら、xは最大値である」となりますが、これは正しい判断でしょうか。xが最大値であるかは、zとの比較も必要なので誤りです。

エの場合も「xはzより大きいのなら、xは最大値である」となり、これも誤りです。このようにxが最大値であるかどうかは、必ず他の2つの変数との比較も必要になります。

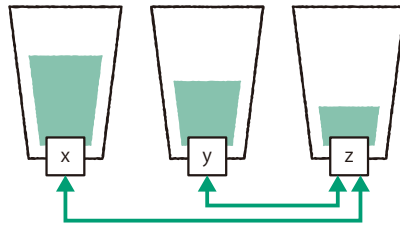


イはxとy、xとzを比較しているため、正しくxが最大値であるかが判断できます。したがって、正解はイです。

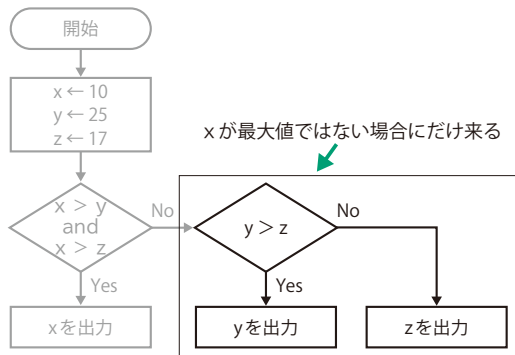
他の選択肢についても見ておきましょう。ウはxとy、yとzの比較だけをしています。例えば、次のようにxが最大値にもかかわらず、「yがzより大きい」を満たしていないという理由だけで、xは最大値ではないと誤って判断されてしまいます。そのため、ウは誤りです。



これはxとz、zとyの比較だけをしています。xが最大値にもかかわらず、「zがyより大きい」を満たしていないという理由だけで、xは最大値ではないと誤って判断されてしまいます。



なお正しい処理では、6行目に来た時点でxが最大値ではないことが判明しています。



そのため、yがzより大きいという理由だけで、yが最大値であることが判明します。6行目の条件も偽 (false) であれば、zが最大値であることが判明します。

正解 **イ**



04

繰り返し



同じ処理を何度も繰り返すことができる方法を学ぼう

- 繰り返しを続ける条件の記述方法を学ぼう
- 処理の最後に「続けるか」を判断する場合の記述方法を学ぼう
- 値が単純に変化する場合に便利な記述方法を学ぼう

繰り返しとは

同じ処理を繰り返す記述について学習します。「繰り返し」は原則として、いつまで繰り返すかという継続条件や、いつ終わるのかという終了条件を明確にしたうえで処理を反復します。

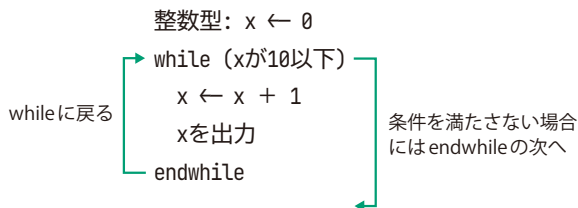
繰り返しが終わらない場合には、いわゆる**無限ループ**となります。プログラミングの現場では意図的に無限ループを使う場面もありますが、科目B試験では必ずループを継続する条件や、ループを終了する条件が存在します。この条件を正しく把握し、それを満たしているかどうかを判断するには、特にトレース表が有効です。

while

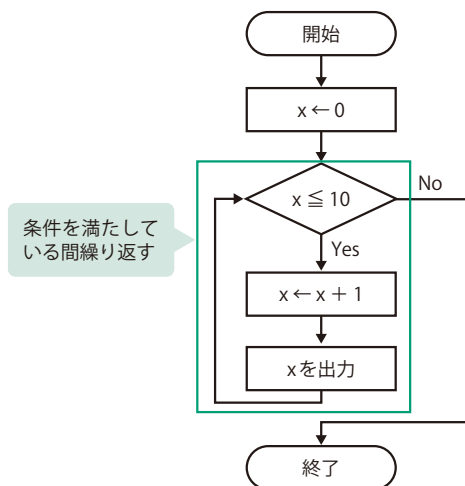
繰り返しを行うにはいくつかの方法があります。まずは^{ホワイル}**while**を使った方法から学習します。次のプログラムの2行目が繰り返しの開始であり、**while**の()**内には繰り返しを継続する条件（継続条件）を記述**します。

```
1  整数型: x ← 0
2  while (xが10以下)
3    x ← x + 1
4    xを出力
5  endwhile
```


2行目で条件を満たしているかを判断し、もし満たしていなければendwhileの次の行に移動します。もし満たしていれば、3行目、4行目を処理し、endwhileに到達したタイミングですぐにwhileに戻ります。



このプログラムを流れ図にすると、次のようになります。「xが10以下」という条件を満たしている間、処理を繰り返します。



繰り返し処理では変数の値がコロコロと変わることが多いため、トレース表を作成してプログラムの流れを追うのが有効です。では先ほどのプログラムのトレース表を作成してみましょう。

1行目でxが0となり、2行目で判定が行われます。条件を満たしているため、3行目に移動してxをカウントアップし、その結果を4行目で出力します。

プログラム	x
1: 整数型: $x \leftarrow 0$	0
2: while (xが10以下)	
3: $x \leftarrow x + 1$	1
4: xを出力	

条件を満たしているため
次の行へ移動

5行目のendwhileで処理をwhileに戻します。whileでは続けてもよいかの判定を行います。今回は条件を満たしているため、再び3行目に入ります。以降も同様に繰り返しを行います。以下はその途中経過です。

プログラム	x
1: 整数型: $x \leftarrow 0$	0
2: while (xが10以下)	
3: $x \leftarrow x + 1$	1
4: xを出力	
2: while (xが10以下)	
3: $x \leftarrow x + 1$	2
4: xを出力	
2: while (xが10以下)	
3: $x \leftarrow x + 1$	3
4: xを出力	
2: while (xが10以下)	
3: $x \leftarrow x + 1$	4
4: xを出力	

endwhileはトレースにおいて無関係であるため省略してください。同様に続けます。

プログラム	x
2: while (xが10以下)	
3: $x \leftarrow x + 1$	8
4: xを出力	
2: while(xが10以下)	
3: $x \leftarrow x + 1$	9
4: xを出力	
2: while (xが10以下)	
3: $x \leftarrow x + 1$	10
4: xを出力	
2: while (xが10以下)	

条件を満たさないため
endwhileの次の行へ移動

whileに記述されている条件を満たさなくなった時点で、endwhileの次の行に移ります。今回は最後の行がendwhileなのでここで処理が終了です。

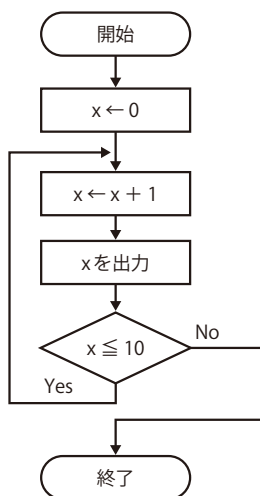
do

doを使うと、繰り返しの継続条件を最後に記述することができるようになります。

```

1  整数型:  $x \leftarrow 0$ 
2  do
3     $x \leftarrow x + 1$ 
4    xを出力
5  while (xが10以下)
```

繰り返しの開始位置にdoを記述し、繰り返しの終了位置にwhileを記述します。



for

ここまでに見たプログラムのように、単純に値をカウントアップしていくのであれば**for**を使うと便利です。同様の処理が、たったの4行になりました。

```
1  整数型: x
2  for (xを1から10まで1ずつ増やす)
3    xを出力
4  endfor
```

forの()内で、変数に初期値を代入し、さらに終了条件の判定を行います。また「1ずつ増やす」と記述がありますが、実際にはendforの行で、値を増やしたり減らしたりします。

次のトレース表のとおり、2行目で初期値を代入し、それ以降はendforがある4行目で値の変更が行われています。

プログラム	x
2 : for (xを1から10まで1ずつ増やす)	1
3 : xを出力	
4 : endfor	2
2 : for (xを1から10まで1ずつ増やす)	
3 : xを出力	
4 : endfor	3

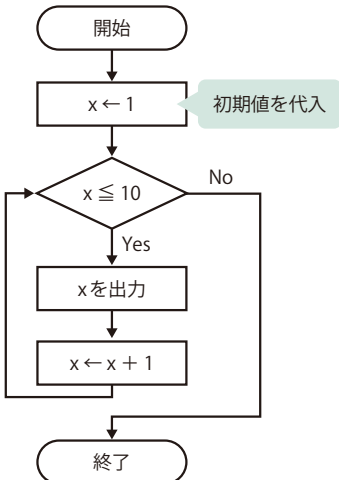
初期値を代入

⋮

2 : for (xを1から10まで1ずつ増やす)	
3 : xを出力	
4 : endfor	9
2 : for (xを1から10まで1ずつ増やす)	
3 : xを出力	
4 : endfor	10
2 : for (xを1から10まで1ずつ増やす)	

終了条件を満たすため
繰り返しを終了

次のような流れ図になりますが、これを理解するよりも、forに書かれた日本語の通りに繰り返しを行うようにしてください。



例題にチャレンジ

例題①

オリジナル問題

次のプログラム中の□に入る正しい答えを、解答群の中から選べ。なお文字列の結合は「+」演算子を使って行う。

以下のプログラムは、文字列を逆順に並び替えて出力する。例えば文字列が「Hello」であれば、出力は「olleH」となる。

プログラム

```
1 文字列型: message ← "Hello"
2 文字列型: reversed ← ""
3 整数型: index ← messageの文字数
4 while (□)
5     reversed ← reversed + messageのindex番目の文字
6     index ← index - 1
7 endwhile
8 reversedを出力
```

解答群

- ア indexが0以上
- イ indexが1以上
- ウ indexが0以下
- エ indexが1以下

解説 まずはシンプルな繰り返し処理から攻略していきましょう。文字列型の変数messageには「Hello」が入っています。これを逆順にした「olleH」が変数reversedに入り、8行目で出力されるのが、正しい動作です。



解説動画

このプログラムは繰り返しにより、変数の値が何度も変化するとと思われるため、トレース表を作って考えてみましょう。

次のトレース表は、**ア**を選んだ場合の途中経過です。3行目でindexの値は5になります(変数messageの文字数)。変数messageの5番目の文字は「o」です。また変数reversedは""(空)です。5行目で"" + "o"を実行して文字列の結合を行い、その結果をreversedに代入します。また6行目でindexをカウントダウンし、4になります。

プログラム	message	reversed	index
1: 文字列型: message ← "Hello"	Hello		
2: 文字列型: reversed ← ""			
3: 整数型: index ← messageの文字数			5
4: while (indexが0以上)			
5: reversed ← reversed + messageのindex番目の文字		o	
6: index ← index - 1			4

同様に繰り返していきますが、次のトレース表の最終行でエラーになります。messageの0番目の文字が存在しないためです。したがって、**A**は誤りです。

4: while (indexが0以上)			
5: reversed ← reversed + messageのindex番目の文字		ol	
6: index ← index - 1			3
4: while (indexが0以上)			
5: reversed ← reversed + messageのindex番目の文字		oll	
6: index ← index - 1			2
4: while (indexが0以上)			
5: reversed ← reversed + messageのindex番目の文字		olle	
6: index ← index - 1			1
4: while (indexが0以上)			
5: reversed ← reversed + messageのindex番目の文字		olleH	
6: index ← index - 1			0
4: while (indexが0以上)			
5: reversed ← reversed + messageのindex番目の文字	messageの0番目の文字は存在しない		

どこで間違ったのでしょうか。エラーを防ぐためには、1文字目を出力したあと、繰り返し処理を終了させるべきです。**イ**の「indexが1以上」にした場合、indexが1になるまで繰り返し、0になると繰り返しから抜けます。上記のトレース表の最終行は実行されず、正しい結果となります。したがって、正解は**イ**です。

正解 **イ**

例題②

オリジナル問題

次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

以下のプログラムは、文字列を逆順に並び替えて出力する。例えば「Hello」であれば「olleH」となる。

プログラム

```
1 文字列型: message ← "Hello"
2 文字列型: reversed ← ""
3 整数型: index
4 for (  )
5     reversed ← reversed + messageのindex番目の文字
6 endfor
7 reversed を出力
```

解答群

- ア indexを0からmessageの文字数まで1ずつ増やす
- イ indexを1からmessageの文字数まで1ずつ増やす
- ウ indexをmessageの文字数から0まで1ずつ減らす
- エ indexをmessageの文字数から1まで1ずつ減らす

解説 例題1のwhileを使ったプログラムと同じ処理を、forに書き換えたものです。



解説動画

最初の繰り返しでは、messageの最後の文字を取得したいはずですが、そのため、indexの初期値が0や1になるのは誤りであり、**ア**と**イ**は候補から削ります。先ほど見たように、messageの0文字目は取得できません。そのため「0まで」となっている**ウ**も誤りです。したがって、正解は**エ**です。

正解 **エ**

例題③

R6 公開問題 - 問2 改変

次のプログラム中の に入れる正しい答えを，解答群の中から選べ。

以下のプログラムは，"0"と"1" だけから成る，1文字以上の文字列binaryを，符号なしの2進数と解釈したときの整数値を出力する。例えば，引数として"10010"を与える と18が出力される。

プログラム

```

1 文字列型: binary ← "10010"
2 整数型: i, length, result ← 0
3 length ← binary の文字数
4 for (iを1からlengthまで1ずつ増やす)
5     result ← 
6 endfor
7 resultを出力

```

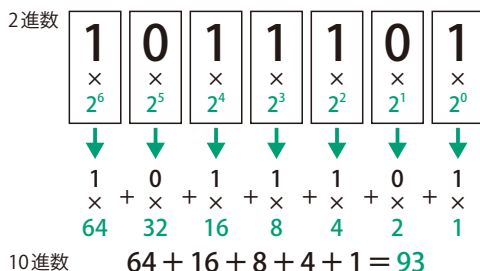
解答群

- ア result + (binaryの (length - i + 1) 文字目の文字) を整数にした値
 イ result + (binaryのi文字目の文字) を整数にした値
 ウ result × 2 + (binaryの (length - i + 1) 文字目の文字) を整数にした値
 エ result × 2 + (binaryのi文字目の文字) を整数にした値

解説 プログラミングの知識に加え、2進数を10進数に変換する方法についての知識も必要です。各桁に対して2の累乗を掛けて、最後にそれらの結果を合算することで、2進数を10進数に変換できます※8。



解説動画



※8 2進数に関する問題は科目Aでも出題されます。

まず右端の桁の値(0または1)に 2^0 を乗じます。次の桁の値には 2^1 を乗じます。同様に3桁目の値に 2^2 を、4桁目の値に 2^3 を乗じます。これを最後まで行い、すべての結果を合計することで10進数に変換できます。

これと同じことをプログラムで実現する必要があります。変数の値が変化していくため、今回もトレース表を作ります。まずは変数の宣言と代入部分です。

プログラム	binary	i	length	result
1: 文字列型: binary ← "10010"	10010			
2: 整数型: i, length, result ← 0				0
3: length ← binary の文字数			5	

forは「iを1からlengthまで1ずつ増やす」と記述されていましたが、lengthは5で固定であるため「iを1から5まで1ずつ増やす」と解釈します。また解答群にある「○文字目」も具体的な値とします。このように**トレース表では、変数が固定値であればその値で置換してプログラム欄に記述してください。**

では空欄に選択肢**A**の内容を入れてトレース表を作成します。なおbinaryとlengthは変化しませんので、トレースの対象から外します。**A**の「(binaryの(length - i + 1)文字目の文字)」は「(binaryの5文字目の文字)」と解釈できますので、resultには0が入ります。

プログラム	i	result
4: for (iを1から5まで1ずつ増やす)	1	
5: result ← result + (binaryの(5)文字目の文字)を整数にした値		0

同様に続けます。

プログラム	i	result
4: for (iを1から5まで1ずつ増やす)	1	
5: result ← result + (binaryの(5)文字目の文字)を整数にした値		0
6: endfor	2	
4: for (iを1から5まで1ずつ増やす)		
5: result ← result + (binaryの(4)文字目の文字)を整数にした値		1
6: endfor	3	
4: for (iを1から5まで1ずつ増やす)		
5: result ← result + (binaryの(3)文字目の文字)を整数にした値		1
6: endfor	4	

途中ですが、ここで処理の誤りに気付くことができます。この処理ではbinaryの桁を1つずつ確認し、もし「1」であれば、整数「1」をresultに加算しているだけです。「10010」に「1」は2つあるため、このまま続けるとresultは2になります。得たい結果は18なので**ア**は誤りです。このように、トレースの途中で「このプログラムが何をしようとしているのか」に気付くと、正しい解答にたどり着きやすくなります。

では**イ**はどうでしょうか。処理する桁の順番が逆になるだけで、結果は**ア**と同じになるはずです。



次に**ウ**をトレースしていきます。

プログラム	i	result	
4: for (iを1から5まで1ずつ増やす)	1		
5: result ← result × 2 + (binaryの(5)文字目の文字)を整数にした値		0	0 × 2 + 0
6: endfor	2		
4: for (iを1から5まで1ずつ増やす)			
5: result ← result × 2 + (binaryの(4)文字目の文字)を整数にした値		1	0 × 2 + 1
6: endfor	3		
4: for (iを1から5まで1ずつ増やす)			
5: result ← result × 2 + (binaryの(3)文字目の文字)を整数にした値		2	1 × 2 + 0
6: endfor	4		
4: for (iを1から5まで1ずつ増やす)			
5: result ← result × 2 + (binaryの(2)文字目の文字)を整数にした値		4	2 × 2 + 0
6: endfor	5		
4: for (iを1から5まで1ずつ増やす)			
5: result ← result × 2 + (binaryの(1)文字目の文字)を整数にした値		9	4 × 2 + 1

resultは9になりました。得たいのは18であるため、**ウ**も誤りです。残った**エ**が正解だと考えられます。**エ**は、**ウ**とは逆順に処理していきます。

プログラム	i	result	
4: for (iを1から5まで1ずつ増やす)	1		
5: result ← result × 2 + (binaryの(1)文字目の文字)を整数にした値		1	$0 \times 2 + 1$
6: endfor	2		
4: for (iを1から5まで1ずつ増やす)			
5: result ← result × 2 + (binaryの(2)文字目の文字)を整数にした値		2	$1 \times 2 + 0$
6: endfor	3		
4: for (iを1から5まで1ずつ増やす)			
5: result ← result × 2 + (binaryの(3)文字目の文字)を整数にした値		4	$2 \times 2 + 0$
6: endfor	4		
4: for (iを1から5まで1ずつ増やす)			
5: result ← result × 2 + (binaryの(4)文字目の文字)を整数にした値		9	$4 \times 2 + 1$
6: endfor	5		
4: for (iを1から5まで1ずつ増やす)			
5: result ← result × 2 + (binaryの(5)文字目の文字)を整数にした値		18	$9 \times 2 + 0$

resultが18となりました。したがって、正解は**エ**です。

正解 **エ**



05

ビット演算



情報の最小単位であるビットを自在に操る方法を学ぼう

- ビットをずらすことで掛け算や割り算を高速に行うことができる理由を学ぼう
- 論理演算を使ったビットを変換する方法を学ぼう
- ビットマスクを使ってあるビット列だけを取り出す方法を学ぼう

ビット演算とは

これまで学習したように、変数に値を代入するという操作は、その値を主記憶装置に記録することを意味します。例えば人間は「age ← 5」というプログラムを見ると「ageに5を代入」と理解します。しかしコンピュータ内部では「5」を2進数である「101」に変換して、主記憶装置に格納します。このようにすべての値は0と1の組合せであるビット列で表現されています。これから学習するビット演算は、このようなビットそのものを直接操作する方法です。

シフト演算

コンピュータ内部では、すべてのデータは0と1の組合せであるビット列で扱われています。そのため、計算もこのビット列の各ビットを操作することで行われます。その操作の1つがシフト演算です。シフト演算は、掛け算や割り算を高速に行う手段です。2進数を10進数に変換する方法は、P.53の例題を通して説明しました。

2進数

1	0	1	1	1	0	1
×	×	×	×	×	×	×
2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
↓	↓	↓	↓	↓	↓	↓
1	0	1	1	1	0	1
×	+	×	+	×	+	×
64	32	16	8	4	2	1

10進数 $64 + 16 + 8 + 4 + 1 = 93$

2進数「1011101」は、10進数では「93」になります。ではこの2進数を左に1ビットだけずらしてみます。

1011101
↓ ↓ ↓ ↓ ↓ ↓ ↓
10111010

ないビットは0とする

「10111010」となりました。今度は上記と同様の方法で10進数に変換してみます。

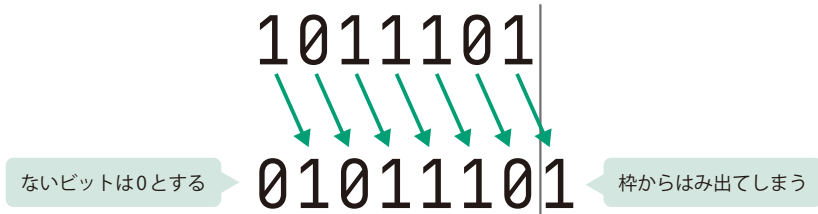
2進数

1	0	1	1	1	0	1	0
×	×	×	×	×	×	×	×
2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
↓	↓	↓	↓	↓	↓	↓	↓
1	0	1	1	1	0	1	0
×	+	×	+	×	+	×	+
128	64	32	16	8	4	2	1

10進数 $128 + 32 + 16 + 8 + 2 = 186$

186になりました。これはちょうど、ずらす前の値(2進数「1011101」)の2倍です。このように、ある2進数を左に1ビットだけずらすことで、値は2倍になります。したがって、値を2倍にしたいときには、左に1ビットずらすだけで済みます。このように、ずらすことで値を操作する演算を**シフト演算**といいます。特に左にシフトすることを**左シフト**といいます。左シフトを1度だけ行くと、もとの

値の2倍になります。さらに左シフトを行うともとの値の4倍、さらに左シフトを行うともとの値の8倍になります。左に1ビットシフトすると値が2倍になるわけですから、**右に1ビットシフトすると値は1/2**になります。これを**右シフト**といいます。



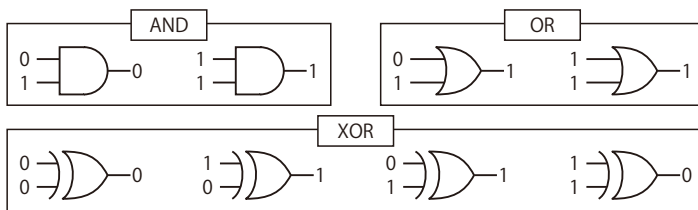
右端のビットが枠からはみ出てしまいました。これは割り算における余りを意味します。もとの値である「1011101」を右シフトすると結果が「0101110」で、余りが「1」になったわけです。10進数で表現すると「46余り1」となり、 $93 \div 2$ の計算結果と一致します。なお、コンピュータ内部にはシフト演算を行うための回路としてシフターが備わっています。シフターのおかげで掛け算や割り算をした結果が一瞬で得られます。

科目Bでは、掛け算または割り算するためにシフト演算を含むプログラムについて出題されることがあります。

論理演算

コンピュータ内部には加算を行うための装置も備わっており、これを**加算器**といいます。加算器は複数の**論理回路**で構成されており、論理回路には種類があります。論理回路には2つの入力端子と1つの出力端子があります。高い電圧をかけた状態を「1」、電圧をかけない（もしくは低い電圧をかける）状態を「0」とすると、動作は次のように変化します。

- AND回路：2つの入力とともに「1」の場合、出力が「1」になる
- OR回路：2つの入力のどちらかが「1」の場合、出力が「1」になる
- XOR回路：どちらか一方の入力だけが「1」の場合、出力が「1」になる



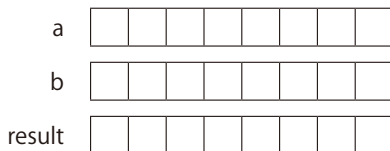
プログラムでは、AND回路やOR回路の動作に相当する処理として**論理演算**があります。論理演算では論理回路と同様に、0または1を入力として与えることで、論理演算の種類に応じた出力を得ることができます。例を見ていきましょう。

```

1  8ビット型: a ← 1
2  8ビット型: b ← 00001111
3  8ビット型: result
4  result ← a ∧ b
5  resultを出力

```

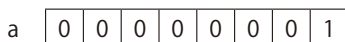
8ビット型という新たな型が出てきました。〇〇ビット型というのは、2進数を扱う型です。8ビット型であるaやb、resultには、8ビットの領域が用意されます。



1行目では、aに1を入れています、次のようにはなりません。



2進数には0と1しかありません。何も入っていない領域は「0」となるため、次のようになります。



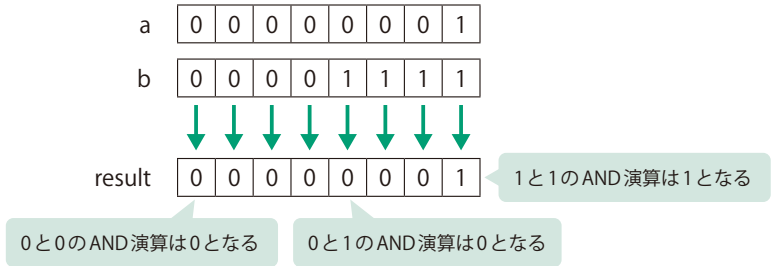
bには00001111を入れているため、次のようになります。



3行目では8ビット型の変数resultを宣言しているだけです。値を代入していませんが、何も入っていない領域は「0」になるため、次のようになります。



4行目の「^」はAND演算を表す記号です。AND演算とは、AND回路と同様の動作を行う演算です。「00000001」と「00001111」についてビットごとに論理演算を行い、結果を変数resultに代入します。



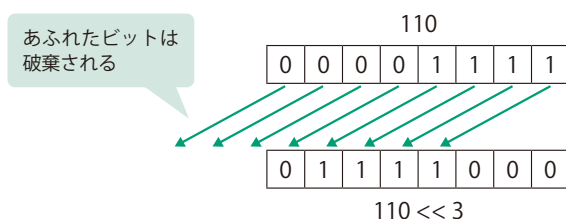
ビット操作を行う演算子には次のようなものがあります※9。

ビット操作を行う演算子

ビット操作	演算子	演算の例	結果
論理積 (AND)	$\wedge$	$0 \wedge 1$	0
論理和 (OR)	$\vee$	$1 \vee 0$	1
左シフト	$\ll$	$110 \ll 3$	110000
右シフト	$\gg$	$11010 \gg 1$	1101

※9 執筆時点の公開問題を見るかぎりでは、これらの演算子の意味は問題文に明記されていました。しかし念のため、軽く暗記しておくといえでしょう。「^」はアルファベットの「A」に似ているため「AND」と結び付けてください。

「110 << 3」は、110を3ビット左シフトする操作です。「11010 >> 1」は、11010を1ビット右シフトする操作です※10。



ビットマスク

公開問題を見るかぎりは、次のように記述することで、目的とする文字だけを得ることができるようです。

1 文字列型: $a \leftarrow \text{ISHIDAの3番目の文字}$

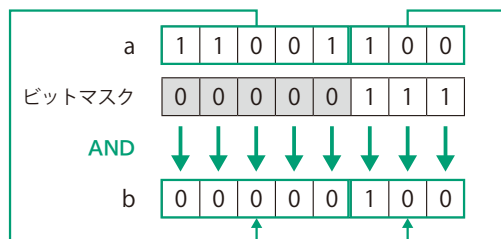
「ISHIDA」の3番目の文字は「H」なので、 a には「H」が代入されます。ビット列でも「3番目のビット」と記述することで、3ビット目だけを得ることができると思われます。またサンプル問題では、ビットマスクを使ってある特定のビット列だけを得る方法も使われていました。**ビットマスク**とは、目隠しするためのマスクの役割を持たせたビット列です。

ここに「11001100」というビット列があったとします。このビット列の右から3ビットだけを得たいと考えました。この場合、右から3ビットを1とし、それ以外を0としたビット列を用意します。これがビットマスクです。

a	1	1	0	0	1	1	0	0
ビットマスク	0	0	0	0	0	1	1	1

そして**もとの値（上記のa）とビットマスクの論理積の演算**を行います。

※10 ビットがあふれることをオーバーフローといいます。これをエラーとするかどうかはプログラミング言語によって異なりますが、もしエラーにならない場合は意図しない値になることがあります。

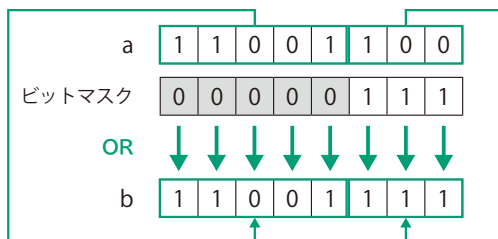


ビットマスクで0にした
ビットは、結果も0となる

ビットマスクで1にしたビットは、
もとのビットがそのまま出てくる

このように、ビットマスクを使うことで目的とするビット列だけを得る処理が使われることがあります。

また**論理和を使ったビットマスク**もよく使われます。この場合、ビットマスクで「0」としたビットは、もとのビットがそのまま出力されます。ビットマスクで「1」としたビットは、結果も必ず「1」となります。



ビットマスクで0にしたビットは、
元のビットと同じになる

ビットマスクで1にした
ビットは、結果も1になる

例題にチャレンジ

例題①

オリジナル問題

次の記述中の に入れる正しい答えを、解答群の中から選べ。

次のプログラムを実行すると が出力される。

なお、演算子 $\wedge$ はビット単位の論理積、演算子 $\vee$ はビット単位の論理和、演算子 $\gg$ は論理右シフト、演算子 $\ll$ は論理左シフトを表す。例えば、`value >> n` は `value` の値を `n` ビットだけ右に論理シフトし、`value << n` は `value` の値を `n` ビットだけ左に論理シフトする。

プログラム

```
1 8ビット型: a ← 11001100
2 8ビット型: b ← 10101010
3 8ビット型: r1 ← a ∧ b
4 8ビット型: r2 ← a ∨ b
5 8ビット型: r3 ← r1 ∧ r2
6 8ビット型: r4 ← r3 >> 1
7 r4を出力
```

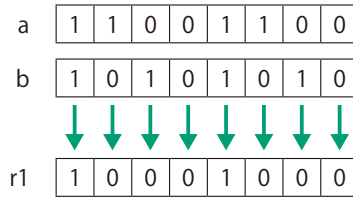
解答群

- ア 00110110
- イ 01000100
- ウ 00010010
- エ 00100010

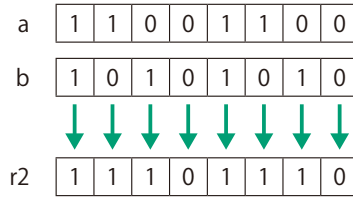
解説 1行目では変数 `a` に「11001100」が、2行目では変数 `b` に「10101010」が代入されます。3行目ではこれらの論理積の演算を行い、結果を `r1` に代入します。



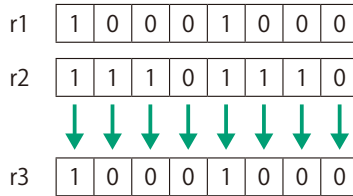
解説動画



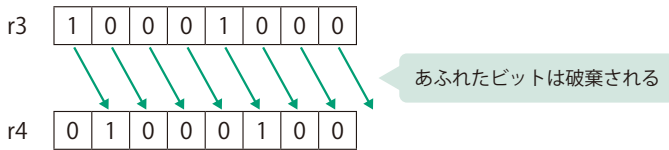
4行目では変数aとbの論理和の演算を行い、結果をr2に代入しています。



これらの結果を使って、5行目では論理積の演算を行い、結果をr3に代入します。



最後にr3を右に1ビットシフトし、結果をr4に代入します。



7行目で「01000100」が出力されるため、正解はイです。

正解 イ

例題②

R4/12月サンプル問題-問6改変

次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

以下のプログラムは8ビット型の変数byteのビットの並びを逆にした値を出力する。例えば、byteの値が01001011の場合には、戻り値は11010010となる。

なお、演算子 $\wedge$ はビット単位の論理積、演算子 $\vee$ はビット単位の論理和、演算子 $\gg$ は論理右シフト、演算子 $\ll$ は論理左シフトを表す。例えば、value $\gg$ nはvalueの値をnビットだけ右に論理シフトし、value $\ll$ nはvalueの値をnビットだけ左に論理シフトする。

プログラム

```

1  8ビット型: byte ← 01001011
2  8ビット型: rbyte ← byte
3  8ビット型: r ← 00000000
4  整数型: i
5  for (iを1から8まで1ずつ増やす)
6      
7
8  endfor
9  rを出力

```

解答群

	プログラム
ア	$r \leftarrow (r \ll 1) \vee (rbyte \wedge 00000001)$ $rbyte \leftarrow rbyte \gg 1$
イ	$r \leftarrow (r \ll 7) \vee (rbyte \wedge 00000001)$ $rbyte \leftarrow rbyte \gg 7$
ウ	$r \leftarrow (rbyte \ll 1) \vee (rbyte \gg 7)$ $rbyte \leftarrow r$
エ	$r \leftarrow (rbyte \gg 1) \vee (rbyte \ll 7)$ $rbyte \leftarrow r$

解説 8ビットのビット列を逆順にするのであれば「1ビット目を8ビット目へ」「2ビット目を7ビット目へ」……「8ビット目を1ビット目へ」のように単純なアルゴリズムで実現できそうです。しかしこの問題のプログラムと各選



解説動画

択肢には「○ビット目を○ビット目へ」という処理は見当たりません。あるのは論理演算だけです。論理演算だけでビット列を逆順にできる理由は、トレースしながら考えていきましょう。なお、1行目では変数byteにビット列を代入していますが、それ以降で値の変更はないため、トレース表への記述は省略します。

プログラム	rbyte	r	i
2: 8ビット型: rbyte ← byte	01001011		
3: 8ビット型: r ← 00000000		00000000	
4: 整数型: i			
5: for (iを1から8まで1ずつ増やす)			1

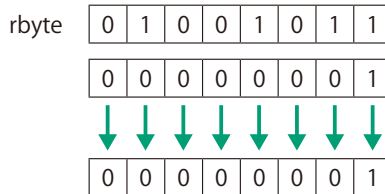
まずは をアのプログラムで埋めてみます。

プログラム	rbyte	r	i
6: $r \leftarrow (r \ll 1) \vee (rbyte \wedge 00000001)$			
7: $rbyte \leftarrow rbyte \gg 1$			

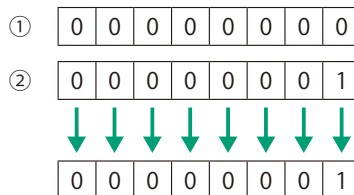
6行目では、「①rを左に1ビットシフトした結果」と「②rbyteと00000001の論理積の結果」を得て、①と②の論理和をrに入れます。

rは00000000なので、「①rを左に1ビットシフトした結果」は00000000です。

「②rbyteと00000001の論理積の結果」は次のように、00000001になります。



さらに①と②の論理和を行い、結果は「00000001」です。



これをrに代入します。

プログラム	rbyte	r	i
6 : r ← (r << 1) ∨ (rbyte ∧ 00000001)		00000001	
7 : rbyte ← rbyte >> 1			

次に7行目で、rbyteを右に1ビットシフトしています。

プログラム	rbyte	r	i
6 : r ← (r << 1) ∨ (rbyte ∧ 00000001)		00000001	
7 : rbyte ← rbyte >> 1	00100101		

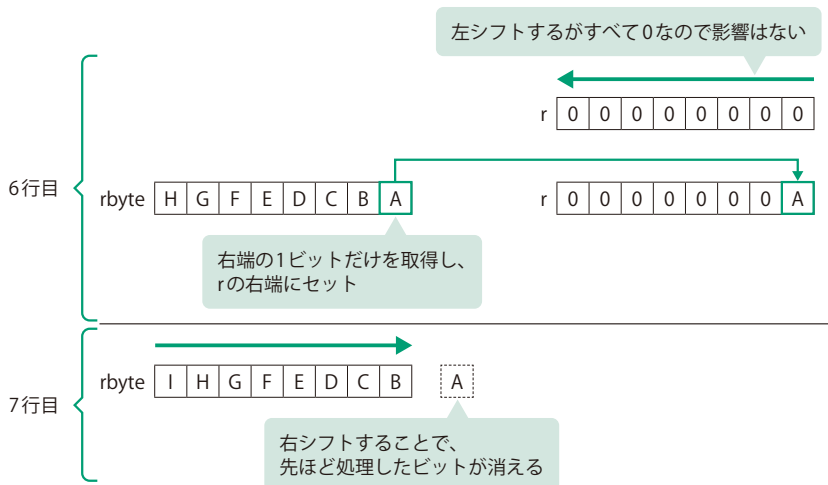
このプログラムの肝は、この6行目と7行目の処理です。まだ繰り返しを1度実行しただけなので、何をしたいのかわからないと思います。繰り返しを進めてみましょう。

プログラム	rbyte	r	i
5 : for (iを1から8まで1ずつ増やす)			2
6 : r ← (r << 1) ∨ (rbyte ∧ 00000001)		00000011	
7 : rbyte ← rbyte >> 1	00010010		
5 : for (iを1から8まで1ずつ増やす)			3
6 : r ← (r << 1) ∨ (rbyte ∧ 00000001)		00000110	
7 : rbyte ← rbyte >> 1	00001001		
5 : for (iを1から8まで1ずつ増やす)			4
6 : r ← (r << 1) ∨ (rbyte ∧ 00000001)		00001101	
7 : rbyte ← rbyte >> 1	00000100		
5 : for (iを1から8まで1ずつ増やす)			5
6 : r ← (r << 1) ∨ (rbyte ∧ 00000001)		00011010	
7 : rbyte ← rbyte >> 1	00000010		
5 : for (iを1から8まで1ずつ増やす)			6
6 : r ← (r << 1) ∨ (rbyte ∧ 00000001)		00110100	
7 : rbyte ← rbyte >> 1	00000001		
5 : for (iを1から8まで1ずつ増やす)			7
6 : r ← (r << 1) ∨ (rbyte ∧ 00000001)		01101001	

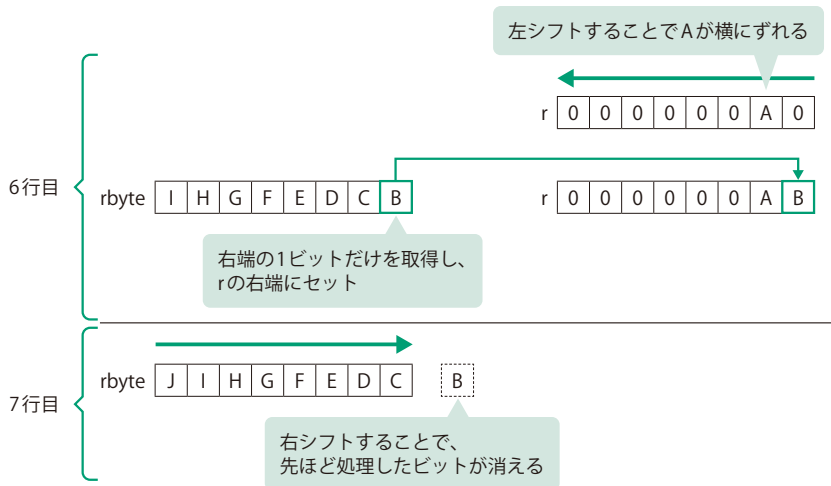
7: rbyte ← rbyte >> 1	00000000		
5: for (iを1から8まで1ずつ増やす)			8
6: r ← (r << 1) ∨ (rbyte ∧ 00000001)		11010010	
7: rbyte ← rbyte >> 1	00000000		

問題文にある通り11010010になったため、正解は**A**です。ここまでトレースすることで、このプログラムはビットマスクを使って、必要なビットだけを処理していることがわかります。ビット列にアルファベットを入れ、どのような流れでシフトされているかを見てみましょう。

1度目の繰り返し処理

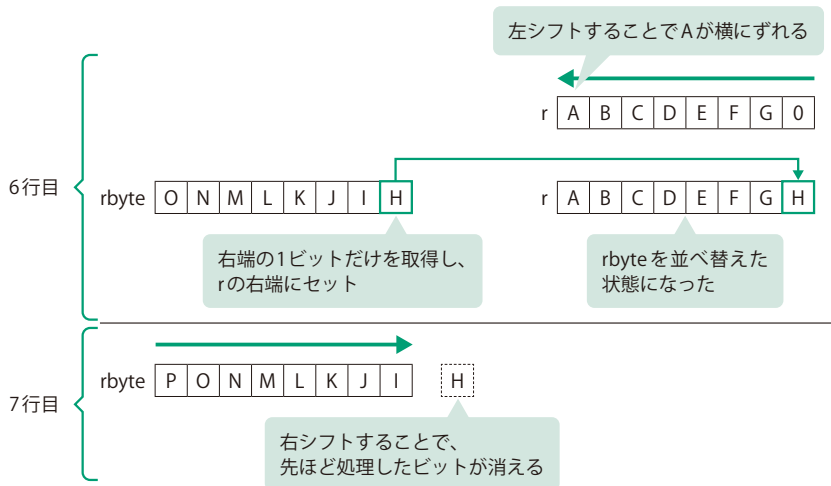


2度目の繰り返し処理



同様に繰り返し処理を行い、最後の処理 (iが8) で並べ替えが完了します。

8度目の繰り返し処理



正解 ア



手続と関数



同じ処理を何度も使い回すことができる方法を学ぼう

- 手続と関数の違いと共通点について学ぼう
- 値を渡す方法と、値を返してもらう方法を学ぼう
- 手続や関数を使った処理をトレースする方法を学ぼう

手続と関数とは

プログラムをひとまとまりにし、名前を付けたものを**手続**または**関数**といいます。擬似言語では、名前を付けたプログラムのまとまりが値を返す場合、関数と呼び分けています。

次の例は、文字を出力するだけの簡単なプログラムです。

```
1 "A"を出力する
```

このプログラムに名前を付けて、必要なときに呼び出せるようにします。次の例では「proc」という名前を付けて手続を作っています。

```
1 Oproc()  
2 "A"を出力する
```

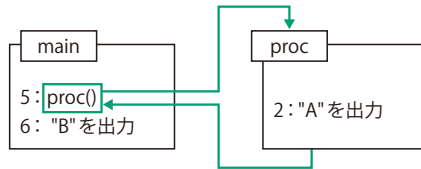
手続名の前にはOを付けます。また手続名のあとには()を付けます。また手続内で行う処理は、インデント(字下げ)してまとまりを作ります。

●呼び出し

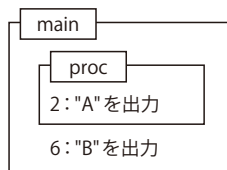
手続は、他の手続からいつでも、何度でも呼び出して実行することができます。

1	○proc()
2	"A"を出力
3	
4	○main()
5	proc()
6	"B"を出力

手続mainでは、手続procを呼び出して実行しています。procの実行が終わると、またmainに戻ります。次の図は、手続mainを実行した場合の動きです。



上の制御の流れがわかりにくければ、以下のようなイメージで考えてください。呼び出している箇所を、呼び出し先の手続そのもので上書きしている様子です。



いずれにしても、2行目の「"A"を出力」を実行し、次に6行目の「"B"を出力」を実行します。

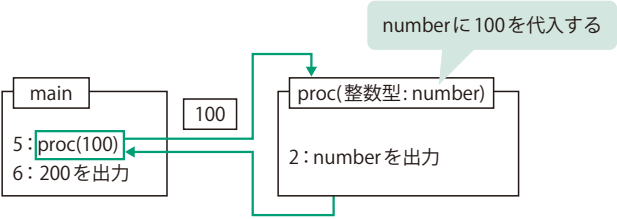
なお、これは手続mainを実行した場合の動きです。もし手続procを実行した場合には「A」だけが出力されていました。どの手続を実行するかは問題文に記述されていますので、しっかりと確認するようにしましょう。

● 引数

手続や関数を呼び出すときに、値を渡すことができます。この値を^{ひきすう}引数^{ひきすう}といいます。先ほどのプログラムを少し修正してみましょう。

1	○proc(整数型: number)
2	numberを出力
3	
4	○main()
5	proc(100)
6	200を出力

引数を受け取る手続や関数の () 内に、引数のデータ型と引数名を記述します。手続mainで手続procを呼び出すとき、100という値を渡しています。procは受け取った100を、整数型の変数numberに代入し、proc内で使用します。



手続mainを実行した場合のトレース表は次のようになります。

プログラム	number
4 : ○main()	
5 : proc(100)	
1 : ○proc(整数型: number)	100
2 : numberを出力	
6 : 200を出力	

引数で受け取った値をnumberに代入する

procは整数型の値を引数として受け取りますが、整数型以外にもさまざまなデータ型を指定できます。また、引数はコンマ区切りで、複数渡すことも可能です。

```

1  ○proc(整数型: a, 整数型: b, 整数型: c)
2    a + b + cの結果を出力
3
4  ○main()
5    proc(25, 10, 30)

```

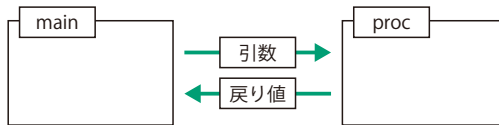
5行目のproc呼び出しでは引数を3つ渡します。それらを1行目で受け取って変数に代入し、2行目で合計した値を出力します。

関数

もし呼び出し元に値を返す場合には、「手続」ではなく「関数」となります。

戻り値

手続や関数では、呼び出し先に対して引数で値を渡すことができますが、その逆に呼び出し元は値を受け取ることが可能です。これを**戻り値**といいます。

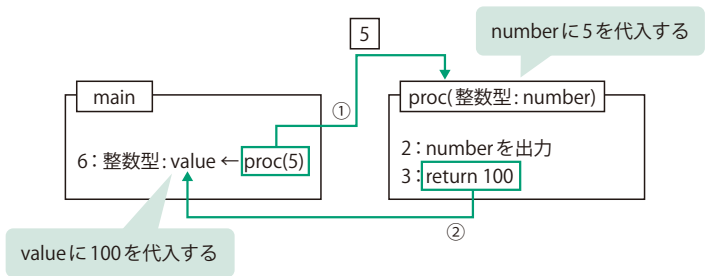


値を戻すときはreturnを使います。

```

1  ○proc(整数型: number)
2    numberを出力
3    return 100
4
5  ○main()
6    整数型: value ← proc(5)

```



トレース表は以下のようになります。

プログラム	number	value
5: ○main()		
6: 整数型: value ← proc(5)		
1: ○proc(整数型: number)	5	
2: numberを出力		
3: return 100		
6: 整数型: value ← proc(5)		100

戻り値は1つだけです。引数のように複数の値を戻すことはできません。

例題にチャレンジ

例題①

R5 公開問題 - 問2

次の記述中の に入れる正しい答えを、解答群の中から選べ。

次のプログラムにおいて、手続proc2を呼び出すと、の順に出力される。

プログラム

```
1  Oproc1()  
2  "A"を出力する  
3  proc3()  
4  
5  Oproc2()  
6  proc3()  
7  "B"を出力する  
8  proc1()  
9  
10 Oproc3()  
11 "C"を出力する
```

解答群

- ア "A", "B", "B", "C"
- イ "A", "C"
- ウ "A", "C", "B", "C"
- エ "B", "A", "B", "C"
- オ "B", "C", "B", "A"
- カ "C", "B"
- キ "C", "B", "A"
- ク "C", "B", "A", "C"

解説 問題文にあるように、まずは手続proc2を呼び出します。proc2では、6行目で手続proc3を呼び出しています。そしてproc3では、11行目で「C」を出力しています。このように1つずつトレースしていくことで、正確に出力順を知ることができます。



解説動画

プログラム
5 : Oproc2()
6 : proc3()
10 : Oproc3()
11 : "C"を出力する
7 : "B"を出力する
8 : proc1()
1 : Oproc1()
2 : "A"を出力する
3 : proc3()
10 : Oproc3()
11 : "C"を出力する

最後までトレースすると、C、B、A、Cの順に出力されることがわかります。したがって、正解は**ク**です。

正解 **ク**

例題②

オリジナル問題

次の記述中の **a** , **b** , **c** に入れる正しい答えを、解答群の中から選べ。

main(18, 24)を実行すると、 α の箇所は "**a**" 回実行される。変数gとlの値は、それぞれ **b** と **c** となる。ここで、 $x \bmod y$ はxをyで割った余りを返す。

プログラム

```

1  ○main(a, b)
2    g ← gcd(a, b)
3    l ← lcm(a, b, g)
4    "GCD="を出力する
5    gを出力する
6    "LCM="を出力する
7    lを出力する
8
9  ○gcd(x, y)
10   while (y ≠ 0)
11     r ← x mod y      // a
12     x ← y
13     y ← r
14   endwhile
15   return x
16
17 ○lcm(p, q, g)
18   return (p ÷ g) × q

```

解答群

	a	b	c
ア	3	3	18
イ	4	3	18
ウ	5	3	18
エ	3	6	72
オ	4	6	72
カ	5	6	72
キ	3	18	144
ク	4	18	144
ケ	5	18	144

解説 1つの手続と、2つの関数で構成されたプログラムです。main(18, 24)を実行した流れをトレースしてみましょう。まずは途中経過です。



解説動画

プログラム	a	b	x	y	r	g	p	q	l
1: $\text{main}(a, b)$	18	24			引数で受け取った値をxとyに代入する				
2: $g \leftarrow \text{gcd}(a, b)$									
9: $\text{gcd}(x, y)$			18	24					
10: while ($y \neq 0$)									
11: $r \leftarrow x \bmod y$					18				
12: $x \leftarrow y$			24						
13: $y \leftarrow r$				18					
10: while ($y \neq 0$)									
11: $r \leftarrow x \bmod y$					6				
12: $x \leftarrow y$			18						
13: $y \leftarrow r$				6					
10: while ($y \neq 0$)									
11: $r \leftarrow x \bmod y$					0				
12: $x \leftarrow y$			6						
13: $y \leftarrow r$				0					
10: while ($y \neq 0$)									

yが0になったので繰り返し処理が終わり、15行目に移ります。

プログラム	a	b	x	y	r	g	p	q	l
15: return x									
2: $g \leftarrow \text{gcd}(a, b)$						6			
3: $l \leftarrow \text{lcm}(a, b, g)$									
17: $\text{lcm}(p, q, g)$						6	18	24	
18: return $(p \div g) \times q$									
3: $l \leftarrow \text{lcm}(a, b, g)$									72

このあとは4～7行目の処理で変数の値が出力されます。トレース表を見るとわかる通り、11行目は3回実行されています。また最終的なgの値とlの値はそれぞれ6と72ですので、正解は**エ**です。

正解 **エ**



07

再帰



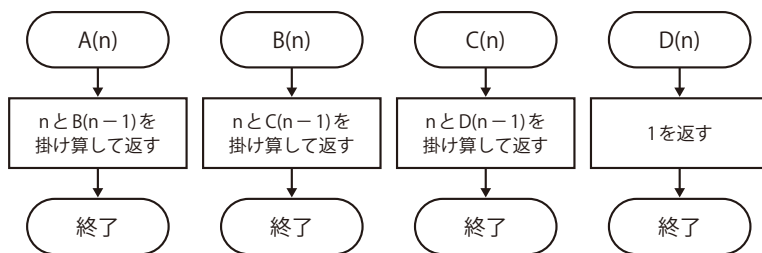
再帰の定義と必要性について学ぼう

- 自分自身を呼び出すとは何かについて学ぼう
- 再帰するプログラムをトレースする際の注意点を学ぼう

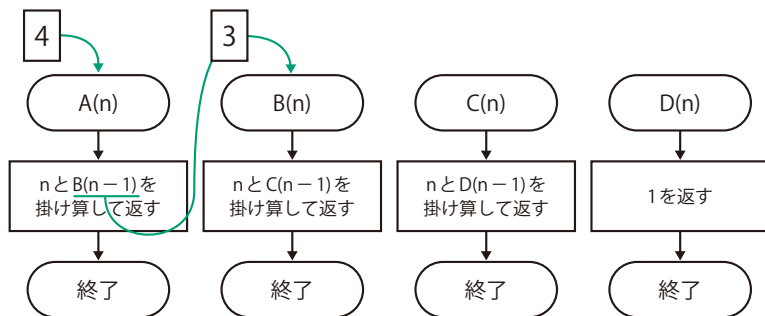
再帰とは

これまで学んだ手続や関数は、ある手続から別の手続を呼び出す仕組みでした。
^{さいき}**再帰**はその応用で、手続（または関数）が自分自身を呼び出すという仕組みです。
 多くの人がつまずく再帰について、しっかりと学習していきましょう。

階乗を計算するプログラムを例として、再帰について考えます。**階乗とは、ある自然数以下のすべての自然数を掛け算することです。**例えば4の階乗は $4 \times 3 \times 2 \times 1$ の計算結果であり、24です。これをプログラムで記述してみます。

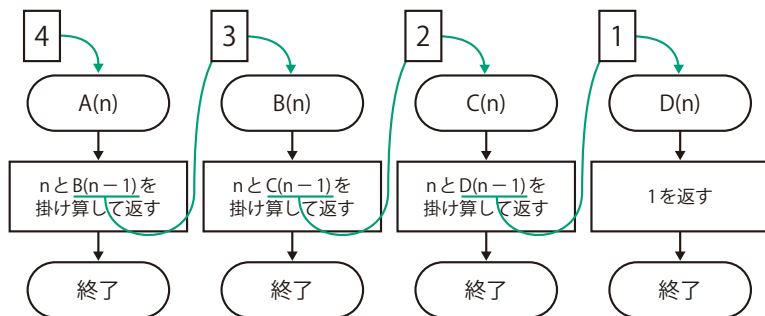


まず4を引数にして関数Aを実行します。関数Aでは「変数 n と $B(n-1)$ を掛け算」しています。 n は4であるため、「4と $B(3)$ を掛け算」と解釈します。

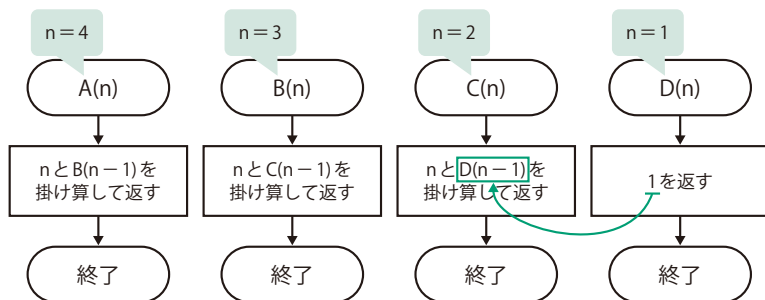


なお、関数Aの変数 n と関数Bの変数 n は別の変数です。**変数名は同じでも、関数や手続が異なる場合には別の変数として扱われます。**

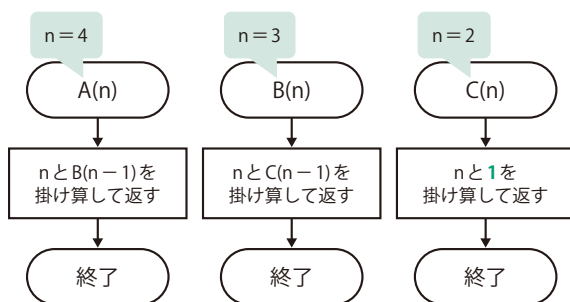
関数Bの変数 n は3であるため、関数Bの処理は「3とC(2)を掛け算」と解釈します。関数Cも同様です。



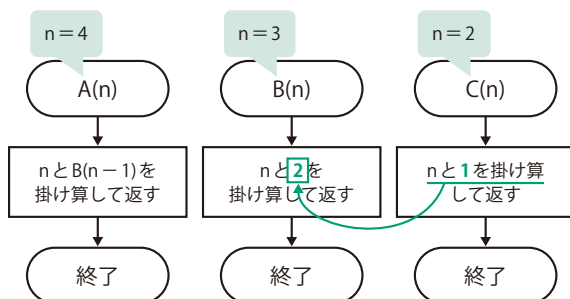
関数Dでは「1を返す」となっているため、その通りにします。



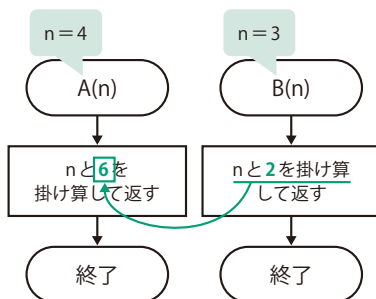
1を返すことで、呼び出し元である関数Cの「 $D(n-1)$ 」が、「1」に置き換えられます。なお、「1を返す」という処理を実行した関数Dは、処理を終えたため終了します。



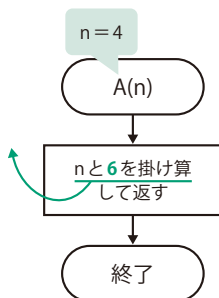
関数Cは「変数nと1を掛け算して返す」と解釈されます。関数Cの変数nは2が入っているため、 2×1 の結果である「2」を関数Bに返します。それにより、呼び出し元の関数Bの「 $C(n-1)$ 」の部分が、「2」に置き換わることになります。



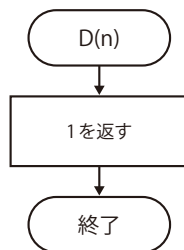
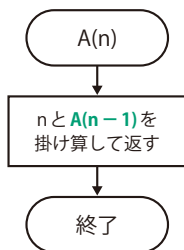
同様に関数Bは、呼び出し元の関数Aに対して、3と2を掛け算した結果を返します。



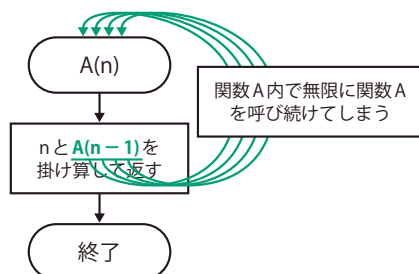
最後に関数Aは、 n と6を掛け算した結果を返して終了です。最終的に返された値は24です。



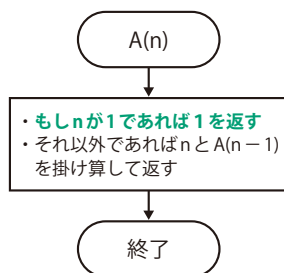
これで4つの関数で構成された階乗のプログラムが完了します。しかしよく見ると、関数A、B、Cはほとんど同じプログラムです。これを1つにまとめてみます。



関数Aの処理には「 $A(n-1)$ 」とあります。このように、手続や関数内から自分自身を呼び出す処理が再帰です。ただし、このままでは処理が終わらない、いわゆる無限ループに陥ってしまいます※11。



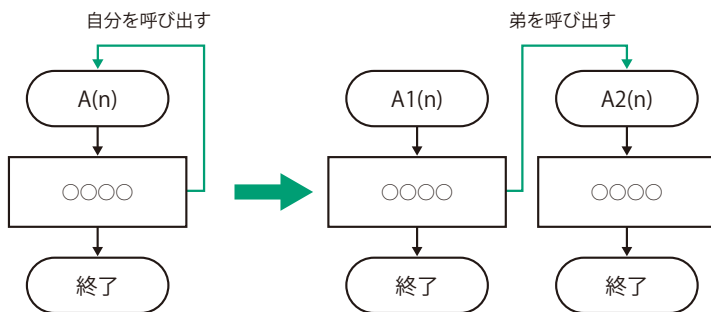
そこで、関数Dも関数Aに組み込みます。関数Dの引数は「1」でした。そのため組み込む際には「引数nが1のときには1を返す」とします。



これで無限ループにはならず、正常に階乗が計算できるようになりました。

再帰は慣れるまでイメージしにくいと思いますので、次のように考えてください。まず「関数A」ではなく「関数A1」と読み換えます。この「関数A1」から呼び出される自分自身を「関数A2」と読み替えます。つまり1度目の関数と、そこから呼ばれる2度目の関数は別の関数として考えるわけです。これは、ある処理をもつ「兄」が、まったく同じ処理を持つ「弟」を呼び出すようなイメージです。

※11 無限ループになった場合、プログラミング言語や実行環境によってはエラーとして扱われることがあります。



このように、まったく同じ処理をする別の関数として捉えることで、再帰の流れが理解しやすくなります。

例題にチャレンジ

例題①

オリジナル問題

次のプログラム中の , に入れる正しい答えの組合せを、解答群の中から選べ。

関数 fact は、引数で与えられた整数の階乗を行う。

プログラム

```

1  ○main(整数型: n)
2    整数型: r ← fact(n)
3    rを出力する
4
5  ○fact(整数型: n)
6    if (nは1と等しい)
7      return 
8    else
9      return n × 
10   endif

```

解答群

	a	b
ア	1	fanc(1)
イ	1	fanc(n)
ウ	1	fanc(n - 1)
エ	n	fanc(1)
オ	n	fanc(n)
カ	n	fanc(n - 1)
キ	n - 1	fanc(1)
ク	n - 1	fanc(n)
ケ	n - 1	fanc(n - 1)

解説 再帰では、どこかで必ず自分自身の呼び出しを終わらせる条件が必要です。階乗の計算では、掛ける数を1ずつ減らしていき、1になったらそれ以上は掛け算をしません。

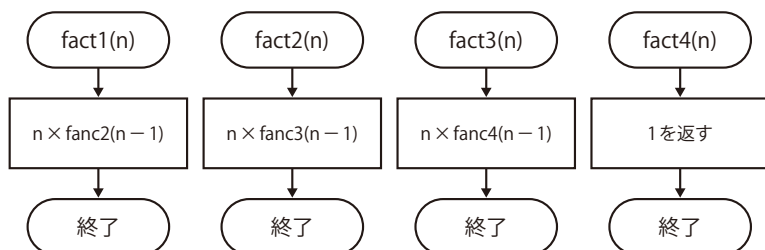


解説動画

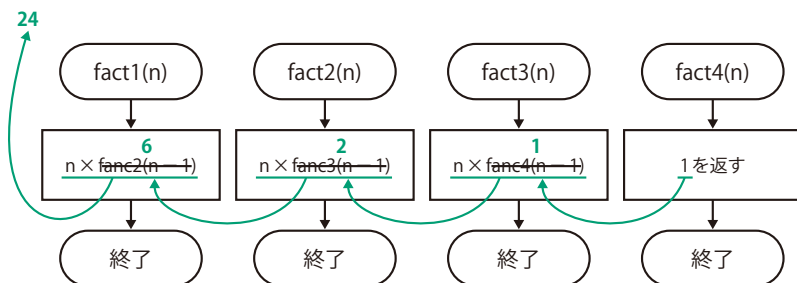
例えば4の階乗であれば、 $4 \times 3 \times 2 \times 1$ という計算を行います。この「終わりの条件」が書かれているのが、6行目の「if (nは1と等しい)」です。この条件が成立すると、自分自身は呼び出さず、「1」を返します。したがって、**a**には1が入ります。

一方、この条件に当てはまらない場合 (nが2以上のとき) は、再び自分自身を呼び出して計算を続けます。ただし、渡された引数の値をそのまま自分自身の呼び出しで使うと、いつまでも処理が終わりません。そのため、引数は1つ小さい数である「 $n - 1$ 」とします。そして、その戻ってきた結果にnを掛けた結果を戻り値として返していくことで、階乗が完成します。つまり、**b**には「fact($n - 1$)」が入ります。したがって、正解はウです。

なお、このプログラムの流れは次のように表現できます。関数名はすべてfactなのですが、説明の都合上fact1~4としています。



手続mainではfact1を、引数を4として呼び出しています。手続mainが受け取る値は24となります。



正解 ウ

例題②

H23 特別午前 - 問 6 改変

次の記述中の に入れる正しい答えを、解答群の中から選べ。

関数 f が以下のように定義されている。 $f(775, 527)$ の値は である、ここで、 $x \bmod y$ は x を y で割った余りを返す。

プログラム

```

1  ○整数型: f(整数型: x, 整数型: y)
2  if (y = 0) then
3      return x
4  else
5      return f(y, x mod y)
6  endif

```

解答群

- ア 0
- イ 31
- ウ 248
- エ 527

解説 関数内で自分自身を呼び出しているため、再帰であることがわかります。

再帰の場合は、無限ループにならないように必ず自分自身を呼び出さないための条件が指定されています。今回は y が 0 であることがその条件です。

再帰は比較的単純なプログラムで出題されます。そのため多くの場合、トレース表を作る必要はありません。戻り値を正確に記録するだけで問題なくトレースできるはずです。

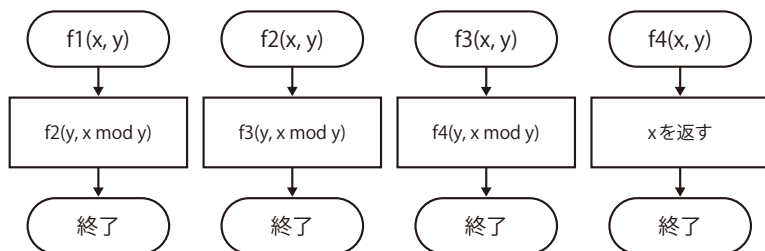


解説動画

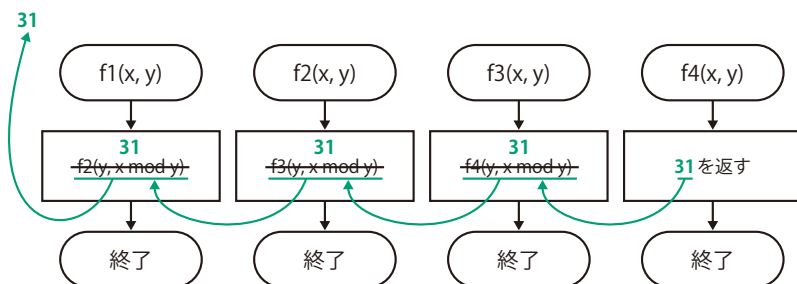
再帰の流れ

呼び出し回数	x の値	y の値
1	775	527
2	527	248 (775 ÷ 527 の余り)
3	248	31 (527 ÷ 248 の余り)
4	31	0 (248 ÷ 31 の余り)

4回目の呼び出しで y が0になり、再帰が終了します。この再帰のプログラムは次のように考えることができます。実際にはすべての関数名が「f」なのですが、判別するために関数名を $f1 \sim f4$ としています。



$f4$ では x を返しています。先ほどのリストの4を見ると、 x は31ですから、次のようになります。



最終的に返された値は31ですので、正解は**イ**です。

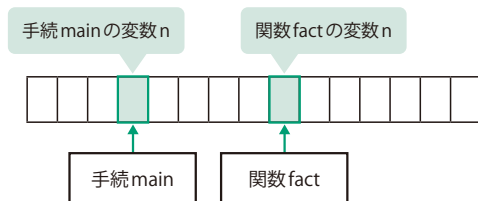
正解 **イ**

変数の有効範囲

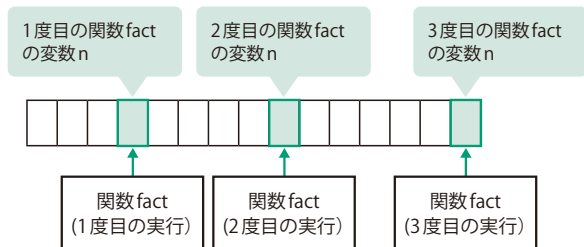
さきほどの例題では次のプログラムを扱いました。

```
1  ○main(整数型: n)
2    整数型: r ← fact(n)
3    rを出力する
4
5  ○fact(整数型: n)
6    if (nは1と等しい)
7      return a
8    else
9      return n × b
10   endif
```

変数nが、手続mainと関数factの両方で宣言されています。同じ変数名ですが、これらは別々のメモリ領域に確保されます。



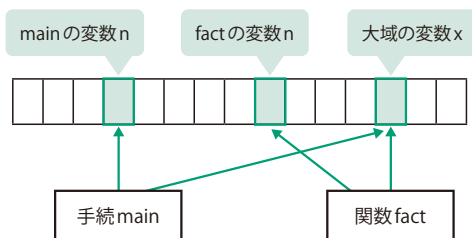
再帰においても、1度目の関数factと2度目の関数factとでは別々のメモリ領域が使われます。



すべての手続や関数で、常に共通の変数を使いたい場合には、特殊な記述を行います。

1 大域: 整数型: x

「大域:」と先頭に付けることで、その変数はすべての手続や関数における共通の変数となります。通常のプログラミング言語ではグローバル変数といわれますが、擬似言語では**大域変数**といいます。



大域変数の動きを理解するために、まずは次のプログラムでmainを呼び出したときの実行結果を考えてみてください。

```

1  OaddFive()
2   整数型: n ← 0
3   n ← n + 5
4   nを出力する
5
6  Omain()
7   addFive()
8   addFive()
9   addFive()

```

7行目で、手続addFiveを実行します。2行目で変数nを宣言して0で初期化します。3行目で0に5を加算し、その結果である「5」が4行目で出力されます。これを3回繰り返しますので、結果は「5、5、5」となります。

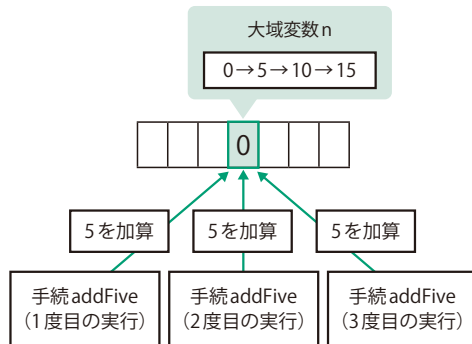
では次に、変数nを大域変数にします。

```

1 大域: 整数型: n ← 0
2 〇addFive()
3   n ← n + 5
4   nを出力する
5
6 〇main()
7   addFive()
8   addFive()
9   addFive()

```

1行目でnを、大域変数として宣言し、初期値を0にしています。このように大域変数は、各手続（または関数）とは別の場所で宣言します。7行目でaddFiveを実行すると、nに5が加算され、その結果が出力されます。同様に8行目のaddFiveでもnに5が加算されます。結果である10が出力されます。さらに9行目で15となり、その結果が出力されます。出力は「5、10、15」です。



このように大域変数を用意した場合、プログラム全体で共用が可能です。

Chapter

2

配列

本章の学習ポイント

- 複数の値を連続して扱いたい場合には配列が便利である
- 配列は1つまたは複数の要素で構成され、それぞれの要素は要素番号を指定してアクセスすることができる
- 配列への要素の追加や値の変更には決まった記述方法がない
- 二次元配列を使うことで、値を表形式で表現できる



01

配列



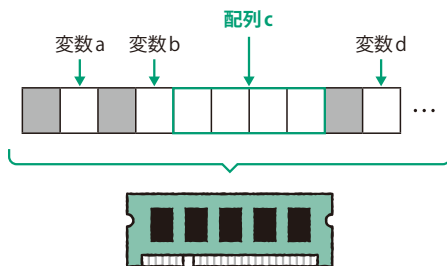
複数の値を1つのグループにまとめて扱う方法を学ぼう

- 配列を構成する要素の扱い方を学ぼう
- 要素にアクセスする方法を学ぼう
- 配列に要素を格納する方法や値の更新方法を学ぼう

配列とは

プログラム上で複数の値をまとめて管理する方法として、**配列**と**連結リスト**があります。どちらも実務や試験対策として必須の知識です。この章では配列について学習していきましょう。

1章で学習したように、プログラムで扱うほとんどの変数の値は主記憶装置に保存されます。通常の変数に代入できる値は1つであり、それらの変数は主記憶装置上にバラバラに配置されます。一方で、配列には複数の値をまとめて保持でき、それらの値は主記憶装置上の連続した領域に保存されます。



このように配列は連続した領域を使うため、何個の値を格納するかをあらかじめ指定する必要があります。これは配列の大きな特徴であると同時に、不便に感じやすい点でもあります。

値を4つ格納する配列を作ります



一方で**配列の最大のメリットは参照が容易である**ということです。これはちょうど、アパートの部屋番号を指定するだけで住居を特定できるのと同じです。アパートが配列で、アパート名が配列名にあたります。



101号室

101

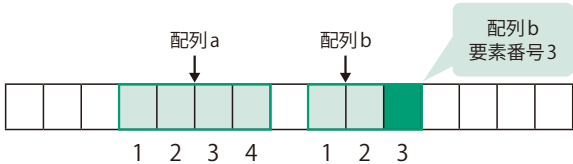


アパートにおける部屋は、配列では**要素**といいます。部屋番号は**要素番号**にあたり、**添え字**と表現されることもあります^{※1}。

アパートと配列

アパート	配列
アパート名	配列名
部屋	要素
部屋番号	要素番号、添え字

ある1つの部屋を特定するためには、アパート名と部屋番号があれば可能です。配列も同様に、配列名と要素番号があれば1つの要素を取り出すことができます。



※1 制度改正前の基本情報技術者試験では「添え字」といわれていましたが、公開問題では「要素番号」が使われています。

配列宣言

配列を用意することを**配列宣言**といいます。また、配列宣言を行うときには必ずデータ型を指定します。データ型はすでに学習した「整数型」「文字列型」などを指定することができ、配列の場合には「**整数型の配列**」「**文字列型の配列**」のように記述します。

次の例では、文字列型の配列を用意しています。配列名は「欲しい物リスト」を英語表記した「wish_list」としました。

```
1 文字列型の配列: wish_list
```

値の格納

宣言した配列の要素に値を格納するときは、次のように記述します。

```
2 wish_list ← {"りんご", "鉛筆", "歯ブラシ"}
```

記述した順番で「りんご」「鉛筆」「歯ブラシ」が格納されていきます。

配列 wish_list	りんご	鉛筆	歯ブラシ
--------------	-----	----	------

配列の個々の要素を操作する場合は、配列[要素番号]という形式で記述します。変数と同様に、値を上書きすることも可能です。次の記述により、要素番号が1と2の要素が更新されます。

```
3 wish_list[1] ← "タオル"
4 wish_list[2] ← "消しゴム"
```

配列 wish_list	タオル	消しゴム	歯ブラシ
	1	2	3

このように、要素に値を格納する方法は2つあります。配列にまとめて要素を格納する場合は、要素番号は指定せず、値をコンマで区切って{}で囲みます。特定の要素に値を格納する場合は、[]を使って要素番号を指定し、値はそのまま記述します。

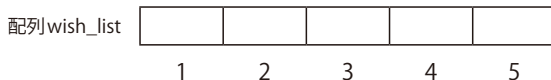
通常の変数と同様に、配列もまた宣言と値の格納を同時に行うことができます。

1 文字列型の配列: `wish_list ← {"りんご", "消しゴム", "歯ブラシ"}`

実は値の格納方法に決まった記述方法はないため、問題文やプログラム内のコメントを読んでその意味を解釈してください。

サンプル問題には、以下のような格納方法もありました。「{5個の未定義の値}」とある通り、5つの要素で構成された配列を定義し、それらの要素には値は設定しません。つまり、要素の値は未定義の状態です。

1 文字列型の配列: `wish_list ← {5個の未定義の値}`



さらに次のような記述もサンプル問題にありました※2。

1 文字列型の配列: `allItem ← orders`に含まれる文字列を
重複なく辞書順に格納した配列

2 // 表の列では{"A", "B", "C", "D", "E"}

要素の追加

一般的なプログラミング言語では、配列に要素を追加するのは少し複雑な処理が必要です。例えば、5つの要素で構成されている配列があるとします。さらに要素を追加して6つにするには、新たに要素数が6の配列を用意し、元の配列の要素をそっくりそのままコピーします。これで要素の値はそのままに、要素数が6の配列ができました※3。

擬似言語の場合、**配列の要素数を増やしたいときは「末尾に〇〇を追加」という記述だけで実現**できます。

1 文字列型の配列: `wish_list ← {"りんご", "消しゴム", "歯ブラシ"}`

2 `wish_list`の末尾に値"タオル"を追加する

※2 さらに「1～10の値を100個ランダムに、一様分布で格納」や「ア～コを、順番に格納」などの記述も考えられます。

※3 多くのプログラミング言語では、簡単に要素を拡張するための関数が用意されていますので、プログラマが1から処理を記述する場面はないはずです。

要素番号

多くのプログラミング言語では、最初の要素番号は「0」です。しかし擬似言語では「1」から始まる場合がほとんどです。最初の要素番号ですから「1」のほうが直感的にわかりやすいと思いますが、問題文に記述されているルールに従いましょう。

要素番号0から開始	りんご	消しゴム	歯ブラシ	タオル	おにぎり
	0	1	2	3	4

要素番号1から開始	りんご	消しゴム	歯ブラシ	タオル	おにぎり
	1	2	3	4	5

なお、IPAが公開している科目Bのサンプル問題では、次のように要素番号が1で始まるのが主流です。

問3 次の記述の中の に入れる正しい答えを、解答群から選べ。ここでは配列の要素番号は1から始まる。

ただし、試験の制度が変わる前（2023年3月以前）では、要素番号が0から始まる問題もあったため、要素番号の開始番号は見逃さないように注意しましょう。

トレースのコツ

配列のトレースは複雑になりがちです。例えば、要素数が5の配列があったとします。これは一般的なプログラミングの場面では決して多くはない数です。それでも変数に換算すれば5つ分ですからトレースするのは大変です。問題によってはトレースを省略するわけにはいきませんので、慣れるまではきちんとトレースするようにしましょう。

```
1: wish_list ← {"りんご", "鉛筆", "歯ブラシ"}
2: wish_listの末尾に値"タオル"を追加する
3: wish_listの末尾に値"おにぎり"を追加する
4: wish_list[2] ← "消しゴム"
```

りんご	鉛筆	歯ブラシ
-----	----	------

りんご	鉛筆	歯ブラシ	タオル
-----	----	------	-----

りんご	鉛筆	歯ブラシ	タオル	おにぎり
-----	----	------	-----	------

りんご	消しゴム	歯ブラシ	タオル	おにぎり
-----	------	------	-----	------

要素の数だけ枠を区切り、枠内に値を書いていってください。本試験では枠を書く時間はないはずなので、要素と要素の間をコンマ区切りにするなどして、間違いがないようにしましょう。

1, 2, 3, 4, 5
りんご, 鉛筆, 歯ブラシ, タオル, おにぎり

例題にチャレンジ

例題①

オリジナル問題

以下は、5つの整数を格納した配列から合計値を求めて表示するプログラムである。出力される値を答えよ。ここで、配列の要素番号は1から始まる。

プログラム

```
1  整数型の配列: numbers ← {5個の未定義の値}
2  整数型: total ← 0
3  整数型: i
4
5  numbers ← {10, 20, 30, 40, 50}
6
7  for (iを1から5まで1ずつ増やす)
8    total ← total + numbers[i]
9  endfor
10
11 totalを出力
```

解答群

- ア 10
- イ 30
- ウ 50
- エ 100
- オ 150

解説 簡単な例題を解いて、配列のトレースに慣れていきましょう。

まずは「未定義」を意味する文字を決めておきます。ここでは△としましたが「未」や「□」などでもかまいません。



解説動画

未定義を表す文字を決めておく

プログラム	numbers	total	i
1: 整数型の配列: numbers ← {5個の未定義の値}	△, △, △, △, △		
2: 整数型: total ← 0		0	
3: 整数型: i			
5: numbers ← {10, 20, 30, 40, 50}	10, 20, 30, 40, 50		

続いて繰り返し部分をトレースしていきます。

プログラム	numbers	total	i
7: for (iを1から5まで1ずつ増やす)			1
8: total ← total + numbers[1]		10	
7: for (iを1から5まで1ずつ増やす)			2
8: total ← total + numbers[2]		30	
7: for (iを1から5まで1ずつ増やす)			3
8: total ← total + numbers[3]		60	
7: for (iを1から5まで1ずつ増やす)			4
8: total ← total + numbers[4]		100	
7: for (iを1から5まで1ずつ増やす)			5
8: total ← total + numbers[5]		150	

答え

したがって、正解は**オ**の150です。

正解 **オ**

例題②

オリジナル問題

以下は、5つの整数を格納した配列から最大値を求めて表示するプログラムである。出力される値を答えよ。ここで、配列の要素番号は1から始まる。

プログラム

```

1  整数型の配列: scores ← {65, 15, 72, 67, 85}
2  整数型: max_score ← 0
3  整数型: i
4  for (iを1から5まで1ずつ増やす)
5    if (scores[i]がmax_scoreより大きい)
6      max_score ← scores[i]
7    endif
8  endfor
9
10 max_scoreを出力

```

解答群

- ア 65
- イ 15
- ウ 72
- エ 67
- オ 85

解説 最大値や最小値を求めるプログラムは非常によく出題されます。

考え方としては、まず初代のチャンピオンを決めます。そして**チャンピオン**と「どっちが強いか」を1ずつ比較し、**チャンピオンより強い要素があれば、それを新たなチャンピオンとします**。なお、最初の要素を初代チャンピオンとする流れが一般的です。

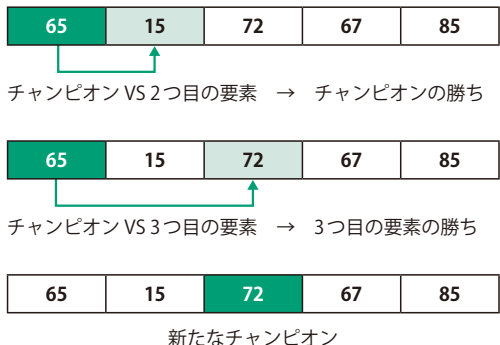


解説動画



チャンピオン

65	15	72	67	85
----	----	----	----	----



ではトレースしていきましょう。2 行目では最も小さい数である 0 を代入しています。これにより最初の要素（このプログラムでは 65）は必ずチャンピオンとなる状態を作っています。なおこのプログラムがマイナス値も扱うのであれば、整数型で扱うことができる最小値を代入します※4。

プログラム	scores	max_score	i
1: 整数型の配列: scores ← {65, 15, 72, 67, 85}	65, 15, 72, 67, 85		
2: 整数型: max_score ← 0	最初の要素は必ず勝つ ようにするために 0 にする	0	
3: 整数型: i			
4: for (i を 1 から 5 まで 1 ずつ増やす)			1
5: if (scores[1] が 0 より大きい)	最初の要素は必ず勝つ		
6: max_score ← scores[1]		65	
5: if (scores[2] が 65 より大きい)	2 番目の要素は 15 なので負ける		2
5: if (scores[3] が 65 より大きい)	3 番目の要素は 72 なので勝つ		3
6: max_score ← scores[3]		72	
5: if (scores[4] が 72 より大きい)	4 番目の要素は 67 なので負ける		4
5: if (scores[5] が 72 より大きい)	5 番目の要素は 85 なので勝つ		5
6: max_score ← scores[5]		85	答え

したがって、正解はオの 85 です。

正解 オ

※4 擬似言語では整数型で扱うことができる最大値や最小値は明示されていません。そのため今回の問題のように正の数だけを扱うか、「扱うことができる最小値を代入」などの説明があるはずですが。

例題③

オリジナル問題

次のプログラム中の **a** , **b** , **c** に入れる正しい答えの組合せを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数sortは、引数で与えられた整数を昇順に整列した結果を返す関数である。

プログラム

```

1  ○整数型の配列: sort(整数型の配列: numbers)
2  整数型: temp
3
4  for (iを1から a まで1ずつ増やす)
5      for (jを1から b まで1ずつ増やす)
6          if ( c がnumbers[j+1]より大きい)
7              temp ← numbers[j]
8              numbers[j] ← numbers[j+1]
9              numbers[j+1] ← temp
10         endif
11     endfor
12 endfor
13 return numbers

```

解答群

	a	b	c
ア	numbers の要素数	numbers の要素数 - j	numbers[i]
イ	numbers の要素数	numbers の要素数 - j	numbers[j]
ウ	numbers の要素数	numbers の要素数 - i	numbers[i]
エ	numbers の要素数	numbers の要素数 - i	numbers[j]
オ	numbers の要素数	numbers の要素数	numbers[i]
カ	numbers の要素数 - 1	numbers の要素数 - j	numbers[j]
キ	numbers の要素数 - 1	numbers の要素数 - j	numbers[i]
ク	numbers の要素数 - 1	numbers の要素数 - i	numbers[j]
ケ	numbers の要素数 - 1	numbers の要素数 - i	numbers[i]
コ	numbers の要素数 - 1	numbers の要素数	numbers[j]

解説 並べ替えを行う整列のアルゴリズムはいくつか種類があります。出題されたプログラムがどのアルゴリズムなのかを見抜くことは非常に重要です。今回のプログラムは、隣の要素と比較しているため、**バブルソート**であると推測できます。



解説動画

バブルソートでは、ある要素と隣の要素を比較し、もし理想通りになっているのであれば何もしません。**もし理想通りでなければ交換して、理想通りの状態にします。**ここでの「理想通り」とは、右に進むにつれて値が大きくなる状態です。

要素が65、15、72、67、10の配列を例に、バブルソートの流れを見てみましょう。まず1番目の要素と2番目の要素を比較します。理想通りになっていないので、交換をして理想通りにします。

隣同士を比較する

65	15	72	67	10
----	----	----	----	----

65	15	72	67	10
----	----	----	----	----



左の要素が小さいことが理想なので交換する

15	65	72	67	10
----	----	----	----	----

次に2番目と3番目を比較します。理想通りになっているので何もしません。

隣同士を比較する

15	65	72	67	10
----	----	----	----	----

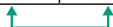
理想通りになっているため交換しない

同様にこれを最後まで行います。

隣同士を比較する

15	65	72	67	10
----	----	----	----	----

15	65	72	67	10
----	----	----	----	----



左の要素が小さいことが理想なので交換する

15	65	67	72	10
----	----	----	----	----

隣同士を比較する

15	65	67	72	10
----	----	----	----	----

15	65	67	72	10
----	----	----	----	----


 左の要素が小さいことが理想なので交換する

15	65	67	10	72
----	----	----	----	----

確定

この時点で最大値である72が最後の要素になりました。このように**最大値がどんどんと上に移動していく様子からバブル（泡）ソート**といわれています。

この時点で確定しているのは最後の要素だけであり、整列はまだ完了していません。そのため同様に比較&交換を行っていきます。**最後の要素はすでに確定しているため、それより前の要素を処理**していきます。

15	65	67	10	72
----	----	----	----	----

理想通りになっているため交換しない

15	65	67	10	72
----	----	----	----	----

理想通りになっているため交換しない

15	65	67	10	72
----	----	----	----	----


 左の要素が小さいことが理想なので交換する

15	65	10	67	72
----	----	----	----	----

15	65	10	67	72
----	----	----	----	----

5番目は最大値であることが確定しているため比較しない

この時点で4番目と5番目が確定しました。次にそれ以前を処理していき、2番目から5番目までが確定した時点で整列は完了します。

10	15	65	67	72
----	----	----	----	----

確定

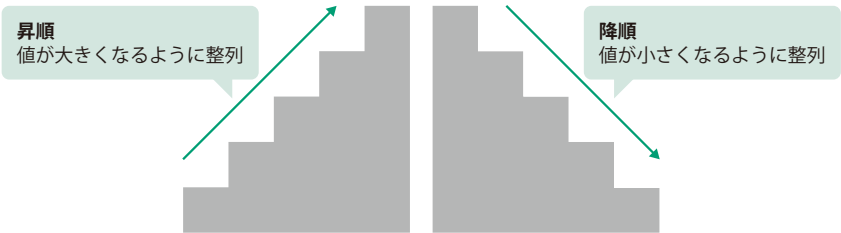
確定

確定

確定

この問題では、先頭の要素から末尾の要素に向かって値が大きくなる昇順という整列方法です。問題によっては値が小さくなる降順の場合もあるため、問題文やプログラムで判断してください。

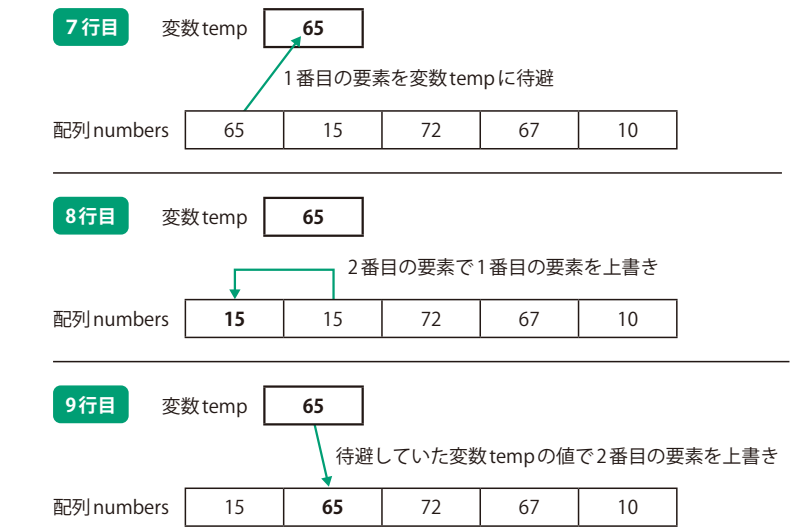
昇順と降順の定義



ではプログラムを見ていきます。交換する処理は次の7～9行目です。

```
7      temp ← numbers[j]
8      numbers[j] ← numbers[j+1]
9      numbers[j+1] ← temp
```

図で表すと次のような形です。



このような交換方法は多いため、覚えておきましょう。

ではプログラムの空欄を埋めていきます。まずは「理想通りでなければ交換する」という判断が必要です。

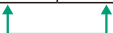
```
6 if (  がnumbers[j+1]より大きい)
```

この条件を満たす場合に交換が行われます。つまり、この条件は「理想通りではない」という判断条件になります。昇順の整列において、理想通りではないのは、左の要素が右の要素よりも大きい状態です。したがって には「numbers[j]」が入ります。

```
6 if (numbers[j]がnumbers[j+1]より大きい)
```

j が1だとすると、要素番号1の要素と、要素番号2の要素を比較して、必要であれば交換することになります。

1	2	3	4	5
65	15	72	67	10



1	2	3	4	5
15	65	72	67	10

次に、2つの繰り返しの条件を完成させます。

```
4 for (iを1から  まで1ずつ増やす)
5   for (jを1から  まで1ずつ増やす)
6     if (numbers[j]がnumbers[j+1]より大きい)
```

5行目から始まる繰り返しで、jの値がカウントアップされています。6行目ではこのjの値で「理想通りか」を判断しているため、jが5になるとエラーになります。

```
6 if (numbers[5]がnumbers[5+1]より大きい)
```

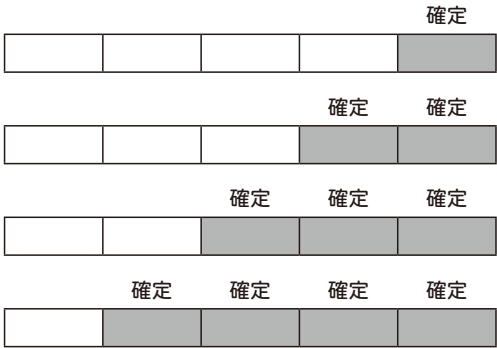
存在しない要素を参照しているためエラーになる

エラーにならないギリギリの値は4です。したがって は4があてはまりそうです。


```
4 for (iを1から a まで1ずつ増やす)
5   for (jを1から4まで1ずつ増やす)
6     if (numbers[j]がnumbers[j+1]より大きい)
```

しかしこれは誤りです。エラーにはならないのですが、無駄な比較が発生してしまいます。バブルソートでは、すでに確定した要素の比較は行いません。

バブルソートでは最後から順に確定していく



外側の繰り返しが1周するごとに、比較する要素番号を減らしていく必要があります。

外側の繰り返し周数	比較する要素番号
1周目	要素番号が1から4までの要素を隣と比較
2周目	要素番号が1から3までの要素を隣と比較
3周目	要素番号が1から2までの要素を隣と比較
4周目	要素番号が1から1までの要素を隣と比較

この表のような動きにするには、bに「4 - i」を入れます。

外側の繰り返し周数	比較する要素番号
i周目	要素番号が1から(4 - i)までの要素を隣と比較

```
4 for (iを1から a まで1ずつ増やす)
5   for (jを1から4-iまで1ずつ増やす)
6     if (numbers[j]がnumbers[j+1]より大きい)
```

外側の繰り返しは先ほどの図のように4回行う必要があります。ただし、この4という数は、配列の要素数が5の場合です。実際には引数で受け取った配列を使うことから、`a`でも引数の配列の要素数を使う必要があります。そのため、`a`は「numbersの要素数 - 1」が入ります。

```
4 for (iを1からnumbersの要素数 - 1まで1ずつ増やす)
5   for (jを1から4 - iまで1ずつ増やす)
6     if (numbers[j]がnumbers[j+1]より大きい)
```

そうなると5行目の「4-i」も誤りであり、`b`は「numbersの要素数 - i」が正解です。

```
4 for (iを1からnumbersの要素数まで1ずつ増やす)
5   for (jを1からnumbersの要素数 - iまで1ずつ増やす)
6     if (numbers[j]がnumbers[j+1]より大きい)
```

このように、まずは小さな配列をサンプルとして使って空欄を埋めていきましょう。

正解 ク

例題④

R4/4月サンプル問題 - 問2

次のプログラム中の **a** と **b** に入れる正しい答えの組合せを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

次のプログラムは、整数型の配列arrayの要素の並びを逆順にする。

プログラム

```

1  整数型の配列: array ← {1, 2, 3, 4, 5}
2  整数型: right, left
3  整数型: tmp
4  for (leftを1から(arrayの要素数÷2の商)まで1ずつ増やす)
5      right ← a
6      tmp ← array[right]
7      array[right] ← array[left]
8      b ← tmp
9  endfor

```

解答群

	a	b
ア	arrayの要素数 - left	array[left]
イ	arrayの要素数 - left	array[right]
ウ	arrayの要素数 - left + 1	array[left]
エ	arrayの要素数 - left + 1	array[right]

解説 プログラムを見るだけで、解答群から2つを削ることができます。仮にイやエを選んだとすると、5～8行目は次のようになります。



解説動画

```

5  right ← a
6  tmp ← array[right]
7  array[right] ← array[left]
8  array[right] ← tmp

```

6行目でarray[right]の要素をいったんtmpに入れ、8行目でまた戻しています。この動作に意味はなく、明らかな誤りだと判断できるので、**イ**と**エ**は除外します。これで **b** はarray[left]であることが確定しました。

```
5   right ← a
6   tmp ← array[right]
7   array[right] ← array[left]
8   array[left] ← tmp
```

例題3で学習した、要素の入れ替えがここでも使われています。要素を逆順にするために必要な入れ替えはこのようになります。



ちょうど真ん中である3番目は、入れ替える必要がありません。その前提で繰り返し条件を見てみましょう。

```
4   for (leftを1から(arrayの要素数÷2の商)まで1ずつ増やす)
```

arrayの要素数は5なので、次のような条件であることがわかります。

```
4   for (leftを1から2まで1ずつ増やす)
```

leftが1の場合には5と入れ替え、leftが2の場合には4と入れ替えるようにプログラムを完成させます。

a に選択肢の1つである「arrayの要素数 - left」を入れてみましょう。

```
5   right ← arrayの要素数 - left
6   tmp ← array[right]
7   array[right] ← array[left]
8   array[left] ← tmp
```

arrayの要素数が5で、さらにleftが1の場合、次のようになります。

```
5   right ← 5 - 1
6   tmp ← array[4]
7   array[4] ← array[1]
8   array[1] ← tmp
```

ここではarray[1]とarray[4]を交換してしまっています。そうではなく、正しいのはarray[1]とarray[5]の交換です。したがって a に入るのは「arrayの要素数 - left + 1」です。念のためトレースしてみましょう。

プログラム	array	right	left	tmp
1: 整数型の配列: array ← {1, 2, 3, 4, 5}	1, 2, 3, 4, 5			
2: 整数型: right, left				
3: 整数型: tmp				
4: for (leftを1から(arrayの要素数÷2の商)まで1ずつ増やす)			1	
5: right ← arrayの要素数 - left + 1		5		
6: tmp ← array[right]				5
7: array[right] ← array[left]	1, 2, 3, 4, 1			
8: array[left] ← tmp	5, 2, 3, 4, 1			

ひとまず先頭と最後が入れ替わりました。次に2番目と4番目を入れ替えます。

プログラム	array	right	left	tmp
9: endfor			2	
4: for (leftを1から(arrayの要素数÷2の商)まで1ずつ増やす)				
5: right ← arrayの要素数 - left + 1		4		
6: tmp ← array[right]				4
7: array[right] ← array[left]	5, 2, 3, 2, 1			
8: array[left] ← tmp	5, 4, 3, 2, 1			

これで逆順になりました。したがって、正解はウです。

正解 ウ



02

二次元配列



表形式で値を扱う方法を学ぼう

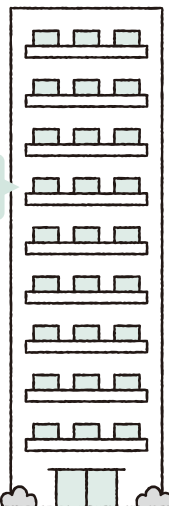
- 二次元配列を扱うと便利な場面について学ぼう
- 二次元配列を構成する要素の扱い方を学ぼう

二次元配列とは

配列の説明ではアパートを例に挙げました。配列は、部屋番号を指定することで特定の1部屋を特定できるイメージです。一方、**二次元配列**は表(テーブル)のように、例えば「上から3番目、左から6番目」といった具合に2つの値を指定することで、初めて1つの場所を特定できるデータ形式です。通常の配列(**一次元配列**)が一階建てのアパートだとすると、二次元配列は高層マンションに例えられます。



101号室

7階の
709号室

行と列

二次元配列はExcelのような表形式のデータを扱うのに適しています。Excelでは、行番号と列番号を指定することで、特定のセルを指定することができます。

	1 列目 ↓	2 列目 ↓	3 列目 ↓	4 列目 ↓	
1行目 →	A	B	C	D	E
2行目 →	1				
3行目 →	2				
4行目 →	3				
5行目 →	4				
	5				

3行目の4列目

二次元配列でも行にあたる番号と、列にあたる番号を指定することで特定の要素を指定します。なお、どちらが行でどちらが列か迷った場合には、次のように漢字の構造で思い出すとよいでしょう。



人気のラーメン店に「行列ができる」などと表現することから、一般的には「行」「列」の順番で特定の要素を指定します。

二次元配列の宣言

二次元配列を宣言する場合は、データ型を指定し「〇〇型の二次元配列」という記述をします。次の例では、文字列型の二次元配列を用意しています。

1 文字列型の二次元配列: student

また「〇〇型配列の配列」という少し変わった記述をする場合もあります。次の例は「文字列型の配列をさらに配列にする」という意味で、上記の例と同じ動作になります。

1 文字列型配列の配列: student

二次元配列を宣言したときは、次のような表をイメージしてください。

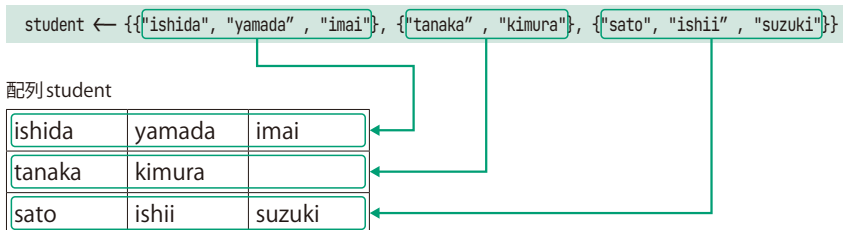
			配列 (一次元配列)
			配列の配列 (二次元配列)

格納

宣言したあとは、次のように記述することで、二次元配列の各要素に値を格納することができます。

```
2 student ← {"ishida", "yamada", "imai"},
3             {"tanaka", "kimura"},
4             {"sato", "ishii", "suzuki"}}
```

二次元配列の1行目と3行目には3つの要素が格納され、2行目には2つの要素が格納されます。



一次元配列と同様に、二次元配列も要素番号を指定して値を格納することができます。**要素番号は、配列[行番号][列番号]のように、行→列の順で指定します※5。**

※5 行→列の順で指定することは必須ではありません。列→行の順で指定する場合でも、プログラム全体でその順序を一貫して使用していると認識していれば問題ありません。しかし、混乱の原因になりやすいため、原則として行→列の順を採用するようにしましょう。


```
5 student[2][1] ← "yamamoto"
6 student[3][2] ← "koyama"
```

これらの処理で「2行目の1列目」と「3行目の2列目」の要素が置換されました。

	1	2	3
1	ishida	yamada	imai
2	yamamoto	kimura	
3	sato	koyama	suzuki

行と列の番号は、次のようにコンマ区切りで記述することもあります。この場合も記述順は「行, 列」です。

```
5 student[2, 1] ← "yamamoto"
6 student[3, 2] ← "koyama"
```

また、宣言と同時に値の格納を行うこともできます。

```
1 文字列型配列の配列: student ← {"ishida", "yamada", "imai"},
2                                {"tanaka", "kimura"},
3                                {"sato", "ishii", "suzuki"}}
```

一次元配列に要素を追加する記述方法には、厳密なルールがないという説明をしました。例えば「重複なく辞書順に追加する」のようにコメント欄に書かれていることがあります。二次元配列も同様ですが、一次元配列よりも構造が複雑であるため、行番号と列番号の指定を間違わないように意識してください。

コメントに「配列studentに配列memberを追加する」とあった場合

配列 student

ishida	yamada	imai
yamamoto	kimura	
sato	koyama	suzuki

配列 member

tamura	hasegawa	oikawa
--------	----------	--------

配列 student

ishida	yamada	imai
yamamoto	kimura	
sato	koyama	suzuki
tamura	hasegawa	oikawa

コメントに「配列student[2]の末尾に"horiai"を追加する」とあった場合

配列 student

ishida	yamada	imai
yamamoto	kimura	
sato	koyama	suzuki

配列 student

ishida	yamada	imai
yamamoto	kimura	horiai
sato	koyama	suzuki

トレースのコツ

二次元配列はトレースが一気に難しくなります。二次元配列が小さければ、他の変数と同じトレース表を使ってください。

プログラム	i	student									
1 : ○○○○○○○○○○○	1	<table><tr><td>ishida</td><td>yamada</td><td>imai</td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	ishida	yamada	imai						
ishida	yamada	imai									
2 : ○○○○○○○○○○○	2	<table><tr><td>ishida</td><td>yamada</td><td>imai</td></tr><tr><td>yamamoto</td><td>kimura</td><td></td></tr><tr><td>sato</td><td>koyama</td><td>suzuki</td></tr></table>	ishida	yamada	imai	yamamoto	kimura		sato	koyama	suzuki
ishida	yamada	imai									
yamamoto	kimura										
sato	koyama	suzuki									
3 : ○○○○○○○○○○○	3	<table><tr><td>ishida</td><td>yamada</td><td>imai</td></tr><tr><td>tanaka</td><td>hasegawa</td><td>tamura</td></tr><tr><td>sato</td><td>koyama</td><td>suzuki</td></tr></table>	ishida	yamada	imai	tanaka	hasegawa	tamura	sato	koyama	suzuki
ishida	yamada	imai									
tanaka	hasegawa	tamura									
sato	koyama	suzuki									

二次元配列のサイズが大きいのであれば、二次元配列だけのトレース表を別の場所に用意したほうがよいでしょう。

プログラム	i
1: ○○○○○○○○○○○○○○	1
2: ○○○○○○○○○○○○○○	2
3: ○○○○○○○○○○○○○○	3

①

②

③

①

ishida	yamada	imai	murata

②

ishida	yamada	imai	murata
yamamoto	kimura		
sato	koyama	suzuki	

③

ishida	yamada	imai	murata
tanaka	hasegawa	tamura	kato
sato	koyama	suzuki	hoshino
bando	abe	kimura	horiai

また二次元配列のすべての要素ではなく、一部分だけをトレースする手もあります。仮に 10×10 などの要素数が多い二次元配列があったとします。このうち左角にあたる 3×3 だけをトレースしていくなどが有効です。

一部分だけトレースする

例題にチャレンジ

例題①

オリジナル問題

次の記述中の に入れる正しい答えを、解答群の中から選べ。

次のプログラムにおいて、手続mainを呼び出すと、元の表が したものがBとなる。

プログラム

```
1  ○main()
2   整数型配列の配列: A ← {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}
3   整数型配列の配列: B
4   B ← transpose(A)
5
6  ○transpose(整数型配列の配列: src)
7   整数型: n ← src[1]の要素数
8   整数型配列の配列: dst ← n×n要素を0で初期化した行列
9   整数型: i, j
10  for (iを1からnまで1ずつ増やす)
11    for (jを1からnまで1ずつ増やす)
12      dst[j][i] ← src[i][j]
13    endfor
14  endfor
15  return dst
```

解答群

- ア 90度右回転
- イ 90度左回転
- ウ 90度右回転してから左右反転
- エ 90度左回転してから上下反転

解説 まずはこの例題で、二次元配列のトレースに慣れましょう。今回は小さなサイズの二次元配列なので、トレース表の中にそのまま表を描きながら確認していきます。



解説動画

手続mainが呼び出されると、2行目で二次元配列Aの各要素に値が代入されます。

プログラム	A	B									
2: 整数型配列の配列: A ← $\{\{1, 2, 3\}, \{4, 5, 6\}, \{7, 8, 9\}\}$	<table> <tr><td>1</td><td>2</td><td>3</td></tr> <tr><td>4</td><td>5</td><td>6</td></tr> <tr><td>7</td><td>8</td><td>9</td></tr> </table>	1	2	3	4	5	6	7	8	9	
1	2	3									
4	5	6									
7	8	9									

4行目でその二次元配列を関数transposeに渡します。関数transposeでは配列srcで受け取り、7行目で「src[1]」を参照しています。src[1]は{1, 2, 3}なので、要素数は3です。

プログラム	src			n	dst		
6：〇transpose(整数型配列の配列：src)	1	2	3				
	4	5	6				
	7	8	9				
7：整数型：n ← src[1]の要素数				3			
8：整数型配列の配列：dst ← n×n要素を0で初期化した行列					0	0	0
					0	0	0
					0	0	0

10～14行目はfor文による繰り返し処理です。12行目で配列srcの要素を配列dstにコピーしています。このときコピー元の要素番号と、コピー先の要素番号が異なりますので、注意しましょう。

1つ目の繰り返し処理では、iとjはともに1であるため、配列srcの左上の要素を、配列dstの同じく左上にコピーすることになります。

プログラム	src	n	dst	i	j																		
10: for (iを1からnまで1ずつ増やす)				1																			
11: for (jを1からnまで1ずつ増やす)					1																		
12: dst[j][i] ← src[i][j]	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table>	1	2	3	4	5	6	7	8	9		<table><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td></tr></table>	1	0	0	0	0	0	0	0	0		
1	2	3																					
4	5	6																					
7	8	9																					
1	0	0																					
0	0	0																					
0	0	0																					

jがカウントアップされて2になり、次のように要素がコピーされました。

プログラム	src	n	dst	i	j																		
10 : for (iを1からnまで1ずつ増やす)				1																			
11 : for (jを1からnまで1ずつ増やす)					2																		
12 : dst[j][i] ← src[i][j]	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table>	1	2	3	4	5	6	7	8	9		<table><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>2</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td></tr></table>	1	0	0	2	0	0	0	0	0		
1	2	3																					
4	5	6																					
7	8	9																					
1	0	0																					
2	0	0																					
0	0	0																					

さらに内側ループが1つ進んでjが3になるとこのようになります。

プログラム	src			n	dst			i	j
12 : dst[j][i] ← src[i][j]	1	2	3		1	0	0		
	4	5	6		2	0	0		
	7	8	9		3	0	0		

内側のループが終わり、外側のループが2周目に入ります。

プログラム	src	n	dst	i	j																		
10 : for (iを1からnまで1ずつ増やす)				2																			
11 : for (jを1からnまで1ずつ増やす)					1																		
12 : dst[j][i] ← src[i][j]	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table>	1	2	3	4	5	6	7	8	9		<table><tr><td>1</td><td>4</td><td>0</td></tr><tr><td>2</td><td>0</td><td>0</td></tr><tr><td>3</td><td>0</td><td>0</td></tr></table>	1	4	0	2	0	0	3	0	0		
1	2	3																					
4	5	6																					
7	8	9																					
1	4	0																					
2	0	0																					
3	0	0																					

これを最後まで続けます。

●外側のループ2周目が終わった状態



●外側のループ3周目が終わった状態



配列srcを90度左に回転させてから、上下に反転させた表になりました。これを15行目のreturnで、呼び出し元に返します。

プログラム	A	B									
4 : B ← transpose(A)		<table><tr><td>1</td><td>4</td><td>7</td></tr><tr><td>2</td><td>5</td><td>8</td></tr><tr><td>3</td><td>6</td><td>9</td></tr></table>	1	4	7	2	5	8	3	6	9
1	4	7									
2	5	8									
3	6	9									

したがって、正解はエの「90度左回転してから上下反転」です。

正解 エ

例題②

R1 秋 - 午前問9 改変

配列Aが図1の状態のとき、次のプログラムを実行すると、配列Bが図2の状態になった。プログラム中の□に入る正しい答えを、解答群の中から選べ。ここで、配列の要素番号は0から始まる。

		j →							
		0	1	2	3	4	5	6	7
i ↓	0		*	*	*	*	*	*	
	1		*						
	2		*						
	3		*	*	*	*			
	4		*						
	5		*						
	6		*						
	7		*						

図1 配列Aの状態

		j →							
		0	1	2	3	4	5	6	7
i ↓	0								
	1	*	*	*	*	*	*	*	*
	2					*			*
	3					*			*
	4					*			*
	5								*
	6								*
	7								

図2 配列Bの状態

プログラム

```

1  整数型:i, j
2  for (iを0から7まで1ずつ増やす)
3    for (jを0から7まで1ずつ増やす)
4      □
5    endfor
6  endfor

```

解答群

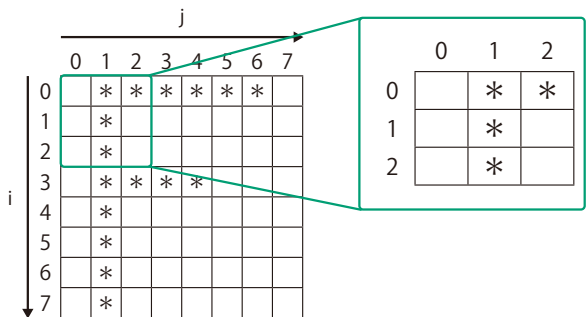
- ア $B[7 - i, 7 - j] \leftarrow A[i, j]$
- イ $B[7 - j, i] \leftarrow A[i, j]$
- ウ $B[i, 7 - j] \leftarrow A[i, j]$
- エ $B[j, 7 - i] \leftarrow A[i, j]$

解説 実際の試験では、要素番号が1から始まる場合がほとんどですが、この例題のように0から始まる場合もありますので、注意しましょう。

図1と図2を見比べてみると、「F」の形である図1を、右に90度回転する処理のようです。選択肢を1つずつ確認していきましょう。今回の配列は8×8と、少々大きめのサイズです。そのため、一部分だけをトレースしていきます。

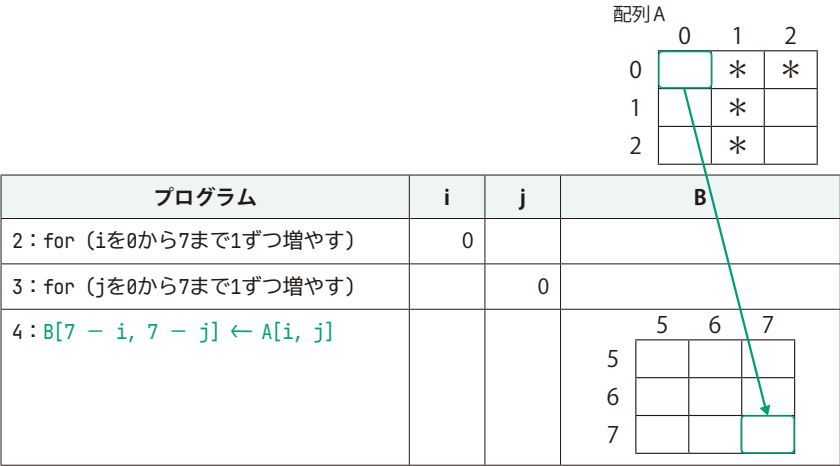


解説動画



また、プログラムや解答群の内容から、配列Aに変化がないことがわかるので、トレース表に記載しません。

まずは**A**からトレースしていきます。4行目では配列Aの左上 ([0, 0]) を、配列Bの左上 ([7, 7]) にコピーしています。



内側のループの2周目はB[7, 6] に、A[0, 1] の要素をコピーします。

配列 A	0	1	2
0		*	*
1		*	
2		*	

プログラム	i	j	B																
2: for (iを0から7まで1ずつ増やす)	0																		
3: for (jを0から7まで1ずつ増やす)		1																	
4: $B[7 - i, 7 - j] \leftarrow A[i, j]$			<table> <tr> <td></td><td>5</td><td>6</td><td>7</td></tr> <tr> <td>5</td><td></td><td></td><td></td></tr> <tr> <td>6</td><td></td><td></td><td></td></tr> <tr> <td>7</td><td></td><td>*</td><td></td></tr> </table>		5	6	7	5				6				7		*	
	5	6	7																
5																			
6																			
7		*																	

配列 A の [0, 1] は、「F」の角にあたる部分です。「F」の角は配列 B では [1, 7] に格納しなければならないため、**ア** は誤りです。このようにいくつかトレースすることで、答えを導き出すためには何をポイントにして考えればよいか、ひらめくことがあります。

では、いまひらめいた方法で考えてみます。「F」の角は配列 A では [0, 1] の要素です。配列 B では [1, 7] に格納する必要があります。

	j							
	0	1	2	3	4	5	6	7
i ↓ 0		*	*	*	*	*	*	
1		*						
2		*						
3		*	*	*	*			
4		*						
5		*						
6		*						
7		*						

図1 配列 A の状態

	j							
	0	1	2	3	4	5	6	7
i ↓ 0								
1	*	*	*	*	*	*	*	*
2				*				*
3				*				*
4				*				*
5							*	
6							*	
7								

図2 配列 B の状態

イ は $B[7 - j, i]$ です。i に 0、j に 1 を入れると [6, 0] となります。[1, 7] とならないため、誤りです。

ウ は $B[i, 7 - j]$ です。i に 0、j に 1 を入れると [0, 6] となります。[1, 7] とならないため、誤りです。

エ は $B[j, 7 - i]$ です。i に 0、j に 1 を入れると [1, 7] となるため、正解です。

正解 **エ**

Chapter

3

オブジェクト指向

本章の学習ポイント

- クラスは設計図で、インスタンスはそれを実体化したもの
- クラスに定義されている変数や配列をメンバ変数、関数をメソッドと呼ぶ
- コンストラクタとは、インスタンスを実体化したタイミングで必ず実行されるメソッドのこと



01

オブジェクト指向



オブジェクト指向の概念と使い方について学ぼう

- オブジェクト指向が試験でどのように登場するのかを理解しよう
- 提示されたクラスをプログラム内でどのように利用するのかを学ぼう

オブジェクト指向とは

オブジェクト指向が登場する以前は、大規模なプログラム開発において、プログラムが複雑化し、可読性が著しく低下するという課題がありました。しかし、オブジェクト指向の導入により、大規模なプログラムも比較的シンプルに扱えるようになりました。具体的な解説に入る前に、多くの人にとって身近なツールであるExcelを例として、オブジェクト指向の基本的な考え方を解説していきます。

オブジェクト指向における**クラス**は、1つのシートに相当します。例えばある店舗で、顧客を管理するために「顧客シート」を作ったとします。これはオブジェクト指向では「顧客クラス」に相当します。

顧客シートには、「顧客番号」「顧客名」「住所」といった項目があります。これらはオブジェクト指向では**メンバ変数**に相当します。

クラスはまだ具体的な値が入っていない、データの構造を定義するものです。

	A	B	C	D	E	F
1						
2		顧客番号	顧客名	住所		
3						
4						
5						
6						
7						
8						

具体的な値を入れたものを**インスタンス**といいます。これは「顧客シート」における、個々の顧客データが該当します。

	A	B	C	D	E	F
1						
2		顧客番号	顧客名	住所		
3		1	北海道一郎	北海道…		
4		2	東京次郎	東京都…		
5		3	大阪三郎	大阪府…		
6		4	名古屋四郎	愛知県…		
7		5	横浜五郎	神奈川県…		
8						
9						
10						
11						

クラスには、メンバ変数以外にも処理を定義することができます。同じくExcelを例に説明します。次のシートにはボタンが設置されています。ボタンをクリックすると、あらかじめプログラムで記述した処理を実行させることが可能です。例えば、採番ボタンをクリックした場合、顧客番号を1から順番に振ることができます。

	A	B	C	D	E	F	G
1							
2		顧客番号	顧客名	住所		顧客検索	
3		1	北海道一郎	北海道…			
4		2	東京次郎	東京都…			
5		3	大阪三郎	大阪府…		顧客登録	
6		4	名古屋四郎	愛知県…			
7		5	横浜五郎	神奈川県…			
8						採番	
9							
10							
11							

顧客検索ボタンをクリックすると、顧客番号を聞かれるので、顧客名を調べたい顧客番号を入力します。

顧客検索

×

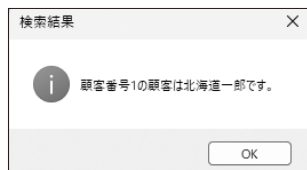
顧客番号を入力してください

OK

キャンセル

11

入力した顧客番号をもとに、顧客シート内を検索して、該当する顧客の名前を表示します。



このようにExcelでは、シートに処理を埋め込んで、データを操作することができます。オブジェクト指向におけるクラスにも処理を記述することができ、それを**メソッド**といいます。

また、Excelでデータを管理する場合には、顧客シート、商品シート、注文シートなど、扱うデータの種類ごとにシートを分けると思います。もし、これらのデータを1つのシートにまとめてしまうと、データを更新するのに手間がかかり、扱いづらくなってしまいます。

	A	B	C	D	E	F	G	H
1								
2		顧客番号	顧客名	住所		商品番号	商品名	価格
3		1	北海道一郎	北海道…		1	鉛筆	110
4		2	東京次郎	東京都…		2	消しゴム	90
5		3	大阪三郎	大阪府…		3	ボールペン	140
6		4	名古屋四郎	愛知県…		4	シャープペン	120
7		5	横浜五郎	神奈川県…		5	マーカー	170
8								
9								
10		注文番号	顧客名	商品番号	注文数			
11		1	5月1日	4	3			
12		2	5月1日	1	10			
13		3	5月2日	4	3			
14		4	5月2日	2	8			
15		5	5月2日	1	9			
16		6	5月3日	1	6			
17		7	5月4日	3	5			
18								
19								
20								
21								

情報システム開発においても、**データの種類ごとにクラスを作成する**のが一般的です。

	A	B	C	D	E
1					
2		顧客番号	顧客名	住所	
3					
4					
5					
6					
7					

	A	B	C	D	E
1					
2		商品番号	商品名	価格	
3					
4					
5					
6					
7					

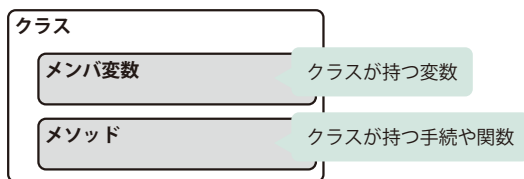
	A	B	C	D	E
1					
2		注文番号	注文日	商品番号	注文数
3					
4					
5					
6					
7					

これからオブジェクト指向の学習を進めていきますが、混乱した場合にはクラスはExcelのシートと似た考え方であることを思い出してください。

擬似言語でのクラス

クラスとはExcelにおけるシートのようなものであり、**通常は管理するモノやコトを1つのクラス**とします。「管理するモノ」が社員であれば「社員クラス」、商品であれば「商品クラス」を作成します。「管理するコト」が注文であれば「注文クラス」、出勤であれば「出勤クラス」を作成します。

そしてクラスには、メンバ変数とメソッドを定義します。



クラスで扱う変数を**メンバ変数**といいます。システム開発の要件定義で、依頼者から「注文に関しては、注文があった日付や顧客、商品や個数を管理したい」などと要望を受けたとします。この場合、次のようなメンバ変数を定義することになります。なお、注文を特定するために必要となる「注文コード」も定義しました。

- 1 文字列型: orderDate
- 2 整数型: customerCode
- 3 整数型: productCode
- 4 整数型: number
- 5 整数型: orderCode

クラスで扱う手順や関数を**メソッド**といいます。メソッドには引数を渡すことができ、必要であれば戻り値を返すこともできます。

本書を執筆している時点では、クラスを定義するプログラムはサンプル問題にありません。問題文にはあらかじめ定義されたクラスの説明があり、それを通常の手続や関数で使います^{※1}。次の表は「あらかじめ定義されたクラスの説明」の例です。

クラスOrderのメンバ変数の説明

メンバ変数	型	説明
orderDate	文字列型	注文があった日付
customerCode	整数型	注文した顧客の顧客コード
productCode	整数型	注文した商品の商品コード
number	整数型	注文した商品の個数
orderCode	整数型	注文を識別するための番号

クラスOrderのメソッドの説明

メソッド	戻り値	説明
insert(文字列型: orderDate, 整数型: customerCode, 整数型: productCode, 整数型: number)	整数型	注文した情報を追加し、メソッド内で採番された注文コードを返す。
update(整数型: orderCode, 文字列型: orderDate, 整数型: customerCode, 整数型: productCode, 整数型: number)	なし	第一引数で渡した注文コードに該当する注文を、その他の引数の値で更新する。
delete(整数型: orderCode)	なし	第一引数で渡した注文コードに該当する注文を削除する。

※1 IPAが公開しているサンプル問題には、クラスについての説明があるだけです。

このように、問題内のプログラムに用意されているクラスについて、日本語で説明されます。

1つ目の表には、メンバ変数の名前、型、説明がまとめられており、クラス Order が5つのメンバ変数を持っていることがわかります。また2つ目の表には、メソッドの名前、戻り値、説明がまとめられており、クラス Order が3つのメソッドを持っていることがわかります。

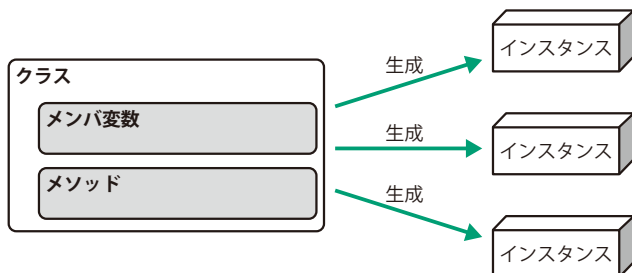
なお、クラス名の先頭は大文字であることがほとんどです。このクラスは注文に関連するデータや処理を扱うため、クラス名は「Order」です。それに対して、メンバ変数やメソッドの名前の先頭は小文字であることがほとんどです。注文日は「orderDate」、顧客コードは「customerCode」という変数名が付けられています。

インスタンス

クラスはあくまで設計図のようなものです。Excelの例では、次のようなクラスを設計しました。

	A	B	C	D	E	F
1						
2		顧客番号	顧客名	住所		
3						
4						
5						
6						
7						
8						

実際にメンバ変数に値を入れたり、メソッドを実行するには、クラスをもとに**インスタンス**を生成する必要があります。インスタンスとは「実体」を意味する言葉です。なお、ある1つのクラスから複数のインスタンスを生成できます。



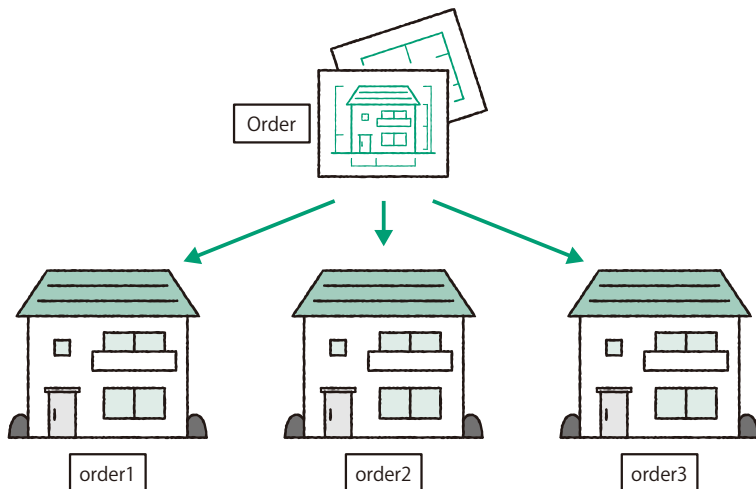
クラスをもとに、インスタンスを生成してみましょう。次のような記述により、インスタンスを入れる変数を用意できます。このインスタンスを入れる変数のことを**インスタンス変数**といいます。

クラス名: インスタンス名

整数型の変数 `number` を宣言する場合は「整数型: `number`」、文字列型の変数 `name` を宣言する場合は「文字列型: `name`」と記述することはすでに学びました。同様に、クラス `Order` のインスタンス変数を宣言するときは、次のように記述します。

```
1 Order: order1
2 Order: order2
3 Order: order3
```

インスタンス変数 `order1`、`order2`、`order3` を宣言しました。これは `Order` という設計図をもとに、`order1`、`order2`、`order3` という3つの家を建てたようなものです。



通常の変数の定義と同じく、インスタンス変数もまとめて定義することが可能です。

```
1 Order: order1, order2, order3
```

ただし、この段階ではインスタンスを入れるための変数を用意しただけで、まだインスタンスは存在していません。通常の変数も「数値型: age」としただけでは、変数ageの中身は空(未定義)の状態です。「age ← 50」と記述することで、変数ageに50が代入されます。インスタンスも同じです。

インスタンスを生成して、それをインスタンス変数に代入する場合は、次のように記述します。

```
インスタンス変数 ← クラス名()
```

クラス名のあとに()を記述する必要があります。これはある種のルールとして暗記したほうがよいでしょう。

次の例では、インスタンス変数order1、order2、order3を宣言し、それぞれに生成したインスタンスを代入しています。

```
1 Order: order1, order2, order3
2 order1 ← Order()
3 order2 ← Order()
4 order3 ← Order()
```

こうすることで、ようやくインスタンス変数order1、order2、order3の準備が終わり、メンバ変数やメソッドを使うことができるようになります。

インスタンスを用意したあとは、以下のように記述することで、メンバ変数やメソッドの操作を行うことができます。

```
インスタンス名.メンバ変数名
インスタンス名.メソッド名(引数)
```

例えば、「order.orderDate」は、「インスタンスorderのメンバ変数orderDate」という意味です。この記述にしたがって、各メンバ変数に値を代入してみます。

```

5 order1.orderDate ← "2025/10/19"
6 order1.customerCode ← 51
7 order1.productCode ← 14
8 order1.number ← 1

```

インスタンス order1 のメンバ変数に値が代入されました。

インスタンス order1

orderDate	customerCode	productCode	number
2025/10/19	51	14	1

メソッドも使ってみましょう。先ほどのメソッドの説明を確認します。

クラス Order のメソッドの説明 (再掲)

メソッド	戻り値	説明
insert(文字列型: orderDate, 整数型: customerCode, 整数型: productCode, 整数型: number)	整数型	注文した情報を追加し、メソッド内で採番された注文コードを返す。
update(整数型: orderCode, 文字列型: orderDate, 整数型: customerCode, 整数型: productCode, 整数型: number)	なし	第一引数で渡した注文コードに該当する注文を、その他の引数の値で更新する。
delete(整数型: orderCode)	なし	第一引数で渡した注文コードに該当する注文を削除する。

表を参考に、インスタンス order2 のメソッドである insert に引数を渡して呼び出します。

```

9 order2.insert("2025/07/08", 21, 14, 9)

```

さらにメソッドの説明を確認すると、メソッド insert には戻り値があるようです。戻り値を受け取って出力してみます。

```

10 整数型: orderCode = order3.insert("2025/08/28", 30, 45, 5)
11 orderCodeを出力

```

10行目で変数orderCodeを宣言すると同時に、order3のメソッドinsertの戻り値を代入しています。さらに11行目でその値を出力しています。ここまでの流れで、インスタンスorder1～3のそれぞれのメンバ変数に値が代入されました。

インスタンス order1

orderDate	customerCode	productCode	number
2025/10/19	51	14	1

インスタンス order2

orderDate	customerCode	productCode	number
2025/07/08	21	14	9

インスタンス order3

orderDate	customerCode	productCode	number
2025/08/28	30	45	5

order1は、それぞれのメンバ変数に値を直接代入しました。order2とorder3はinsertを呼び出したことによって、メンバ変数に値が代入されました。クラスOrderの説明をもとに、メソッドの処理内容を推測して、プログラムを記述しました。**試験でも、説明欄の記述をもとに処理を推測してください。**

インスタンスのネスト構造

メンバ変数には、文字や数値などの単純な値だけではなく、インスタンスを含めることも可能です。混乱しやすいため、1つずつ処理を見ていきます。まず2人の人をインスタンス化しました。クラスはPersonです。

```
1 Person: person1 ← Person()
2 Person: person2 ← Person()
3 person1.id ← 1
4 person1.name ← "北海太郎"
5 person1.age ← 19
6 person2.id ← 80
7 person2.name ← "北海次郎"
8 person2.age ← 17
```

インスタンス person1

id	name	age
1	北海太郎	19

インスタンス person2

id	name	age
80	北海次郎	17

ここまでのプログラムでは、インスタンス person1 と person2 が個別に存在している状態です。実はこの2人は兄弟なので、情報を紐付けたいと思います。

person1 にメンバ変数 youngerBrotherId を用意し、弟の id をセットしました。これで person1 の弟は person2 であることがわかります。

インスタンス person1

id	name	age	youngerBrotherId
1	北海太郎	19	80

インスタンス person2

id	name	age
80	北海次郎	17

この方法以外にも、person1 のメンバ変数に person2 を代入して、兄弟であることを明示する方法があります^{※2}。

インスタンス person1

id	name	age	youngerBrotherId
1	北海太郎	19	person2

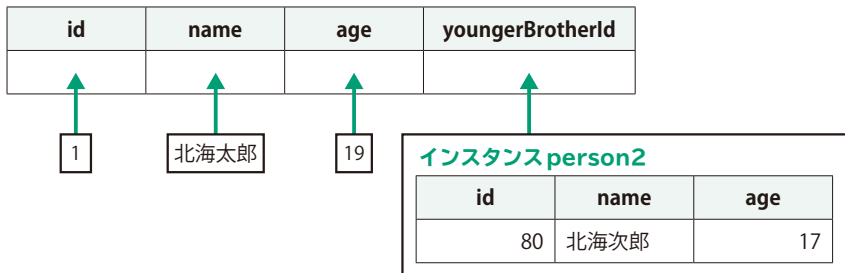
インスタンス person2

id	name	age
80	北海次郎	17

※2 実際にはメンバ変数に、インスタンス変数の値を代入しているのではなく、インスタンス変数へのポインタを代入していますが、あまり気にする必要はありません。

少し不思議に思うかもしれませんが、メンバ変数のデータ型には、整数型や文字列型だけではなく、クラスも指定できます。次のように考えると、イメージしやすいかもしれません。

インスタンス person1



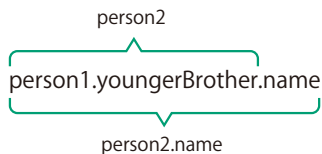
メンバ変数にインスタンスを代入するには、次のように記述します。

```
9 person1.youngerBrother ← person2
```

また、次のようにしてメンバ変数を使います。

```
10 person1.nameを出力
11 person1.youngerBrother.nameを出力
```

10行目では「北海太郎」、11行目では「北海次郎」と出力されます。「person1.youngerBrother」は「person2」のことで、「person1.youngerBrother.name」は「person2.name」と同じ意味になります。



インスタンスの配列

1つのクラスから複数のインスタンスを生成する場合、それらのインスタンスを配列として扱ったほうが便利な場合もあります。整数型の配列では「整数型の配列: numbers ← {5個の未定義の値}」のように記述して宣言していました。インスタンスを配列にする場合でも、同様に記述します。

```
1 Orderの配列: orders ← {5個の未定義の値}
```

さらにインスタンスを生成します。このとき、for文を使った繰り返し処理でインスタンスを生成し、配列の個々の要素に代入できます。

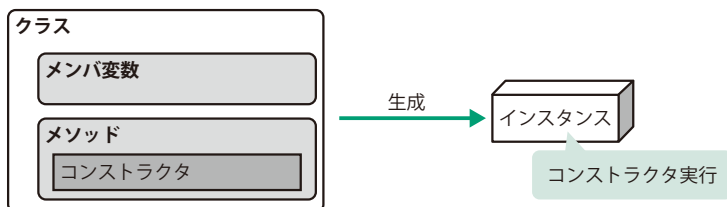
```
2 for (iを1から5まで1ずつ増やす)
3   orders[i] ← Order()
4 endfor
```

各インスタンスのメソッドを実行します。

```
5 orders[1].insert("2025/10/19", 51, 14, 1)
6 orders[2].insert("2025/07/08", 21, 14, 9)
7 orders[3].insert("2025/08/28", 30, 45, 5)
8 orders[4].insert("2025/11/07", 1, 10, 1)
9 orders[5].insert("2025/10/20", 7, 11, 1)
```

コンストラクタ

インスタンスを生成したタイミングで、自動的にメソッドを実行することができます。このメソッドが**コンストラクタ**です。



コンストラクタについてはメソッドと同様に表で説明されます。以下はOrderクラスのコンストラクタの説明です。「コンストラクタ」欄には、クラス名と同じ名前のメソッドが記載されます。

このように、**コンストラクタはクラスと同じ名前のメソッドであることが特徴**です。

クラスOrderのコンストラクタの説明例 1

コンストラクタ	説明
Order()	メンバ変数orderDateに現在の日付を設定する。

メソッドに引数を渡すことができるように、コンストラクタにも引数を渡すことができます。

クラスOrderのコンストラクタの説明例 2

コンストラクタ	説明
Order(文字列型: orderDate, 整数型: customerCode, 整数型: productCode, 整数型: number)	引数の値でメンバ変数を初期化する。

インスタンスを生成する際に、引数を渡します。

```
1 Order: order1 ← Order("2025/07/08", 21, 14, 9)
```

説明欄にあるとおり、このクラスはコンストラクタを使うことで、インスタンスを生成する際に、メンバ変数を初期化することができます。

例題にチャレンジ

例題①

R4/12月サンプル問題-問5改変

次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

関数 calc は、正の実数 x と y を受け取り、 $\sqrt{x^2 + y^2}$ の計算結果を返す。クラス Math は、計算に使うメソッドが複数定義されている。クラス Math のメソッドの説明を表に示す。

クラス Math のメソッドの説明

メソッド	戻り値	説明
pow(実数型: a, 実数型: b)	実数型	第1引数として正の実数 a を、第2引数として実数 b を受け取り、 a の b 乗の値を実数型で返す。

プログラム

```
1 ○実数型: calc(実数型: x, 実数型: y)
2   Math: math ← Math()
3   return 
```

解答群

- ア $(\text{math.pow}(x, 2) + \text{math.pow}(y, 2)) \div \text{math.pow}(2, 0.5)$
- イ $(\text{math.pow}(x, 2) + \text{math.pow}(y, 2)) \div \text{math.pow}(x, y)$
- ウ $\text{math.pow}(2, \text{math.pow}(x, 0.5)) + \text{math.pow}(2, \text{math.pow}(y, 0.5))$
- エ $\text{math.pow}(\text{math.pow}(\text{math.pow}(2, x), y), 0.5)$
- オ $\text{math.pow}(\text{math.pow}(x, 2) + \text{math.pow}(y, 2), 0.5)$
- カ $\text{math.pow}(x, 2) \times \text{math.pow}(y, 2) \div \text{math.pow}(x, y)$
- キ $\text{math.pow}(x, y) \div \text{math.pow}(2, 0.5)$

解説 「 x の 2 乗」と「 y の 2 乗」を加算して、その結果の平方根になるように を埋めます。

x の 2 乗をするには、クラス Math のメソッド pow を実行します。その際、第1引数に x 、第2引数に 2 を渡します。これで x の 2 乗の結果が得られます。同様に y の 2 乗も実行して両者を加算します。



解説動画

xの2乗		math.pow(x, 2)
+		+
yの2乗		math.pow(y, 2)

さらにその結果の平方根を計算します。平方根は0.5乗で得られます。

$$\underbrace{\text{math.pow}(x, 2) + \text{math.pow}(y, 2)}_{\text{math.pow}(\text{ }, 0.5)}$$

2行目でクラスMathはインスタンスを生成しており、解答群のプログラムもインスタンスmathを使ったもののみです。正解は**オ**の「math.pow(math.pow(x, 2) + math.pow(y, 2), 0.5)」です。

正解 **オ**

例題②

オリジナル問題

次の記述中の に入れる正しい答えを、解答群の中から選べ。

手続mainを実行すると出力結果は となる。

プログラム

```

1  ○main()
2    Company: c ← Company()
3    Person: takahashi ← Person()
4    takahashi.name ← "高橋"
5    Person: imai ← Person()
6    imai.name ← "今井"
7    Person: ishida ← Person()
8    ishida.name ← "石田"
9    c.ceo ← takahashi
10   imai.boss ← takahashi
11   ishida.boss ← imai
12   ishida.boss.boss.nameを出力

```

解答群

- ア Company
- イ Person
- ウ 高橋
- エ 今井
- オ 石田

解説 2行目でクラスCompanyのインスタンスを生成しています。同様に、3、5、7行目ではクラスPersonのインスタンスを生成しています。4、6、8行目ではメンバ変数に値を代入しているため、8行目を実行した時点での各インスタンスは次のような状態になります。



解説動画

インスタンス takahashi

name
高橋

インスタンス imai

name
今井

インスタンス ishida

name
石田

9行目では、インスタンスcのメンバ変数ceoに値を代入しています。この問題にはクラスの説明がありませんが、ceoに代入しているのはクラスPersonのインスタンスであるtakahashiです。そのため、メンバ変数ceoの型もPersonであることがわかります。

10行目ではインスタンスimaiのメンバ変数bossに、先ほどと同じくクラスPersonのインスタンスであるtakahashiを代入しています。したがって、メンバ変数bossの型もPersonであることがわかります。11行目まで実行した時点では、次のようになります。

インスタンスc

ceo
takahashi



インスタンス takahashi

name	boss
高橋	

インスタンス imai

name	boss
今井	takahashi

インスタンス ishida

name	boss
石田	imai

12行目では「ishida.boss.boss.name」を出力しています。インスタンス ishidaのメンバ変数bossはインスタンス imaiです。また、インスタンス imaiのメンバ変数bossはインスタンス takahashiです。したがって、12行目は「takahashi.name」とであると解釈できるため、「高橋」が出力されることとなります。

正解 ウ

例題③

オリジナル問題

次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

関数 main は、駅の名前と緯度及び経度を登録し、指定した緯度、経度における最寄りの駅の名前を得る手続である。クラス Station では駅に関する情報を管理している。クラス Station のメソッドの説明を表に示す。main を実行することで が出力される。関数 abs は、引数の絶対値を実数型で返す。

クラス Station の説明

メンバ変数	型	説明
name	文字列型	駅の名前。
position	実数型	札幌駅からの距離。

コンストラクタ	説明
Station(文字列型: name, 実数型: position)	引数の値でメンバ変数を初期化する。

プログラム

```

1  大域: Stationの配列: station
2
3  ○setStations()
4      station[1] ← Station("札幌", 0.0)
5      station[2] ← Station("大通", 1.2)
6      station[3] ← Station("すすきの", 2.0)
7      station[4] ← Station("中島公園", 2.8)
8
9  ○実数型: calcDistance(実数型: a, 実数型: b)
10     実数型: d ← abs(a - b)
11     return d
12
13  ○findNearestStation(実数型: myPos)
14     実数型: min // 非常に大きな値
15     整数型: index ← 未定義
16     整数型: i ← 1

```

```

17  while (iが4以下)
18      実数型: d ← calcDistance(myPos, station[i].position)
19      if (dがmin以下)
20          min ← d
21          index ← i
22      endif
23      i ← i + 1
24  endwhile
25  station[index].nameを出力
26
27  Omain()
28  setStations()
29  実数型: myPos ← 1.5
30  findNearestStation(myPos)

```

解答群

- ア 札幌
- イ 大通
- ウ すすきの
- エ 中島公園

解説 1行目では、クラスStationのインスタンス配列stationを大域変数として宣言しています。関数setStationsが呼び出されると、4行目から7行目でインスタンス配列stationの各要素に、駅名と路線上の位置(km)を持つstationインスタンスを生成して代入しています。クラスStationの説明にあるコンストラクタの引数は、それぞれ次のような情報を表しています。



解説動画

- 駅の名前
- 札幌駅からの距離

これらは、インスタンス生成時に対応するメンバ変数へ格納されます。

関数mainが呼び出されると、28行目で関数setStationsが呼び出され、4つの駅の情報がインスタンス配列stationに登録されます。

station[1]

name	position
札幌	0.0

station[3]

name	position
すすきの	2.0

station[2]

name	position
大通	1.2

station[4]

name	position
中島公園	2.8

29行目では、自分の現在位置を示す数値(1.5)を変数myPosに格納し、それを30行目で関数findNearestStationの引数として渡しています。

関数findNearestStationでは、繰り返し処理によって配列内の各インスタンス(駅)を1つずつ参照し、引数で受け取った現在地との距離を、関数calcDistanceを使って算出します。変数dには、現在地と駅との距離の差の絶対値が代入されます。

9	○実数型: calcDistance(実数型: a, 実数型: b)	
10	実数型: d ← abs(a - b)	station[1]の場合: 1.5
11	return d	station[2]の場合: 0.3
		station[3]の場合: 0.5
		station[4]の場合: 1.3

算出された距離が、それまでに記録された最小距離よりも小さい場合は、indexにその駅の番号が保存されます。すべての駅を調べ終えた時点で、indexに記録された駅の名前が出力され、もっとも近い駅が表示されます。1.5(自分の現在位置)にもっとも近いのは、1.2の大通です。したがって、正解は**イ**です。

正解 **イ**

例題④

オリジナル問題

次の記述中の に入れる正しい答えを、解答群の中から選べ。

手続mainは、注文情報を登録した後に返品処理を行う。このプログラムを実行すると出力結果は出力順に となる。なお手続mainには引数として2を渡している。

クラスOrderのメンバ変数の説明

メンバ変数	型	説明
orderCode	文字列型	注文の識別子。
orderDate	文字列型	注文があった日付。
customerCode	文字列型	注文した顧客の識別子。
itemCode	文字列型	注文した商品の識別子。
number	整数型	注文した商品の個数。

クラスOrderのメソッドの説明

メソッド	戻り値	説明
canReturn(整数型: n)	論理型	渡された個数nがこの注文のnumber以下であれば、真を返す。
returnItem(文字列型: date, 文字列型: reason, 整数型: number)	OrderReturn	指定された返品日、返品理由、個数で返品情報 (OrderReturn) を生成し、返す。

クラスOrderReturnのメンバ変数の説明

メンバ変数	型	説明
order	Order	対象となる注文インスタンス。
number	整数型	返品された商品の個数。
reason	文字列型	返品の理由。
date	文字列型	返品処理が行われた日。

プログラム

```
1  ○main(整数型: k)
2    Order: o ← Order()
3    o.orderCode ← "D999"
4    o.orderDate ← "2025/10/20"
5    o.customerCode ← "A18"
6    o.itemCode ← "P31"
7    o.number ← 3
8    if (o.canReturn(k))
9      OrderReturn: r
10     r ← o.returnItem("2025/11/07", "サイズが違う", k)
11     r.numberを出力
12     r.order.numberを出力
13   else
14     "返品できません"を出力
15   endif
```

解答群

- ア 1と2
- イ 2と3
- ウ 3と2
- エ 2と1

解説 2行目でインスタンスを生成し、3～7行目でメンバ変数に値を代入しています。これ以降、インスタンスoのメンバ変数の値に変更はありません。



解説動画

インスタンスo

orderCode	orderDate	customerCode	itemCode	number
D999	2025/10/20	A18	P31	3

続く8行目には条件分岐があります。インスタンスoのメソッドcanReturnの戻り値によって分岐するため、このメソッドの説明を確認してみましょう。

canReturn(整数型: n)	論理型	渡された個数nがこの注文のnumber以下であれば、真を返す。
-------------------	-----	---------------------------------

8行目では、メソッドcanReturnに引数として変数kを渡しています。変数kは1行目で受け取っており、問題文にある通り2が代入されています。そのため、「渡された個数2がこの注文のnumber以下であれば、真を返す。」と解釈できます。では次に、numberの値を考えます。説明には「この注文の」とあります。「この」は自分自身を示していると考えられます。自分自身とはこのインスタンス自身です。8行目の条件分岐を見てみると「o.canReturn」を実行していますので、「自分自身」とはインスタンスoのことです。

インスタンスo ←		
メンバ変数	型	説明
orderCode	文字列型	注文の識別子。
orderDate	文字列型	注文があった日付。
customerCode	文字列型	注文した顧客の識別子。
itemCode	文字列型	注文した商品の識別子。
number	整数型	注文した商品の個数。
メソッド	戻り値	説明
canReturn(整数型: n)	論理型	渡された個数nがこの注文のnumber以下であれば、真を返す。
returnItem(文字列型: date, 文字列型: reason, 整数型: number)	OrderReturn	指定された返品日、返品理由、個数で返品情報(OrderReturn)を生成し、返す。

インスタンスoのnumberは、7行目で3を代入しているため、さらに「渡された個数2が3以下であれば、真を返す。」と解釈できます。この条件を満たしているため、9行目に移動します。

9行目ではクラスOrderReturnのインスタンス変数を宣言しています。仮に「OrderReturn: r ← OrderReturn()」と記述されていれば、2行目と同様にすぐに使える状態でした。しかし単に「OrderReturn: r」とインスタンス変数を宣言しただけの状態です。続く10行目で、インスタンスoのメソッドreturnItemを実行し、戻り値をrに代入しています。引数は文字列「2025/11/07」、文字列「サイズが違う」、変数kです。説明を見ると、「返品情報(OrderReturn)を生成」とあります。どうやら内部では、次のようなプログラムが記述されているようです。変数名は仮にxとしました。

```
OrderReturn x ← OrderReturn()
```

さらに「指定された返品日、返品理由、個数で」とあるので、引数で受け取った値を各メンバ変数に代入しているようです。

インスタンスr

order	number	reason	date
	2	サイズが違う	2025/11/07

こうなると気になるのはメンバ変数orderです。説明には「対象となる注文のインスタンス」とあることから、2行目で生成され、3〜7行目で値を代入していたインスタンスoであると推測します。

インスタンスr

order	number	reason	date
インスタンスo	2	サイズが違う	2025/11/07

```
2 Order: o ← Order()
3 o.orderCode ← "D999"
  ⋮
7 o.number ← 3
8 if (o.canReturn(k))
9   OrderReturn: r
10  r ← o.returnItem("2025/11/07", "サイズが違う", k)
```

これでインスタンスrのすべてのメンバ変数に値が入りました。説明にある通り、OrderReturn型のインスタンスを戻り値として10行目で受け取っています。最後に、11行目で「r.number」、12行目で「r.order.number」を出力しています。

インスタンスr

order	number	reason	date
インスタンスo	2	サイズが違う	2025/11/07

orderCode	orderDate	customerCode	itemCode	number
D999	2025/10/20	A18	P31	3

2、3の順番で出力されるため、正解はイの「2と3」です。

正解 イ

Chapter

4

データ構造

本章の学習ポイント

- 連結リストは各要素が次の要素への参照を持っている構造であり、配列に比べて追加と削除は効率的だが、参照は非効率である
- 連結リストには単方向リストと双方向リストがある
- キューは先に入れた要素から順に取り出すことしかできず、スタックは逆に後から入れた順でしか取り出すことができない構造である
- 親子関係があるデータ構造が木構造であり、種類ごとに特徴がある

01

連結リスト



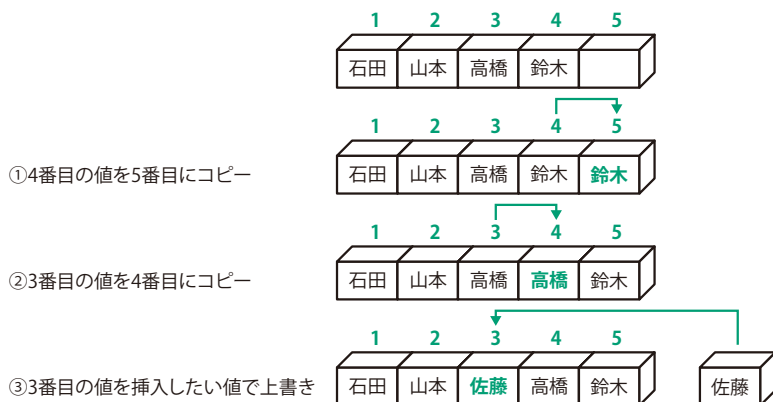
配列に比べた場合のメリットとデメリットを学習しよう

- 連結リストと配列の一番の違いを理解しよう
- 追加と削除が効率的に行うことができる理由を理解しよう
- 参照が非効率である理由を理解しよう

連結リストとは

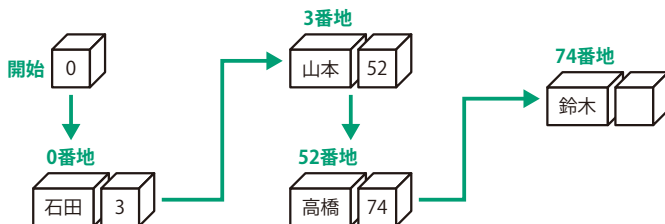
複数のデータを扱うのに、配列はとても便利でした。配列はデータを連続した領域に並べて保存するため、「3番目のデータは？」といった指定があれば、要素番号（添え字）を使って参照することができます。

一方で、配列の途中に新しい要素を挿入したい場合は、後ろに並んでいる要素を1つずらす必要があり、処理に手間がかかります。次の図は、2番目と3番目の間に値を挿入する手順です。まず3番目以降の値を1つずつ後ろにコピーしていきます。次に3番目の値を挿入したい値で上書きします。



また、途中の要素を削除して、その分だけ前に詰める場合も、同じように1つずつ前にコピーしていきます。これは非常に非効率で手間がかかる作業です※1。

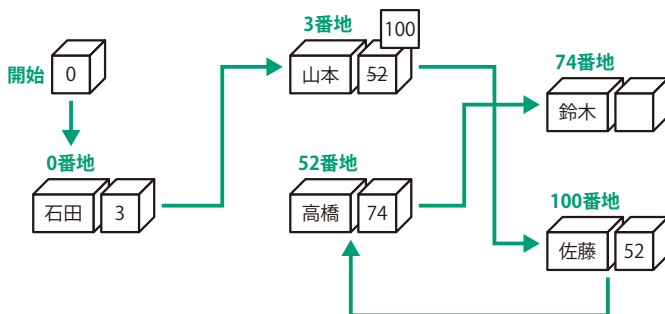
そこで登場するのが**連結リスト**です。連結リストは、個々のデータが次のデータがある場所の情報を保持しているデータ構造です。この「次のデータの場所」を**ポインタ**といいます。一般的には、次のデータが保存されている主記憶装置上のアドレス（住所）を、ポインタとして保持しています。



このように、データをポインタでつなぐことで、途中に要素を挿入したり、途中の要素を削除したりする操作が柔軟に行えます。

単方向リスト

先ほどの配列の例と同様に、連結リスト内の「山本」のあとに「佐藤」を挿入してみましょう。「山本」のポインタを「佐藤」にし、「佐藤」のポインタを「高橋」にするだけで挿入が完了します。



※1 多くのプログラミング言語では、配列を便利に操作するためのライブラリが用意されているため、実際にはプログラマが配列に対する操作を1から記述する場面は少ないと思います。

データが主記憶装置上のどこにあるかという物理的な位置にとらわれず、順序をポインタで表現できるのが連結リストの最大の特徴です。連結リストにはいくつか種類があり、次のデータのポインタだけを保持しているものは**単方向リスト**と呼ばれます。

JavaやPythonなどのプログラミング言語では、連結リストを扱うためのクラスがあらかじめ用意されています。しかし、擬似言語ではそのようなクラスは標準で用意されていません。そのため、次のようなメンバ変数を持つクラスを定義し、それを前提とした問題が作られています。

クラス **ListElement** のメンバ変数の説明

メンバ変数	型	説明
value	文字列型	要素の値。
next	ListElement	次の要素の参照。次の要素がないときの状態は未定義。

メンバ変数は「データそのもの (value)」と「次のデータを示すポインタ (next)」で構成されています。それでは、連結リストを扱うプログラムを擬似言語で記述してみましょう。まず連結リストのクラスのインスタンスを生成し、その連結リストにデータを追加してみます。

```
1 ListElement: student1 ← ListElement()
2 student1.value ← "ishida"
3 ListElement: student2 ← ListElement()
4 student2.value ← "yamamoto"
```

1行目と3行目で、ListElement クラスのインスタンスを2つ用意しました。

student1

value	next
ishida	

student2

value	next
yamamoto	

続いて、student1 の次がstudent2であることをポインタで示すプログラムを記述します。メンバ変数nextはListElement型なので、次のデータを持つインスタンスそのものを代入します。


```
5 student1.next ← student2
```

student1

value	next
ishida	student2

student2

value	next
yamamoto	

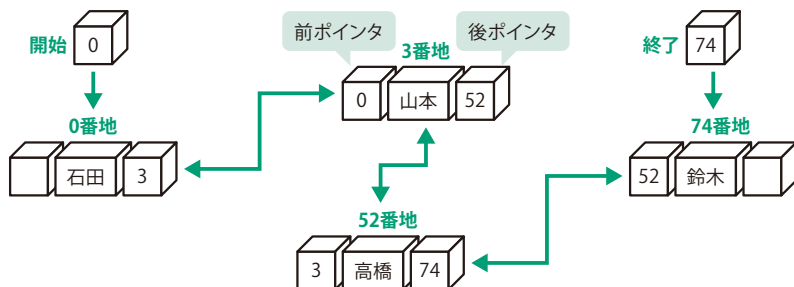
これで2つの要素で構成される連結リストができました。

なお問題によっては、いったんインスタンス変数に代入することなく、直接メンバ変数に代入することもあります。

```
1 ListElement: student1 ← ListElement()
2 student1.value ← "ishida"
3 student1.next ← ListElement() //インスタンスをメンバ変数に代入
4 student1.next.value ← "yamamoto"
```

双方向リスト

ここまでで説明した連結リストは、次のデータを示すポインタ（後ポインタ）だけを持つ単方向リストです。後ポインタに加え、前のデータを示すポインタ（前ポインタ）も持つ構造を**双方向リスト**といいます。



この連結リストにおいて、52番地の次の要素の番地は、後ポインタを参照することですぐにわかります。同様に前の要素の番地も、前ポインタを参照することでわかります。このように、自分の次だけではなく前の要素もすぐにわかるのが双方向リストのメリットです※2。

※2 ある要素にポインタが2つ格納されるため、配列に比べると必要な記憶容量が多くなるというデメリットがあります。ただし現在のコンピュータは記憶容量が十分に大きいため、それほど影響を受けることはありません。

例題にチャレンジ

例題①

オリジナル問題

次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

手続mainは、2つの要素からなる連結リストを生成し、先頭の要素とその次の要素の値を表示する。クラスListElementの説明を図に示す。ListElement型の変数はクラスListElementのインスタンスの参照を格納するものとする。

図 クラスListElementの説明

メンバ変数	型	説明
val	文字列型	リストの要素の値。
next	ListElement	リストの次の文字を保持するインスタンスの参照。初期状態は未定義である。

コンストラクタ	説明
ListElement(文字型: qVal)	引数 qVal でメンバ変数 val を初期化する。

プログラム

```

1  Omain()
2  ListElement: a ← ListElement("X")
3  ListElement: b ← ListElement("Y")
4  a.next ← b
5  ListElement: p ← a
6  p.valを出力
7  p ← 
8  p.valを出力

```

解答群

- ア p.val
- イ p.next
- ウ a.val
- エ a.next

オ b.val
カ b.next

解説 2行目と3行目でインスタンスを生成しています。コンストラクタで受け取った引数は、そのインスタンスのメンバ変数valに代入されます。



解説動画

a

val	next
X	

b

val	next
Y	

4行目で、インスタンスaのメンバ変数nextにbを代入します。

a

val	next
X	b



b

val	next
Y	

5行目ではさらにインスタンスを生成しています。このインスタンスpにはインスタンスaそのものを代入しているため、インスタンスaがそのままコピーされます。

a

val	next
X	b

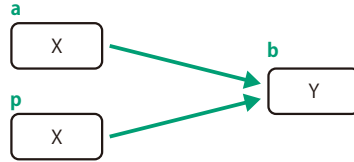
p

val	next
X	b

b

val	next
Y	

これでaとpが先頭で、それらの次がbという構造になりました。



問題文にある通り、6行目で先頭の値である「X」が出力されます。もし7行目がなければ、8行目でも「X」が出力されてしまいますが、問題文には「先頭の要素とその次の要素の値を表示」とあります。したがって8行目では「Y」が出力されなければなりません。インスタンスpのメンバ変数nextにはbが入っていますので、8行目で「p.next.val」を出力することで目的は達成されます。

しかし、8行目で出力しているのは「p.val」であり、このままでは要件を満たしません。そのため、7行目で「p ← p.next」と記述して代入を行い、「p」を「p.next」に変更します。

```
7    p ← p.next
```

これにより、インスタンスp自身がインスタンスbの複製になるため、p.valでYが出力されます。

したがって、答えは**イ**です。

正解 **イ**

例題②

オリジナル問題

次のプログラム中の a , b に入れる正しい答えの組合せを、解答群の中から選べ。

手続 searchBack は、引数で与えられた末尾の要素と探索したい文字を基に双方向リストから逆順に探し、要素の値を返す手続である。各要素は、クラス Node を用いて表現する。クラス Node のメンバ変数の説明を図に示す。

図 クラス Node のメンバ変数の説明

メンバ変数	型	説明
val	文字列型	要素の値。
next	Node	リストの次の文字を保持するインスタンスの参照。初期状態は未定義である。
prev	Node	リストの前の文字を保持するインスタンスの参照。初期状態は未定義である。

プログラム

```

1  ○文字型: searchBack(Node: tail, 文字列型: key)
2    Node: curr ← a
3    while (curr ≠ 未定義)
4      if (curr.val = key)
5        return curr.val
6      endif
7      curr ← curr. b
8    endwhile
9    return 未定義

```

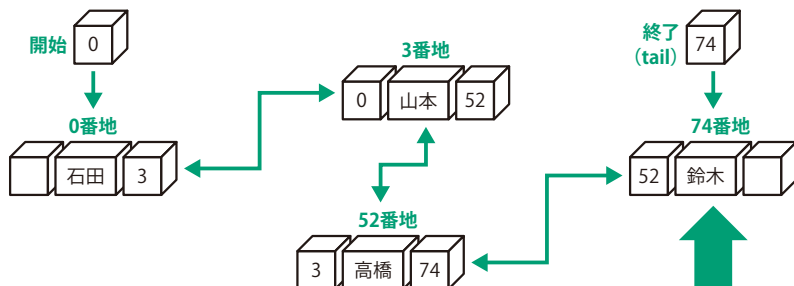
解答群

	a	b
ア	tail	val
イ	tail	next
ウ	tail	prev
エ	key	val
オ	key	next
カ	key	prev
キ	node	val
ク	node	next
ケ	node	prev

解説 双方向リストを逆順に探索します。仮に次のような双方向リストの場合、探索の開始は末尾の要素である「鈴木」からです。



解説動画



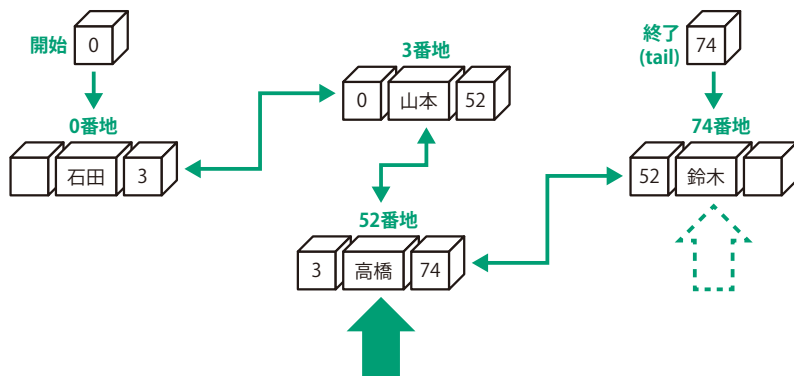
手続searchBackを実行する際、引数として末尾の要素と探索したい文字列が与えられます。そのため、Node型の変数tailに末尾の要素、文字列型の変数keyに探索したい文字列が入ります。上記の双方向リストの場合、tailには74番地の要素が入ります。

問題文を踏まえると、プログラムの2行目でインスタンスcurrに、これから探索するインスタンスを代入するものと思われます。4行目の「if (curr.val = key)」で探したい値と比較しており、もし比較結果が同じであれば、要素の値を5行目で戻り値にしているためです。

まずはtailから探索したいため、2行目でcurrにtailを代入します。そのため、**a**はtailです。

```
2 Node: curr ← tail
```

3～8行目は繰り返し処理です。currのvalが、探したい値であるkeyと同じであれば、currのvalを戻り値とします。同じではない場合、次の繰り返しで1つ前の要素を調べられるようにする必要があります。



上の図では52番地に移動しています。つまり、現在のcurrを前の要素で更新している状態です。「リストの前の文字を保持するインスタンスの参照」はメンバ変数prevに入っているため、currにcurr.prevを代入することで、curr自身を前の要素に更新できます。したがって、**b**はprevが入り、正解は**ウ**であることがわかります。

```
2 curr ← curr.prev
```

正解 **ウ**

例題③

R4/4月サンプル問題 - 問3

次のプログラム中の `a` , `b` に入れる正しい答えの組合せを, 解答群の中から選べ。

手続appendは, 引数で与えられた文字を単方向リストに追加する手続である。単方向リストの各要素は, クラスListElementを用いて表現する。クラスListElementの説明を図に示す。ListElement型の変数はクラスListElementのインスタンスの参照を格納するものとする。大域変数listHeadは, 単方向リストの先頭の要素の参照を格納する。リストが空のときは, listHeadは未定義である。

図 クラスListElementの説明

メンバ変数	型	説明
val	文字列型	リストの要素の値。
next	ListElement	リストの次の文字を保持するインスタンスの参照。初期状態は未定義である。

コンストラクタ	説明
ListElement(文字型: qVal)	引数 qVal でメンバ変数 val を初期化する。

プログラム

```

1  大域: ListElement: listHead ← 未定義の値
2  ○append(文字型: qVal)
3      ListElement: prev, curr
4      curr ← ListElement(qVal)
5      if (listHeadが a)
6          listHead ← curr
7      else
8          prev ← listHead
9          while (prev.nextが未定義でない)
10             prev ← prev.next
11         endwhile
12         prev.next ← b
13     endif

```

解答群

	a	b
ア	未定義	curr
イ	未定義	curr.next
ウ	未定義	listHead
エ	未定義でない	curr
オ	未定義でない	curr.next
カ	未定義でない	listHead

解説 プログラムを読み解くには、まず特徴的な変数や処理に注目し、そこから役割を推測するのが有効です。この例題の場合は、大域変数listHeadが特徴的であると感じるはずです。listHeadは問題文から、リストの先頭(head)のデータを示していることがわかります。



解説動画

listHeadが使われているのは、5、6、8行目です。5行目でlistHeadが「未定義」か（解答群のア、イ、ウ）、または「未定義でない」場合（解答群のエ、オ、カ）に、6行目でlistHeadにcurrを代入します。

currはクラスListElementのインスタンスで、4行目でqValを引数にしてコンストラクタを呼び出して生成されます。つまり、追加しようとしているデータそのものです。このcurrを6行目でlistHeadに代入することで、「追加するデータを先頭にする」という意味になります。

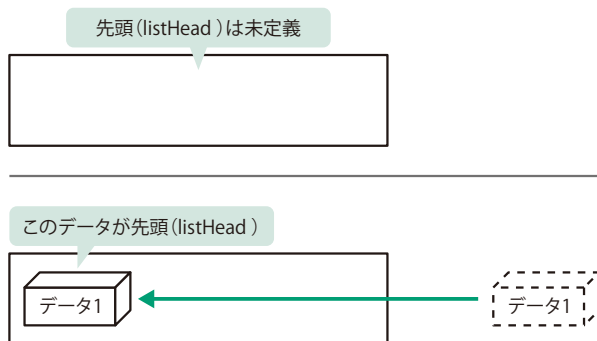
●最初のデータを追加

listHeadが未定義の状態です。データ1を追加するとします。



最初に入れるデータなので、データ1は先頭になります。最初のデータであれば6行目でlistHeadに代入し、先頭の要素にする必要があることから、5行目は「これは最初のデータか？」という判断条件になるはずです。解答群を見ると、**a**は「未定義」または「未定義ではない」のどちらかです。もしこれから入れるデータが最初のデータなのであれば、listHeadは「未定義」の状態です。したがって、**a**は「未定義」です。

5 if (listHeadが未定義)



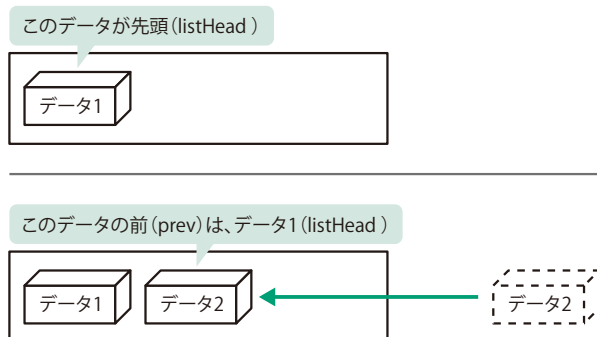
これで最初のデータを追加した場合の処理が完成です。

● 2 番目のデータを追加

続いては、listHeadに値が入っている（未定義ではない）状態で、データを追加する場合の処理です。listHeadには先頭のデータが入っており、5 行目の if 文の条件を満たさないため、8 行目に移動します。

8	prev ← listHead
9	while (prev.nextが未定義でない)
10	prev ← prev.next
11	endwhile
12	prev.next ← b

8 行目では、最初のデータ（listHead）を prev に代入しています。prev はおそらく「前」を意味する previous の略です。このように、変数名からある程度その用途を推測することができます。



追加しようとしている2番目のデータ（データ2）は、4行目でcurrに代入しています。つまり、現状は次のような状態です。

- prev: 前のデータ（最初のデータであるデータ1）
- curr: いま追加しようとしている2番目のデータ（データ2）

9行目で「前のデータのnextは未定義であるか？」と問われています。メンバ変数nextに値が入るのは、12行目を実行した場合だけなので、この時点では未定義です。したがって、9～11行目の繰り返し処理は1度も行われずに、12行目に移動します。

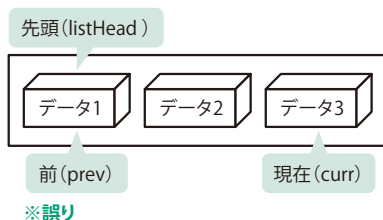
12行目では「前のデータのnext」にある値が代入されます。この時点では、「最初のデータの次」にあたるもの、つまり追加しようとしている「2番目のデータ」です。この2番目のデータはcurrに格納されています。よって、**b**に入るのはcurrです。

```
12      prev.next ← curr
```

したがって、正解は**A**です。今回の例題は、変数名をもとに役割を推測しました。もしそれができなければ、もう少し苦労したはずです。サンプル問題を使って、変数名とその役割を整理しておくといよいでしょう^{※3}。

● 3番目のデータを追加

知識を深めるために、3番目のデータも追加してみます。4行目で3番目のデータがcurrに入り、8行目でprevにlistHeadが入ります。「3番目のデータ」の前は「先頭データ」となるため、このままでは正しい状態ではありません。

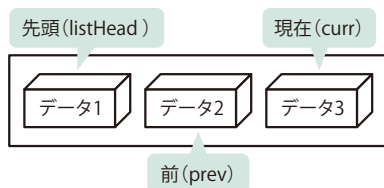


その誤りを9～11行目の繰り返し処理で正していきます。

```
9      while (prev.nextが未定義でない)
10      prev ← prev.next
11      endwhile
```

※3 1章P.21に、よく使われる変数名とその意味を表にまとめてあるので、ぜひ参考にしてください。

10行目でprevにprev.nextを入れています。前の図のようにprevはデータ1ですので10行目の処理でこれが上書きされることになります。「prev」は先頭のデータですから、「prev.next」は先頭の次のデータ、つまり2番目のデータです。この2番目のデータをprevに入れています。



これにより、これから追加しようとしている「3番目のデータの前のデータ」が2番目のデータとなり、正しい状態になりました。データ2 (prev) のnextは未定義であるため、繰り返し処理を終了して12行目に移ります。12行目では、prev.nextに現在のデータであるデータ3 (curr) を入れています。これで「データ2の次がデータ3」である状態が完成しました。

正解 ア



02

キューとスタック



キューとスタックがどのような場面で役立つのかを学ぼう

- 先入先出法とも言われるキューの特徴を学ぼう
- 後入先出法とも言われるスタックの特徴を学ぼう
- キューとスタックを擬似言語で活用する方法を学ぼう

キューとスタック

キューとスタックは、データを扱う順番を決める仕組みです。キューは並んだ順に処理していく仕組みですので、コンビニのレジに並んだ行列をイメージしてください。スタックはその逆で、先に並んだ人は後回しにされますので、積まれた皿を上から洗うところをイメージしましょう※4。

● キュー

キューとスタックでは、おそらくキューのほうがイメージしやすいと思います。コンビニの行列は、並んだ順に処理されます。プリンタ印刷も、印刷を開始した順に印刷されます。到着した順に処理をする仕組みが**キュー**です。



※4 キュー (queue) は「列 (順番に並ぶ列)」、スタック (stack) は「積む・積み重ねる」という意味の英単語が語源です。

連結リストと同様に、JavaやPythonなどのプログラミング言語には最初からキューの仕組みが備わっていますが、擬似言語の場合にはキューを扱うクラスが用意され、それを手続や関数で利用します。

クラスQueueの説明

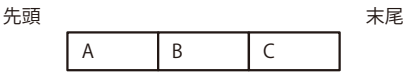
コンストラクタ		説明
Queue()		空のキューを生成する。

メソッド	戻り値	説明
enqueue(文字列型: s)	なし	キューに、文字列sを要素として追加する。
dequeue()	文字列型	キューから最初に追加された要素を取り出して返す。

クラスQueueが定義されている前提でプログラムを記述します。

```
1 Queue: queue ← Queue()
2 queue.enqueue("A")
3 queue.enqueue("B")
4 queue.enqueue("C")
```

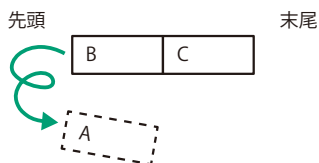
1行目でインスタンスを生成しています。2行目でAを行列に並べせます。3行目と4行目の処理でも同様に、BとCを行列に並べせます。



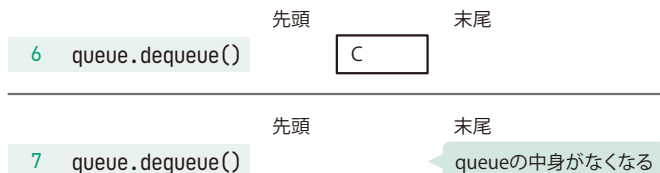
クラスの説明を見ると、キュー(変数queue)から要素を取り出すにはメソッドdequeueを使うようです。

```
5 queue.dequeue()
```

説明欄に「要素を取り出す」とあります。取り出すと、その要素はqueueから削除されます。



この状態でさらにメソッド `dequeue` を実行すると「B」が取り出され、さらに実行すると「C」が取り出されるため、`queue` には何もなくなります。



● スタック

スタックは積まれた皿を上から洗うイメージです。**スタック**では後に到着した要素から順に取り出します^{※5}。



キューと同様に、擬似言語ではスタックを実現するためのクラスが用意されています。スタックの場合には追加をメソッド `push`、取り出しをメソッド `pop` で行うことが多いはずです。

クラス `Stack` の説明

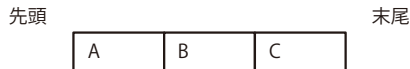
コンストラクタ	説明
<code>Stack()</code>	空のスタックを生成する。

メソッド	戻り値	説明
<code>push(文字列型:s)</code>	なし	スタックに、文字列 <code>s</code> を要素として追加する。
<code>pop()</code>	文字列型	スタックから最後に追加された要素を取り出して返す。

クラス Stack が定義されている前提でプログラムを記述します。

```
1 Stack: stack ← Stack()
2 stack.push("A")
3 stack.push("B")
4 stack.push("C")
```

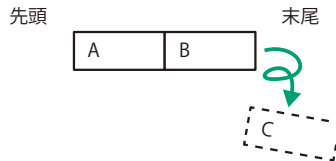
このプログラムを実行すると、A→B→Cの順番でスタック(変数stack)に要素が追加されます。



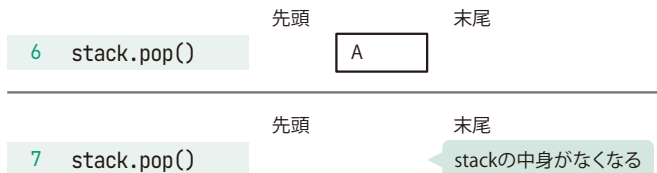
さらにpopで要素を取り出します。

```
5 stack.pop()
```

最後に追加されたのはCですので、これを取り出します。



このあとにpopを2回実行すると、stackは空になります。



※5 ブラウザでWebページを見ると履歴がどんどん積み上がっていきますが、「戻る」を押すと最後に見たWebページが表示されます。これもスタックの仕組みです。

例題にチャレンジ

例題①

R4/12月サンプル問題-問8

次の記述中の に入れる正しい答えを、解答群の中から選べ。

優先度付きキューを操作するプログラムである。優先度付きキューとは扱う要素に優先度を付けたキューであり、要素を取り出す際には優先度の高いものから順番に取り出される。クラス `PrioQueue` は優先度付きキューを表すクラスである。クラス `PrioQueue` の説明を図に示す。ここで、優先度は整数型の値 1, 2, 3 のいずれかであり、小さい値ほど優先度が高いものとする。

手続 `prioSched` を呼び出したとき、出力は の順となる。

クラス `PrioQueue` の説明

コンストラクタ	説明
<code>PrioQueue()</code>	空の優先度付きキューを生成する。

メソッド	戻り値	説明
<code>enqueue(文字列型: s, 整数型: prio)</code>	なし	優先度付きキューに文字列 <code>s</code> を要素として、優先度 <code>prio</code> で追加する。
<code>dequeue()</code>	文字列型	優先度付きキューからキュー内で最も優先度の高い要素を取り出して返す。最も優先度の高い要素が複数あるときは、そのうち最初に追加された要素を一つ取り出して返す。
<code>size()</code>	整数型	優先度付きキューに格納されている要素の数を返す。

プログラム

```

1  OprioSched()
2  PrioQueue: prioQueue ← PrioQueue()
3  prioQueue.enqueue("A", 1)
4  prioQueue.enqueue("B", 2)
5  prioQueue.enqueue("C", 2)
6  prioQueue.enqueue("D", 3)
7  prioQueue.dequeue() /* 戻り値は使用しない */
8  prioQueue.dequeue() /* 戻り値は使用しない */
9  prioQueue.enqueue("D", 3)
10 prioQueue.enqueue("B", 2)
11 prioQueue.dequeue() /* 戻り値は使用しない */
12 prioQueue.dequeue() /* 戻り値は使用しない */
13 prioQueue.enqueue("C", 2)
14 prioQueue.enqueue("A", 1)
15 while (prioQueue.size()が0と等しくない)
16     prioQueue.dequeue()の戻り値を出力
17 endwhile

```

解答群

- ア "A", "B", "C", "D"
- イ "A", "B", "D", "C"
- ウ "A", "C", "C", "D"
- エ "A", "C", "D", "D"

解説 通常のキューよりも少しだけ複雑な「優先度付きキュー」がテーマです。キューとの違いはメソッド dequeue の説明欄にある通りです。

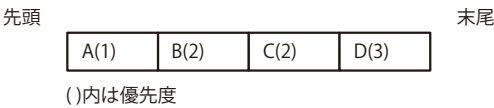


解説動画

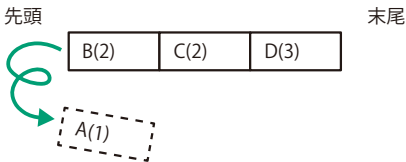
- (1) 取り出しは、優先度が最も高いものとする（ただし値の低いものが優先度が高い）
- (2) 優先度が同じである要素が複数ある場合には、早く到着したものとする

つまり、P.170～172で学んだキューは、(2)の性質です。今回の仕組みでは(2)の「到着順」よりも先に(1)の「優先度」を比較し、もっとも優先度が高い要素を取り出します。注意しなければならないのは、値が低いほど優先度が高いということです。「1」がもっとも優先度が高く、「4」がもっとも優先度が低いことになります。以上を踏まえて、処理の流れを追っていきましょう。

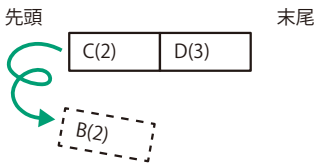
2行目ではキューを扱うクラスのインスタンスを生成し、3行目～6行目でそのキューに要素を追加しています。



7行目でメソッド dequeue を実行し、要素を取り出します。「参照」は要素を見ることですが、「取り出し」はその要素を削除します。取り出す要素は優先度がもっとも高いもの（優先度の値がもっとも低いもの）なので、Aが該当します。



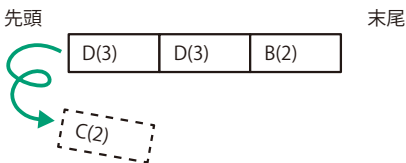
さらに8行目で取り出しを行います。もっとも優先度が高いのは2です。複数あるため、先に追加した要素であるBを取り出します。



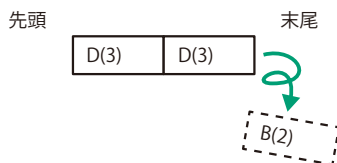
9行目と10行目で要素を追加します。



11行目で要素を取り出します。優先度がもっとも高い（優先度の値がもっとも低い）要素はCとBの2つです。先に追加したのはCであるため、これを取り出します。



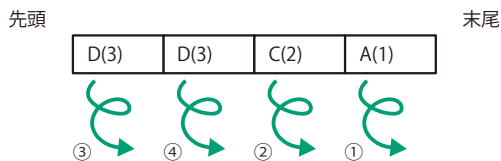
さらに12行目で要素を取り出しますが、優先度がもっとも高いのはBです。



13、14行目で要素を追加します。



そして最後に、15行目から17行目の繰り返し処理で、4回取り出しを行います。



A→C→D→Dの順番で取り出されるため、正解はエです。

正解 エ

例題②

オリジナル問題

次の記述中の に入れる正しい答えを、解答群の中から選べ。

キューとスタックを操作するプログラムである。クラス Queue はキューを表すクラスであり、クラス Stack はスタックを表すクラスである。クラスの説明を図に示す。

手続 proc を呼び出したとき、出力は となる。

図 1 クラス Queue の説明

コンストラクタ	説明
Queue()	空のキューを生成する。

メソッド	戻り値	説明
enq(文字列型: s)	なし	キューに、文字列 s を要素として追加する。
deq()	文字列型	キューから最初に追加された要素を取り出して返す。

図 2 クラス Stack の説明

コンストラクタ	説明
Stack()	空のスタックを生成する。

メソッド	戻り値	説明
push(文字列型: s)	なし	スタックに、文字列 s を要素として追加する。
pop()	文字列型	スタックから最後に追加された要素を取り出して返す。

プログラム

```

1  Oproc()
2  Queue: queue ← Queue()
3  Stack: stack ← Stack()
4  stack.push("a")
5  stack.push("b")
6  queue.enq(stack.pop())
7  queue.enq("c")
8  stack.push("d")
9  stack.push(queue.deq())
10  整数型: x ← stack.pop()
11  xを出力

```

解答群

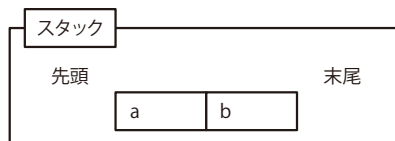
- ア a
 イ b
 ウ c
 エ d

解説 この例題のキューとスタックは、P.170～173の説明と同じ処理を行います。処理の流れを追っていきましょう。

手続procが呼び出されると、キューとスタックを生成し、2行目でインスタンスqueue、3行目でインスタンスstackを生成します。続く4行目と5行目でインスタンス変数stackに要素を追加しています。



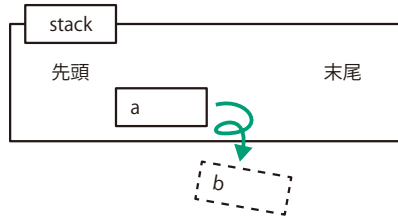
解説動画



6行目ではインスタンス変数queueに要素を追加しています。追加する要素を引数に記述しますが、6行目では引数が「stack.pop()」となっています。

```
6  queue.enq(stack.pop())
```

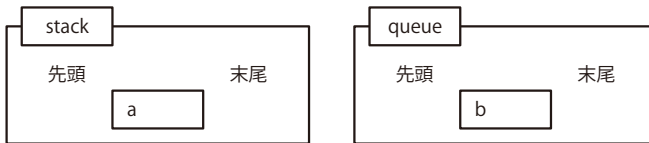
stack.pop()によって、スタックに最後に追加した要素が取り出されます。



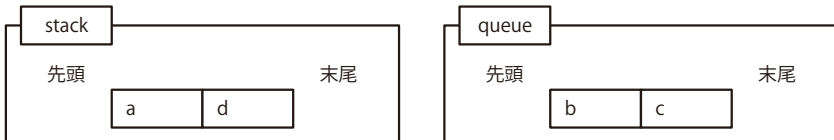
この取り出した値をqueueに追加しますので、6行目は次のように解釈することができます。

6 `queue.enq("b")`

つまり、queueに「b」が追加されます。

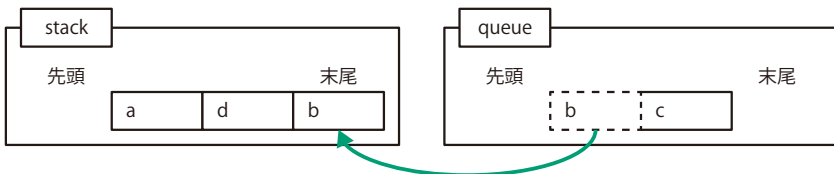


7行目でqueueに「c」を追加し、8行目でstackに「d」を追加しています。

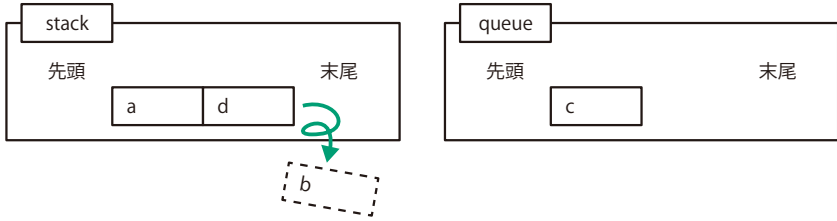


9行目では、queueから値を取り出して、その値をstackに追加しています。

9 `stack.push(queue.deq())`



最後に10行目でスタックから取り出して、11行目でその値を出力しています。



出力される値は「b」なので、正解は**イ**です。

正解 **イ**



03

木構造

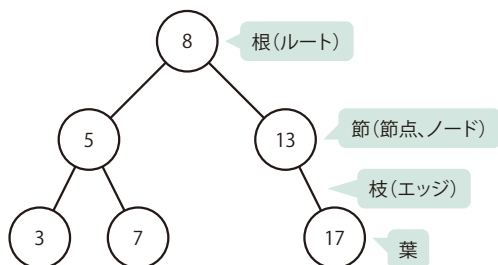


木構造を使った効率的な探索方法について理解を深めよう

- 木構造で格納されたデータを探索する方法を学ぼう
- 二分木の特徴と二分木を構成する方法を学ぼう

木構造とは

次のようなデータ構造を**木構造**といいます。



本書を逆さまに持ってみてください。「8」が根となる木のような形になっていることがわかんと思います。「8」「5」「13」などのデータを**節**や**節点**または**ノード**といいます。特に最上位の節を**根**または**ルート**、最下層の節を**葉**といいます。また節間をつなぐ線を**枝**や**エッジ**といいます。

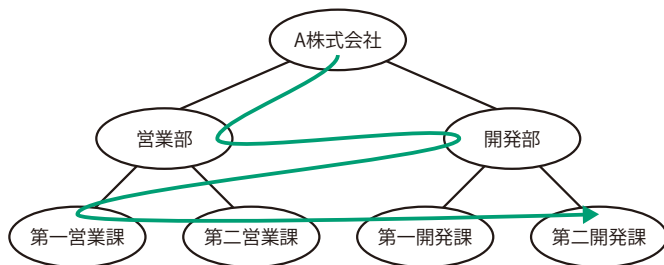
なお、木構造のデータには親子関係があり、8の子は5と13、5の子は3と7です。逆に5の親は8で、3と7の親は5にあたります。

探索順

木構造でデータを格納した場合、そこからデータを探す場面があります。例えば、部署を木構造で管理した場合は「営業課はどこに属しているのか」などと探すことがあるでしょう。無作為に探すのは非効率なので、必然的にいくつかの方法に大別されることになります。

● 幅優先探索

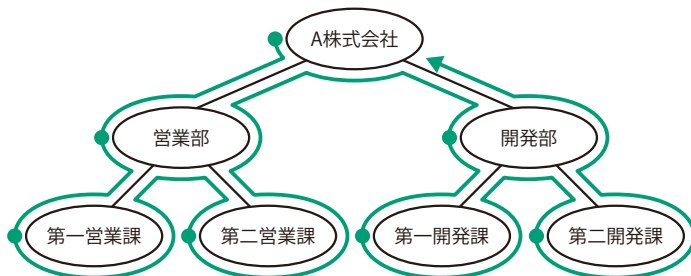
まずは**幅優先探索**です。これは次の順番でデータを探す方法です。



「幅」というより「横」と考えたほうがわかりやすいかもしれません。親子関係は無視をして、とにかく横に探していく方法です。

● 深さ優先探索

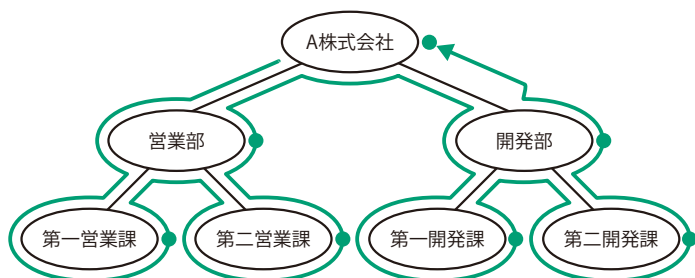
深さ優先探索は次のような順番でデータを探します。



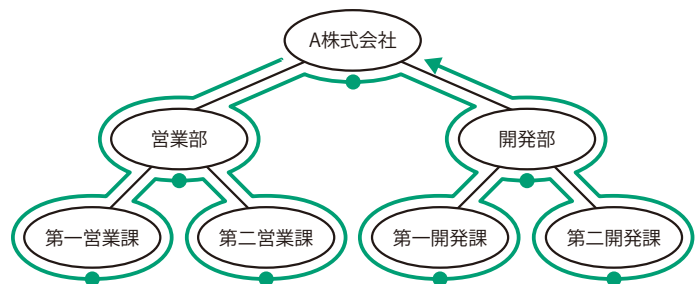
まず反時計回りに線を引きします。線が節の左側を通ったときにその節を探索する（調べる）流れです。

この探索の流れは、深さ優先探索のなかでも**先行順（行きがけ順）**という探索方法です。節に到着したタイミングで、すぐにその節を探索します。反時計回りに線を引いて、節の左に丸を付けてもよいでしょう。

後行順（帰りがけ順）は、複数回節に到着した場合の最後にその節を探索します。例えば「営業部」は3度到着していますが、3度目に探索します。1度目や2度目は何もしません。反時計回りに線を引いて、節の右に丸を付けてもよいでしょう。



中間順（通りがけ順）では複数回到着するうちの真ん中の到着で探索します。反時計回りに線を引いて、節の下に丸を付けてもよいでしょう。



ただし、この探索は1つずつ節を探索するため非効率な場合があります。そこで、格納場所や構造にいくつか条件を付けることで探索に適した形にすることができます。その代表的なものに二分木があります。

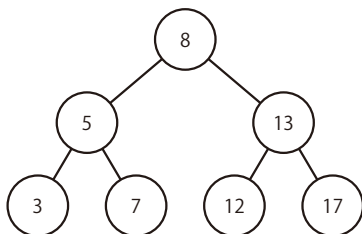
二分木

すべての節において、子の数が2つ以下に制限されている木を**二分木**といいます。

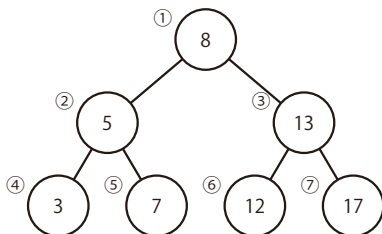
● 完全二分木

二分木のうち、次の条件を満たしている木を**完全二分木**といいます。

- 条件1：葉以外の節には子が必ず2つある
- 条件2：根から葉の深さがすべて同じ



完全二分木は、配列との相性がよい構造です。上図の木構造に番号を付けてみます。



①の左の子の番号は②で、右の子の番号は③です。②の左の子の番号は④で、右の子の番号は⑤です。これらの番号には、親の番号が「 i 」だとすると次の法則があります。

- 左の子の番号： $2i$
- 右の子の番号： $2i + 1$

これらの番号を「要素番号」「添え字」と解釈すると、この木構造をそのまま配列に置き換えることが可能です。

要素番号	①	②	③	④	⑤	⑥	⑦
値	8	5	13	3	7	12	17

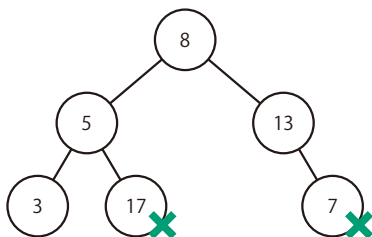
つまり完全二分木の構造だと、シンプルなプログラムで配列に格納することができます。

● 二分探索木

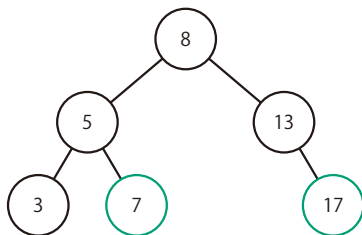
二分木のうち、次の条件を満たす二分木のことを**二分探索木**といいます。これは探索効率を上げるために、値にルールを設けた木構造です。

- 条件1：左の子孫は自分よりも小さい
- 条件2：右の子孫は自分よりも大きい

次の図は条件を満たしていないため、二分探索木ではありません。



「17」は、その親である「5」よりも大きいため条件を満たしています。しかし、祖父にあたる「8」よりも大きいため、条件を満たしません。二分探索木では、「8」の左にある節はすべて「8」より小さい状態にする必要があります。また「7」は、その親である「13」より小さいため条件を満たしていません。二分探索木にするためには、次のように訂正します。



決められたルールに従っているため、探索がしやすくなります。例えば「13」を見つけない場合に、「8」の左は探索する必要はありません。「8」の右だけを探索すればよく、これは探索効率がよいことを意味しています。ただし、「13」の子は1つしかいないため、根から葉までの高さがすべて同じとはならず、完全二分木ではありません。そのため配列に格納するには、少しプログラムが複雑になります。

例題にチャレンジ

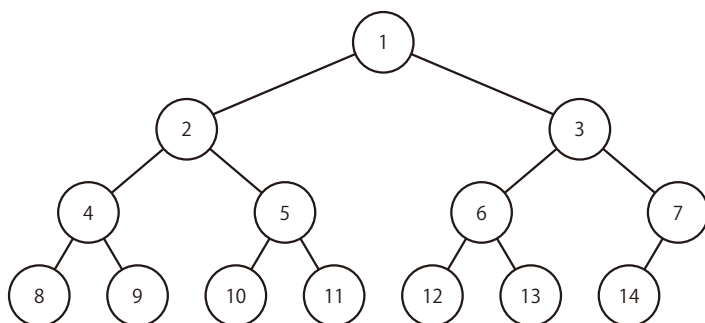
例題①

オリジナル問題

次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

手続mainは、図の二分木の、引数で指定したノードを根とする部分木をたどりながら、全てのノードの値を合計する。大域の配列treeが図の二分木を表している。

図 二分木



注記 ○の中の値は節番号である。

クラスNodeはノードを定義している。クラスNodeのメンバ変数の説明を表に示す。

表 クラスNodeのメンバ変数の説明

メンバ変数	型	説明
value	整数型	ノードの値。
right	Node	ノードの右の子ノード。
left	Node	ノードの左の子ノード。

手続mainを実行すると出力結果は105となる。ノードの値の合算にあたり、ノードは で探索している。

プログラム

```
1  大域: Nodeの配列: tree
2
3  ○整数型: sumTree(Node: node)
4    整数型: total ← node.value
5    if (node.left ≠ 未定義)
6      total ← total + sumTree(node.left)
7    endif
8    if (node.right ≠ 未定義)
9      total ← total + sumTree(node.right)
10   endif
11   return total
12
13  ○main()
14    tree ← {14個の未定義の値}
15    for (iが1から14まで1ずつ増加)
16      tree[i] ← Node()
17      tree[i].value ← i
18    endfor
19    tree[1].right ← tree[3]
20    tree[1].left  ← tree[2]
21    tree[2].right ← tree[5]
22    tree[2].left  ← tree[4]
23    tree[3].right ← tree[7]
24    tree[3].left  ← tree[6]
25    tree[4].right ← tree[9]
26    tree[4].left  ← tree[8]
27    tree[5].right ← tree[11]
28    tree[5].left  ← tree[10]
29    tree[6].right ← tree[13]
30    tree[6].left  ← tree[12]
31    tree[7].left  ← tree[14]
32    整数型: result ← sumTree(tree[1])
33    resultを出力
```

解答群

- ア 幅優先探索
- イ 深さ優先探索の先行順
- ウ 深さ優先探索の中間順
- エ 深さ優先探索の後行順

解説 1行目で大域変数として配列treeを宣言しています。「Nodeの配列」となっていることからわかる通り、treeはインスタンスNodeの配列です。また、木構造を配列で表現していることから、この木構造は完全二分木であることが推測できます。しかし、図の二分木を見ると「7」の右の子が存在しないため、完全二分木ではありません。そのため、このプログラムでは「右の子があるが、左の子はいない」「左の子があるが、右の子はいない」という場合を想定した、少し複雑なプログラムになっています。



解説動画

14行目で配列nodeに要素数が14個の配列を代入しています。15行目から18行目の繰り返しで、それぞれのノードに値を代入しています。さらに19行目から31行目で、左の子と右の子をそれぞれのノードに設定しています。これで問題文にある木構造が再現できました。配列treeの要素（インスタンスNode）は、次のような状態です。

value	right	left
1	tree[3]	tree[2]

value	right	left
2	tree[5]	tree[4]

value	right	left
3	tree[7]	tree[6]

value	right	left
4	tree[9]	tree[8]

value	right	left
5	tree[11]	tree[10]

value	right	left
6	tree[13]	tree[12]

value	right	left
7		tree[14]

value	right	left
8		

value	right	left
9		

value	right	left
10		

value	right	left
11		

value	right	left
12		

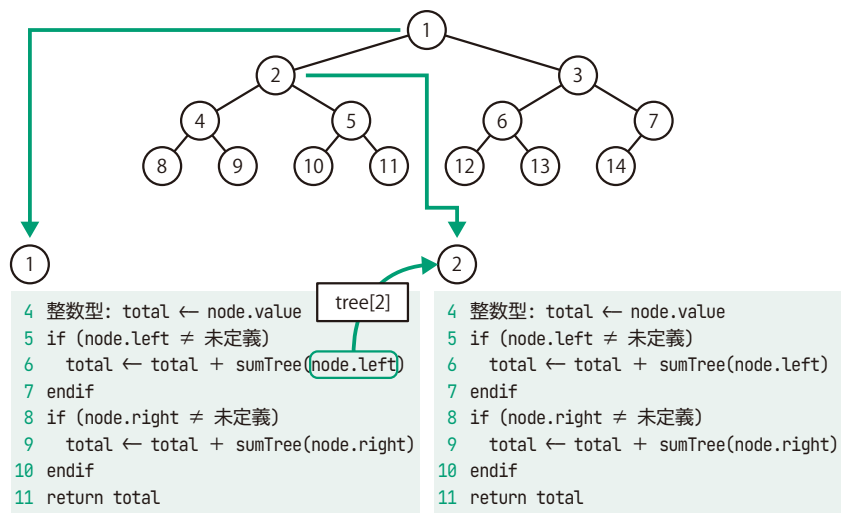
value	right	left
13		

value	right	left
14		

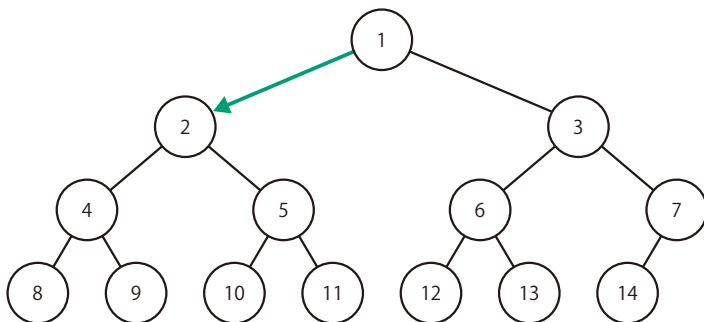
この状態で32行目の「sumTree(tree[1])」を実行します。引数であるtree[1]は根のノードです。

手続sumTreeでは、最初に4行目でtotalにnode.valueを代入します。node.valueは、tree[1].valueのことで、1が代入されます。

続いて5行目で、左のノードが未定義かを確認します。未定義であれば8行目に移動しますが、tree[1]のleftはtree[2]であり、未定義ではないため6行目を実行します。6行目では自分自身(手続sumTree)を呼び出しており、**再帰**(P.80参照)になっています。ここでは引数をtree[2]として、sumTreeを呼び出します。再帰は慣れるまでイメージが付きにくいので、次の図のように、要素ごとにプログラムが存在することをイメージしてください。説明上、ここでは「プログラム①」や「プログラム②」と表現します。



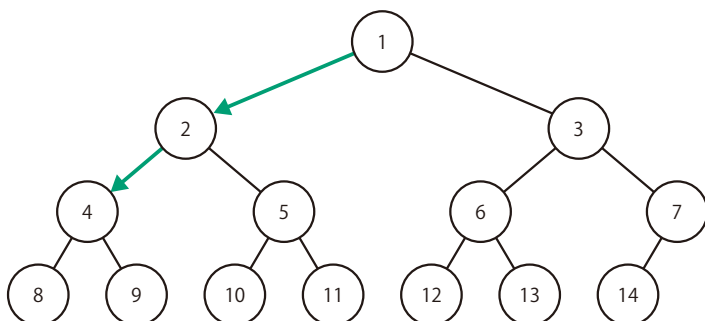
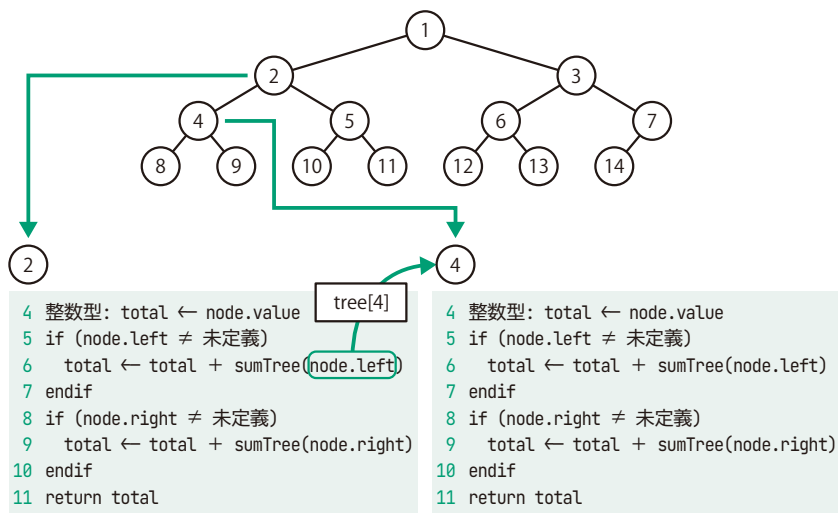
プログラム①からプログラム②を呼び出しています。これは、1→2の順にノードをたどったことを意味しています。



ただし、この問題ではノードをたどった順序が問われているわけではありません。問われているのは探索する方法であり、探索した順番からどのような探索方法であるのかを考えます。探索方法を考えるために、引き続き処理を追っていきましょう。

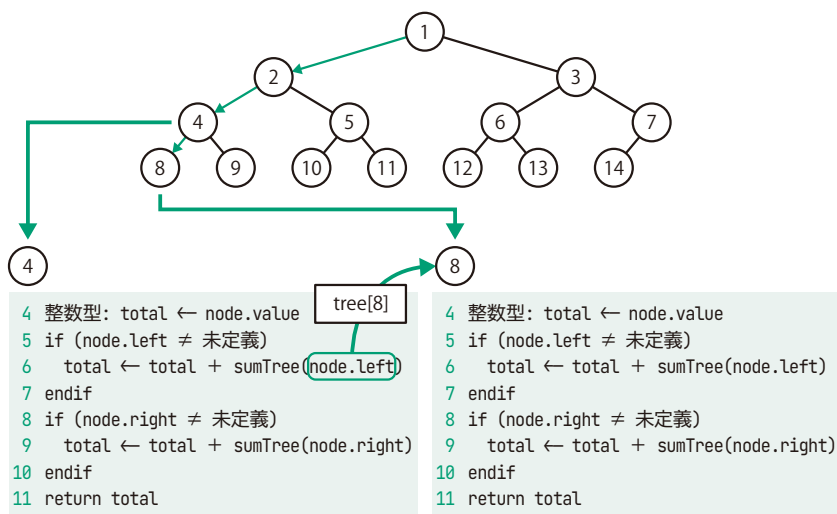
●プログラム②

totalに2を代入したあと、5行目でleftを確認します。未定義ではないため、tree[2].leftを引数としてsumTreeを呼びます。tree[2].leftはtree[4]です。これはプログラム②からプログラム④に移動したイメージになります。



●プログラム④

totalに4を代入したあと、leftは未定義ではないことから、プログラム⑧に進みます。

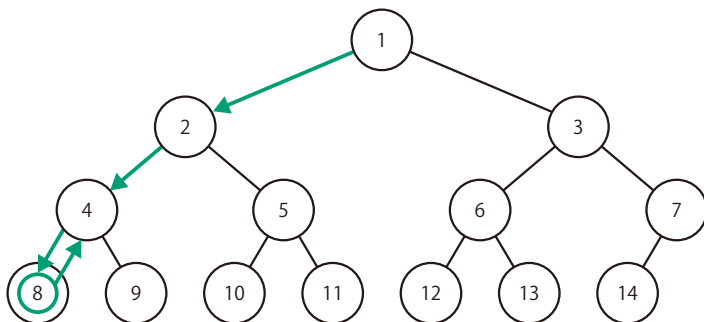


●プログラム⑧

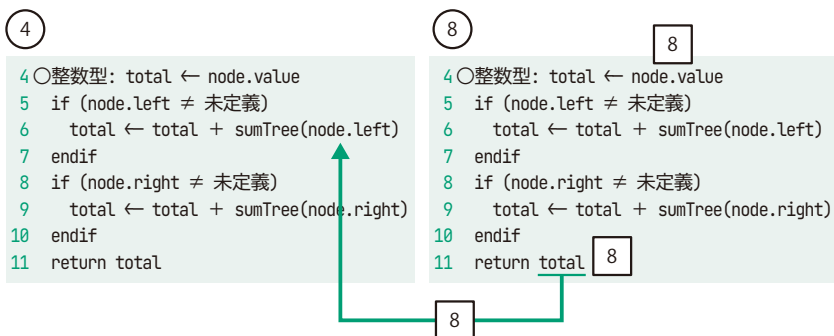
totalに8を代入したあと、左のノード (left) が未定義かを確認します。ここまでleftは、常に未定義ではなかったのですが、ここで初めて未定義となりました。これは左の子が存在しないことを意味しています。

leftが未定義であるため8行目に進みます。8行目ではrightが未定義であるかを判断しています。つまり右の子が存在するかどうかの判断です。rightは未定義であるため、11行目に進みtotalを戻り値とします。

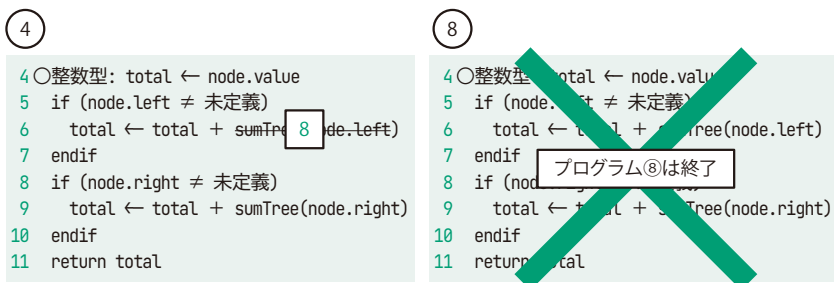
処理を開始してから要素の値を使ったのはこれが初めてです。使ったことを丸で表現しておきましょう。



totalは、3行目でtree[8].valueが代入されているため8を戻り値とします。



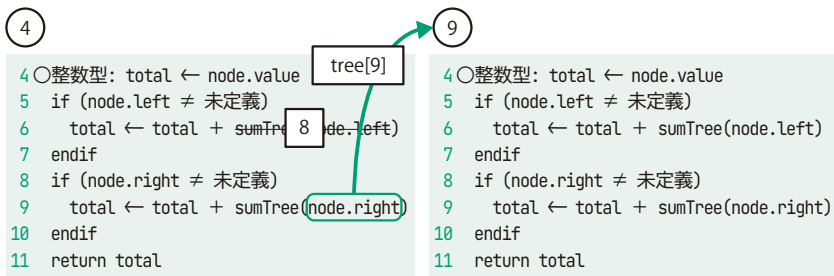
戻り値として得た値を、さらに呼び出し元に戻り値として返すことで、プログラム④の呼び出し部分が8に置換されるイメージです。



●プログラム④

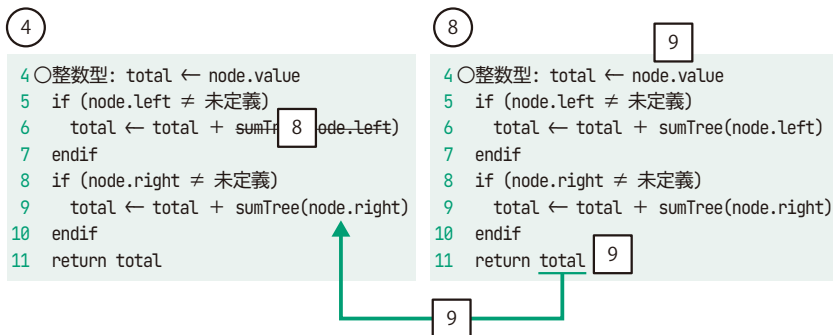
プログラム④に戻ってきました。totalは4行目で4が入っているため、6行目は「4 + 8」と解釈でき、totalはここで12となります。

次に8行目の判定を行います。rightは未定義ではないため、9行目に進みます。

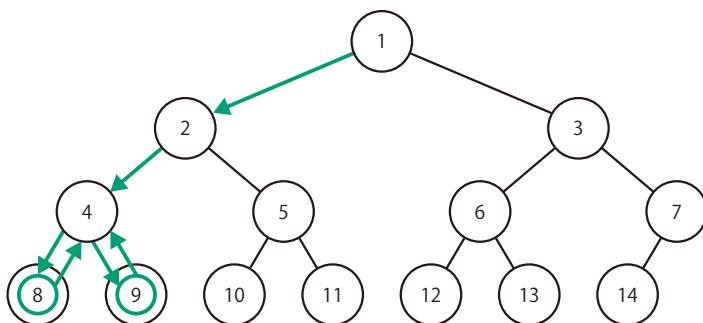


●プログラム⑨

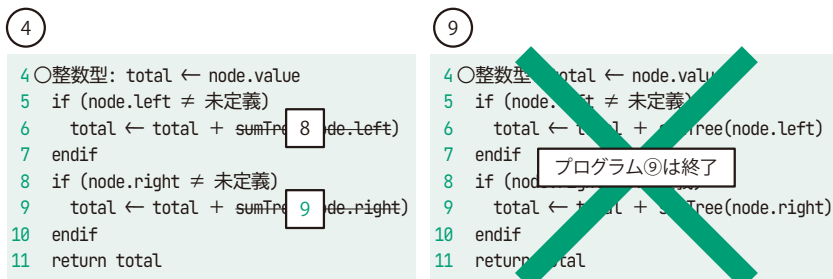
要素9は左右の子がともに未定義であるため、totalを戻り値とします。



戻り値にしたため要素9に丸を付けます。

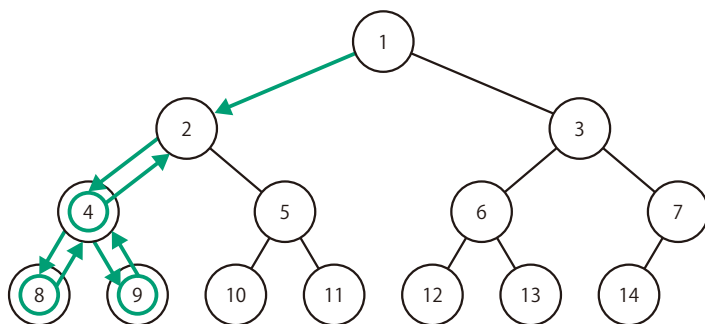


呼び出した箇所が9に置換されます。



●プログラム④

再びプログラム④に戻ってきました。totalは8行目で $12 + 9 = 21$ となり、11行目で戻り値としてtotalを返します。これでプログラム④から抜けるため、丸を付けます。



現時点でたどった順番は $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 4 \rightarrow 9 \rightarrow 4$ で、丸を付けた順番は $8 \rightarrow 9 \rightarrow 4$ です。「たどった順番」において「4」は3回登場していますが、戻り値を返したのは最後です。3回到着したうち、最後の3回目に値を参照していますので、後行順に該当します。したがって、正解は**エ**です。

正解 **エ**

例題②

R4/12月サンプル問題-問9

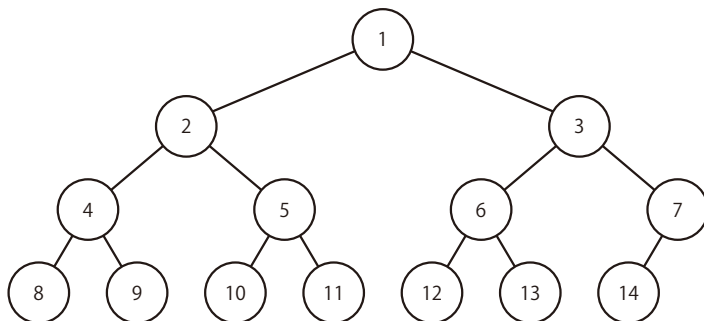
次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

手続orderは、図の二分木の、引数で指定した節を根とする部分木をたどりながら、全ての節番号を出力する。大域の配列 tree が図の二分木を表している。

配列treeの要素は、対応する節の子の節番号を、左の子、右の子の順に格納した配列である。例えば、配列treeの要素番号1の要素は、節番号1の子の節番号から成る配列であり、左の子の節番号2、右の子の節番号3を配列「{2, 3}」として格納する。

手続orderをorder(1)として呼び出すと、の順に出力される。

図 二分木



注記1 ○の中の値は節番号である。

注記2 子の節が一つの場合は、左の子の節とする。

プログラム

```
1  大域: 整数型配列の配列: tree ← {{2, 3}, {4, 5}, {6, 7}, {8, 9},
2                                {10, 11}, {12, 13}, {14}, {}, {}, {},
3                                {}, {}, {}, {}} // {}は要素数0の配列
4
5  Oorder(整数型: n)
6    if (tree[n]の要素数が2と等しい)
7      order(tree[n][1])
8      nを出力
9      order(tree[n][2])
10   elseif (tree[n]の要素数が1と等しい)
11     order(tree[n][1])
12     nを出力
13   else
14     nを出力
15   endif
```

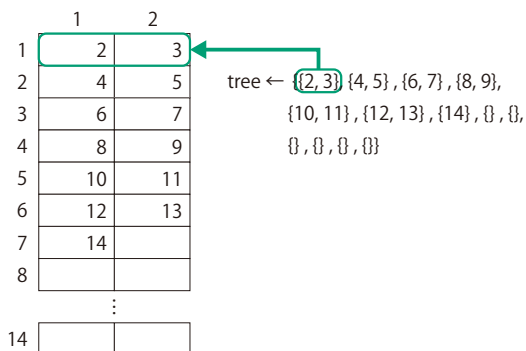
解答群

- ア 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14
- イ 1, 2, 4, 8, 9, 5, 10, 11, 3, 6, 12, 13, 7, 14
- ウ 8, 4, 9, 2, 10, 5, 11, 1, 12, 6, 13, 3, 14, 7
- エ 8, 9, 4, 10, 11, 5, 2, 12, 13, 6, 14, 7, 3, 1

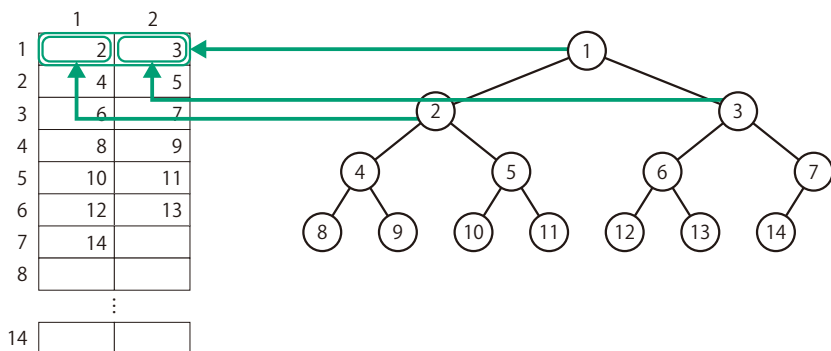
解説 問題文の正確な読解が必要になりますが、二分木の特性がわかっていれば比較的スムーズに理解することができるはずです。まず大域変数である二次元配列treeを、表形式にしてみましょう。



解説動画



問題文には「配列treeの要素番号1の要素は、節番号1の子の節番号から成る配列」とあります。言い回しが複雑ですが、これは次の状態を説明した文章です。



配列treeの要素番号1は、配列{2,3}が格納されています。これは二分木の節番号1の子が、節番号2と3であることを表しています。同様に、要素番号2の要素は配列{4, 5}です。要素番号8は節番号8に子がないことから要素数が0の状態です。つまり、配列treeが二分木の構造を表しているのです。これを踏まえて、プログラムを見ていきましょう。

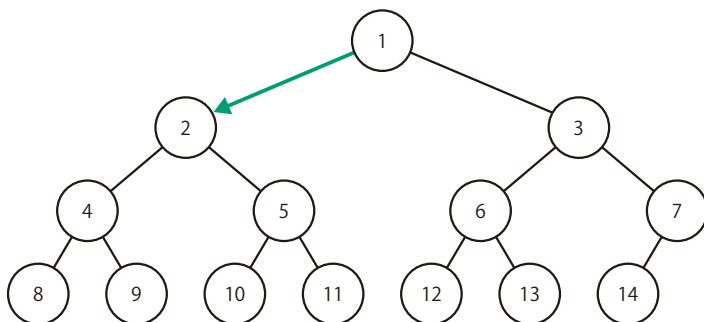
まずは手順orderをorder(1)として呼び出すことで、引数nには1が入ります。

6行目でtree[n]の要素数を調査し、2であれば7行目に移動します。異なる場合は、10行目に移動し、要素数が1であれば11行目に入ります。引数nが1の場合はどうでしょうか。tree[1]の要素数は2であるため、7行目に移動します。

tree[1][1]は2であるため、7行目は次のように解釈されます。

7 order(2)

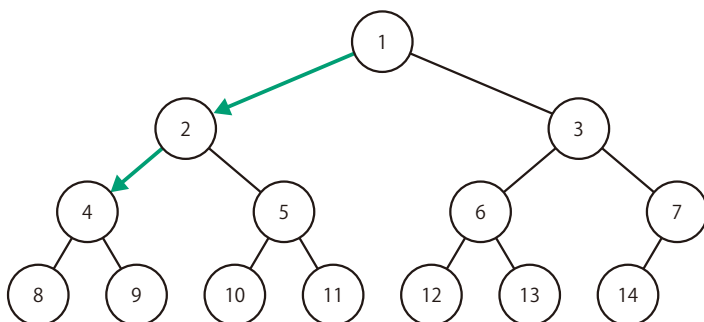
最初はorder(1)が呼び出され、次にorder(2)が呼び出されるため、この時点で節は1→2の順にたどっています。



同様に6行目でtree[n]の要素数を調査します。引数は2なので、nには2が入ります。そのため、tree[n]はtree[2]という意味になり、要素数は2であることがわかります。tree[2][1]は4であるため、7行目は次のように解釈されます。

7 order(4)

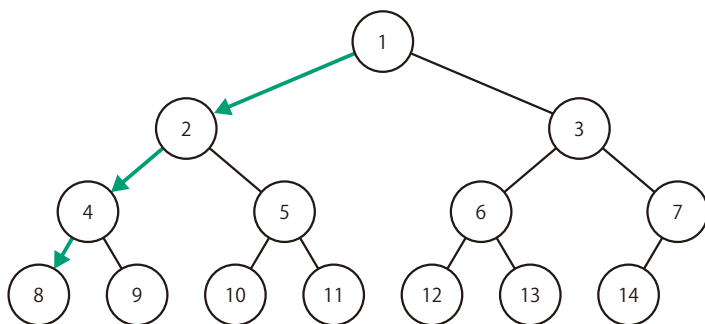
この時点では、節を1→2→4の順にたどっています。



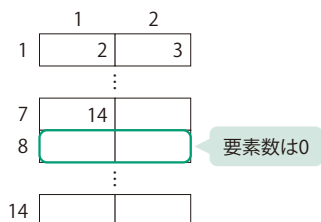
しかし、この段階ではまだ一度も出力が行われていません。節「1」、節「2」、節「4」のそれぞれで「要素数が2か？」と聞いただけです。要素数は「その節にぶら下がっている節の数」を表しているため「子は2つか？」と聞いているだけです。

さらに先に進みましょう。tree[4]の要素数(子の数)は2ですので、7行目に進みます。7行目はtree[4][1]を引数にして、orderを実行するため、次のように解釈されます。

7 order(8)

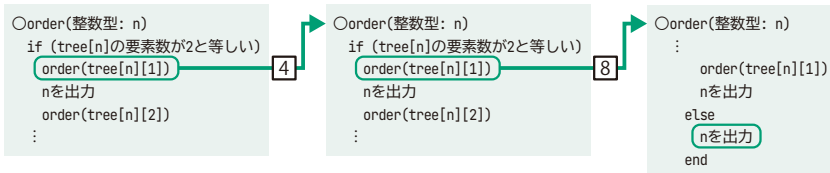


ここで、いままで繰り返してきた流れに変化が生まれます。初めて要素数(子の数)が2ではなくなるのです。



6行目の条件を満たさず、10行目の条件である「要素の数は1か?」も満たしません。そこで14行目の「nを出力」を実行し、初めて値が出力されます。nは8ですから、最初に出力されたのは「8」であり、選択肢はウとエのどちらかになります。次に出力されるのは、「4」と「9」のどちらでしょうか。それさえわかれば答えが得られます。

「8」を出力したプログラムは、関数の実行を終了します(下図の右端のプログラム)。そして呼び出し元に戻ります(下図の中央のプログラム)。



呼び出し先(上図の右端のプログラム)が終了するのを待っていた呼び出し元(上図の中央のプログラム)のプログラムは、次の行の「nを出力」を実行します。

```

Order(整数型: n)
if (tree[n]の要素数が2と等しい)
    order(tree[n][1])
    nを出力
    order(tree[n][2])
:

```

4

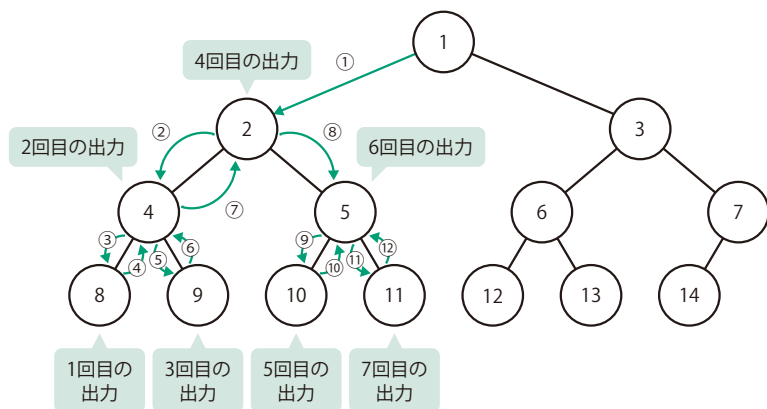
```

Order(整数型: n)
if (tree[n]の要素数が2と等しい)
    order(tree[n][1])
    nを出力
    order(tree[n][2])
:

```

中央のプログラムにおけるnは「4」であるため、「4」が出力されます。8→4の順で出力されたため、答えはウです。

なおトレースを続けていくと、次のよう出力されることになります(途中経過)。



先ほどの例題と同様に節4に注目してみます。3回到着したうちの2回目で出力していますので、探索順は深さ優先探索の中間順となります。

正解 ウ

Chapter

5

演習問題

本章の学習ポイント

- 多くの演習を行い、擬似言語に慣れる
- トレース表はできるだけ省略する
- 選択肢をできるだけ消去法で削り、正解の可能性を高める
- わからない問題は飛ばしてどんどん進む

演習問題にチャレンジ

問題①

オリジナル問題

次の記述中の に入れる正しい答えを、解答群の中から選べ。

手続 proc2 を呼び出すと、 の順に出力される。

プログラム

```
1  Oproc1(整数型: x)
2    "A"を出力する
3    proc3(x + 1)
4
5  Oproc2()
6    proc3(1)
7    "B"を出力する
8    proc1(2)
9
10 Oproc3(整数型: n)
11  "C"を出力する
12  if (n = 2)
13    "D"を出力する
14  endif
```

解答群

- ア A → C → D → B
- イ C → B → A → C
- ウ C → D → B → A
- エ C → B → C → A
- オ A → B → C → D
- カ C → A → C → B
- キ B → C → A → C
- ク C → A → D → B

解説 手続が呼び出される流れを見ていきましょう。まずはproc2からです。



解説動画

●手続proc2

proc2を呼び出すと、6行目でproc3(1)が呼び出されます。proc3を実行することになりますが、**proc2は6行目で一時停止**します。これは後にproc3の終了によって再開します。

●手続proc3

proc3では最初に「C」が出力されます。proc3における引数nは1なので、12行目のif文の条件「 $n = 2$ 」は成立せず、「D」は出力されません。そのままproc3は終了します。

●手続proc2 (再開)

proc3が終了したことで、proc2が再開します。7行目で「B」が出力され、8行目でproc1(2)が呼び出されます。ここで再びproc2は一時停止します。

●手続proc1

proc1ではまず「A」が出力されます。次に3行目でproc3($x + 1$)が呼び出されます。xは2なので、proc3(3)が呼び出されます。なお、現在proc2が一時停止中ですが、さらにproc1もここで一時停止します。

●手続proc3

11行目で再び「C」が出力されます。このときのnは3なので、やはりif文の条件($n = 2$)は成立せず、「D」は出力されません。proc3は終了します。

●手続proc1 (再開)

proc3の終了により制御が再開し、そのままproc1も終了します。

●手続proc2 (再開)

proc1の終了により制御が再開し、そのままproc2も終了します。

以上により、出力の順番はC→B→A→Cです。したがって、正解は**イ**です。



ある手続や関数を呼び出した際には、呼び出し元の制御はそこで一時停止していることを忘れないでください。この考え方は特に再帰で重要となります。

正解 **イ**

問題2

オリジナル問題

次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

関数 selectMax は、異なる三つの整数を引数で受け取り、そのうちの最大値を返す。

プログラム

```
1  ○整数型: isLarger(整数型: a, 整数型: b)
2    if (a > b)
3      return 1
4    else
5      return 0
6    endif
7
8  ○整数型: selectMax(整数型: x, 整数型: y, 整数型: z)
9    if (  )
10     return x
11   elseif (isLarger(y, z) = 1)
12     return y
13   else
14     return z
15   endif
```

解答群

- ア isLarger(x, y) = 1
- イ isLarger(x, y) = 1 and isLarger(x, z) = 1
- ウ x > y and y > z
- エ isLarger(x, z) = 1 or isLarger(y, z) = 1
- オ isLarger(z, x) = 1 and isLarger(z, y) = 1
- カ isLarger(x, y + z) = 1

解説 関数 `isLarger` は、`a` が `b` より大きいときには 1 を、それ以外の場合は 0 を返す関数です。この関数を使って、3 つの値のうち最大のものを返すのが関数 `selectMax` です。



解説動画

関数 `selectMax` の 10 行目で `x` を返していることから、 には「`x` が 3 つの中で最大のとき」に該当する条件が入ります。`x` が最大ということは、「`x > y` かつ `x > z`」を満たす状態です。`isLarger` を使って書くとこのようになります。

- `x > y` : `isLarger(x, y) = 1`
- `x > z` : `isLarger(x, z) = 1`

この両方の条件を満たすようにするには、両方を `and` でつなぎます。

```
9  if (isLarger(x, y) = 1 and isLarger(x, z) = 1)
```

したがって、正解は **イ** です。この条件を満たさない場合は、次に「`y > z`」の比較を行います。`isLarger(y, z) = 1` であれば `y` が返され、そうでなければ `z` が返されます。この判定の流れにより、3 つの値のうち最大のものを返す構造になっています。

正解 **イ**

問題③

オリジナル問題

次のプログラム中の `a` 及び `b` に入れる正しい答えを、解答群の中から選べ。

手続 `calcCoins` は、金額 `change` に対して、硬貨の枚数 (100 円玉, 10 円玉, 1 円玉) を計算するものである。ここで、`x mod y` は `x` を `y` で割った余りを返す。

プログラム

```
1  ○整数型: calcCoins(整数型: change)
2  整数型: c100, c10, c1
3  c100 ← change ÷ 100
4  change ← change mod 100
5  c10 ←  a
6  change ←  b
7  c1 ← change
8  return c100 + c10 + c1
```

解答群

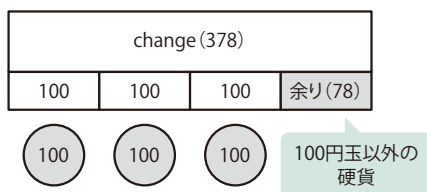
	a	b
ア	$\text{change} \div 10$	$\text{change} \bmod 10$
イ	$\text{change} \div 10$	$\text{change} \bmod 1$
ウ	$\text{change} \bmod 10$	$\text{change} \div 10$
エ	$\text{change} \bmod 10$	$\text{change} \bmod 10$
オ	$\text{change} \div 1$	$\text{change} \bmod 10$
カ	$\text{change} \div 10$	$\text{change} \bmod 100$

解説 変数名を踏まえると、c100には100円玉、c10には10円玉、c1には1円玉の枚数が入ると考えられます。



解説動画

3行目では「 $\text{c100} \leftarrow \text{change} \div 100$ 」でchangeに「100」がいくつ含まれるかを計算しており、100円玉の枚数を得ることができます。100円玉の枚数はこれで確定できるので、その余りを使うと10円玉と1円玉の枚数を求められます。その余りを求める計算は、4行目の「 $\text{change} \leftarrow \text{change} \bmod 100$ 」です。例えば、378を引数にして手続calcCoinsを呼び出した場合、 $378 \div 100$ の余りは78で、changeには78が入ります。



5行目の **a** では、changeから10円玉の枚数を計算します。100円玉の枚数計算と同様に、10で割った商が10円玉の枚数です。changeが78円の場合、10円玉は $78 \div 10 = 7$ 枚だとわかります。この計算に対応する式は「 $\text{change} \div 10$ 」です。選択肢は**ア**、**イ**、**カ**に絞られました。

6行目の **b** では、10円玉の枚数を得た余りの金額を求めます。changeが78円のとき、10円玉を7枚(70円)を使った余りは $78 \bmod 10 = 8$ です。この計算に対応する式は「 $\text{change} \bmod 10$ 」なので、正解は**ア**です。

正解 **ア**

問題④

オリジナル問題

次のプログラム中の `a` 及び `b` に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まるとする。

関数 findMaxIndex は、ランダムな整数が格納された配列 data[1]~data[10] の中で、最大の値を持つ要素の要素番号を返すものである。

プログラム

```
1  ○整数型: findMaxIndex()
2  整数型の配列: data /* 一様分布の乱数 (0~99) を格納 */
3  整数型: idx ← 1
4  整数型: max ← a
5  整数型: i
6  for (iを2から10まで1ずつ増やす)
7    if (b)
8      max ← data[i]
9      idx ← i
10   endif
11   endfor
12   return idx
```

解答群

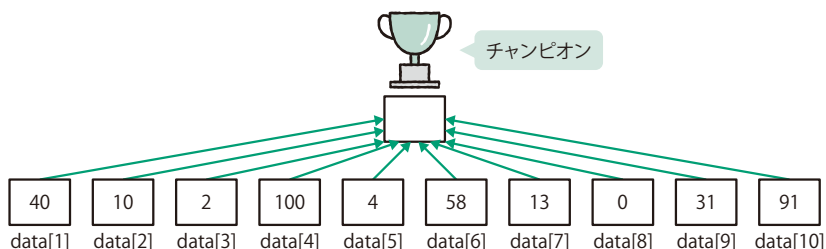
	a	b
ア	0	data[i] < max
イ	0	data[i] = max
ウ	0	data[i] > max
エ	data[idx]	data[i] < max
オ	data[idx]	data[i] = max
カ	data[idx]	data[i] > max
キ	data[idx + 1]	data[i] < max
ク	data[idx + 1]	data[i] = max
ケ	data[idx + 1]	data[i] > max

解説 2行目の配列dataには、0～99の一樣分布の乱数が格納されます。このように**コメントで格納内容が明示されることもあります**ので、記述されている内容にしてください。



解説動画

最大値を得るには、1人のチャンピオンと複数のチャレンジャを1つずつ比較する方法が一般的です。



●初代チャンピオンを決める方法①

初代チャンピオンを決める方法はいくつかありますが、そのうちの1つが変数を取り得る最小値を初代チャンピオンにする方法です。今回のプログラムでは、0～99の範囲の値がdataの要素になるため、最小値は0です。そのため、初代チャンピオンを0にしておきます。これが想定される **a** です（実際には誤りなので、後に説明します）。

4 整数型: max ← 0

そして7行目で「チャレンジャがチャンピオンよりも大きい」という比較を行い、8行目でチャンピオンを交代する（maxを更新する）というのが正しい処理であると考えられます。そのため、**b** には「配列dataのi番目の要素がmaxよりも大きい」という条件の「data[i] > max」を入れます。

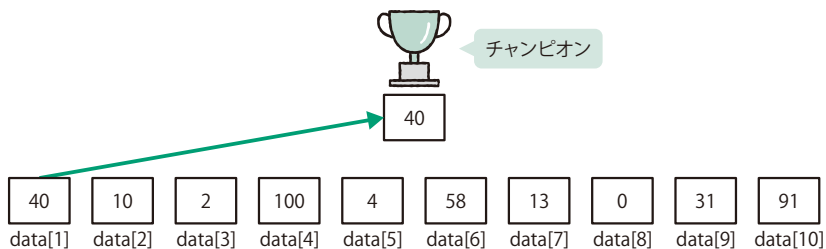
```
6 for (iを2から10まで1ずつ増やす)
7   if (data[i] > max)
8     max ← data[i]
```

maxの初期値は0ですから、1度目の繰り返し処理の比較で、必ず最初のチャレンジャがチャンピオンになります。

しかし、これは誤りです。6行目からの繰り返しは、「iを2から10まで1ずつ増やす」繰り返しになっているためです。つまり、**dataの要素番号が1の要素（最初のチャレンジャ）は、チャンピオンに挑戦しない状態になっています。**

●初代チャンピオンを決める方法②

そこで、初代チャンピオンを最小値の0にするのではなく、dataの要素番号1の要素を初代チャンピオンにしておきます。



この状態で繰り返し処理に入ると、dataの要素番号2 (2番目のチャレンジャ) から順にチャンピオンに挑戦することになり、正しく最大値が得られます。つまり4行目はこのようになります。

```
4  整数型: max ← data[1]
```

ただしdata[1]は解答群にないため、data[idx]を答えとします。変数idxは3行目で1が代入されるため、同様の値となります。

```
4  整数型: max ← data[idx]
```

したがって、aは「data[idx]」、bは「data[i] > max」の組み合わせである**力**が正解です。

正解 **力**

問題5

オリジナル問題

次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

関数 sumToN は、1 から n までの整数の和を再帰的に計算する関数である。

プログラム

```
1  ○整数型: sumToN(整数型: n)
2  整数型: result
3  if (n = 1)
4    result ← 1
5  else
6    result ← n + 
7  endif
8  return result
```

解答群

- ア sumToN(n - 1)
- イ sumToN(n + 1)
- ウ sumToN(n)
- エ 1
- オ n - 1
- カ sumToN(1)

解説 この問題では、sumToN(n + 1) のような式を選びたくなるかもしれませんが。たしかに「1 から n を合計する」という目的を考えると、n + 1 を使えば「1 + 2 + 3 … + n」が実現できるような気がします。しかし、注意すべきは3行目の終了条件である if(n = 1) です。sumToN(n + 1) のように呼び出してしまうと、n はどんどん大きくなり、終了条件に永遠に合致せず、無限に再帰してしまいます。

「1 から n までの和」を「n から 1 までの和」と解釈しなおすと、答えが導き出せます。つまり、自分自身を呼び出すごとに 1、2、3 と増えるのではなく、n から始まって 3、2、1 と減っていく必要があります。したがって、正解は「sumToN(n - 1)」の **ア** です。



解説動画

正解 **ア**

問題6

オリジナル問題

次のプログラム中の `a` , `b` に入れる正しい答えを、解答群の中から選べ。

関数 `countWords` は、最大 20 文字の文字列を引数 `text` として受け取り、その中に含まれる単語の個数を返す。単語は空白で区切られており、空白が複数連続する場合もある。

プログラム

```
1  ○整数型: countWords(文字列型: text)
2    整数型: count ← 0
3    論理型: inWord ← false
4    整数型: i
5    for (iを1からtextの文字数まで1ずつ増やす)
6      if (textのi番目の文字が a )
7        if ( b )
8          count ← count + 1
9          inWord ← true
10       endif
11     else
12       inWord ← false
13     endif
14   endfor
15   return count
```

解答群

	a	b
ア	" "と等しい	inWordがtrueと等しい
イ	" "と等しい	inWordがfalseと等しい
ウ	" "と等しい	inWordが" "と等しい
エ	" "と等しくない	inWordがtrueと等しい
オ	" "と等しくない	inWordがfalseと等しい
カ	" "と等しくない	inWordが" "と等しい

解説 7行目の **b** から考えてみましょう。9行目では、条件を満たしたときに変数inWordにtrueを代入しています。そのため、7行目の条件が「inWordがtrueと等しい」は誤りです。また、変数inWordは論理型の変数なので、**ウとカ**のように、" "と比べるのは誤りです。したがって、**b** は**イとオ**の「inWordがfalseと等しい」が入ります。



解説動画

次に **a** を考えます。**b** が確定したため、試さなければならないパターンが減りました。ここからはトレース表を作成して考えてみます。引数textは「a pen」とします。

まず **a** が「" "と等しい」の場合です。なお以下のトレース表では、iの更新を5行目の「for」で行っていますが、実際の更新は「endfor」で行われます。今回は「endfor」の行の記述を省略したため、「for」でiを更新しています。このようにトレース表作成は、臨機応変に行ってください。

プログラム	count	inWord	i
2: 整数型: count ← 0	0		
3: 論理型: inWord ← false		false	
4: 整数型: i			
5: for (iを1からtextの文字数まで1ずつ増やす)			1
6: if (textのi番目の文字が" "と等しい)			
12: inWord ← false		false	
5: for (iを1からtextの文字数まで1ずつ増やす)			2
6: if (textのi番目の文字が" "と等しい)			
7: if (inWordがfalseと等しい)			
8: count ← count + 1	1		
9: inWord ← true		true	
5: for (iを1からtextの文字数まで1ずつ増やす)			3
6: if (textのi番目の文字が" "と等しい)			
12: inWord ← false		false	
5: for (iを1からtextの文字数まで1ずつ増やす)			4
6: if (textのi番目の文字が" "と等しい)			
12: inWord ← false		false	
5: for (iを1からtextの文字数まで1ずつ増やす)			5
6: if (textのi番目の文字が" "と等しい)			
12: inWord ← false		false	

単語は2つにもかかわらず、countは1であるため誤りです。では、**a** を「" "と等しくない」で埋めてみます。

プログラム	count	inWord	i
2: 整数型: count ← 0	0		
3: 論理型: inWord ← false		false	
4: 整数型: i			
5: for (iを1からtextの文字数まで1ずつ増やす)			1
6: if (textのi番目の文字が" "と等しくない)			
7: if (inWordがfalseと等しい)			
8: count ← count + 1	1		
9: inWord ← true		true	
5: for (iを1からtextの文字数まで1ずつ増やす)			2
6: if (textのi番目の文字が" "と等しくない)			
12: inWord ← false		false	
5: for (iを1からtextの文字数まで1ずつ増やす)			3
6: if (textのi番目の文字が" "と等しくない)			
7: if (inWordがfalseと等しい)			
8: count ← count + 1	2		
9: inWord ← true		true	
5: for (iを1からtextの文字数まで1ずつ増やす)			4
6: if (textのi番目の文字が" "と等しくない)			
7: if (inWordがfalseと等しい)			
5: for (iを1からtextの文字数まで1ずつ増やす)			5
6: if (textのi番目の文字が" "と等しくない)			
7: if (inWordがfalseと等しい)			

countが正しく「2」になったので、正解は**オ**です。なお、この問題は次のような考え方です。

a		p	e	n
---	--	---	---	---

inWordの初期値がfalseなので
カウントアップ

前の文字が" "なので
カウントアップ



複雑なプログラムの場合には、すべてのパターンをトレースするのは非現実的です。そのため「このプログラムでは何をしようとしているのか」を見いだすことで選択肢を削り、残った選択肢だけでトレースしてください。

正解 **オ**

問題7

オリジナル問題

次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

関数 popCount は、8 ビット型の引数 byte に対して、「1」のビット数を返す。

プログラム

```
1  ○整数型: popCount(8ビット型: byte)
2  8ビット型: rbyte ← byte
3  整数型: count ← 0
4  整数型: i
5  for (iを1から8まで1ずつ増やす)
6    if ()
7      count ← count + 1
8    endif
9    rbyte ← rbyte >> 1
10  endfor
11  return count
```

解答群

- ア $\text{rbyte} \wedge 00000001 = 00000001$
- イ $\text{rbyte} \wedge 00000001 = 00000000$
- ウ $\text{rbyte} \vee 00000001 = 00000001$
- エ $\text{rbyte} \wedge 11111110 = 00000000$
- オ $\text{rbyte} >> 1 = 00000001$
- カ $\text{rbyte} = 00000001$

解説 変数 byte には、もとのビット列が格納され、それ以降変化はありません。一方で、変数 byte の値が代入された変数 rbyte は、繰り返し処理の 9 行目で値が変化していきます。このように**変数の変化が激しい場合には、トレース表を作ると便利**です。

変数 byte が「10101100」の前提でトレース表を作成してみました。まずはアからです。



解説動画

10101100

プログラム	i	rbyte	count
2: 8ビット型: rbyte ← byte		10101100	
3: 整数型: count ← 0			0
4: 整数型: i			
5: for (iを1から8まで1ずつ増やす)	1		
6: if (rbyte ∧ 00000001 = 00000001)			
9: rbyte ← rbyte >> 1		01010110	

1度目の繰り返しまでをトレースしました。「10101100」と「00000001」のAND演算を行うと、結果は「00000000」です。6行目の判定条件の結果は偽となるため、countに変化はありません。9行目でrbyteを右に1ビットシフトするため、rbyteは「01010110」となります。

続いて、2度目の繰り返しを行います。

01010110

プログラム	i	rbyte	count
5: for (iを1から8まで1ずつ増やす)	2		
6: if (rbyte ∧ 00000001 = 00000001)			
9: rbyte ← rbyte >> 1		00101011	

同じく6行目の判定条件の結果は偽となるため、countに変化はありません。9行目でrbyteの右シフトを行い「00101011」となります。

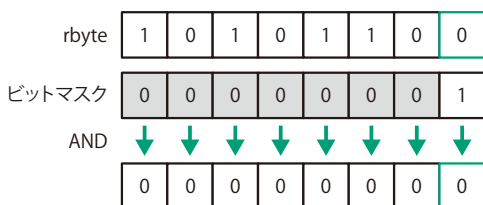
さらに、3度目の繰り返しを行います。

00101011

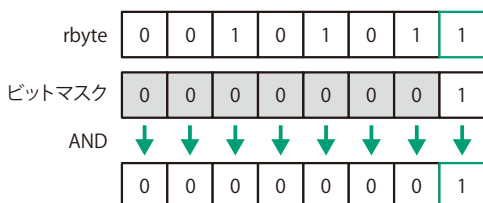
プログラム	i	rbyte	count
5: for (iを1から8まで1ずつ増やす)	3		
6: if (rbyte ∧ 00000001 = 00000001)			
7: count ← count + 1			1
9: rbyte ← rbyte >> 1		00010101	

6行目の判定条件の結果が真となり、countがカウントアップされ1になりました。ここまでトレースすることで、**A**を選択した場合のプログラムが何を目的としているのかを推測できるかもしれません。

rbyteの最下位ビット(右端のビット)が1であるかどうかを確認するために、「00000001」をビットマスクとしてAND演算を行っています。その結果が「00000001」であれば、最下位ビットが「1」であったことになるため、countを1増やします。1度目の繰り返しでは最下位ビットが「0」でした。



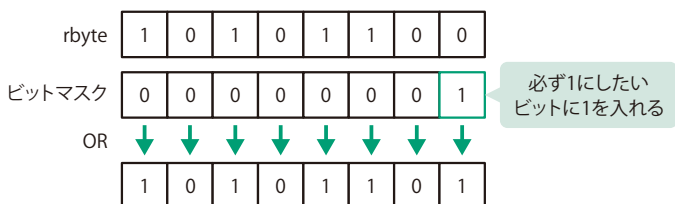
確認が終わったら右に1ビットシフトし、同様に次の繰り返しで右端が1であるかを確認しています。さらに右に1ビットシフトすると、次のように最下位ビットが「1」となりましたので、変数countをカウントアップします。



このような処理を8回行うことで、ビット列全体における「1」の数が得られます。したがって、正解は**ア**です。

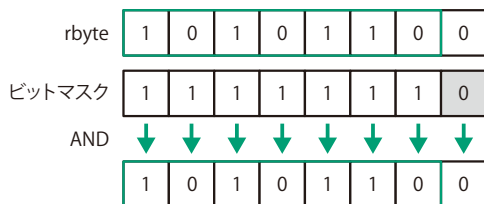
イは、「00000001」をビットマスクとしてAND演算を行った結果が「0」である場合に、countをカウントアップするという意味になります。このプログラムの目的は「1のビット数を数える」ことですが、**イ**を採用してしまうと「0のビット数」を数えてしまうことになり、誤りです。

ウは、ビットマスクとのOR演算を行っているため誤りです。



あるビット列の右端だけを得るには、右端だけを「1」とした8ビットのビットマスクが必要です。そのビットマスクとAND演算することで、目的とするビットを得ることができます。

エは、ビットマスクが「11111110」となっています。このビットマスクとAND演算すると左から7ビットだけが得られ、右端は必ず0になってしまうため誤りです。



オを採用すると「右に1ビットシフトした結果が"00000001"であるか」という判断を行うことになります。シフトは9行目で行っているため、無駄なシフトが行われることになります。

力を採用すると「rbyteが"00000001"であるか」という判断を行うことになります。この場合countは1となり、目的とする結果となりません。以下はiの値とrbyteの対応表です。

i	rbyte							
1	1	0	1	0	1	1	0	0
2	0	1	0	1	0	1	1	0
3	0	0	1	0	1	0	1	1
4	0	0	0	1	0	1	0	1
5	0	0	0	0	1	0	1	0
6	0	0	0	0	0	1	0	1
7	0	0	0	0	0	0	1	0
8	0	0	0	0	0	0	0	1

初めて条件に一致

このように、トレース表を作成しなくても、よりシンプルな表で正解が判断できるのであれば、それに越したことはありません。



トレース表は変数が頻繁に動くときに特に便利ですが、作成に時間がかかります。そのため、メモ書き程度で把握できるのであればわざわざトレース表を作る必要はありません。実際の試験で完全なトレース表を作らなければならない問題は少ないはずです。

正解 ア

次の記述中の に入れる正しい答えを、解答群の中から選べ。

キューとスタックを操作するプログラムである。クラス Queue はキューを表すクラスであり、クラス Stack はスタックを表すクラスである。クラスの説明を図1及び図2に示す。

手続 proc を呼び出したとき、出力は の順となる。

図1 クラス Queue の説明

コンストラクタ	説明
Queue()	空のキューを生成する。

メソッド	戻り値	説明
enq(文字列型:s)	なし	キューに、文字列sを要素として追加する。
deq()	文字列型	キューから最初に追加された要素を取り出して返す。

図2 クラス Stack の説明

コンストラクタ	説明
Stack()	空のスタックを生成する。

メソッド	戻り値	説明
push(文字列型:s)	なし	スタックに、文字列sを要素として追加する。
pop()	文字列型	スタックから最後に追加された要素を取り出して返す。
isEmpty()	論理型	スタックに要素がなければtrueを返す。

プログラム

```

1  Oproc()
2      Stack: stack ← Stack()
3      Queue: queue ← Queue()
4      stack.push("a")
5      stack.push("b")
6      queue.enq(stack.pop())
7      queue.enq("c")
8      stack.push("d")
9      queue.enq(stack.pop())
10     stack.push("e")
11     while (stack.isEmpty() = false)
12         queue.enq(stack.pop())
13     endwhile
14     queue.deq()を出力
15     queue.deq()を出力

```

解答群

- ア a → b
- イ b → c
- ウ c → d
- エ e → a
- オ b → d
- カ c → b

解説 キューとスタックの問題は頻出です。ただし、単純なキューとスタックはあまり出題されないと考えられます。公開されている問題では優先度付きキューについて出題されていますし、今回のようにisEmptyなどのメソッドが追加されている場合もあります。クラスの説明をよく読んで答えましょう。

キューとスタックの問題は、トレース表を作成すると簡単に正解がわかります。まずは途中経過です。



解説動画

プログラム	stack	queue
4 : stack.push("a")	a	
5 : stack.push("b")	ab	
6 : queue.enq(stack.pop())	a	b
7 : queue.enq("c")		bc
8 : stack.push("d")	ad	
9 : queue.enq(stack.pop())	a	bcd
10 : stack.push("e")	ae	

11行目の繰り返し条件は「stack.isEmpty() = false」です。メソッドisEmptyの説明を見ると「スタックに要素がなければtrueを返す」となっていますので、繰り返し条件は「スタックに要素がある間は繰り返す」ということになります。

stackを見てみると「ae」となっており、要素が存在しているため、繰り返し処理の中に入ります。

12 queue.enq(stack.pop())

stack.pop()によりスタックから「e」が取り出され、それをキューに追加しています。

プログラム	stack	queue
12 : queue.enq(stack.pop())	a	bcde

再び11行目の繰り返し条件を確認します。スタックには要素があるため、繰り返し処理を続けます。同じくスタックから「a」を取り出してキューに追加します。

プログラム	stack	queue
12 : queue.enq(stack.pop())		bcdea

スタックに要素がなくなったため、繰り返し処理を終了します。

14行目と15行目でキューの要素を取り出して出力します。キューは「bcdea」の順で格納されているため、出力結果は「b→c」となり、正解は**イ**です。

正解 **イ**

問題9

R4/12月サンプル問題 - 問11改変

次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は0から始まるとする。ここで、 $x \bmod y$ は x を y で割った余りを返す。

関数 binSortMod3 を `binSortMod3()` として呼び出すと、戻り値の配列には、未定義の要素が含まれていない。

プログラム

```
1  ○整数型の配列: binSortMod3(整数型の配列: data)
2    整数型: n ← dataの要素数
3    整数型の配列: bins ← {3個の未定義の値}
4    整数型: i
5    for (iを0から(n - 1)まで1ずつ増やす)
6      整数型: r ← data[i] mod 3
7      bins[r] ← data[i]
8    endfor
9    return bins
```

解答群

- ア {1, 2, 3}
- イ {3, 6, 9}
- ウ {2, 4, 5}
- エ {1, 4, 7}

解説 与えられた配列dataの各要素の値を、別の配列に格納しなおしています。

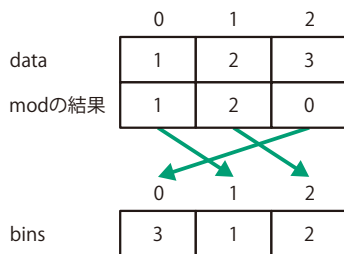
配列に値を格納するには、どこに格納するかを示す要素番号を決めなければなりません。今回の問題では「値を3で割った余り」が、格納先の配列の要素番号です。これは、mod演算を使ったハッシュ関数の基本的な考え方とよく似ています。

値が仮に1だとします。6行目の計算で「 $1 \div 3$ 」の余りである1が変数rに代入されます。そして7行目でbins[1]に「1」が代入されます。同様に値が3の場合には「 $3 \div 3$ 」の余りが0であるため、7行目でbins[0]に「3」が代入されます。

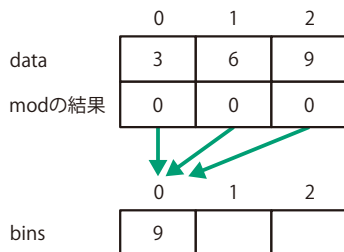
では、解答群を1つずつ試してみましょう。まずはアです。 $1 \div 3$ の余りは1、 $2 \div 3$ の余りは2、 $3 \div 3$ の余りは0です。binsに「未定義」はないため、アが正解です。



解説動画



他の選択肢も見てみましょう。**イ**はこのようになります。



3、6、9ともに余りは0です。まず3が要素番号0の要素に格納されます。次に6で同じ要素が上書きされ、次に9で上書きされます。結果的にbins[1]とbins[2]は未定義のままであるため誤りです。

ウの場合には、2、4、5がそれぞれ要素番号2、1、2になります。要素番号0は未定義のままであり誤りです。

エの場合には、1、4、7はすべて要素番号1になるため誤りです。

なお、**すでに値がある要素に別の値を入れようとする**ことを**衝突 (シノニム)**といいます^{※1}。

正解 **ア**

※1 実際のハッシュを使った格納では、計算された位置にすでに値がある場合、空いている場所を線形探索によって見つけ、そこに格納します。

問題10

オリジナル問題

次の記述中の `a` と `b` に入れる正しい式の組合せを、解答群の中から選べ。ここで、配列の要素番号は0から始まるとする。

関数 `splitWords` を呼び出すと、戻り値の配列には、文字列 `s` に含まれる単語がすべて格納されている。単語間の区切りは半角空白とする。

プログラム中で使われている関数の説明を図に示す。

図 関数の説明

関数	戻り値	説明
<code>index(文字列型: s, 文字型 c)</code>	整数型	<code>c</code> が <code>s</code> の中で最初に出現する位置を返す。見つからなければ0を返す。
<code>cut(文字列型: s, 整数型: a, 整数型: b)</code>	文字列型	<code>s</code> の <code>a</code> 文字目から <code>b</code> 文字目までの部分文字列を返す。

プログラム

```

1  ○文字列型の配列: splitWords(文字列型: s)
2  文字列型の配列: words ← {十分な数の未定義の値}
3  整数型: i ← 0
4  while (index(s, " ")が0と等しくない)
5      整数型: pos ← index(s, " ")
6      words[i] ← cut(s, a, b)
7      s ← cut(s, pos + 1, sの文字数)
8      i ← i + 1
9  endwhile
10 words[i] ← s
11 return words

```

解答群

	a	b
ア	0	pos
イ	0	pos - 1
ウ	1	pos
エ	1	pos - 1
オ	pos	pos
カ	pos	pos - 1

解説 関数 `splitWords` では、与えられた文字列 `s` に含まれる複数の単語を配列に格納していきます。単語間は半角空白で区切られています。仮に「This is a pen」が渡された前提でトレースしてみましょう。



解説動画

プログラム	words	i	pos
3: 整数型: <code>i ← 0</code>		0	
4: while (<code>index(s, " ")</code> が0と等しくない)			
5: 整数型: <code>pos ← index(s, " ")</code>			5

「This is a pen」の最初に半角空白が出現する位置は5です。したがって `pos` には5が格納されました。

それでは、選択肢を1つずつ空欄に当てはめてみます。ただしアとイは誤りです。

a に0を入れると「`cut(s, 0, b)`」となりますが、0文字目は存在しないためです。

ウの場合でトレースを続けます。なお `words` は配列であるため、本来であればすべての要素を記述すべきですが、トレースを省力化するために、`words[i]` だけを記述します。また、前後の空白をわかりやすくするために、「」で囲みました。このようにトレースは柔軟に行いましょう。

プログラム	words[i]	i	pos
6: <code>words[i] ← cut(s, 1, pos)</code>	「This」		

`words[0]` の値が「This」となりました。`pos` が5であるため、1文字目から5文字目だけを切り出したのですが、最後に半角空白が入っています。得たいのは「This」だけであるため、**ウ**は誤りです。

工の場合には「cut(s, 1, pos - 1)」となります。posに5を入れると「cut(s, 1, 4)」となり、正しく「This」だけが得られます。したがって、正解は**工**です。

なお、**オ**、**カ**はともに5文字目から切り出しているため誤りです。

正解 **工**

問題⑪

オリジナル問題

次のプログラム中の **a** 及び **b** に入れる正しい答えを、解答群の中から選べ。ここで、文字列の結合は+演算子で行い、 $x \bmod y$ は x を y で割った余りを返す。

関数toReversedBinaryは、整数型の引数 n (正の整数)を2進数に変換し、その各ビットを逆順に並べた文字列を戻り値として返す。たとえば10進数「13」の2進数は「1101」であり、これを逆順に並べると「1011」となる。これを10進数で解釈した文字列「11」が戻り値となる。

プログラム

```

1  ○文字型: bitToChar(整数型: bit)
2    if (bit = 0) then
3      return "0"
4    else
5      return "1"
6    endif
7
8  ○文字列型: toReversedBinary(整数型: n)
9    文字列型: result ← ""
10   while (n ≠ 0)
11     整数型: b ← a
12     n ← b
13     result ← result + bitToChar(b)
14   endwhile
15   return result

```

解答群

	a	b
ア	$n \bmod 2$	$n \div 2$
イ	$n \div 2$	$n \div 2$
ウ	$n \times 2$	$n \div 2$
エ	$n \bmod 2$	$n \bmod 2$
オ	$n \div 2$	$n \bmod 2$
カ	$n \times 2$	$n \bmod 2$
キ	$n \bmod 2$	$n - 2$
ク	$n \div 2$	$n - 2$
ケ	$n \times 2$	$n - 2$

解説 aにある3種類の処理は、2進数を扱う処理においてよく使われるものばかりです。



解説動画

● $n \bmod 2$

2で割った余りなので、必ず0と1のどちらかになります。この結果は、**2進数に変換したときの最下位ビットの値と一致**します。例えば、13を2進数で表現すると「1101」であり、最下位ビットは「1」です。これは $13 \div 2$ の余りと一致します。

● $n \div 2$

$n \div 2$ は、 **n の2進数表現を1ビット右にシフトした値と一致**します。例えば、13を2進数で表現すると「1101」です。 $13 \div 2$ の結果である6を2進数で表現すると「0110」です。最下位ビットが消えてしまいましたが、 $13 \div 2$ で余り（または小数点以下）を切り捨てたときと同じ状態です。

● $n \times 2$

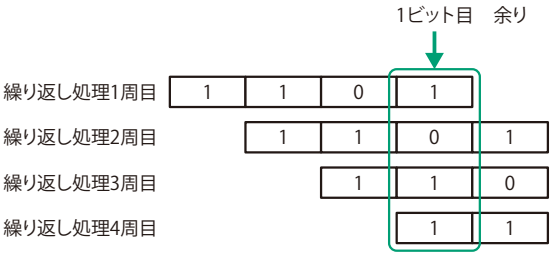
n の2進数表現を1ビット左にシフトした値と一致します。 13×2 の結果である26を2進数で表現すると「11010」ですが、これは13の2進数である「1101」を左に1ビットシフトしたものと一致します。

この3つの中で、ある特定のビットだけを取り出すのは「 $n \bmod 2$ 」です。まだ確証はありませんが、これが正しい前提で考えてみましょう。

最初は最下位である1ビット目だけを取り出します。次に2ビット目だけを取り出します。

そして3ビット目、4ビット目と順に取り出す必要があります。これらを10行目からの繰り返し処理で行っています。

a が「 $n \bmod 2$ 」だとすると、常に最下位ビットを得ているわけですから、12行目ではビット列を右にシフトする必要があります。こうすることで各ビットをすべて処理することができるようになります。



右シフトする必要があるため、**b** は「 $n \div 2$ 」だと思われます。これでいったんテストしてみましょう。

プログラム	n	result	b
8: ○文字列型: toReversedBinary(整数型: n)	13		
9: 文字列型: result ← ""		""	
10: while (n ≠ 0)			
11: 整数型: b ← n mod 2			1
12: n ← n ÷ 2	6		
13: result ← result + bitToChar(b)		"1"	
10: while (n ≠ 0)			
11: 整数型: b ← n mod 2			0
12: n ← n ÷ 2	3		
13: result ← result + bitToChar(b)		"10"	
10: while (n ≠ 0)			
11: 整数型: b ← n mod 2			1
12: n ← n ÷ 2	1		
13: result ← result + bitToChar(b)		"101"	
10: while (n ≠ 0)			
11: 整数型: b ← n mod 2			1
12: n ← n ÷ 2	0		
13: result ← result + bitToChar(b)		"1011"	

resultが正しく「1011」となりました。正解はアです。



とりあえず当たりを付けて、それが正しいと仮定してトレースを行うことで、試すパターンの数を減らすことができます。

正解 ア

問題12

H20 秋 - 問 6 改変

次のプログラム中の `a` 及び `b` に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は0から始まるものとし、 $x \bmod y$ は x を y で割った余りを返す。

関数 `convert` は、整数型の引数 `num` (9桁以下) を、3桁区切り形式の文字列に変換し、その結果を戻り値として返す。

次のルールに従って変換を行う：

- ①整数値が負の場合は、先頭にマイナス符号を付ける。
- ②数値の下位から3けたごとにコンマ(,)を挿入する。

関数 `convert` に与える `num` および戻り値の例を表に示す。プログラムでは、配列の領域外を参照してはならないものとする。また、整数 `num` は9桁以下の値とする。

表 関数 `convert` に与える `num` および戻り値の例

整数	3桁区切り形式の文字列
1234567	1,234,567
-57482	-57,482
63	63
-999999	-999,999

プログラム

- 1 ○文字列型: `convert(整数型: num)`
- 2 論理型: `minus ← false`
- 3 整数型: `i ← 0, j ← 0`

```
4  文字列型の配列: table ← {"0", "1", "2", "3", "4",
5                                "5", "6", "7", "8", "9"}
6  文字列型の配列: result ← {20個の未定義の値} // 十分な長さで仮確保
7
8  if (num < 0)
9      minus ← true
10     num ← num × (-1)
11 endif
12
13 do
14     result[j] ← table[num mod 10]
15     j ← j + 1
16     a
17     i ← i + 1
18     if (b = 0 and num ≠ 0)
19         result[j] ← ","
20         j ← j + 1
21     endif
22 while (num ≠ 0)
23
24 if (minus)
25     result[j] ← "-"
26     j ← j + 1
27 endif
28
29 for (iを0からj ÷ 2 - 1まで1ずつ増やす)
30     文字型: tmp ← result[i]
31     result[i] ← result[j - i - 1]
32     result[j - i - 1] ← tmp
33 endfor
34
35 return result
```

解答群

	aに関する式	bに関する式
ア	$\text{num} \leftarrow \text{num} \bmod 10$	$i \bmod 3$
イ	$\text{num} \leftarrow \text{num} \bmod 10$	$(i + 1) \bmod 3$
ウ	$\text{num} \leftarrow \text{num} \bmod 10$	$(j + 2) \bmod 3$
エ	$\text{num} \leftarrow \text{num} \bmod 10$	$3 \bmod j$
オ	$\text{num} \leftarrow \text{num} \times (-10)$	$i \bmod 3$
カ	$\text{num} \leftarrow \text{num} \times (-10)$	$(i + 1) \bmod 3$
キ	$\text{num} \leftarrow \text{num} \times (-10)$	$(j + 2) \bmod 3$
ク	$\text{num} \leftarrow \text{num} \times (-10)$	$3 \bmod j$
ケ	$\text{num} \leftarrow \text{num} \div 10$	$i \bmod 3$
コ	$\text{num} \leftarrow \text{num} \div 10$	$(i + 1) \bmod 3$
サ	$\text{num} \leftarrow \text{num} \div 10$	$(j + 2) \bmod 3$
シ	$\text{num} \leftarrow \text{num} \div 10$	$3 \bmod j$

解説 3桁でコンマ区切りをする処理は、多くのプログラムで必要となります。

例にある通り、1234567であれば1,234,567となります。複雑なのはマイナス値です。-57482であれば-57,482としなければなりません。



解説動画

2行目に登場する変数 `minus` は、変数名から見て「値がマイナスであるかどうか」を保持する目的で使われていると推測されます。8行目で「0未満か」を判断し、0未満であれば9行目で変数 `minus` に `true` を代入しています。初期値は `false` ですから、やはり変数 `minus` は、引数 `num` がマイナスかどうかの判断に使っているようです。

このプログラムにおいて、14行目の「`num mod 10`」が特徴的な計算に見えると思います。「ある値を10で割った余り」は非常によく使われますので覚えておきましょう。両者にどんな関係がありそうでしょうか。

「ある値を10で割った余り」の例

値	10で割った余り
123	3
987	7
5	5
-123	3

例からわかるように、ある値を10で割った余りは、その値の1桁目の数値を意味しています。

また、配列tableの要素番号0には文字の「0」が、要素番号1には文字の「1」が格納されています。

要素番号	0	1	2	3	4	5	6	7	8	9
値	"0"	"1"	"2"	"3"	"4"	"5"	"6"	"7"	"8"	"9"

つまり14行目の「table[num mod 10]」では以下の2つの処理を行っています。

- 1桁目の数値を得る
- その数値を文字型にする

numが「1234」の場合で、13行目からの繰り返しだけをトレースしてみると、result[0]に「4」が代入されました。

プログラム	num	result	j	i
13: do	1234	△,△,△,△,△,△,△,△,△,△,△,△, △,△,△,△,△,△,△,△,△,△	0	0
14: result[j] ← table[num mod 10]		"4",△,△,△,△,△,△,△,△,△,△,△, △,△,△,△,△,△,△,△,△,△		
15: j ← j + 1			1	

仮に16行目がないものとして処理を続けてみます。また18～21行目の判定処理はコマ区切りの挿入ですので、これもいったん外してみます。

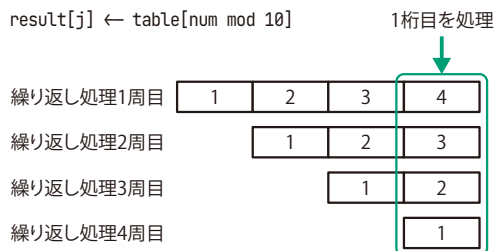
13	do
14	result[j] ← table[num mod 10]
15	j ← j + 1
16	a
17	i ← i + 1
18	if (b = 0 and num ≠ 0)
19	result[j] ← ","
20	j ← j + 1
21	endif
22	while (num ≠ 0)

このプログラムでトレースを続けます。

プログラム	num	result	j	i
17: i ← i + 1				1
22: while (num ≠ 0)				
14: result[j] ← table[num mod 10]		"4", △, △, △, △, △, △, △, △, △, △, △, △, △, △, △, △, △, △, △		
15: j ← j + 1			2	
17: i ← i + 1				2
22: while (num ≠ 0)				
14: result[j] ← table[num mod 10]		"4", "4", △, △, △, △, △, △, △, △, △, △, △, △, △, △, △, △, △, △		
15: j ← j + 1			3	
17: i ← i + 1				3
22: while (num ≠ 0)				
14: result[j] ← table[num mod 10]		"4", "4", "4", △, △, △, △, △, △, △, △, △, △, △, △, △, △, △, △, △, △		
15: j ← j + 1			4	
17: i ← i + 1				4

すべての繰り返しで、「num mod 10」の計算結果が「4」になってしまいます。そのためresultには毎回「4」が格納されます。そうではなく、すでに「4」は処理を行ったので、次に「3」を処理したいはずです。

14行目では「1桁目を文字にして、それをresultに配置」していることから、14行目に至る前に「1234」を「123」に変更する処理が必要になります。



「1234」を「123」に、「123」を「12」に、「12」を「1」に変更する処理をどこかに入れるとすると、先ほど一時的に省略した16行目しかありません。目的とする結果になるかを、解答群から1つずつ試してみましょう。

● $\text{num} \leftarrow \text{num} \bmod 10$ の場合

num が「1234」の状態では「 $\text{num} \bmod 10$ 」の計算をすると、1桁目の値が取得できません。これを num に代入すると「4」になってしまいます。理想的なのは「123」なのでこれは誤りです。

● $\text{num} \leftarrow \text{num} \times (-10)$ の場合

「 $\text{num} \times (-10)$ 」の計算をすると、 num には「-12340」が代入され、これも誤りです。

● $\text{num} \leftarrow \text{num} \div 10$ の場合

「 $\text{num} \div 10$ 」の計算をすると num には「123」が代入されます。これは理想通りです。したがって、解答群の **ケ**、**コ**、**サ**、**シ** のいずれかが正解です。

このまま処理を続けると、 $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$ の順に result に追加していくため、 result には逆順で格納されることになります。

	0	1	2	3		18	19
result	4	3	2	1	...		

では次に、18～21行目の処理を復活させて考えてみます。仮に18行目の条件がないと、繰り返し処理の1周ごとにコンマを代入する処理が実行されます。

	0	1	2	3	4	5	6	7		18	19
result	4	,	3	,	2	,	1	,	...		

繰り返しごとにコンマを入れるのではなく、3周ごとに代入するのが正しい動作なので、その判断条件が必要になります。

では、「現在が何周目か」を判断するにはどうしたらいいでしょうか。変数 j と変数 i を使う方法がありそうですが、変数 j は20行目でもカウントアップしています。1周ごとにカウントアップするのは変数 i なので、これを使うことにします。

「〇周ごと」の判断には、割り算の余りを使うのが一般的です。「3周ごと」は、3で割り切れるかどうかを確認することで判断できます。

周	3で割った余り
1	1
2	2
3	0
4	1
5	2
6	0
7	1
8	2
9	0

bには「 $i \bmod 3$ 」が入ります。これで3周ごとに「,」が入るようになります。したがって、正解は**ケ**です。

	0	1	2	3	4	5	6	7		18	19
result	4	3	2	,	1				...		

引き続きプログラムを読解してみましょう。24行目の判定で「マイナスである」となった場合には、resultに「-」を追加します。

```

24  if (minus)
25      result[j] ← "-"
26      j ← j + 1
27  endif

```

変数jは13行目から22行目の繰り返し処理により、最後の要素の次の要素の要素番号になっています。そのため「-」は必ず一番最後の要素に代入されることになります。引数numが「-1234」の場合は、次のようになります。

	0	1	2	3	4	5	6	7		18	19
result	4	3	2	,	1	-			...		

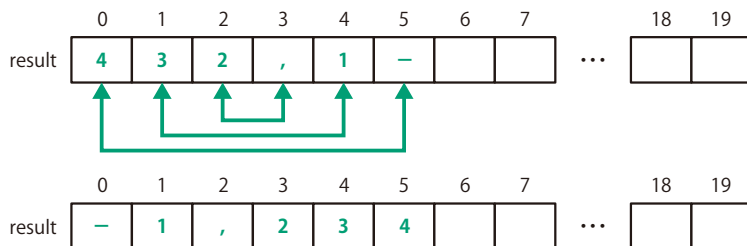
このままでは逆順であるため28行目からの処理で正しい順序に直しています。


```

29  for (iを0からj ÷ 2 - 1まで1ずつ増やす)
30    文字型: tmp ← result[i]
31    result[i] ← result[j - i - 1]
32    result[j - i - 1] ← tmp
33  endfor

```

要素番号0と5を、1と4を、2と3を交換することで並べ替えを行っています。



ココに
気をつけて!

いったんプログラムの一部を省略してトレースすることで、処理の流れが理解しやすくなります。今回は16行目を省略したところ、おかしい結果になったことを確認しました。そこで「どうすれば正しい結果になるか」を考え、16行目に入れるべき処理を検討しました。その結果、正しい答えを導くことができました。

正解 ケ

次のプログラム中の a 及び b に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は0から始まるものとする。

手続invertは、引数の配列inに格納された8行×6列の文字パターンを、整数invtypeの値(1～4)にしたがって、回転又は反転して出力する。

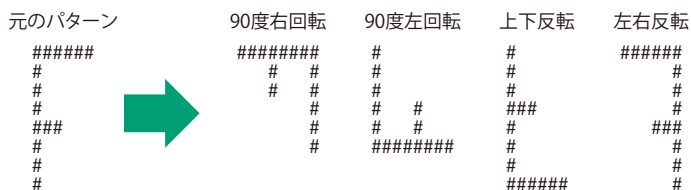
引数invtypeの値と文字パターンの回転又は反転種別の関係は、下表のとおりである。

表 値と文字パターンの回転又は反転種別の関係

invtype	回転・反転種別
1	90度右回転
2	90度左回転
3	上下反転
4	左右反転

手続mainの実行結果をまとめた文字パターンの印字例を図1に示す。

図1 文字パターンの印字例



回転および反転操作の手順を、8行6列の配列dat[8][6]を例として説明する。

- 90度回転では、回転後は行と列の数が入れ替わることになる。
- 90度右回転の場合、先頭の行データ群は右端の列データ群へ移動する。次の行データ群が右から2番目の列に移り、この手順が続く。最後の行データ群が左端の列に移る。

先頭行: dat[0][0], dat[0][1], ..., dat[0][4], dat[0][5]

↓ ↓ ↓ ↓

右端列: dat[0][7], dat[1][7], ..., dat[4][7], dat[5][7]

- 90度左回転の場合には、先頭の行データ群は、左端の列データ群 `dat[5][0]`, `dat[4][0]`, ..., `dat[1][0]`, `dat[0][0]` に移る。次の行データ群が左から二つ目の列に移り、同様の手順で移動が進み、最後に最下段の行データ群が右端の列データ群に移る。
- 上下反転では、先頭の行データ群は、最下段の行データ群 `dat[7][0]`, `dat[7][1]`, ..., `dat[7][4]`, `dat[7][5]` に移る。次の行データ群が下から二つ目の行に移り、同様の手順で移動が進み、最後に最下段の行データ群が先頭の行データ群に移る。
- 左右反転では、行内で入れ替わるので、先頭の行データ群は、`dat[0][5]`, `dat[0][4]`, ..., `dat[0][1]`, `dat[0][0]` に移る。以降の行データ群も同様の手順で行内で順序が入れ替わる。

プログラム

```

1  大域: 整数型: RSZ ← 6
2  大域: 整数型: CSZ ← 8
3
4  ○invert(整数型: invtype, 文字型の二次元配列: in)
5      整数型: row ← RSZ
6      整数型: col ← CSZ
7      整数型: i, j
8
9      if (invtype = 1またはinvtype = 2)
10         row ← CSZ
11         col ← RSZ
12     endif
13
14     for (iを0からcol - 1まで1ずつ増やす)
15         for (jを0からrow - 1まで1ずつ増やす)
16             if (invtype = 1)
17                 in[CSZ - j - 1][i] を出力      // 90度右回転
18             else if (invtype = 2)
19                 a を出力      // 90度左回転
20             else if (invtype = 3)
21                 in[CSZ - i - 1][j] を出力      // 上下反転
22             else if (invtype = 4)
23                 b を出力      // 左右反転
24             endif
25         endfor
    endfor

```

```

26     改行を出力
27     endfor
28
29     ○main()
30     文字型の二次元配列: src ← {
31         {"#", "#", "#", "#", "#", "#"},
32         {"#", " ", " ", " ", " ", " "},
33         {"#", " ", " ", " ", " ", " "},
34         {"#", "#", "#", " ", " ", " "},
35         {"#", " ", " ", " ", " ", " "},
36         {"#", " ", " ", " ", " ", " "},
37         {"#", " ", " ", " ", " ", " "},
38         {"#", " ", " ", " ", " ", " "}
39     }
40
41     invert(1, src)
42     invert(2, src)
43     invert(3, src)
44     invert(4, src)

```

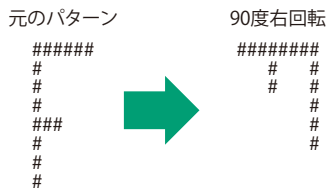
解答群

	a	b
ア	in[RSZ - i - 1][j]	in[RSZ - i - 1][j]
イ	in[RSZ - j - 1][i]	in[RSZ - i - 1][j]
ウ	in[i][CSZ - j - 1]	in[RSZ - i - 1][j]
エ	in[i][RSZ - j - 1]	in[RSZ - i - 1][j]
オ	in[j][CSZ - i - 1]	in[RSZ - i - 1][j]
カ	in[j][RSZ - i - 1]	in[RSZ - i - 1][j]
キ	in[RSZ - i - 1][j]	in[i][RSZ - j - 1]
ク	in[RSZ - j - 1][i]	in[i][RSZ - j - 1]
ケ	in[i][CSZ - j - 1]	in[i][RSZ - j - 1]
コ	in[i][RSZ - j - 1]	in[i][RSZ - j - 1]
サ	in[j][CSZ - i - 1]	in[i][RSZ - j - 1]
シ	in[j][RSZ - i - 1]	in[i][RSZ - j - 1]

解説 invertの第1引数に1を渡すとします。この場合、第2引数で渡した配列が、90度右回転します。



解説動画



なお、問題文にあるとおり、二次元配列の1つ目の要素番号が行番号を、2つ目の要素番号が列番号を示しています。

●変数iとjは行と列のどちらを扱っているのか

どの要素がどの要素にコピーされれば、90度右回転になるのでしょうか。ひとまず「F」の角に着目してみます。

	0	1	2		5	6	7
0	#	#	#		0	#	#
1	#				1		#
2	#				2		#

90度右回転の場合には、「0行目、0列目」の値が「0行目、7列目」にコピーされるのが正しい処理です。17行目の処理は、そのようになっているはずですが、どうでしょうか。

```
17      in[CSZ - j - 1][i] を出力    // 90度右回転
```

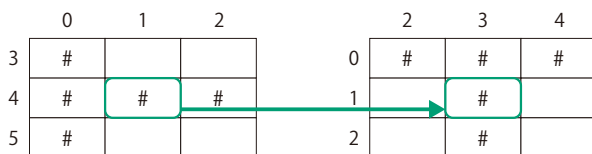
変数iと変数jはコピー元の要素番号ですが、どちらが行番号で、どちらが列番号を表しているかが不明です。そこでまずは変数iと変数jが何を意味するかを調べてみます。変数iが行番号で、変数jが列番号の場合には、次のようになります。

```
17      in[8 - 0 - 1][0] を出力    // 90度右回転
```

逆に、変数iが列番号で、変数jが行番号の場合には、次のようになります。

```
17      in[8 - 0 - 1][0] を出力    // 90度右回転
```

両方の場合でin[7][0]になってしまいましたので、このままでは調査は進みません。「0, 0」のように、同じ値で調査したのが不適切だったようです。そこで4行1列目のコピー先を探ってみることにします。



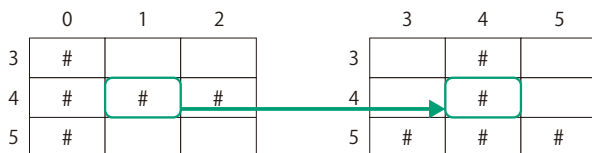
仮に*i*が列番号で、*j*が行番号だとすると、次のように解釈できます。

17 `in[8 - 4 - 1][1]` を出力 // 90度右回転

`in[3][1]`を出力することになりますが、これは正しい場所です。*i*が列番号で、*j*が行番号のようです。これを覚えておきましょう。

● 90度左回転させるには

では `a` を埋めていきます。90度左回転するには「4行目、1列目」は「4行目、4列目」に移動するべきです。



そのようになるプログラムを解答群から選びます。

ア、キの「`in[RSZ - i - 1][j]`」は、`in[6 - 1 - 1][4] = in[4][4]`と解釈できます。さっそくですが正しい値になりました。念のため、他の選択肢も見てみましょう。

[1, 4] が [4, 4] に移動するかどうか

選択肢	プログラム	説明
ア、キ	<code>in[RSZ - i - 1][j]</code>	<code>in[6 - 1 - 1][4] = in[4][4]</code> となるため 正しい
イ、ク	<code>in[RSZ - j - 1][i]</code>	<code>in[6 - 4 - 1][1] = in[1][1]</code> となるため誤り
ウ、ケ	<code>in[i][CSZ - j - 1]</code>	<code>in[1][8 - 4 - 1] = in[1][3]</code> となるため誤り
エ、コ	<code>in[i][RSZ - j - 1]</code>	<code>in[1][6 - 4 - 1] = in[1][1]</code> となるため誤り
オ、サ	<code>in[j][CSZ - i - 1]</code>	<code>in[4][8 - 1 - 1] = in[4][6]</code> となるため誤り
カ、シ	<code>in[j][RSZ - i - 1]</code>	<code>in[4][6 - 1 - 1] = in[4][4]</code> となるため 正しい

残念ながら、**ア**と**キ**に加え、**カ**と**シ**も候補に残ってしまいました。では別の場所でも探ってみましょう。

[2, 4]、は[4, 3]に移動するべきです。

[2, 4]、は[4, 3]に移動するかどうか

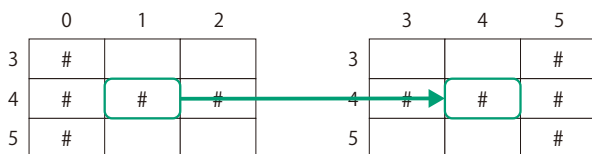
選択肢	プログラム	説明
ア、キ	<code>in[RSZ - i - 1][j]</code>	<code>in[6 - 2 - 1][4] = in[3][4]</code> と解釈できる。 正しくは <code>in[4][3]</code> であるため、誤り
カ、シ	<code>in[j][RSZ - i - 1]</code>	<code>in[4][6 - 2 - 1] = in[4][3]</code> と解釈できるため正しい

これで**カ**と**シ**に絞り込みました。

●左右反転させるには

次に `b` です。左右反転するためには、次のどちらを当てはめるべきでしょうか。

- ・ **カ** `in[RSZ - i - 1][j]`
- ・ **シ** `in[j][RSZ - i - 1]`



左右反転の場合には、行番号に変化はありません。上記の図では*i* (列番号) が1で*j* (行番号) が4です。移動後も行番号は4のままであればなりません。そうになっているのは**シ**です。具体的な値を当てはめて検証します。

`in[4][6 - 1 - 1] → in[4][4]`

正しい移動位置が計算できました。したがって、正解は**シ**です。

正解 **シ**

次のプログラム中の a 及び b に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は0から始まるものとする。

手続InsertSortは、整数型の配列 $A[0], \dots, A[N]$ ($N > 0$) を、挿入ソートにより昇順に整列する処理を行う。

- (1) $A[0]$ と $A[1]$ を整列し、次に $A[0]$ から $A[2]$ までを整列し、その次に $A[0]$ から $A[3]$ までというように、整列する区間の要素を一つずつ増やしていき、最終的に $A[0]$ から $A[N]$ までを整列する。
- (2) 整列する区間が $A[0]$ から $A[M]$ ($1 \leq M \leq N$) までのとき、 $A[M]$ をすでに整列された列 $A[0], \dots, A[M-1]$ の中の適切な位置に挿入する。その手順は次のとおりである。
 - (a) $A[M]$ の値を、作業領域 Tmp に格納する。
 - (b) $A[M-1]$ から $A[0]$ に向かって Tmp と比較し、 Tmp よりも大きな値を順次（要素番号の大きい方）に移動する。
 - (c) (b) で最後に移動した値の入っていた配列要素に Tmp の内容を格納する。(b) で移動がなかった場合には $A[M]$ に格納する。

手続InsertSortの引数の仕様を次に示す。

手続InsertSortの引数の仕様

引数	データ型	入力／出力	意味
A	整数型の配列	入出力	整列対象の配列
N	整数型	入力	整列する区間の最後の要素番号

プログラム

```

1  ○InsertSort(整数型の配列: A, 整数型: N)
2    整数型: Idx1, Idx2, Tmp
3    論理型: Loop
4
5    for (Idx1を1からNまで1ずつ増やす)
6      Tmp ← A[Idx1]
7      Idx2 ← Idx1 - 1
8      Loop ← true
9
10     while (Idx2 ≥ 0 and Loop = true)
11       if (A[Idx2] > Tmp)
12         a
13         Idx2 ← Idx2 - 1
14       else
15         Loop ← false
16       endif
17     endwhile
18
19     b
20   endfor

```

解答群

	a	b
ア	$A[\text{Idx2} - 1] \leftarrow A[\text{Idx2}]$	$A[\text{Idx2}] \leftarrow A[\text{Idx2} + 1]$
イ	$A[\text{Idx2} - 1] \leftarrow A[\text{Idx2}]$	$A[\text{Idx2}] \leftarrow A[\text{Idx2} - 1]$
ウ	$A[\text{Idx2} - 1] \leftarrow A[\text{Idx2}]$	$A[\text{Idx2}] \leftarrow \text{Tmp}$
エ	$A[\text{Idx2} - 1] \leftarrow A[\text{Idx2}]$	$A[\text{Idx2} + 1] \leftarrow A[\text{Idx2}]$
オ	$A[\text{Idx2} - 1] \leftarrow A[\text{Idx2}]$	$A[\text{Idx2} + 1] \leftarrow \text{Tmp}$
カ	$A[\text{Idx2} + 1] \leftarrow A[\text{Idx2}]$	$A[\text{Idx2}] \leftarrow A[\text{Idx2} + 1]$
キ	$A[\text{Idx2} + 1] \leftarrow A[\text{Idx2}]$	$A[\text{Idx2}] \leftarrow A[\text{Idx2} - 1]$
ク	$A[\text{Idx2} + 1] \leftarrow A[\text{Idx2}]$	$A[\text{Idx2}] \leftarrow \text{Tmp}$
ケ	$A[\text{Idx2} + 1] \leftarrow A[\text{Idx2}]$	$A[\text{Idx2} + 1] \leftarrow A[\text{Idx2}]$
コ	$A[\text{Idx2} + 1] \leftarrow A[\text{Idx2}]$	$A[\text{Idx2} + 1] \leftarrow \text{Tmp}$

解説 挿入ソートの説明通りにプログラムを完成させれば正解になりますが、
ここでついでに挿入ソートを学習しましょう。

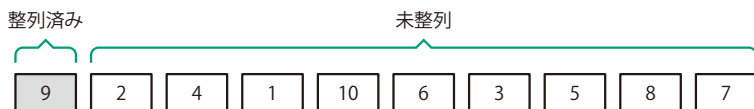


解説動画

挿入ソートとは、要素を適切な位置に順番に挿入していく整列アルゴリズム
です。仮に1から10の要素があるとして。



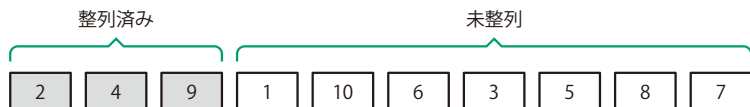
まず先頭である「9」の理想的な場所を考えます。「8」と「10」の間が理想的なのですが、配列全体の中から「理想的な場所はどこか」をチェックするわけではありません。「整列済み」の中からチェックすることになります。この段階ではまだ「整列済み」がないため「9」はこのまま置いておきます。この「9」が唯一の「整列済み」の要素となります。



次に「未整列」のうちの先頭である「2」を対象に、挿入場所を探します。理想的なのは「1」と「3」の間なのですが、配列全体から理想的な場所を探すではありません。「整列済み」の中からだけ理想的な場所を探します。「2」の理想的な場所は「9」の前です。ここに「2」を移動します。



同様に「未整列」のうちの先頭である「4」を対象に処理を行います。理想的な場所は「2」と「9」の間です。



これを最後まで繰り返していきます。この処理では、大きな繰り返しの中に小さな繰り返しが含まれています。

●大きな繰り返し

整列の対象となる要素を1つずつ取り出していく処理です。

最初の1つ目はそのままにしておき、2つ目の要素から順番に、「これから整列済みの部分に挿入する要素(=対象の要素)」を選びます。3つ目、4つ目…と、配列の最後まで、1つずつ対象の要素を選んで処理していきます。

●小さな繰り返し

対象の要素について、「整列済み」の部分と1つずつ比較しながら、自分が入るべき位置を探します。これが理想的な位置を探す処理です。

プログラムを見ると、5行目からの大きな繰り返しの中に、10行目からの小さな繰り返しが存在していることがわかります。5行目からの大きな繰り返しは1~N(最後の要素)まで行っているのに対し、10行目からの繰り返しは、理想的な場所が見つかったら繰り返しを終了する処理になっているはずです。小さな繰り返し部分のプログラムを見てみましょう。

```

10  while (Idx2 ≥ 0 and Loop = true)
11    if (A[Idx2] > Tmp)
12      a
13      Idx2 ← Idx2 - 1
14    else
15      Loop ← false
16    endif
17  endwhile

```

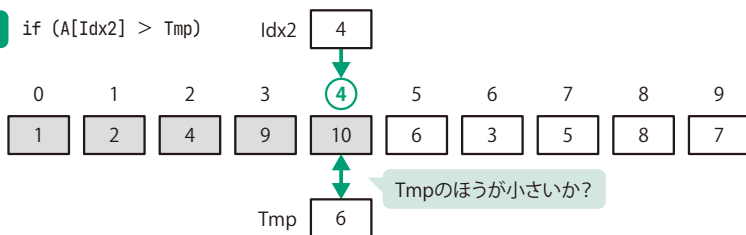
「Idx2が0以上、かつ、Loopがtrueである」ことが「小さな繰り返し」の継続条件なので、終了条件はその逆の意味となる「Idx2が0以上」ではない、または、「Loopがtrue」ではない」となります。そのため、理想的な場所が見つかった場合には、変数Loopがfalseになると考えられます。

その代入が15行目の「Loop ← false」です。したがって11行目は「理想的な場所ではないか?」の判断であることがわかります。

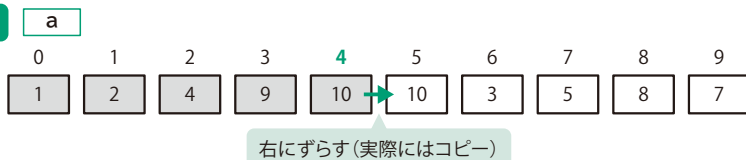
もしこの条件に合致するのであれば「理想的な場所ではなかった」とことになります。理想的な場所が見つかるまでの間繰り返し返さなければならない処理は、問題文の(2)(b)にあるように「要素を右に移動して隙間をあける」ことです。

以下は途中からの流れです。「6」の理想的な場所を「整列済み」から探しますが、見つかるまで1つずつ要素を右にずらしていきます。

11行目 if (A[Idx2] > Tmp)



12行目

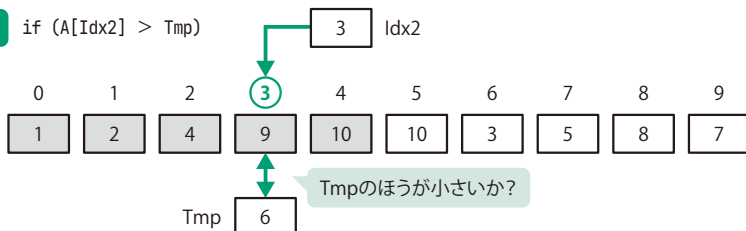


13行目 Idx2 ← Idx2 - 1

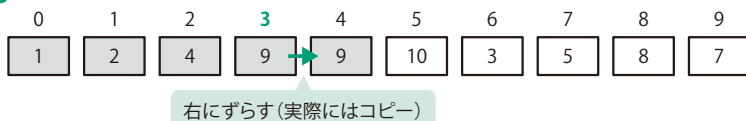
Idx2 3

右の要素にコピーする処理「A[Idx2 + 1] ← A[Idx2]」が a です。解答群ではカ
〜コが該当します。同様の処理を続けます。

11行目 if (A[Idx2] > Tmp)



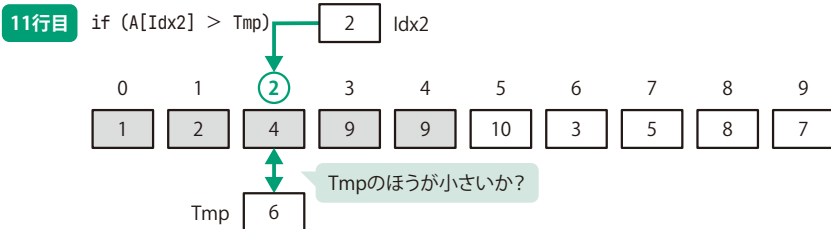
12行目 A[Idx2 + 1] ← A[Idx2]



13行目 Idx2 ← Idx2 - 1

Idx2 2

この状態で11行目の判断を行います。



「6」と「4」を比較した場合に、「 $4 > 6$ 」は誤りであるため11行目の条件に一致せず、15行目に遷移します。15行目では変数Loopがfalseとなり10行目からの小さな繰り返しが終わります。

この時点で変数Idx2は「2」ですが、実際に「6」を挿入したい位置は「3」です。そのため19行目では、「Idx2」ではなく「Idx2 + 1」の位置に挿入するようなプログラムにしなければなりません。「 $A[\text{Idx2} + 1] \leftarrow \text{Tmp}$ 」となっている□が正解です。

正解 □

次のプログラム中の a ~ c に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は0から始まるものとする。

中置表記法による正しい数式を、スタックを用いて後置表記法に変換する副プログラム toPostfix である。

- (1) 例えば、中置表記法による数式 $4 \times (9 + 3)$ は、toPostfix の処理の結果、後置表記法による数式 $493 + \times$ に変換される。
- (2) 中置表記法によるここでの数式は、1 けたの数字 "0" ~ "9", 演算子 "+", "-", "×", "÷" 及び括弧 "(", ")" の各要素からなる。各要素は表 1 に示す変換の優先度をもつ。ここで、数値が大きいほど優先度が高い。プログラムでは、各要素を文字として扱い、文字型の配列 Exptext に 1 要素ずつ格納している。また、表 1 に示す要素以外に、スタックの底を示すスタック制御文字 EOS を使用し、その優先度を -1 とする。

表 1 数式の要素と優先度

数式の要素	優先度
(	4
0, 1, ..., 9	3
×, ÷	2
+, -	1
)	0

- (3) 変換結果の後置表記法の数式は、文字型の配列 Postfix に 1 要素ずつ格納される。
- (4) toPostfix は、スタックを初期化して、Exptext の先頭の要素から順番に次の①、②の処理を繰り返し、Exptext の要素をすべて処理したら、スタックに残っているすべての数式の要素を順に取り出して Postfix に格納する。
- ① Exptext[i] の優先度がスタックの先頭要素の優先度以下で、かつスタックの先頭要素が "(" でない間、スタックから要素を取り出して Postfix に格納する。
- ② Exptext[i] が ")" でなければ、Exptext[i] をスタックに積み、")" であれば、スタックから要素を一つだけ取り出す。
- (5) toPostfix の引数の仕様を、表 2 に示す。

表 2 toPostfixの引数の仕様

変数	型	入力/出力	意味
Exptext	文字型の配列	入力	中置表記法による数式が格納されている配列
Textlen	整数型	入力	中置表記法による数式の要素数
Postfix	文字型の配列	出力	後置表記法に変換後の数式が格納される配列
Postfixlen	整数型	出力	後置表記法に変換後の数式の要素数

(6) toPostfixは、スタックを初期化する手続initStack、数式の要素をスタックに積む手続push、スタックから取り出す関数pop、スタックの先頭要素の値を返却する関数top、及び引数で指定された要素の優先度を返す関数getPriorityを使用する。initStack、push、pop、top及びgetPriorityの仕様を表3に示す。

表 3 toPostfixが使用する手続/関数の仕様

手続/関数	仕様
initStack()	スタックを初期化し、スタックにEOSを格納する。
push(文字型:element)	スタックにelementを格納する。
文字型:pop()	スタックから要素を取り出し、その要素を返却する。
文字型:top()	スタックの先頭要素の値を返却する。
整数型:getPriority(文字型:element)	引数で指定された要素elementの優先度を返却する。

プログラム

```
1  OtoPostfix(文字型の配列: Exptext, 整数型: Textlen,  
2           文字型の配列: Postfix, 整数型: Postfixlen )  
3   整数型: k, i  
4   論理型: sflag  
5   文字型: work  
6  
7   initStack()  
8   k ← 0  
9  
10  for (iを0からTextlen - 1まで1ずつ増やす)  
11    while (getPriority(Exptext[i]) ≤ getPriority(top())  
12          and top() ≠ "(")  
13      Postfix[k] ← pop()  
14      a  
15    endwhile  
16  
17    if (Exptext[i] ≠ ")")  
18      push(Exptext[i])  
19    else  
20      pop() // 取り出した要素は使わない  
21    endif  
22  endfor  
23  
24  sflag ← true  
25  while (sflag = true)  
26    work ← pop()  
27    if (b)  
28      sflag ← false  
29    else  
30      Postfix[k] ← work  
31      a  
32    endif  
33  endwhile  
34  
35  Postfixlen ← c
```


解答群

	a	b	c
ア	$k \leftarrow k + 1$	work = EOS	k
イ	$k \leftarrow k + 1$	work = EOS	$k + 1$
ウ	$k \leftarrow k + 1$	work = EOS	$k - 1$
エ	$k \leftarrow k + 1$	work $\neq$ EOS	k
オ	$k \leftarrow k + 1$	work $\neq$ EOS	$k + 1$
カ	$k \leftarrow k + 1$	work $\neq$ EOS	$k - 1$
キ	$k \leftarrow k - 1$	work = EOS	k
ク	$k \leftarrow k - 1$	work = EOS	$k + 1$
ケ	$k \leftarrow k - 1$	work = EOS	$k - 1$
コ	$k \leftarrow k - 1$	work $\neq$ EOS	k
サ	$k \leftarrow k - 1$	work $\neq$ EOS	$k + 1$
シ	$k \leftarrow k - 1$	work $\neq$ EOS	$k - 1$

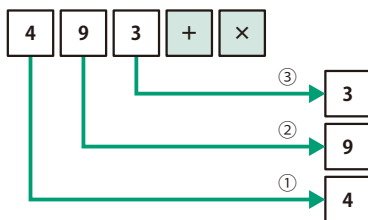
解説 中置表記法とは、我々が日常で使う計算式の記述方法であり、次のようなものです。



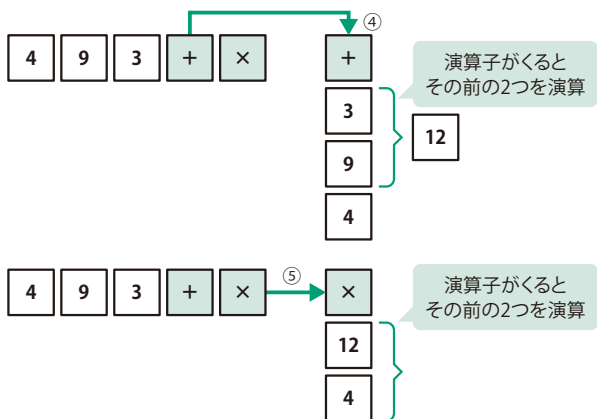
解説動画

$$4 \times (9 + 3)$$

これはコンピュータにとって苦手な記述方法です。人間の場合、一目で () があることがわかるので、() から計算し、その結果と4を掛け算します。しかし、コンピュータは1文字ずつ確認して処理を行うため、人間のようには判断できません。そこで、コンピュータが処理しやすい順番に並べ替える必要があります。それが後置表記法です^{※2}。



※2 人間がプログラミングする際には中置表記法で記述します。これをコンピュータが理解できる「機械語」という言語に変換する過程で後置表記法に置き換わります。



この流れはまさにスタックの性質そのものです。**文字を積み上げていき、演算子が登場したのなら、そこから直前の2つを取り出して計算に使っています。**

表3に記載されている通り、手続initStackでスタックを初期化（すべてクリア）できます。手続pushでスタックに文字を追加し、関数popでスタックから文字を取り出します。また関数topを使うと、一番上に積み上がった文字を得ることができるようです。popは文字を取り出すのに対し、topは文字を参照するだけなので、文字は消えません。

それでは、中置表記法で記述された計算式を後置表記法に変更するプログラムを完成させましょう。

```
10 for (iを0からTextlen - 1まで1ずつ増やす)
```

Textlenには、中置表記法で記述した式の文字数が入っています。「 $4 \times (9 + 3)$ 」の場合、Textlenは「7」で、10行目は「iを0から6まで1ずつ増やす」と解釈します。それでは1文字ずつ処理していきます。変換後の計算式は、1文字ずつ配列Postfixに格納されることを覚えておいてください。

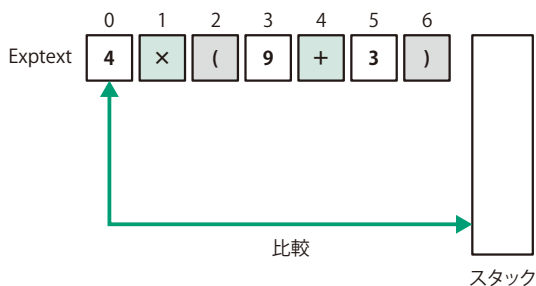
●1文字目

まずiが0、つまり1文字目を処理する前提で見えていきます。

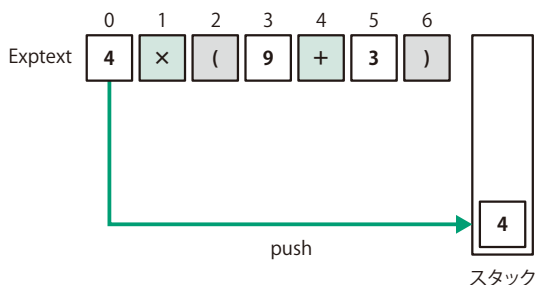
```
11 while (getPriority(Exptext[i]) ≤ getPriority(top()))
12     and top() ≠ "("
```

配列Exptextには、中置表記法の文字が入っています。関数topでは、現在スタックの一番上に積まれている文字が入っています。「中置表記法の1文字目」と「スタックの一

番上の文字」の優先順位を比較し、もし「中置表記法の1文字目」が「スタックの一番上の文字」よりも優先順位が低ければ、13行目に遷移します。しかしスタックは空です。

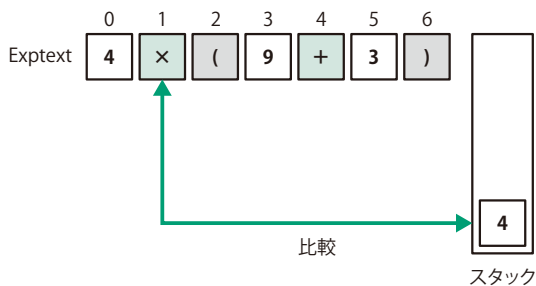


そのためこの繰り返し処理は何もせずに17行目に遷移します。17行目では、1文字目が「)」であるかを確認し、異なっていれば1文字目をスタックに追加します。

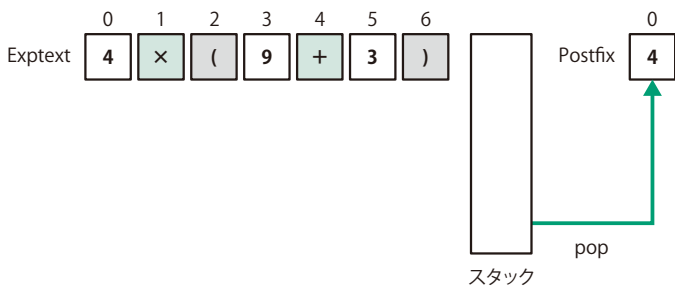


●2文字目

2文字目も同様に、スタックの一番上の文字と優先順位を比較します。

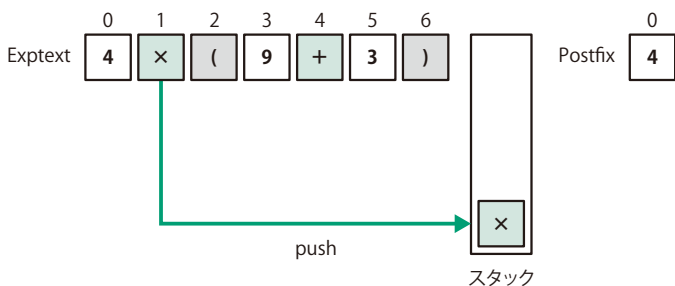


「4」と「x」では、「4」の方が優先順位が高いため、13行目に遷移します。13行目では「4」を配列Postfixに追加します。



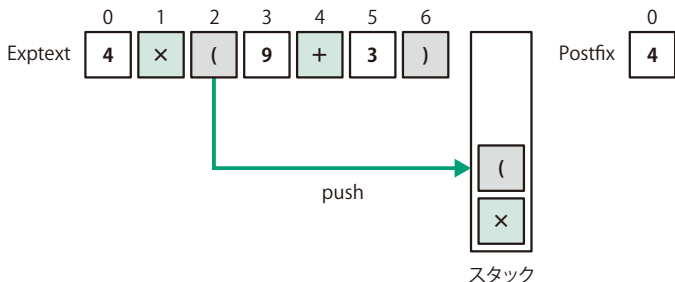
Postfixの要素番号を示す変数kがこのまま変わらないと、どんどんとPostfix[0]が上書きされてしまいます。そこで、この後のどこかでkの値をカウントアップしなければなりません。それが14行目です。aは「 $k \leftarrow k + 1$ 」となります。

スタックが空になったため、先ほどと同様に17行目に遷移します。2文字目「x」は「)」とは異なるため、2文字目をスタックに積み上げます。



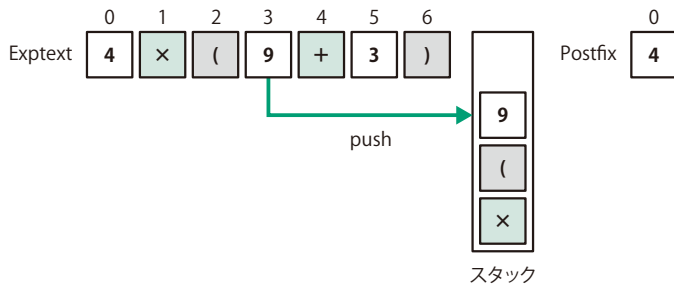
● 3文字目

ここで最も優先順位が高い「(」が登場します。11行目の繰り返し条件を満たさないため、配列Postfixに追加されることなく17行目に遷移します。そして18行目の処理により、スタックに積み上げられます。



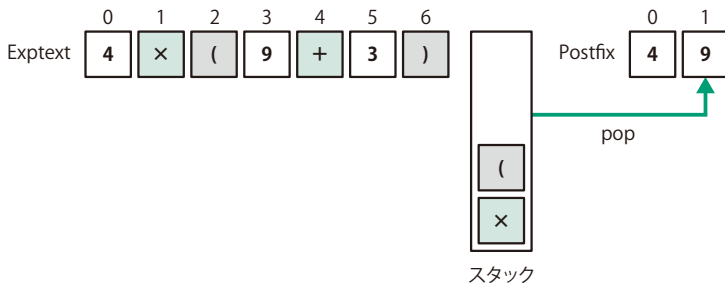
● 4文字目

11行目の繰り返し条件には「top() ≠ "("が含まれています(12行目に記載)。スタックの一番上の文字は「(」であるためこの条件は満たされず、17行目に遷移し、18行目でスタックに4文字目が追加されます。

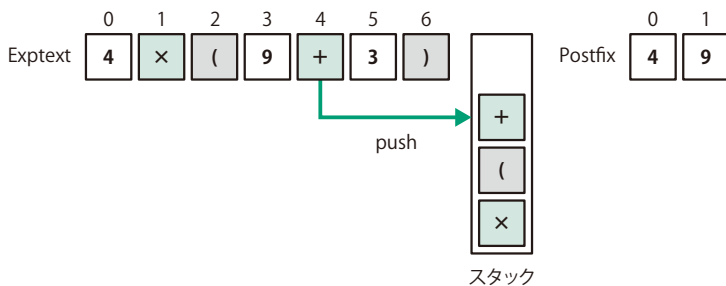


● 5文字目

「+」と「9」では「9」の方が優先順位が高いため、繰り返し条件を満たします。13行目の処理で「9」が配列Postfixに追加されます。

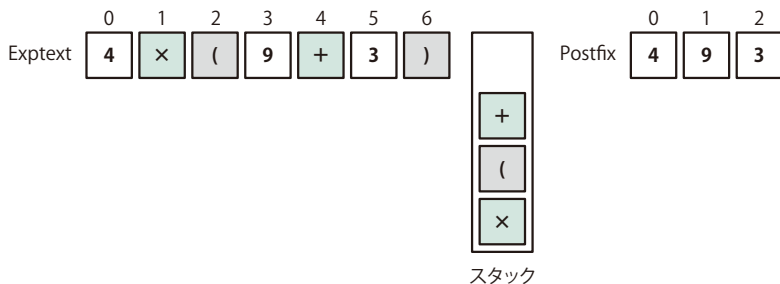


スタックはまだ空ではないため、11行目の繰り返しをさらに続けます。しかし「9」と「(」を比較した場合、「9」の方が大きいため今回もここで繰り返しは終了です。17行目に遷移し、18行目の処理で「+」がスタックに積まれます。



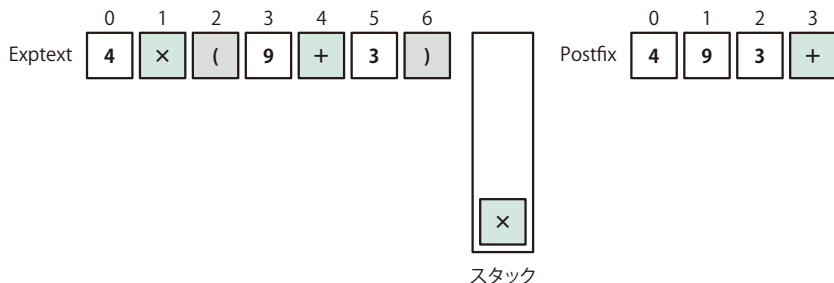
●6文字目

いままで見てきたように、数字はいったんスタックに積むものの、すぐに取り出して Postfix に追加しています。この法則が理解できていれば、トレースを簡略化することができます。Postfix に「3」を追加します。



●7文字目

最後は、優先順位が最も低い「)」です。11 行目の繰り返し条件で「+」との比較を行います。条件を満たすため 13 行目に遷移し、スタックの一番上の文字「+」が Postfix に追加されます。7 文字目は「)」であるため、17 行目の「) ではない」という条件には一致しません。そのため、20 行目の処理でスタックの一番上の文字を削除します。



ここで変数kの値を確認しておきます。aを埋めた結果、14行目でカウントアップします。つまり、配列Postfixに要素を追加するたびにカウントアップするということです。要素は4回追加しているため、この時点で変数kは「4」になっています。

●最終処理

これで10行目から22行目までの繰り返し処理が終了ですが、「×」がスタックに残ったままです。これをPostfixに追加する処理が24行目から33行目までです。

```
24  sflag ← true
25  while (sflag = true)
26    work ← pop()
27    if (b)
28      sflag ← false
29    else
30      Postfix[k] ← work
31      k ← k + 1
32    endif
33  endwhile
```

26行目でスタックにあった「×」がworkに代入されます。このworkの値を、30行目の処理で配列Postfixに追加しなければなりません。そのようになるよう27行目を完成させます。

「work = EOS」の場合で考えてみます。EOSとはスタックの底を示す制御文字です。つまり「スタックが空である」場合に「work = EOS」となります。しかしworkは「×」になっていますからEOSではなく、30行目に遷移します。30行目で「×」が配列Postfixに追加され、正しい状態になりました。さらに31行目でkが「5」にカウントアップされます。

では「work ≠ EOS」の場合で考えてみます。workは「×」になっていますからEOSではなく、28行目に遷移します。sflagがfalseになり25行目の繰り返し条件を満たさず、35行目に遷移してしまいます。まだPostfixが未完成にもかかわらず、繰り返し処理を終えてしまいますので「work ≠ EOS」は誤りです。bは「work = EOS」です。

最後に35行目でPostfixlenに値を代入します。Postfixlenは、配列Postfixの要素数です。変数kはこの時点で「5」になっており、これは配列Postfixの要素数と一致します。特に加減算をせずにそのまま代入すればよいため、cは「k」となります。

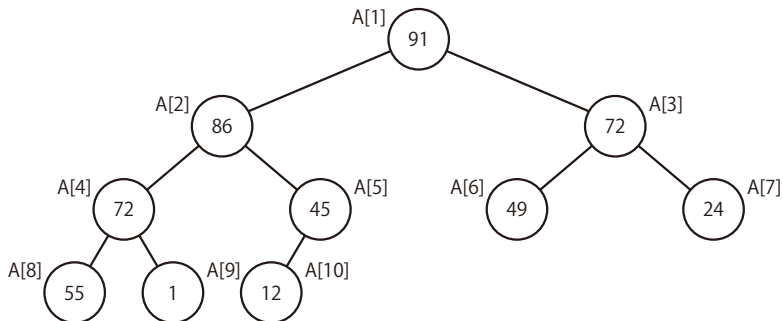
正解 A

次のプログラム中の a 及び b に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まるものとする。

手続HeapSortは、配列に格納されている整数値をヒープソートで昇順に整列する。

- (1) 整列するNum個 ($\text{Num} \geq 2$) の整数値は、大域変数の配列 $A[1]$, $A[2]$, ..., $A[\text{Num}]$ に格納されている。
- (2) ヒープソートは、2分木を用いてデータを整列する。2分木を配列で表現するには、ある節が $A[i]$ に対応するとき、その左側の子の節を $A[2 \times i]$ に、右側の子の節を $A[2 \times i + 1]$ に対応させる。図では、丸が節を、丸中の数字が節の値を示し、実際に値が格納されている配列の要素を丸の脇に示している。
- (3) ヒープとは、図のように、各節の値が自分の子節の値以上になっている2分木である。

図 2分木



- (4) 整列の手順は、次のとおりである。
- ①配列Aの中の、 $A[1]$, $A[2]$, ..., $A[\text{Num}]$ を整列対象とする。
 - ②整列対象の各要素が、(2)で述べたような2分木を表現しているものとして、要素の値を入れ替えてヒープを作る。その結果、木の根 ($A[1]$) には整列対象の要素の最大値が入る。
 - ③木の根 ($A[1]$) の値と、木の最後の節 (整列対象の最後の要素に対応する節) の値とを交換する。
 - ④木の最後の節を2分木から取り除く (整列対象の要素を一つ減らす)。
 - ⑤④の結果が、木の根だけになるまで、②~④の処理を繰り返す。

(5) ヒープを作り直す手順は、次のとおりである。

- ①木の根を親の節とする。
- ②子の節がなければ終了する。
- ③二つの子の節のうち、値が大きい方と親の節の値を比較し、親の節の方が小さいときは、値を交換する。親の節の値が子の節の値以上のときは、何もしないで終了する。
- ④値を交換した子の節を根とする部分木に対して、①～③の処理を繰り返す。

(6) 手順の引数の仕様を表1～4に示す。

表1 HeapSortの引数の仕様

引数名	データ型	入力/出力	意味
Num	整数型	入力	木の最後の節に対応する配列要素の要素番号

表2 InitHeapの引数の仕様

引数名	データ	入力/出力	意味
Last	整数型	入力	最初にヒープを作る木の、最後の節に対応する配列要素の要素番号

表3 MakeHeapの引数の仕様

引数名	データ	入力/出力	意味
Top	整数型	入力	ヒープを作り直す部分木の、木の根に対応する配列要素の要素番号
Last	整数型	入力	ヒープを作り直す部分木の、最後の節に対応する配列要素の要素番号

表4 Swapの引数の仕様

引数名	データ型	入力/出力	意味
X	整数型	入力	A[Y]と交換する配列要素の要素番号
Y	整数型	入力	A[X]と交換する配列要素の要素番号

プログラム

- 1 大域: 整数型の配列: $A \leftarrow \{1000000 \text{ 個の未定義の値}\}$
- 2
- 3 $\text{〇HeapSort(整数型: Num)}$
- 4 整数型: Idx
- 5 InitHeap(Num) // 配列Aの要素を使ってヒープを構成する

```

6   for (IdxをNumから2まで1ずつ減らす)
7       Swap(1, Idx)
8       MakeHeap(1, Idx - 1)
9   endfor
10
11  ○MakeHeap(整数型: Top, 整数型: Last)
12      整数型: L, R
13      a
14       $R \leftarrow L + 1$ 
15      if ( $R \leq \text{Last}$ ) // 3個比較
16          if ( $A[L] < A[R]$ ) // 右が大きい
17              if ( $A[\text{Top}] < A[R]$ )
18                  Swap(Top, R)
19                  MakeHeap(R, Last)
20              endif
21          else // 左が大きい
22              if ( $A[\text{Top}] < A[L]$ )
23                  Swap(Top, L)
24                  MakeHeap(L, Last)
25              endif
26          endif
27      else
28          if (b) // 2個比較
29              if ( $A[\text{Top}] < A[L]$ )
30                  Swap(Top, L)
31                  MakeHeap(L, Last)
32              endif
33          endif
34      endif
35
36  ○Swap(整数型: X, 整数型: Y)
37      整数型: Tmp
38       $\text{Tmp} \leftarrow A[X]$ 
39       $A[X] \leftarrow A[Y]$ 
40       $A[Y] \leftarrow \text{Tmp}$ 

```

解答群

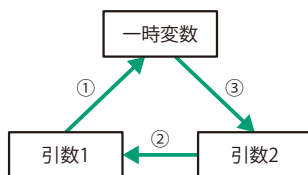
	a	b
ア	$L \leftarrow \text{Top}$	$L \leq \text{Last}$
イ	$L \leftarrow \text{Top}$	$L < \text{Last}$
ウ	$L \leftarrow \text{Top}$	$R \leq \text{Last} - 1$
エ	$L \leftarrow \text{Top}$	$R \leq \text{Last} - 2$
オ	$L \leftarrow \text{Top} + 1$	$L \leq \text{Last}$
カ	$L \leftarrow \text{Top} + 1$	$L < \text{Last}$
キ	$L \leftarrow \text{Top} + 1$	$R \leq \text{Last} - 1$
ク	$L \leftarrow \text{Top} + 1$	$R \leq \text{Last} - 2$
ケ	$L \leftarrow \text{Top} \times 2$	$L \leq \text{Last}$
コ	$L \leftarrow \text{Top} \times 2$	$L < \text{Last}$
サ	$L \leftarrow \text{Top} \times 2$	$R \leq \text{Last} - 1$
シ	$L \leftarrow \text{Top} \times 2$	$R \leq \text{Last} - 2$

解説 手続Swapは、与えられた2つの引数の値を交換する役割を持ちます。

交換処理は、今回のように手続として分離するのではなく、プログラム中に直接記述されるケースが多いと思います。**引数1をいったん一時的な変数に待避しておき、そのうえで引数1を引数2で上書きします。最後に引数2を一時変数で上書きします。**これで入れ替えが完成します。よく使われる方法ですので、覚えておきましょう。



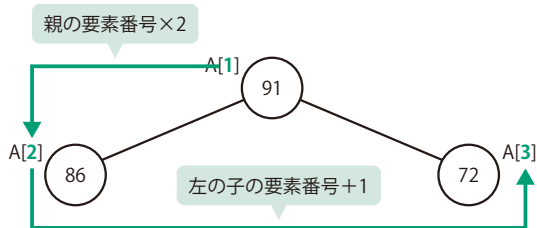
解説動画



●左の子と右の子の要素番号

親子間での要素番号には法則があります。

- ・左の子の要素番号：親の要素番号の2倍
- ・右の子の要素番号：左の子の要素番号に1を加算



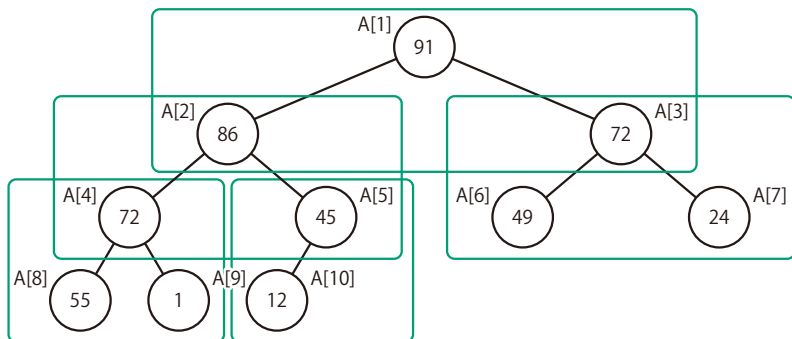
手続MakeHeap内の変数Rには、14行目で「L + 1」の計算結果を代入しています。この変数Rは16行目や17行目で要素番号として使われていることから、節の要素番号であると考えられます。

変数Rはその名前から、「右」の子の要素番号であると推測します。上の図から、「右の子の要素番号」は「左の子の要素番号 + 1」であることがわかるので、変数Lは「左の子の要素番号」だと判断できます。

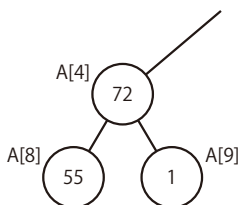
「左の子の要素番号」は「親の要素番号 × 2」です。13行目以前で使うことができる変数はTopとLastだけですので、このどちらかが「親の要素番号」を表現しているはずですが。表3の説明を見ると「Top」は「木の根に対応する」となっており、「Last」は「最後の節」とあります。「親」のことを「木の根」と表現するのは正しいのか気になるところですが（以降で考えます）、2択となると適切なのは「Top」です。したがって、a は「 $L \leftarrow \text{Top} \times 2$ 」が入ります。なお、今回の問題で解答群にあるのは「Top」だけでした。

● MakeHeapでは部分木を扱う

続いて先ほど保留にした「Topの扱い」について考えます。そもそも手続MakeHeapは何を行うものなのでしょうか。表3には手続MakeHeapの引数の説明があります。「Top」「Last」とともに「ヒープを作り直す"部分木"」とあります。部分木は、全体の木の一部分のことですから、例えば次の枠のような範囲です。



このうち、親を「72」とする左下の部分木だけに着目します。この場合MakeHeapの引数Topの要素番号は「4」です。つまり、表3のTopの説明にある「木の根」とは全体の根（要素番号が1）ではなく、部分木の根を意味していることになります。



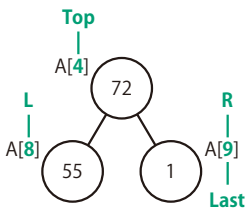
●子の数により処理を分ける

Topの扱いがわかったところで次に進みましょう。15行目のコメント「3個比較」と、28行目のコメント「2個比較」が特徴的です。いったい「3個比較」「2個比較」とはどういう状態なのでしょう。

```
15    if (R ≤ Last) // 3個比較
```

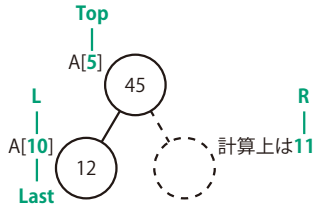
変数Rが変数Last以下の場合には「3個比較」に該当するようです。先ほど見たように変数Topは「部分木における根の要素番号」、変数Rは「その右の子の要素番号」です。さらに変数Lastは説明によると「部分木の、最後の節の要素番号」とのことです。

先ほどの根を「72」とした部分木の最後の節の要素番号は「9」です。これは通常、右の子の要素番号と一致します。



●子が2つの場合の処理

しかし唯一「R」と「Last」が一致しない部分木があります。



変数Rは「L + 1」で計算された値が入りますので、上の図では「11」となっています。一方Lastは「10」です。このように、**右の子が存在しない場合にだけ、LastがRより小さくなる**のです。

15行目の条件「 $R \leq \text{Last}$ 」を満たす場合には、その部分木の根には子が2つ存在していることになります（3個比較）。その際には16行目からの処理で、「親」「左の子」「右の子」の3つの節での整理が行われます。

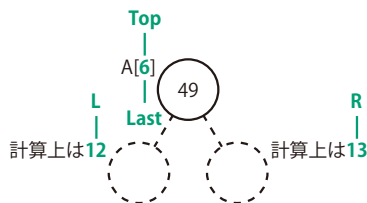
では b を考えます。候補は4つです。

- $L \leq \text{Last}$
- $L < \text{Last}$
- $R \leq \text{Last} - 1$
- $R \leq \text{Last} - 2$

28行目に来たということは、「 $R \leq \text{Last}$ 」を満たしていないということですから、右の子は存在しないということです。そのため、変数Rを使った3番目と4番目は候補から外れます。これで候補は2つに絞られました。

- $L \leq \text{Last}$
- $L < \text{Last}$

15行目のコメントが「3個比較」となっていることを考えると、28行目の条件を満たすことで「2個比較」となるようです。実は部分木の節の数は3つと2つだけではありません。次のように、節が1つの場合もあります。



28行目の条件によって、子を持つか（左だけ）、親だけなのかを判断することになります。候補を1つずつ検証していきます。

仮に `b` に「 $L < \text{Last}$ 」を入れるとします。左に子があると、 L と Last が同じ値になるはずですが「 $L < \text{Last}$ 」では正しく判断してくれません。残った候補は「 $L \leq \text{Last}$ 」です。これで L と Last が同じ値の場合に29行目に遷移するようになりました。

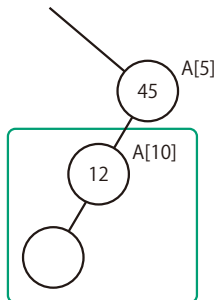
```

28     if ( $L \leq \text{Last}$ ) // 2個比較
29         if ( $A[\text{Top}] < A[L]$ )
30             Swap(Top, L)
31             MakeHeap(L, Last)
32         endif
33     endif
34 endif

```

29行目では値の大小関係を比較し、必要であれば30行目で交換をします。したがって、正解は**ケ**です。

なお、「左の子」がさらに親になっている場合もあるため、31行目で再帰を行い、次のような部分木で MakeHeap を実行します。



今回の問題は、かなり難しかったかと思います。すべてのプログラムを完全に理解する必要はありません。この問題は12択ですが、「これは明らかに違う」と気づける選択肢を削っていき、最終的に2択や3択まで絞ることができたとしたら、それだけでも十分価値があります。満点を取ろうと無理に背伸びする必要はありません。基本情報技術者試験は、6割正解すれば合格できる試験です。そのことを忘れず、できる問題を確実に取る意識を持って取り組んでいきましょう。

正解 **ケ**

次のプログラム中の a 及び b に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は 0 から始まるものとする。

関数 mergeSort は、1 次元配列に連続して格納されている 2^n 個の整数型データ (n は整数で $n > 0$) に対して、併合を n 回繰り返すことによって整列を行う関数である。配列 input の要素を昇順に並べて配列 output に格納する。

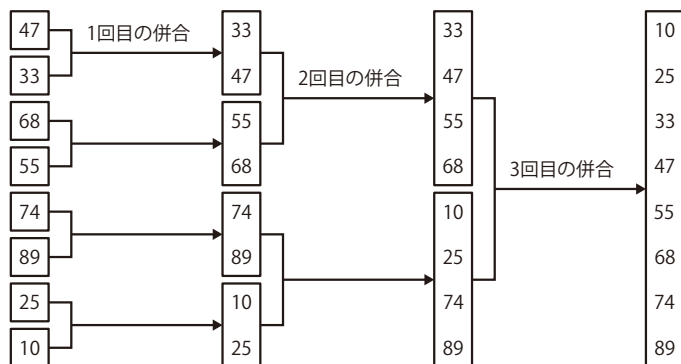
表 mergeSort の引数の仕様

変数	型	入力/出力	意味
input[]	整数型	入力	整列するデータ
output[]	整数型	出力	整列結果を格納する領域
size	整数型	入力	配列の要素数

図は、配列に格納されている 8 個のデータを 3 回の併合で整列する例である。

図 併合による整列の例

入力データ



プログラム

```
1  OmergeSort(整数型の配列: input, 整数型の配列: output, 整数型: size)
2    整数型: span_size, span_idx, write_idx
3    整数型の配列: temp ← {size ÷ 2個の未定義の値}
4    整数型: a_idx, b_idx
5    論理型: a_yet, b_yet
6    span_size ← 2 // 併合対象領域の大きさ
7    output ← input // inputの内容をoutputにすべてコピーする
8    while (span_size ≤ size)
9      span_idx ← 0
10     write_idx ← 0
11     while (span_idx < size)
12       a_idx ← span_idx
13       b_idx ← span_idx + span_size ÷ 2
14       for (iを(a_idx - span_idx)から
15         (b_idx - a_idx - 1)まで1ずつ増やす)
16         temp[i] ← output[i + span_idx]
17       endfor
18       a_yet ← true
19       b_yet ← true
20       while (a)
21         if (b_yet = falseまたは(a_yet = trueかつb_yet = trueかつ
22           temp[a_idx - span_idx] ≤ output[b_idx]))
23           output[write_idx] ← temp[a_idx - span_idx]
24           a_idx ← a_idx + 1
25           if (a_idx ≥ span_idx + span_size ÷ 2)
26             a_yet ← false
27           endif
28         else
29           output[write_idx] ← output[b_idx]
30           b_idx ← b_idx + 1
31           if (b_idx ≥ span_idx + span_size)
32             b_yet ← false
33           endif
34         endif
```

35	b
36	endwhile
37	span_idx ← span_idx + span_size
38	endwhile
39	span_size ← span_size × 2
40	endwhile

解答群

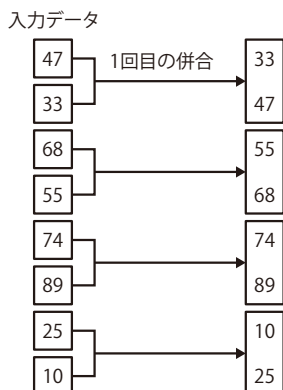
	a	b
ア	a_yet = false and b_yet = false	b_idx ← span_idx
イ	a_yet = false and b_yet = false	write_idx ← b_idx + 1
ウ	a_yet = false and b_yet = false	write_idx ← write_idx + 1
エ	a_yet = false and b_yet = true	b_idx ← span_idx
オ	a_yet = false and b_yet = true	write_idx ← b_idx + 1
カ	a_yet = false and b_yet = true	write_idx ← write_idx + 1
キ	a_yet = true or b_yet = true	b_idx ← span_idx
ク	a_yet = true or b_yet = true	write_idx ← b_idx + 1
ケ	a_yet = true or b_yet = true	write_idx ← write_idx + 1

解説 非常に難しい問題です。すべてのプログラムを完全に理解する必要はありません。あたりをつけてから選択肢を絞り込み、確認のためにトレースをする方法で進めてみます。



解説動画

この問題のテーマは、高速に整列を行うことができるアルゴリズムの**マージソート**です。問題文で説明されている通り、併合（マージ）を繰り返して整列を行います。併合する際に整列を行いますが、この方法がなぜ高速なのかを疑問に感じるかもしれません。まず1回目の併合を見てみましょう。



「47」と「33」の併合では、「33、47」の順になるように整列を行います。このとき比較するのは2つだけですから、ifを使い大小関係を比較しているはずです。該当する箇所をプログラムから探してみましょう。

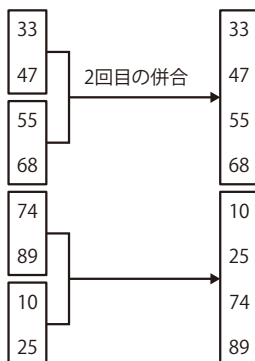
ifは21、22行目と25行目と31行目にあります。26行目と32行目では、それぞれa_yetとb_yetにfalseを設定しているだけですから、おそらく25行目と31行目ではありません。

21、22行目では複雑な条件が記述されています。

- b_yetがfalse
- a_yetとb_yetがtrueで、かつ、temp[a_idx - span_idx]がoutput[b_idx]以下

この段階ではいったい何をしたいのかが不明ですが、とにかくここで大小関係を比較し、もし理想通りになっていないようであれば、小さい順に並べ替えていると思われます。

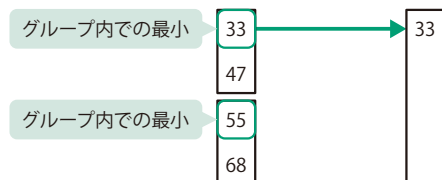
次に2回目の併合の上だけを見てみましょう。



「33」「47」「55」「68」を以下のステップで並べ替えて併合しています。

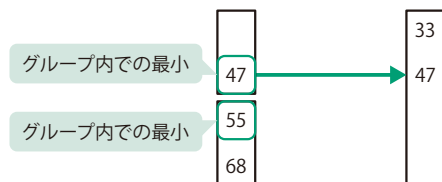
●ステップ1

「33」「55」のうち、小さい方である「33」を右に配置します。「33」「47」のグループ内はすでに1度目の併合で昇順に整列されています。「55」「68」のグループもまた整列済みです。したがって、それぞれのグループの先頭である「33」「55」だけを比較すればよいことになります。「47」「68」は絶対に最小値ではないからです。



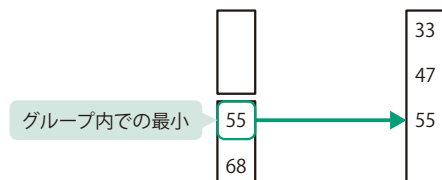
●ステップ2

次に「47」と「55」だけを比較します。同じ理屈で絶対に「68」が次に小さい値ではないためです。「47」が小さいため、これを配置します。



●ステップ3

1つ目のグループから値が消えたので、「55」を無条件で配置します。



●ステップ4

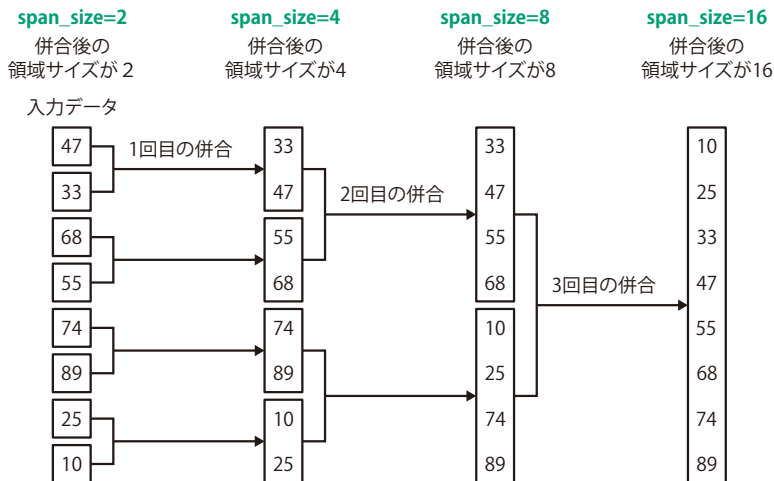
同様に残った「68」を配置し、併合が完了です。これを他でも行います。

この併合が高速である理由は、常に2つの値の大小関係だけを判断しているためです。バブルソートや挿入ソートなどでは、ある値が理想的な状態になるように1つずつ確認する線形探索を行っていますが、マージソートでは線形探索をしていません。

●変数 `span_size` の用途

このプログラムでは特徴的な処理がいくつかありますが、34行目の「`span_size × 2`」に着目してみます。繰り返しごとに、ある数を2倍にしていく処理はかなり特徴的かと思います。

6行目の変数宣言のコメントを見ると「`span_size`」は「併合対象領域の大きさ」とあります。ある領域が2倍ずつ広がっているわけですから、おそらく各グループのことだと思います。ただし6行目で `span_size` は2に初期化されていますので、現在の領域サイズではなく、併合後の領域サイズを表しているようです。



「併合後の領域サイズ」と「要素数」が同じになった場合に繰り返し処理が終了するはず（上の図では3回目の併合が最後）。これは8行目の繰り返しの継続条件と一致します。

```
8 while (span_size ≤ size)
```

●変数 `a_yet` と `b_yet` の用途

次に気になるのは「`a_idx`」「`b_idx`」や「`a_yet`」「`b_yet`」です。なぜ「`c`」や「`d`」はなく、「`a`」「`b`」の2つだけなのでしょう？「2つ」ということから、これは先ほど説明した「2つの値の大小比較」が思いつくはずです。

「`a_idx`」「`b_idx`」は変数名に「`idx`（インデックスの略）」が含まれていることから、それぞれ比較対象となる値の要素番号を示していると考えられます。

また、「`yet`」は「まだ」を意味することから、「`a_yet`」「`b_yet`」はそれぞれのグループ内にまだ処理すべき値が残っているかどうかを示す変数だと推測できます。

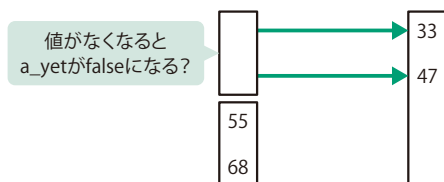
実際にその推測が正しいかどうかを確かめるために、これらの変数がどのような条件で `true` から `false` に変化するのを確認してみましょう。

```

25      if (a_idx ≥ span_idx + span_size ÷ 2)
26          a_yet ← false
27      endif

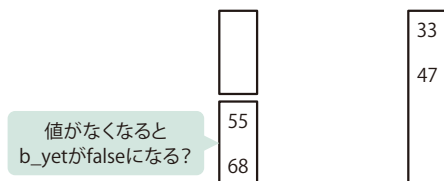
```

「span_idx + span_size ÷ 2」の結果が「a_idx」以下であれば「片方の領域には値がない」と判断されるようです。



span_idxの用途を知らなければ正確なことはわかりません。しかしa_idxが24行目でカウントアップされ、ある値以上になることでa_yetがfalseになることはわかります。やはりa_yetがtrueであれば処理は続き、falseになると処理を終えるようです。その場合 **a** は「a_yetがtrueの間は処理を続ける」を意味する「a_yet = true」になりそうです。しかし解答群に「a_yet = true」はありません。すべての選択肢でb_yetの確認もしています。

b_yetはもう1つのグループの存在を表現しているはずです。



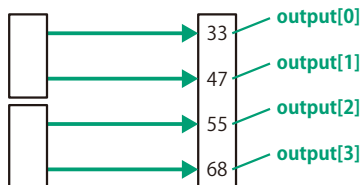
この図の状態では処理はまだ続けなければなりません。つまり**処理を続けるための条件は、a_yetとb_yetのどちらか一方にでも値が残っていることです**。したがって **a** は「a_yet = true or b_yet = true」です。ただa_yetやb_yetが「値がないか」を意味していることはまだ推測です。この後実際にトレースをして確認を得ることにします。

●変数write_idxの用途

次に **b** です。このままではwrite_idxに変化はなく、常に初期値である0のままです。そのため **b** で、write_idxを変化させる必要があります。候補は以下のどちらかです。

- `write_idx ← b_idx + 1`
- `write_idx ← write_idx + 1`

`write_idx`は、配列`output`の要素番号として用いているように見えます。配列`output`は問題文によると「整列結果を格納する領域」です。



マージソートでは、グループ間で大小関係进行比较し、小さい方を配列`output`に配置していきます。その際には単純に「0番目に配置」「1番目に配置」「2番目に配置」……と順に配置します。そのため配列`output`の要素番号として使っている`write_idx`は、単純にカウントアップすればよいことになります。`b`は「`write_idx ← write_idx + 1`」です。

●トレース表を作って検証する

ここまでの考察には、推測に基づく部分が多く含まれていました。この推測が正しいかどうかを確認するために、これからトレース表を作成して検証していきます。

なお、**実際の試験では時間に限りがあるため、ある程度の推測に基づいて解答を導くのもやむを得ない場面があります。**もし余裕があれば、以下のようにトレースを行ってください。

まずは、初期設定などの準備処理を行っている部分を確認します。

プログラム	input	output	size	temp	span_size
1: <code>○mergeSort</code> (整数型の配列: <code>input</code> , 整数型の配列: <code>output</code> , 整数型: <code>size</code>)	47,33,68,55, 74,89,25,10		8		
2: 整数型: <code>span_size</code> , <code>span_idx</code> , <code>write_idx</code>					
3: 整数型の配列: <code>temp</code> ← { <code>size</code> ÷ 2個の未定義の値}				△, △, △, △	
4: 整数型: <code>a_idx</code> , <code>b_idx</code> 5: 論理型: <code>a_yet</code> , <code>b_yet</code>					
6: <code>span_size</code> ← 2					2
7: <code>output</code> ← <code>input</code>		47,33,68,55, 74,89,25,10			

次に8行目からの繰り返し処理に入ります（紙面の都合上、省略している列があります）。

プログラム	temp	span_idx	write_idx	a_idx	b_idx	i
8: while (span_size ≤ size)						
9: span_idx ← 0		0	0			
10: write_idx ← 0						
11: while (span_idx < size)						
12: a_idx ← span_idx				0		
13: b_idx ← span_idx + span_size ÷ 2					1	
14: for (iを(a_idx - span_idx)から						0
15: (b_idx - a_idx - 1)まで1ずつ増やす)						
16: temp[i] ← output[i + span_idx]	47, △, △, △					

いよいよ先ほどの説明で触れたa_yetとb_yetが登場します。

プログラム	a_yet	b_yet
18: a_yet ← true	true	
19: b_yet ← true		true
20: while (a_yet = true or b_yet = true)		
21: if (b_yet = falseまたは(a_yet = trueかつb_yet = trueかつ		
22: temp[a_idx - span_idx] ≤ output[b_idx]))		

21、22行目はかなり複雑な判定条件です。

```

21  if (b_yet = falseまたは(a_yet = trueかつb_yet = trueかつ
22      temp[a_idx - span_idx] ≤ output[b_idx]))

```

b_yetはtrueですから「または」以降だけを抜粋します。

```

21  if (a_yet = trueかつb_yet = trueかつ
22      temp[a_idx - span_idx] ≤ output[b_idx])

```

さらにa_yetとb_yetはtrueですから、最後だけを抜粋します。

```

22  if (temp[a_idx - span_idx] ≤ output[b_idx])

```

各変数の箇所を値で置換してみます。

```

22  if (temp[0 - 0] ≤ output[1])

```


つまり、配列tempの要素番号0の要素と、配列outputの要素番号1の要素の大小関係を比較しています。temp[0]は47で、output[1]は33なので、この条件は満たさず29行目に遷移します。

プログラム	output	b_idx	b_yet
29: output[write_idx] ← output[b_idx]	33,33,68,55,74,89,25,10		
30: b_idx ← b_idx + 1		2	
31: if (b_idx ≥ span_idx + span_size)			
32: b_yet ← false			false

29行目では、output[1]の値でoutput[0]の値を上書きしました。これは、temp[0] (output[0]と同じ値)とoutput[1]を比較し(21、22行目)、output[1]の方が小さかったために、これを先頭に配置した流れです。つまり、2つのグループのうち、2つ目を採用したのです。

入力データ



30行目ではb_idxをカウントアップし「2」になりました。それにより31行目の条件を満たさなくなります。つまりこれは、2つ目のグループに値がなくなったことを意味しています。そこで32行目の処理により、b_yetがfalseとなります。

プログラム	write_idx
35: write_idx ← write_idx + 1	1
20: while (a_yet = true or b_yet = true)	
21: if (b_yet = falseまたは(a_yet = trueかつb_yet = trueかつ 22: temp[a_idx - span_idx] ≤ output[b_idx]))	

35行目でwrite_idxをカウントアップし、2周目の繰り返し処理に入ります。

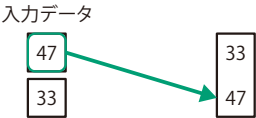
a_yetはtrueなので、20行目の条件は満たされ、21、22行目に遷移します。

さきほど、b_yetがfalseになったため、21、22行目の条件も満たされます。さきほどはここで満たされずに27行目に遷移しましたが、今回は満たされたため23行目に遷移します。

プログラム	output	a_idx	a_yet
23: output[write_idx] ← temp[a_idx - span_idx]	33,47,68,55,74, 89,25,10		
24: a_idx ← a_idx + 1		1	
25: if (a_idx ≥ span_idx + span_size ÷ 2)			
32: a_yet ← false			false

23行目ではoutput[1]をtemp[0]で上書きします。temp[0]は、output[0]の値を待避したものです(15行目)。

32行目の処理でa_yetもfalseとなり、1番目と2番目の併合が完了したことが表現されます。



20行目の「while(a_yet = true or b_yet = true)」は満たさなくなるため繰り返しを終了し、37行目に遷移します。

途中までですが正しい動作となっていることが確認できました。

正解 ケ

Chapter

6

情報セキュリティ

本章の学習ポイント

- 情報セキュリティは物理的、環境的、技術的、運用的側面から対策する必要がある
- マルウェアにはウイルスやワーム、トロイの木馬などさまざまなものがあり、それらの特徴に応じた対策が必要である
- CIAと呼ばれる「機密性」「完全性」「可用性」の確保が特に重要である



01

情報セキュリティの 要求事項



セキュリティの確保に必要な4つの側面を学ぼう

- 建物やサーバ、その他ハードウェアを守るための対策を学ぼう
- 温度や湿度をはじめとするさまざまな環境に対する対策を学ぼう
- ソフトウェアやハードウェアを使った技術的な対策を学ぼう
- 人為的なミスを防ぐためのルール整備について学ぼう

情報セキュリティ要求事項

情報セキュリティと聞くと、マルウェアの対策を行う、パスワードを推測困難なものにするなどが思い付くと思います。もちろんそれらも非常に重要なのですが、情報セキュリティを確保するためには技術的な対策だけでなく、建物や設備、人の行動にまで目を向ける必要があります。ここでは情報セキュリティで求められる4つの側面について学習します。

物理的セキュリティ

物理的セキュリティは、建物やサーバ、ネットワーク機器を守るための対策です。例えば、盗難・破壊・不正侵入を防ぐために行います。物理的セキュリティは、システム全体を守るための、もっとも基本的でありながら見落とされがちな分野です。どのような対策があるのかを覚えておきましょう。

● 入退室管理

サーバ室などの機密性の高いエリアには、ICカードや暗証番号、**生体認証**^{※1}などを使って、許可された人だけが入れるようにします。また誰がいつ入ったかを記録しておくことも大切です。

※1 生体認証はバイOMETRICS認証ともいわれ、指紋、顔、光彩、静脈などを使って認証します。iPhoneではTouchIDやFaceIDが該当します。

● 施錠や監視カメラの設置

サーバラックや保管庫には鍵をかけておきます。さらに監視カメラを設置して録画しておくことで、不正行為の抑止にもなります。この場合、あえて監視カメラが見えるように設置します。

● 災害対策

火災や地震、落雷などの災害にも備えなければなりません。防火対策が施されたラックを使う、免震構造の床を導入することで対策できます。落雷による被害を防ぐためには、避雷設備の設置や雷サージ対策を施した電源タップを使用する方法もあります。

環境的セキュリティ

環境的セキュリティとは、情報機器が置かれている場所や状況によって生じる悪影響から、それらを守るための対策を意味します。

● 温度管理

空調管理サーバは高温や低温に弱いため、室温は一定に保つ必要があります。また、空調設備が停止してしまった場合に備えて、警報装置を設けることも有効です。

● 湿度管理

湿度が高すぎると結露の発生につながり、逆に湿度が低すぎると静電気の発生につながります。これらは機器故障の原因になるため、適切な湿度管理が必要です。

● 無停電電源装置 (UPS)

停電が発生しても**無停電電源装置 (UPS^{※2})**を使うことで、短時間ですが電力供給を継続することができます。これにより、突然の電源断によるサーバの故障を防いだり、処理中のデータが失われたりすることを防ぎます。

● 電源設備の冗長化

電源を複数系統用意しておくことで、片方が故障してももう一方で電源供給を継続できます。

※2 UPSは電源障害時に一時的な電力供給を行い、システムの安全なシャットダウンを可能にします。ただし対応できるのは多くの場合数分程度です。

技術的セキュリティ

技術的セキュリティとは、ソフトウェアやハードウェアを用いて、情報への不正アクセスや情報漏えいを防止するための技術的な対策を指します。

● ファイアウォールの設置

不正な通信を遮断するために、外部ネットワークと内部ネットワークの間に**ファイアウォール**を設置します。これにより、許可されていない通信を防ぐことができます。どの通信を通して、どの通信を通さないかを細かく設定できますが、設定ミスにより、不正な通信を許可してしまう危険があります。

● ウイルス対策ソフトの導入

マルウェアの検出と隔離を行うために、最新の**ウイルス定義ファイル**（**シグネチャファイル**）を適用した対策ソフトを導入します。端末ごとのインストールでは更新や設定に手間がかかるだけでなく、設定ミスなどのリスクもあります。そのため、管理者が一括で制御できる仕組みが有効です。

● アクセス制御の設定

ユーザごとにアクセス可能なフォルダやシステムの範囲を設定します。「**最小権限の原則**」にもとづき、必要最小限の権限のみを与えるのが基本です。

● 多要素認証の導入

多要素認証を導入し、IDとパスワードに加えて、スマートフォン認証やワンタイムパスワードなどを組み合わせることで、本人確認を強化します。なりすましによる不正アクセスを防ぐために、クラウドサービスや外部からの接続において特に重要です。

● 暗号化

機密性の高いデータは、保存時や通信時に暗号化を行います。例えばファイルを暗号化して保存したり、TLSで通信を保護したりします。

技術的セキュリティは、多層的に対策を重ねることが重要です。1つの技術だけではなく、いくつかの技術を組み合わせることを**多層防御**と呼びます。

運用的セキュリティ

運用的セキュリティは、人のミスや不注意で情報が漏れたり、不正利用されたりしないようにルールを整備して対策することです。運用的セキュリティは、技術だけでは防げない「人」に起因するリスクを減らすための重要な取り組みです。

● 情報セキュリティポリシーの策定と周知

組織としての情報セキュリティに関する基本的な方針や行動基準を**情報セキュリティポリシー**として定め、関係者全員に周知することが重要です。例えば「私物のUSBメモリを業務PCに接続しない」「離席時はスクリーンロックを必ず行う」「外部からのメールに添付されたファイルを安易に開かない」といった基本的ルールを明文化しておきます。

● アカウント管理

アカウントの追加や削除、権限の見直しを定期的 to 実施します。退職者や異動者のアカウントが放置されたままになっていると、不正利用の原因になります。

● 操作マニュアルの整備

システムの利用方法や対応手順を明文化し、誰が実施しても同様の対応ができるようにしておきます。これにより運用が標準化され、一定のセキュリティレベルを維持できます。

● セキュリティ教育と訓練

従業員のセキュリティ意識を高めるために、定期的な研修や訓練を実施します。知識だけでなく、緊急時に適切な行動が取れるような実践的な教育が求められます。

● 定期的な運用の見直し

情報セキュリティポリシーや手順書が時代遅れになっていないか、実際の運用とずれていないかを定期的に確認し、必要に応じて更新します。

例題にチャレンジ

例題①

R6 公開問題-問6

A社は従業員450名の商社であり、昨年から働き方改革の一環として、在宅でのテレワークを推進している。A社のシステム環境を図1に示す。

図1 A社システムの環境（抜粋）

- ・従業員には、一人に1台デスクトップPC（以下、社内PCという）を貸与している。
- ・従業員が利用するシステムには、自社で開発しA社に設置している業務システムのほかに、次の二つのSaaS（以下、二つのSaaSをA社利用クラウドサービスという）がある。
 1. メール機能、チャット機能及びクラウドストレージ機能をもつグループウェア（以下、A社利用グループウェアという）
 2. オンライン会議サービス
- ・テレワークでは、従業員の個人所有PC（以下、私有PCという）の業務利用（BYOD）を許可している。
- ・テレワークでは、社内PC及び私有PCのそれぞれに専用のアプリケーションソフトウェア（以下、専用アプリという）を導入し、社内PCのデスクトップから私有PCに画面転送を行うリモートデスクトップ方式を採用している。
- ・専用アプリには、リモートデスクトップからPCへのファイルのダウンロード及びファイル、文字列、画像などのコピー＆ペーストを禁止する機能（以下、保存禁止機能という）があり、A社では私有PCに対して当該機能を有効にしている。
- ・業務システムには、社内PCのデスクトップから利用者ID及びパスワードを入力してログインしている。
- ・A社利用クラウドサービスへのログインは、A社利用クラウドサービス側の設定によってA社の社内ネットワークからだけ可能になるように制限している。ログインには利用者ID及びパスワードを用いている。

テレワークの定着が進むにつれて、社内PCからインターネットへの接続が極端に遅くなり、業務に支障をきたしているので改善できないかと、従業員から問合せがあった。A社の社内ネットワークとインターネットとの間の通信量を調査したところ、テレワーク導入前に比べ、業務時間帯で顕著に増加していることが判明した。そのため、情報システム部では、テレワークでA社利用クラウドサービスに接続する場合には、A社の社内ネットワークも社内PCも介さずに直接接続することを可能にするネットワークの設定変更を実施することにした。

設定変更に当たり、情報セキュリティ上の問題がないかをA社の情報セキュリティリーダーであるBさんが検討したところ、幾つか問題があることが分かった。その一つは、A社利用クラウドサービスへの不正アクセスのリスクが増加することである。そこでBさんは、リスクを低減するために、情報システム部に対策を依頼することにした。

設問

次の対策のうち、情報システム部に依頼することにしたものはどれか。解答群のうち、最も適切なものを選べ。

解答群

- ア A社の社内ネットワークからA社利用クラウドサービスへの通信を監視する。
- イ A社の社内ネットワークとA社利用クラウドサービスとの間の通信速度を制限する。
- ウ A社利用クラウドサービスにA社外から接続する際の認証に2要素認証を導入する。
- エ A社利用クラウドサービスのうち、A社利用グループウェアだけを直接接続の対象とする。
- オ 専用アプリの保存禁止機能を無効にする。

解説 テレワークを行うために社外からクラウドサービスに直接接続できるようにした結果、情報セキュリティ上のリスクが高まった状況が題材です。IDとパスワードが悪意のある第三者に知られてしまった場合、なりすましによる不正ログインが可能になります。このようなリスクに対して有効なのが多要素認証です。

これはパスワードに加えて、ワンタイムパスワードやスマートフォンを使った生体認証の確認など、異なる認証方法を組み合わせることで本人確認を強化します。なお多要素認証のうち、2つの要素だけを使った認証を特に2要素認証といいます。ほかの選択肢は問題の本質的な解決策にはならず、むしろ逆効果となる可能性があります。

- ア 通信を監視することで不正アクセスを検知できますが、防止する効果はありません。不正アクセスを防ぐ施策としては不適切です。
- イ 通信速度を制限することで、ネットワーク負荷を抑えることができますが、不正アクセスそのものを防ぐことはできません。
- エ 一部サービスだけを直接接続の対象にしても、不正アクセスの経路が残るため抜本的な対策にはなりません。
- オ 保存禁止機能を無効化すると個人所有PCへのファイル保存が可能になり、情報漏えいのリスクが高まります。

したがって、2要素認証を導入する**ウ**が正解です。

正解 ウ



02

マルウェアからの保護



マルウェアの種類について学習しよう

- あるマルウェアに対してどのような対策が有効かを理解しよう
- マルウェアの主な侵入経路とその遮断方法について学習しよう

マルウェアとは

マルウェアとは「悪意あるソフトウェア (malicious software)」の総称であり、コンピュータやネットワークに損害を与える目的で作成された不正なプログラムのことです。マルウェアには次のような種類があります。

代表的なマルウェア

種類	説明
ウイルス	他のファイルに寄生して増殖し、ファイルの破壊や改ざんを行う。
ワーム	ネットワークを通じて自己増殖する。
トロイの木馬	正規のソフトに見せかけて不正な動作を行う。
スパイウェア	利用者の情報を収集する。
ランサムウェア	ファイルを暗号化し、復号と引き換えに身代金を要求する。
キーロガー	キーボード入力を記録し、パスワードなどを盗む。
ルートキット	システムの深部に潜伏し、不正なアクセスや制御を可能にするソフトウェアをまとめたパッケージ。
バックドア	攻撃者が外部から侵入するための裏口。
ボット	外部から遠隔操作されるプログラム。
ロジックボム	特定の条件が満たされたときに発動する。
スケアウェア	偽の警告画面を表示して不安をあおり、マルウェアをインストールさせる。

マクロウイルス	ExcelやWordなどのマクロ機能を悪用し、ファイルを開くだけで感染する。
ドロッパー	他のマルウェアをダウンロード・実行するためのプログラム。

● 侵入経路

業務用のPCがマルウェアに感染すると、個人情報や業務データが漏えいする恐れがあります。また感染したPCをそのままにしておくと他のPCにも感染が拡大してしまい、業務が停止するなどの深刻な被害につながる場合があります。どのような経路でマルウェアが侵入してくるのか、代表的なものを押さえておきましょう。

- ・ **フィッシングメール**などの添付ファイルや本文中のリンクを不用意に開く
- ・ 不正なWebサイトへのアクセス
- ・ USBメモリや外付けハードディスクなどの外部媒体
- ・ ソフトウェアやアプリケーションの偽装インストーラ

● 主な対策

マルウェアの被害を防ぐには、次のような工夫が効果的です

- ・ ウイルス対策ソフトの導入と、ウイルス定義ファイルの自動更新
- ・ OSやソフトウェアを常に最新の状態に保つ（**パッチ**の適用）
- ・ 不審なメールや添付ファイル、リンクを不用意に開かない
- ・ 信頼できないソフトウェアや外部媒体を使用しない
- ・ アクセス権限を最小限に制限する（最小権限の原則）

これらの対策は1つだけでなく、組み合わせて実施することが重要です。特に企業や組織では、ユーザ教育もあわせて行う必要があります。

例題にチャレンジ

例題①

R4/12月サンプル問題 - 問18

A社はIT開発を行っている従業員1,000名の企業である。総務部50名、営業部50名で、ほかは開発部に所属している。開発部員の9割は客先に常駐している。現在、A社におけるPCの利用状況は図1のとおりである。

図1 A社におけるPCの利用状況

- 1 A社のPC
 - ・総務部員、営業部員及びA社オフィスに勤務する開発部員には、会社が用意したPC（以下、A社PCという）を一人1台ずつ貸与している。
 - ・客先常駐開発部員には、A社PCを貸与していないが、代わりに客先常駐開発部員がA社オフィスに出社したときに利用するための共用PCを用意している。
- 2 客先常駐開発部員の業務システム利用
 - ・客先常駐開発部員が休暇申請、経費精算などで業務システムを利用するためには共用PCを使う必要がある。
- 3 A社のVPN利用
 - ・A社には、VPNサーバが設置されており、営業部員が出張時にA社PCからインターネット経由で社内ネットワークにVPN接続し、業務システムを利用できるようになっている。規則で、VPN接続にはA社PCを利用することが定められている。

A社では、客先常駐開発部員が業務システムを使うためだけにA社オフィスに出社するのは非効率であると考え、客先常駐開発部員に対して個人所有PCの業務利用（BYOD）とVPN接続の許可を検討することにした。

設問

客先常駐開発部員に、個人所有PCからのVPN接続を許可した場合に、増加する又は新たに生じると考えられるリスクを二つ挙げた組合せは、次のうちどれか。解答群のうち、最も適切なものを選べ。

- (一) VPN接続が増加し、可用性が損なわれるリスク
- (二) 客先常駐開発部員がA社PCを紛失するリスク
- (三) 客先常駐開発部員がフィッシングメールのURLをクリックして個人所有PCがマルウェアに感染するリスク
- (四) 総務部員が個人所有PCをVPN接続するリスク
- (五) マルウェアに感染した個人所有PCが社内ネットワークにVPN接続され、マルウェアが社内ネットワークに拡散するリスク

解答群

- | | |
|------------|------------|
| ア (一), (二) | イ (一), (三) |
| ウ (一), (四) | エ (一), (五) |
| オ (二), (三) | カ (二), (四) |
| キ (二), (五) | ク (三), (四) |
| ケ (三), (五) | コ (四), (五) |

解説 客先常駐の開発部員が個人所有PCから自社にVPN接続する場合、どんなリスクが考えられるでしょうか。(一)～(五)のリスクを見ながら確認していきましょう。

- (一) VPN接続数の増加によりサーバやネットワークの負荷が増加する可能性があり、可用性が損なわれるリスクがあります。なお可用性はセキュリティのCIAの「A」に該当し、セキュリティ確保において非常に重要です。
- (二) この問題の前提となっているPCは個人所有PCであり、A社のPCを貸与しているわけではありません。
- (三) 個人所有のPCが、フィッシングメールによる被害を受ける可能性に変化はありません。
- (四) 設問では「客先常駐開発部員」のBYODを想定しており、総務部員は対象ではありません。
- (五) 感染したPCを通じてA社全体へ被害が及ぶという最悪のケースです。技術的セキュリティ上、もっとも避けるべきリスクのひとつです。

以上より(一)と(五)の組み合わせがもっとも適切です。したがって、正解は**エ**です。

正解 エ



03

バックアップとログ



バックアップの重要性を理解しよう

- フルバックアップ、差分バックアップ、増分バックアップの特徴を学ぼう
- バックアップを行うための注意点について学ぼう
- ログを監視する目的と効果について学ぼう

バックアップとは

バックアップとは、ファイルやデータベースなどのデータやシステムなどのコピーを取っておくことです。システム障害や人的ミス、自然災害、サイバー攻撃などの予期せぬ事態によるデータ喪失に対する対策です。大事なデータが喪失してしまった場合、業務が止まるだけでなく、お客様の信頼も失いかねません。もしデータが喪失しても、バックアップを使って復旧させることが可能です。

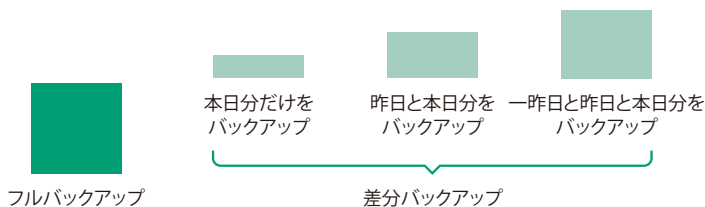
● バックアップの種類

バックアップの種類は大きく分けると、**フルバックアップ**、**差分バックアップ**、**増分バックアップ**の3種です。

フルバックアップは、すべてのデータを丸ごとコピーします。

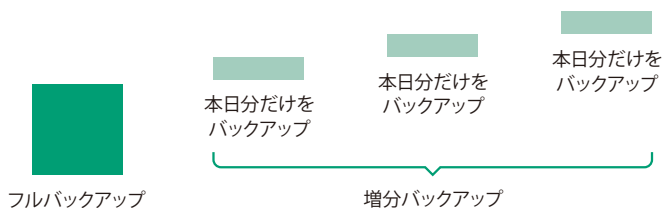


差分バックアップは、前回のフルバックアップ以降に変更された部分だけをバックアップします。復元にはフルバックアップに加えて、差分バックアップも必要です。



1日に1度だけバックアップを行う場合

増分バックアップは、直前のフルバックまたは差分バックアップ以降に変更された部分だけをバックアップします。



バックアップの実施における留意点

バックアップを行う際は、保管方法などを決めて実施する必要があります。

● 保存先の分散

バックアップを同じ装置や建物内に保存すると、災害時にバックアップも含めて失われる恐れがあります。バックアップをした外部メディアを他の場所に保管したり、クラウドにバックアップするなどが有効です。

● 保存期間と世代管理

バックアップは一定期間保持します。世代管理を行うことで、誤って上書きされた場合にも過去の状態に戻すことが可能です。

● バックアップの暗号化とアクセス制御

バックアップデータも情報資産です。盗難や漏えいを防ぐために暗号化やアクセス制限が必要です。

ログの取得と監視

現代の情報システムでは、日々の業務で多くのログが自動的に作られています。実は家庭のPCやスマートフォン、ルーターなどでも、知らないうちに利用履歴やアクセス情報が記録されています。ログはシステムやネットワークの状態を記録したものであり、セキュリティインシデントなどのトラブルが発生した際、原因調査に使用します。ログを取得する目的は主に次の2つです。

● 運用管理

サーバやネットワーク機器の稼働状況、エラー発生状況の把握などに活用されます。例えば、サーバのCPU使用率やメモリ使用率の監視、ネットワークトラフィック量やシステムの稼働時間の監視が挙げられます。

● セキュリティ管理

不正アクセスの痕跡や不審な操作の特定、内部不正の抑止などに活用されます。外部からの不正アクセスを検知する場合は、ファイアウォールのログや**IDS**（侵入検知システム）や**IPS**（侵入防止システム）が通知するアラートが役立ちます。また、組織内部の不正を追跡する場合は、ファイルサーバのアクセスログやメールサーバの送受信ログが重要な証拠となります。

ログの取得と監視における課題と対応策

ログの取得と管理をするうえで、考慮すべき課題があります。

● ログ容量の増加

大規模システムでは1日あたり数十GBから数TB単位のログが生成されることもあり、すぐにディスク容量が足りなくなるリスクがあります。要件によっては数年間にわたるログ保存が求められる場合もあり、その際にはさらに容量管理への配慮が必要になります。

● プライバシーへの配慮

アクセスログや通信履歴には個人情報が含まれることがあり、これらを適切に扱わないとプライバシーの侵害につながります。そのため、ログの取得・利用時には法令や社内規程に従い、慎重に運用する必要があります。

例題にチャレンジ

例題①

H30 秋 情報セキュリティマネジメント試験午後 - 問2改変

A社は、オンラインで物品販売を行う中規模の小売業者である。顧客情報やクレジットカード情報は、ファイルサーバ上で管理している。ファイルサーバのデータは週1回、磁気テープ装置を用いてバックアップされており、テープは2世代分、同じ建物内のキャビネットで保管されている。

昨今、企業を狙ったランサムウェア攻撃が増加しており、A社でもこうした脅威に備える必要がある。ランサムウェアによりファイルサーバ内のデータが暗号化されてしまうと、バックアップデータを用いた復旧が唯一の手段となる場合がある。

先日、社内で情報セキュリティ対策の点検が実施され、その結果、ファイルサーバおよびバックアップに関する以下のようなリスクと対策が情報システム部で整理された。

表1 リスクと対策

No	ファイルサーバとそのデータのバックアップに関するリスク	対策
1	ファイルサーバ上のデータを誤操作で消したり、ランサムウェアによって暗号化されたりした結果、データを利用できなくなるリスク	a
2	ファイルサーバ周辺で火災が発生した結果、データを利用できなくなるリスク	バックアップを2組み取得し、うち1組みを遠隔地に保管する。
3	バックアップの取得が失敗していることに気付かないリスク	b
4	バックアップ対象の設定を誤り、必要なデータのバックアップが取得されないリスク	c

設問

表1中の a, b, c に入れるのに最も適切な3つの選択肢の組合せを、次の解答群の中から順不同で1つ選べ。

- (一) 一時的に構築した情報システムに、バックアップテープの全ファイルをリストアし、ファイル比較ツールを使用してファイルサーバのバックアップ対象ファイルと比較し、ファイルが減っていないことを確認する。
- (二) 現在のバックアップに加え、日次で増分バックアップを行い、増分バックアップを6世代分取得し、世代ごとに別のバックアップテープに保存する。

- (三) テープバックアップ装置を、より高速な製品に交換する。
- (四) バックアップ先の媒体をバックアップテープからハードディスクに変更する。
- (五) バックアップ中にエラーが発生したら電子メールでシステム管理者に通知するツールを導入する。
- (六) バックアップテープをエラーの起きにくい信頼性の高い製品に変更する。
- (七) ファイルサーバに対策ソフトを導入する。
- (八) ファイルサーバのファイル一覧を出力した後、ファイルを全て消去し、バックアップテープのデータをファイルサーバにリストアして出力したファイル一覧と照合し、ファイルが減っていないことを確認する。

解答群

- | | |
|-----------------|-----------------|
| ア (一), (二), (五) | イ (一), (六), (三) |
| ウ (二), (七), (六) | エ (二), (四), (八) |
| オ (三), (五), (七) | カ (三), (六), (八) |
| キ (四), (五), (六) | ク (四), (七), (三) |
| ケ (五), (六), (八) | コ (六), (七), (八) |

解説 この問題は、ファイルサーバのバックアップに関する3つのリスクと、それぞれの対策がテーマです。

No.1では、ファイルサーバ上のデータが誤って削除されたり、ランサムウェアによって暗号化されたりして、利用できなくなるリスクが挙げられています。有効な対策は(二)です。現在の週1回のフルバックアップに加えて、日次の増分バックアップを複数世代分取得することで、過去の任意の時点に遡ってデータを復元できるようになります。

No.3では、バックアップの取得が失敗していても、それに誰も気が付かないリスクが挙げられています。バックアップは自動で行われることが多いため、処理が失敗していることに気が付かないことがあります。このようなリスクに対しては、バックアップ中にエラーが発生した際に管理者へ通知が届くようにすることが有効です。選択肢(五)の対策はメール通知によって失敗をすぐに把握できるため、非常に有効です。

No.4では、バックアップの設定ミスによって、本来取得すべきデータがバックアップされていないリスクが挙げられています。このリスクに対しては、実際のバックアップ内容が正しく取得されているかを検証する仕組みが有効です。選択肢(一)は、一時的な検証環境を用意し、バックアップを復元したうえでファイルサーバ上のデータと比較し、ファイル数が減っていないかを確認する方法です。この対策は、設定ミスによる取得漏れを発見するのに適しています。

したがって、正解は**ア**の(一), (二), (五)の組み合わせです。

正解 **ア**

例題②

H30 秋 情報セキュリティマネジメント試験午後 - 問1改変

F社は従業員70名の商社で、インテリアやギフト用品の仕入れ・販売を行っている。F社には、総務部、企画管理部、商品部、営業部がある。

〔インターネットバンキングサービスの利用〕

F社は、C銀行に口座をもち、C銀行が提供する法人向けインターネットバンキングサービス（以下、IBサービスという）を次の目的で利用している。

- ・自社の口座の残高及び入出金明細の照会
- ・取引先への商品代金の振込、運送業者への輸送費の振込、従業員への給与振込など

〔B社からの問合せ〕

10月2日朝、取引先B社から「先月末入金予定の800万円が未入金」との連絡があった。経理課のL課長が確認したところ、F社の会計記録上は振込済であった。M主任に確認したところ、未入金の理由を把握できなかったが、9月末にB社経理部長から口座変更の連絡を受け取っており、振込先を変更していたことが分かった。

図1 B社の経理部長からのメール

日時 : 2018年9月27日 16:36
 差出人 : YYYYY <YYYYY@interiar-bsha.com>
 宛先 : M@f-sha.com
 CC : ZZZZ@interiar-bsha.com
 件名 : Re: Re: 【ご相談】支払条件の件

M様
 弊社の銀行口座を下記のとおり変更いたしました。
 今月末の入金から新口座宛てにお振り込みをお願いします。
 新口座：〇〇〇銀行〇〇支店 普通 XXXXXXXX 名義 ビーシャ

B社の経理部長からのメールに表示されていたメールアドレスを表1に示す。

表1 B社の経理部長からのメールに表示されていたメールアドレス

役職	図1のメールに表示されていたメールアドレス	正しいメールアドレス
B社の経理部長	YYYYY@interiar-bsha.com	YYYYY@interior-bsha.com
B社の社長	ZZZZ@interiar-bsha.com	ZZZZ@interior-bsha.com

M主任がB社経理部長に確認したところ、B社は口座を変更しておらず、変更通知メールも送っていないかった。また、F社から第三者の口座に代金を振り込んでいたことが判明したため、F社は銀行に連絡するとともに警察へ被害届を提出した。

〔手口と対策〕

後日、警察から、9月末にB社を退職した元従業員を被疑者として逮捕し、犯罪手口に関する供述を得たとの連絡があった。被疑者は指定した口座に振り込ませるよう、偽メールを送信したとのことであった。被疑者は8月にB社の経理部長の手帳からメール受信用のパスワードを盗み見て以来、職場の自分のPCで経理部長のメールを不正に閲覧していた。①B社の情報システム部が自社のログ収集システムに保管していたログからこのことが分かり、被疑者特定の手掛かりになった。

設問

本文中の下線①について、被疑者を特定するために最も有効だったと考えられるものを、解答群の中から選べ。

解答群

- ア B社の経理部長が使っているPCで記録されたメール送受信ログ
- イ B社のメールサーバで記録されたメールクライアントソフトからの大量のログイン失敗ログ
- ウ B社のメールサーバで記録されたメールクライアントソフトからのメール受信要求ログ
- エ B社のメールサーバで記録されたメールクライアントソフトからのメール送信ログ
- オ 被疑者が自宅で使っていた個人所有のPCで記録された操作ログ

解説 B社のメールサーバには受信用パスワードが設定されていましたが、被疑者はそれを不正入手しています。それによりパスワード認証をクリアしたため、経理部長になりすましてメール送信が可能でした。この問題では被疑者がメールを不正閲覧していたことを、どのようにして知ることができたかを答えます。

- ア 被疑者は自分の職場PCで不正閲覧していたため、経理部長のPCのログでは証拠となりません。
- イ 大量のログイン失敗ログは、ブルートフォース攻撃^{※3}などの痕跡を示すものですが、今回のケースでは手帳からパスワードを入手しています。
- エ 送信ログは偽メール送信の証拠になりますが、設問では不正閲覧を知った理由が問われています。
- オ 被疑者は職場PCで不正を行っているため、個人所有PCのログは無関係です。

正解はウです。メール受信要求ログには、誰がどの端末から、いつメールを閲覧したかの情報が記録されており、被疑者特定の手掛かりとなります。

正解 ウ

※3 パスワードを片っ端から試す攻撃です。

情報の転送における 情報セキュリティの維持



ネットワークセキュリティについて学習しよう

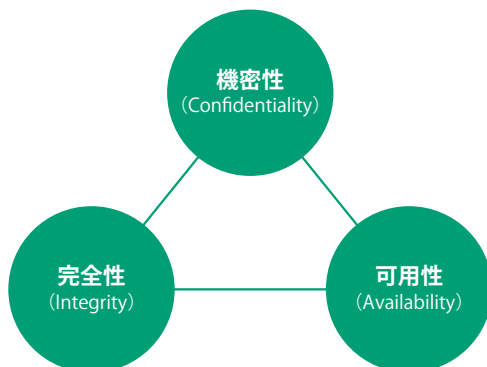
- セキュリティ確保のためのCIAについて学ぼう
- ネットワークにおける暗号化について学ぼう
- ネットワーク運用に関する最適解を学ぼう

安全な情報転送のための技術

インターネットや社内ネットワークで情報を送るときは、常に盗聴・改ざん・なりすましなどの危険にさらされています。特にクラウドサービスやテレワークの普及により、社外との通信が増加し、従来よりも大きなセキュリティリスクが存在するようになりました。

情報セキュリティでは、次の基本原則を守ることが重要です。

- **機密性**：情報を許可された人だけがアクセスできるようにすること
- **完全性**：情報が正確で改ざんされないようにすること
- **可用性**：必要なときに情報やシステムが利用できること



これら3つの英語表記の頭文字をとって、**CIA**と呼ぶこともあります。CIAを守るためには、次のような対策が必要です。

● 暗号化技術

通信内容を暗号化することで、盗聴による情報漏えいを防ぎます。代表的な技術として、**TLS**(Transport Layer Security)があります。TLSは通信経路上のデータを暗号化します。TLSはWebブラウザとWebサーバ間の**HTTPS**通信、メール送受信時の**SMTPTS**や**IMAPS**、ファイル転送の**FTPS**や**SFTP**などに使われている技術です。

● VPN (Virtual Private Network)

VPN^{※4}は、インターネット上に仮想の専用線を構築する技術です。VPNでは**IPsec**などのプロトコルを用いることで、データの暗号化や認証が行われ、遠隔地から社内ネットワークへ安全にアクセスできるようになります。

● デジタル証明書

デジタル証明書は、通信相手の正当性を保証するために利用されます。これにより、なりすまし攻撃を防止できます。企業内では電子メールの送受信に**S/MIME**を活用するケースもあります。さらに、HTTPSにおけるWebサイトの認証や、VPN接続時のクライアント認証などにも利用され、安全性を確保しています。

情報転送における注意点

安全な情報転送を実現するための技術を利用するには、適切な運用が求められます。注意すべき点や、関連する用語を見ておきましょう。

● 暗号化の運用管理

すべての通信を暗号化するには、証明書を適切に管理する必要があります。証明書の期限切れや設定ミスはセキュリティの低下を招くため、定期的な確認が重要です。

※4 もっとも安全なのは専用線なのですが、敷設に膨大なコストがかかります。そこで既存のインターネット回線を使って、専用線のように安全な通信を実現するのがVPNです。

● 性能低下

VPNや暗号化はセキュリティを高めますが、通信遅延や処理負荷を引き起こすことがあります。ハードウェアやネットワーク設計で対策することが可能です。例えば、暗号化処理に対応した専用ハードウェアの導入や、**負荷分散装置（ロードバランサー）**を用いて処理を分散させる方法があります。

● ユーザ教育の必要性

安全な転送手段を用意しても、利用者が適切に使えなければ意味がありません。フィッシング詐欺などを防ぐためにも、ユーザへの啓発が重要です。こうした取り組みは、情報セキュリティ委員会やCSIRTなどの社内組織が中心となって進めていきます。

● 個人情報提供時の情報加工

外部業者へ、自社で管理している個人情報を提供する際は、個人を特定できないように情報を加工する必要があります。これは**個人情報保護法**にも**匿名加工情報**として規定されています。適切に加工することで、事業者は本人の同意なしで第三者提供が可能になります。

● 情報セキュリティ委員会

情報セキュリティ委員会は、組織全体の情報セキュリティ対策を統括する意思決定機関のことです。情報セキュリティポリシーの策定、リスク評価、教育・啓発活動の企画などを行います。通常、最高情報セキュリティ責任者（**CISO**^{※5}）が委員長を務めます。

● CSIRT

CSIRTはComputer Security Incident Response Teamの略称で、情報セキュリティインシデントの対応を専門に行うチームのことです。インシデントの検知、対応、原因分析、再発防止策の検討などを担当します。インシデント発生時、円滑な対応を行うためには、CSIRTを設置しておいたほうがよいでしょう。

※5 Chief Information Security Officerの略称です。

例題にチャレンジ

例題①

オリジナル問題

C社は、従業員120名の電気機器メーカーであり、国内外の企業向けにネットワーク機器を製造・販売している。C社は直販サイト（以下、C社ECサイトという）を運営しており、取引先とのファイル共有や注文管理に利用している。C社ECサイトは10年前に開設され、現在では売上の約6割を占めている。登録ユーザ数は50万件を超えている。

C社には総務部、開発部、営業部、情報システム部、カスタマーサポート部の5つの部門があり、このうち情報システム部がECサイトの運営と情報セキュリティ対策を担当している。

〔トラブル発生〕

C社では、社員が社外から社内ネットワークに接続できるようにVPNを導入していた。VPN接続にはID・パスワード認証のみを使用していたが、最近、社員のID・パスワードが漏洩し、攻撃者が正規社員になりすましてVPNに接続した。その結果、社内システムへの不正アクセスが発生し、社外秘の設計データが外部に持ち出される被害が発生した。

〔対応〕

情報システム部は、再発防止策としてVPN接続における認証方法を強化することを検討した。複数の案が提示され、その中から最も適切な対策を選定することになった。

設問

C社がVPN接続の認証方法を強化するために採用すべき対策として、最も適切なものはどれか。

解答群

- ア VPNサーバの接続ログを定期的に監視し、不審なログインを検知した場合は手動で対応する。
- イ VPN接続時に社員PCにインストールされたクライアント証明書による端末認証を追加し、ID・パスワード認証と組み合わせる。
- ウ 社員PCに最新のアンチウイルスソフトをインストールし、マルウェア感染リスクを低減する。
- エ 社員PCにVPNクライアントソフトをインストールし、特定のVPNサーバのみ接続可能とする設定を適用する。
- オ VPN経由の通信速度を制限し、不正利用による被害を最小化する。

解説 VPN (Virtual Private Network) は、インターネットなどの公衆回線を利用して安全な通信経路を確立する技術です。これにより、社外からでも社内ネットワークに安全にアクセスできるようになります。しかし、VPNの認証がIDとパスワードのみの場合、それらが漏えいしたときに不正アクセスされるリスクがあります。

このリスクに対処するため、より強固な認証方式が推奨されます。そのひとつが**クライアント証明書**による端末の認証です。社員が利用するPCにクライアント証明書をインストールし、VPNサーバが接続時にその証明書の有効性を確認することで、承認された端末以外からの接続を拒否できるようになります。

- ア** ログ監視は不正アクセスの検知には有効ですが、根本的な防止策にはなりません。実際に侵入されてから気付くことになるため、被害の発生を完全には防げません。
- イ** クライアント証明書は端末ごとに発行され、PCにインストールして使用します。VPNサーバは、接続時にクライアント証明書の有効性を確認します。これにより、たとえID・パスワードが漏えいしても、証明書のない端末では接続できず、不正アクセスを防止できます。
- ウ** アンチウイルスソフトは端末のマルウェア感染防止には有効ですが、VPN接続時の認証強化には意味がありません。
- エ** WindowsやMacには標準でVPNクライアント機能が搭載されているため、専用ソフトのインストール自体は認証の強化にはなりません。
- オ** 通信速度の制限ではID・パスワードの不正利用を阻止することはできず、また速度低下によって正規の業務にも支障が出る恐れがあります。

正解 **イ**



05

脆弱性管理と アクセス管理



システムの欠陥と権限について学ぼう

- 脆弱性を悪用した手口と対策について学ぼう
- 適切なアクセス権限の設定方針について学習しよう
- 定期的なアクセス権限のチェック方法を理解しよう

脆弱性管理

脆弱性とは、システムやソフトウェアに存在する不備や欠陥です。これを攻撃者に悪用されることで、情報漏えいやサービス停止などの被害を引き起こすことがあります。この脆弱性を特定してから対処するまでの一連のプロセスを脆弱性管理といいます。サイバー攻撃を未然に防ぐためにも、脆弱性管理は情報セキュリティにおいてとても重要です。

● 脆弱性管理のプロセス

脆弱性管理のプロセスは、大きく情報入手→評価→対処に分けられ、これを継続的に行う必要があります。



最初のステップは、使っているシステムに関する脆弱性の情報を集めることです。OSやミドルウェアの開発者からの公開情報、**JVN** (Japan Vulnerability Notes^{※6}) のような脆弱性情報サイトが主な情報源です。次のように、JVNには脆弱性に関する記事が多く掲載されています。

※6 JVNは日本の脆弱性情報データベースで、情報処理技術者試験を主催しているIPAとJPCERT/CCが運営しています。

The screenshot shows the JVN website interface. The main content area displays a list of vulnerability entries with columns for JVN ID, product name, vulnerability description, and date. The right sidebar contains links to various resources like HOME, JVNとは, and filters for vulnerability status and severity.

<https://jvn.jp/>

次に、発見された脆弱性のリスクを評価します。**CVSS**（共通脆弱性評価システム）のスコアを参考にし、影響の大きさや緊急性に応じて、対応の優先順位を決めます。

優先順位を決めたあとは、優先度の高い順番に対応していきます。緊急性の高い脆弱性は速やかに対応する、低リスクのものについては定期メンテナンス時に対応するなど、優先順位によって対応方針が変わります。具体的な対応としては、開発者が公開したパッチの適用が一般的です。ただしパッチがまだ提供されていない場合には、設定を変えたり、サービスを一時停止したり、IDS/IPSを入れるなどの暫定的な対策が必要です。

また、定期的な脆弱性診断も重要です。診断にはWebアプリケーション診断とプラットフォーム診断があります。Webアプリケーション診断は、**SQLインジェクション**^{※7}や**クロスサイトスクリプティング (XSS)**^{※8}など、Webアプリケーションに関わる脆弱性について診断を行います。プラットフォーム診断は、OSやミドルウェア、ネットワーク機器などの脆弱性を検出するために行います。これらの診断は、システムの導入時や大規模な更新時に加え、定期的実施することが望ましいとされています。

※7 悪意あるSQL文を送り込み、データベースを不正操作する攻撃です。

※8 Webサイトの脆弱性を突き、悪意あるスクリプトを実行させる攻撃です。

● 脆弱性診断基準

脆弱性管理においては、組織が定めた脆弱性診断基準を遵守することが重要です。基準では、診断の実施タイミングや頻度、結果の報告方法、対応期限などが定められており、これに従うことで全社的なセキュリティレベルを維持できます。また、委託先が基準に従って脆弱性診断を実施しているかどうかは定期的に確認する必要があります。特に外部委託では、契約書に基準遵守の義務を明記し、進捗状況をモニタリングすることが有効です。

アクセス管理

情報システムの安全性を維持するためには、利用者ごとに適切なアクセス権限を設定することが重要です。業務に必要な権限を与えたままにすると、内部不正や操作ミスによる情報漏えいのリスクが高まります。そのためシステム管理者は、業務に合わせて必要最低限の権限だけを付与する「最小権限の原則」を意識する必要があります。

● アクセス権限管理の重要性

アクセス権限管理には、大きく2つの側面があります。

1つは利用者認証です。これは正当な利用者だけがシステムにアクセスできるようにする仕組みで、パスワード認証や多要素認証（異なる認証要素を組み合わせた認証方式）が代表例です。もう1つは利用者ごとの権限制御です。利用者ごとに「閲覧のみ」「入力可能」「承認権限あり」のような操作レベルを細かく設定します。

過剰な権限付与は大きなリスクになります。例えば、業務上閲覧だけで十分な利用者に対して入力権限を付与してしまうと、意図しないデータ追加が発生する可能性があります。これを防ぐためにも、最小権限の原則を適用することが重要です。

● アクセス権限の管理

権限は一度付与すれば終わりではなく、適切なタイミングで見直しを行うことが重要です。例えば、利用者が退職した場合には、アカウントの無効化や権限の削除などを速やかに行う必要があります。これを忘れると、辞めた人がシステムに入れてしまい、不正や情報漏えいの危険が高まります。また異動時も、異動先の業務に不要な権限は削除し、新しい業務に必要な権限のみを付与します。

さらに定期的な権限レビューを実施します。これは、付与されている権限が現状の業務内容と一致しているかを確認するプロセスです。長期間放置された不要な権限は、重大なセキュリティインシデントの原因となることがあります。定期的な見直しによって、こうしたリスクを未然に防ぐことができます。

運用状況の点検

情報システムの運用ルールは、一度決めたとしてもそれで終わりではありません。組織の成長やシステムの変更、さらに外部環境の変化（法律改正など）によって、当初のルールが形骸化したり、新たなリスクが発生したりすることがあります。そのため、定期的に運用状況を確認し、情報セキュリティポリシーや規程が現場で適切に守られているかチェックすることが重要です。

運用状況の点検では、次のようなポイントを見ていきます。

● 情報セキュリティポリシーの遵守

状況現場の担当者が定められた手順に従って運用しているか、ガイドラインに沿った対応ができているかを確認します。例えば、不要な利用者アカウントが残ったままになっているケースなどが、問題として指摘されることになります。

● 監視・測定・分析・評価

ログの監視、インシデント対応の記録、バックアップの取得などが適切に行われているかを確認します。単にログが保存されているだけでなく、適切に分析され、不審な動きがあれば迅速に対応しているかが重要です。

● 脆弱性・侵入検査

システムに対して定期的に脆弱性を確認し、未知のリスクがないかを調べます。必要に応じて**ペネトレーションテスト**（侵入検査）※⁹を行い、外部からの攻撃にどの程度耐えられるかも確認します。

こうした点検により、組織が「ルール通りに運用できているかどうか」を具体的に把握できます。点検や監査で問題が見つかった場合は、上位者に報告して相談し、改善策を講じることが求められます。

※⁹ ペネトレーションテストは実際に攻撃を試みることで防御力を確認する方法です。

例題にチャレンジ

例題①

R4/4月情報セキュリティマネジメント試験サンプル問題-問58

国内外に複数の子会社をもつA社では、インターネットに公開するWebサイトについて、A社グループの脆弱性診断基準（以下、A社グループ基準という）を設けている。A社の子会社であるB社は、会員向けに製品を販売するWebサイト（以下、B社サイトという）を運営している。B社サイトは、会員だけがB社の製品やサービスを検索できる。会員の氏名、メールアドレスなどの会員情報も管理している。

B社では、11月に情報セキュリティ活動の一環として、A社グループ基準を基に自己点検を実施し、その結果を表1のとおりまとめた。

表1 B社自己点検結果（抜粋）

項番	点検項目	A社グループ基準	点検結果
(一)	Webアプリケーションプログラム（以下、Webアプリという）に対する脆弱性診断の実施	- インターネットに公開しているWebサイトについて、Webアプリの新規開発時、及び機能追加時に行う。 - 機能追加などの変更がない場合でも、年1回以上行う。	3年前にB社サイトをリリースする1か月前に、Webアプリに対する脆弱性診断を行った。リリース以降は実施していない。 3年前の脆弱性診断では、軽微な脆弱性が2件検出された。
(二)	OS及びミドルウェアに対する脆弱性診断の実施	インターネットに公開しているWebサイトについて、年1回以上行う。	- 毎年4月及び10月に、B社サイトに対して行っている。 - 今年4月の脆弱性診断では、脆弱性が3件検出された。
(三)	脆弱性診断結果の報告	Webアプリ、OS及びミドルウェアに対する脆弱性診断を行った場合、その結果を、診断後2か月以内に各社の情報セキュリティ委員会に報告する。	3年前にWebアプリに対する脆弱性診断を行った2週間後に、結果を情報セキュリティ委員会に報告した。 - OS及びミドルウェアに対する脆弱性診断の結果は、4月と10月それぞれの月末の情報セキュリティ委員会に報告した。

(四)	脆弱性診断結果の対応	• Webアプリ、OS及びミドルウェアに対する脆弱性診断で、脆弱性が発見された場合、緊急を要する脆弱性については、速やかに対応し、その他の脆弱性については、診断後1か月以内に対応する。指定された期限までの対応が困難な場合、対応の時期を明確にし、最高情報セキュリティ責任者(CISO)の承認を得る。	• 3年前に検出したWebアプリの脆弱性2件について、B社サイトのリリースの1週間前に対応した。 • 今年4月に検出したOS及びミドルウェアに対する脆弱性のうち、2件は翌日に対応した。残り1件は、恒久的な対策は来年1月のB社サイトの更改時に対応するものとし、それまでは設定変更による暫定対策をとるという対応計画について、脆弱性診断の10日後にCISOの承認を得た。
-----	------------	--	---

設問

表1中の自己点検の結果のうち、A社グループ基準を満たす項番だけを全て挙げた組合せを、解答群の中から選べ。

解答群

- ア (一), (二)
- イ (一), (二), (三)
- ウ (一), (二), (三), (四)
- エ (一), (二), (四)
- オ (一), (三), (四)
- カ (一), (四)
- キ (二), (三)
- ク (二), (三), (四)
- ケ (三), (四)

解説 表1の各項目を確認していきましょう。

A社グループの脆弱性診断基準では、Webアプリケーションについてはインターネットに公開しているWebサイトで、新規開発や機能追加時に脆弱性診断を行うことになっています。また変更がない場合でも、年1回以上の脆弱性診断を行う基準になっています。しかしB社では、3年前のリリースの1か月前に診断を行った後は実施しておらず、(一)は満たしていません。

OS及びミドルウェアについては毎年4月と10月に診断を実施しており、年1回以上という基準を満たしていますので、(二)は問題ありません。

次に(三)の診断結果の報告についてです。Webアプリケーション診断の結果を2週間後に、OSとミドルウェアの診断結果を4月・10月末に情報セキュリティ委員会に報告しており、いずれも基準の「2か月以内」に該当しています。(三)も満たしています。

最後に(四)の診断結果への対応についてです。緊急性の高い脆弱性は迅速に対応し、その他の脆弱性も期限内に対処しています。また恒久対策が困難なものはCISOの承認を得て暫定対応しています。(四)も基準を満たしています。

以上のことから、基準を満たす項目は(二)、(三)、(四)であり、正解はクです。

正解 ク

例題②

R4/12月サンプル問題-問19

A社は従業員200名の通信販売業者である。一般消費者向けに生活雑貨、ギフト商品などの販売を手掛けている。取扱商品の一つである商品Zは、Z販売課が担当している。

〔Z販売課の業務〕

現在、Z販売課の要員は、商品Zについての受注管理業務及び問合せ対応業務を行っている。商品Zについての受注管理業務の手順を図1に示す。

図1 受注管理業務の手順

商品Zの顧客からの注文は電子メールで届く。

(1) 入力

販売担当者は、届いた注文(変更、キャンセルを含む)の内容を受注管理システム¹⁾(以下、Jシステムという)に入力し、販売責任者²⁾に承認を依頼する。

(2) 承認

販売責任者は、注文の内容とJシステムへの入力結果を突き合わせて確認し、問題がなければ承認する。問題があれば差し戻す。

注1 A社情報システム部が運用している。利用者は、販売責任者、販売担当者などである。

注2 Z販売課の課長1名だけである。

〔Jシステムの操作権限〕

Z販売課では、Jシステムについて、次の利用方針を定めている。

〔方針1〕ある利用者が入力した情報は、別の利用者が承認する。

〔方針2〕販売責任者は、Z販売課の全業務の情報を閲覧できる。

Jシステムでは、業務上必要な操作権限を利用者に与える機能が実装されている。

この度、商品Zの受注管理業務が受注増によって増えていることから、B社に一部を委託することにした（以下、商品Zの受注管理業務の入力作業を行うB社従業員を商品ZのB社販売担当者といい、商品ZのB社販売担当者の入力結果を閲覧して、不備があればA社に口頭で差戻しを依頼するB社従業員を商品ZのB社販売責任者という）。

委託に当たって、Z販売課は情報システム部にJシステムに関する次の要求事項を伝えた。

[要求1] B社が入力した場合は、A社が承認する。

[要求2] A社の販売担当者が入力した場合は、現状どおりにA社の販売責任者が承認する。

上記を踏まえ、情報システム部は今後の各利用者に付与される操作権限を表1にまとめ、Z販売課の情報セキュリティリーダーであるCさんに確認してもらった。

表1 操作権限案

利用者 \ 付与される操作権限	Jシステム		
	閲覧	入力	承認
(省略)	○		○
Z販売課の販売担当者	(省略)	(省略)	(省略)
a1	○		
a2	○	○	

注記 ○は、操作権限が付与されることを示す。

設問

表1中の「a1」, 「a2」に入れる字句の適切な組合せを、解答群から選べ。

解答群

	a1	a2
ア	Z販売課の販売責任者	商品ZのB社販売責任者
イ	Z販売課の販売責任者	商品ZのB社販売担当者
ウ	商品ZのB社販売責任者	Z販売課の販売責任者
エ	商品ZのB社販売責任者	商品ZのB社販売担当者
オ	商品ZのB社販売担当者	商品ZのB社販売責任者

解説 この問題では、A社とB社が一緒に受注管理をする中で、誰にJシステムの承認権限を持たせるかを考えます。

まず閲覧権限しかない利用者は誰かを検討します。「Z販売課の販売責任者」は、A社の販売担当者の入力データを承認する立場にありますので、閲覧権限のみではありません。したがって、**ア**と**イ**は誤りです。「商品ZのB社販売担当者」はA社から委託を受けてデータを入力する作業を行いますので、これも誤りです。**a1**には「商品ZのB社販売責任者」が入ります。この利用者はB社の販売担当者が入力したデータを閲覧し、不備があればA社に差戻しを依頼する役割を担います。**ウ**と**エ**のどちらかが正解です。

次に閲覧と入力ができる利用者を探します。候補は「Z販売課の販売責任者」と「商品ZのB社販売担当者」のどちらかです。このうち「商品ZのB社販売担当者」は「要求1」により、入力権限が必要です。「Z販売課の販売責任者」については、問題文に入力作業を行う記載がないため、入力権限はないと思われます。いずれにしても「商品ZのB社販売担当者」は入力権限が必要なので、正解は**エ**です。

正解 **エ**

例題③

R4/12月サンプル問題-問20

A社は栄養補助食品を扱う従業員500名の企業である。A社のサーバ及びファイアウォール（以下、FWという）を含む情報システムの運用は情報システム部が担当している。

ある日、内部監査部の監査があり、FWの運用状況について情報システム部のB部長が図1のとおり説明したところ、表1に示す指摘を受けた。

図1 FWの運用状況

- FWを含め、情報システムの運用は、情報システム部の運用チームに所属する6名の運用担当者が担当している。
- FWの運用には、FWルールの編集、操作ログの確認、並びに編集後のFWルールの確認及び操作の承認（以下、編集後のFWルールの確認及び操作の承認を操作承認という）の三つがある。
- FWルールの編集は事前に作成された操作指示書に従って行う。
- FWの機能には、FWルールの編集、操作ログの確認、及び操作承認の三つがある。
- FWルールの変更には、FWルールの編集と操作承認の両方が必要である。操作承認の前に操作ログの確認を行う。
- FWの利用者IDは各運用担当者に個別に発行されており、利用者IDの共用はしていない。
- FWでは、機能を利用する権限を運用担当者の利用者IDごとに付与できる。
- 現在は、6名の運用担当者とも全権限を付与されており、運用担当者はFWのルールの編集後、編集を行った運用担当者が操作に誤りがないことを確認し、操作承認をしている。

- FWへのログインにはパスワードを利用している。パスワードは8文字の英数字である。
- FWの運用では、運用担当者の利用者IDごとに、ネットワークを経由せずコンソールでログインできるかどうか、ネットワークを経由してリモートからログインできるかどうかを設定できる。
- FWは、ネットワークを経由せずコンソールでログインした場合でも、ネットワークを経由してリモートからログインした場合でも、同一の機能を利用できる。
- FWはサーバールームに設置されており、サーバールームにはほかに数種類のサーバも設置されている。
- 運用担当者だけがサーバールームへの入退室を許可されている。

表1 内部監査からの指摘

指摘	指摘内容
指摘1	FWの運用の作業中で職務が適切に分離されていない。
指摘2	(省略)
指摘3	(省略)
指摘4	(省略)

B部長は表1の指摘に対する改善策を検討することにした。

設問

表1中の指摘1について、FWルールの誤った変更を防ぐための改善策はどれか。解答群のうち、最も適切なものを選べ。

解答群

- ア Endpoint Detection and Response (EDR) をコンソールに導入し、監視を強化する。
- イ FWでの運用担当者のログインにはパスワード認証の代わりに多要素認証を導入する。
- ウ FWのアクセス制御機能を使って、運用担当者をコンソールからログインできる者、リモートからログインできる者に分ける。
- エ FWの運用担当者を1人に限定する。
- オ 運用担当者の一部を操作ログの確認だけをする者とし、それらの者には操作ログの確認権限だけを付与する。
- カ 運用担当者を、FWルールの編集を行う者、操作ログを確認し、操作承認をする者に分け、それぞれに必要最小限の権限を付与する。
- キ 作業を行う運用担当者を、曜日ごとに割り当てる。

解説 この問題では、A社のファイアウォール運用に対して内部監査が指摘した改善点について、どの対策が最適かを考えます。

指摘1には「FWの運用の作業の中で、職務が適切に分離されていない」と書かれているので、重要なのは「職務分離」により編集と承認を異なる担当者が行うようにすることです。現状では6名の運用担当者全員がFWルールの編集・確認・承認のすべてを行うことができるため、リスクが残ります。この前提をもとに各選択肢を確認します。

- ア** **EDR** (Endpoint Detection and Response) は不審な挙動の検知・対応に役立ちますが、FWルールの誤変更防止や職務分離にはつながらず、誤りです。
- イ** 多要素認証は不正ログイン防止に非常に有効ですが、運用担当者内の職務分離には影響せず、誤りです。
- ウ** コンソールとリモートログインで担当者を分ける案ですが、1人が編集と操作承認ができるという現状に変化はないため、誤りです。
- エ** 担当者を1人に限定すると、作業ミスや不正の監視は不可能になり、誤りです。
- オ** 運用担当者の一部は操作ログ確認権限だけが付与されますが、それ以外の担当者には依然として全権限が付与されているため誤りです。
- カ** 運用担当者を「FWルールの編集を行う者」「操作ログを確認する者」「承認を行う者」に分け、それぞれに必要最小限の権限だけを付与することで、編集と承認を同一担当者が行わないようにできます。これが指摘1の改善策として適切です。
- キ** 曜日ごとに担当者を割り当てても、1人が編集と操作承認ができることには変わりなく、誤りです。

正解 **カ**

Chapter

7

模擬試験問題

※実際の試験問題では、プログラムに行番号は記載されていません。
本書中の解答解説でわかりやすくするため、行番号を挿入しています。

問題

問1 次の記述中の□に入る正しい答えを、解答群の中から選べ。

プログラムを実行すると、"□"と出力される。

〔プログラム〕

```
1  整数型: a ← 5
2  整数型: b ← 7
3  整数型: c ← 9
4  整数型: tmp ← a
5  a ← c
6  c ← b
7  b ← tmp
8  aの値とbの値とcの値をこの順にコンマ区切りで出力する
```

解答群

- ア 5,7,9
- イ 5,9,7
- ウ 7,5,9
- エ 9,5,7
- オ 9,7,5
- カ 7,9,5

問2 次のプログラム中の と に入れる正しい答えの組合せを、解答群の中から選べ。

関数 function1 が受け取る引数と、関数 function2 が受け取る引数とが同じとき、二つの関数は同じ値を返す。ここで、引数 n と引数 m は正の整数であり、引数 m は引数 n よりも 10 以上大きい数とする。

[プログラム]

```
1  ○整数型: function1(整数型: n, 整数型: m)
2      整数型: count ← 0
3      整数型: i
4      for (iをnからmまで1ずつ増やす)
5          if ((i mod 4)が0と等しい)
6              count ← count + 1
7          endif
8      endfor
9      return count
10
11 ○整数型: function2(整数型: n, 整数型: m)
12     整数型: count ← 0
13     整数型: tempN ← n
14     整数型: i, j
15     for ()
16         if ((tempN mod 4)が0と等しい)
17             繰返し処理を終了する
18         endif
19         tempN ← tempN + 1
20     endfor
21
22     for ()
23         count ← count + 1
24     endfor
25     return count
```

解答群

	a	b
ア	iを1から2まで 1ずつ増やす	jをnから始めてmを超えない範囲で tempN ずつ増やす
イ	iを1から2まで 1ずつ増やす	jを tempN からmまで1ずつ増やす
ウ	iを1から2まで 1ずつ増やす	jを tempN から始めてmを超えない範囲で 4 ずつ増やす
エ	iを1から3まで 1ずつ増やす	jをnから始めてmを超えない範囲で tempN ずつ増やす
オ	iを1から3まで 1ずつ増やす	jを tempN からmまで1ずつ増やす
カ	iを1から3まで 1ずつ増やす	jを tempN から始めてmを超えない範囲で 4 ずつ増やす

問3 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数 makeNewArray は、要素数2以上の整数型の配列を引数にとり、整数型の配列を返す関数である。関数 makeNewArray を makeNewArray({3, 2, 1, 6, 5, 4}) として呼び出したとき、戻り値の配列の要素番号5の値は となる。

[プログラム]

```
1  ○整数型の配列: makeNewArray(整数型の配列: in)
2    整数型の配列: out ← {}    // 要素数0の配列
3    整数型: i, tail
4    outの末尾にin[1]の値を追加する
5    for (iを2からinの要素数まで1ずつ増やす)
6        tail ← out[outの要素数]
7        outの末尾に(tail + in[i])の結果を追加する
8    endfor
9    return out
```

解答群

- ア 5
- イ 6
- ウ 9
- エ 11
- オ 12
- カ 17
- キ 21

問4 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。
ここで、配列の要素番号は1から始まる。

次のプログラムは、整数型の配列 array の要素の並びを、指定された回数だけ右方向に循環シフトするものである。例えば、array が {1, 2, 3, 4, 5}, shift が 2 の場合、プログラム実行後の配列は {4, 5, 1, 2, 3} となる。

〔プログラム〕

```
1  整数型の配列: array ← {80, 25, 0, 150, 72}
2  整数型: shift ← 2
3  整数型: i, j, n ← arrayの要素数
4  整数型の配列: tmp ← {n個の未定義の値}
5  for (iを1からnまで1ずつ増やす)
6      j ← 
7      tmp[j] ← array[i]
8  endfor
9  for (iを1からnまで1ずつ増やす)
10     array[i] ← tmp[i]
11 endfor
12 for (iを1からnまで1ずつ増やす)
13     array[i]を出力
14 endfor
```

解答群

- ア $((i + \text{shift}) \bmod n)$
- イ $((i - \text{shift}) \bmod n)$
- ウ $((i + \text{shift} - 1) \bmod n) + 1$
- エ $((i - \text{shift} - 1) \bmod n) + 1$

問5 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。
ここで、配列の要素番号は1から始まる。

図1に示すグラフの頂点には、1から順に整数で番号が付けられている。グラフは無向グラフであり、各頂点間には高々1つの辺がある。一つの辺は両端の頂点の番号を要素にもつ要素数2の整数型の配列で表現できる。例えば、{1, 3}は頂点1と頂点3を端点とする辺を表す。グラフ全体は、グラフに含まれる辺を表す要素数2の配列を全て格納した配列（以下、辺の配列という）で表現できる。辺の配列の要素数はグラフの辺の個数と等しい。図1のグラフは整数型配列の配列{{1, 3}, {1, 4}, {3, 4}, {2, 4}, {4, 5}}と表現できる。

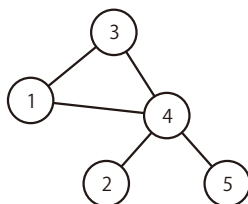


図1 グラフの例

関数 `edgesToMatrix` は、辺の配列を隣接行列に変換する。隣接行列とは、グラフに含まれる頂点の個数と等しい行数及び列数の正方形行列で、 i 行 j 列の成分は頂点 i と頂点 j を結ぶ辺があるときに1となり、それ以外は0となる。行列の対角成分は全て0で、無向グラフの場合は対称行列になる。図1のグラフを表現する隣接行列を図2に示す。

$$\begin{pmatrix} 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

図2 図1のグラフを表現する隣接行列

関数 `edgesToMatrix` は、引数 `edgeList` で辺の配列を、引数 `nodeNum` でグラフの頂点の個数をそれぞれ受け取り、隣接行列を表す整数型の二次元配列を返す。

〔プログラム〕

```
1  ○整数型の二次元配列: edgesToMatrix(整数型配列の配列: edgeList,  
2                                     整数型: nodeNum)  
3  整数型の二次元配列: adjMatrix ← {nodeNum行nodeNum列の0}  
4  整数型: i, u, v  
5  for (iを1からedgeListの要素数まで1ずつ増やす)  
6      u ← edgeList[i][1]  
7      v ← edgeList[i][2]  
8        
9  endfor  
10 return adjMatrix
```

解答群

- ア adjMatrix[u, u] ← 1
- イ adjMatrix[u, u] ← 1
adjMatrix[v, v] ← 1
- ウ adjMatrix[u, v] ← 1
- エ adjMatrix[u, v] ← 1
adjMatrix[v, u] ← 1
- オ adjMatrix[v, u] ← 1
- カ adjMatrix[v, v] ← 1

問6 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数mergeは、昇順に整列された整数型の配列data1及びdata2を受け取り、これらを併合してできる昇順に整列された整数型の配列を返す。

関数mergeをmerge({2, 3}, {1, 4})として呼び出すと、`/** α **/`の行は である。

[プログラム]

```
1  ○整数型の配列: merge(整数型の配列: data1, 整数型の配列: data2)
2      整数型: n1 ← data1の要素数
3      整数型: n2 ← data2の要素数
4      整数型の配列: work ← {(n1 + n2)個の未定義の値}
5      整数型: i ← 1
6      整数型: j ← 1
7      整数型: k ← 1
8
9      while (i ≤ n1 and j ≤ n2)
10         if (data1[i] ≤ data2[j])
11             work[k] ← data1[i]
12             i ← i + 1
13         else
14             work[k] ← data2[j]
15             j ← j + 1
16         endif
17         k ← k + 1
18     endwhile
19
20     while (i ≤ n1)
21         work[k] ← data1[i]
22         i ← i + 1
23         k ← k + 1
24     endwhile
25
```

```
26   while (j ≤ n2)
27       work[k] ← data2[j]  /** a **/
28       j ← j + 1
29       k ← k + 1
30   endwhile
31
32   return work
```

解答群

- ア 実行されない
- イ 1回実行される
- ウ 2回実行される
- エ 3回実行される

問7 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数 binSort を binSort() として呼び出すと、戻り値の配列には未定義の要素は含まれておらず、値は昇順に並んでいる。

〔プログラム〕

```
1  ○整数型の配列: binSort(整数型の配列: data)
2      整数型: n ← dataの要素数
3      整数型の配列: bins ← {n個の未定義の値}
4      整数型: i
5      for (iを1からnまで1ずつ増やす)
6          bins[data[i]] ← data[i]
7      endfor
8      return bins
```

解答群

- ア {2, 6, 3, 1, 4, 5}
- イ {3, 1, 4, 4, 5, 2}
- ウ {4, 2, 1, 5, 6, 2}
- エ {5, 3, 4, 3, 2, 6}

問8 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。
ここで、配列の要素番号は1から始まる。

関数 simRatio は、引数として与えられた要素数1以上の二つの文字型の配列 s1 と s2 を比較し、要素数が等しい場合は、配列の並びがどの程度似ているかの指標として、(要素番号が同じ要素の文字同士が一致する要素の組みの個数 ÷ s1 の要素数) を実数型で返す。例えば、配列の全ての要素が一致する場合の戻り値は1、いずれの要素も一致しない場合の戻り値は0である。なお、二つの配列の要素数が等しくない場合は、-1を返す。

関数 simRatio に与える s1, s2 及び戻り値の例を表に示す。プログラムでは、配列の領域外を参照してはならないものとする。

表 関数 simRatio に与える s1, s2 及び戻り値の例

s1	s2	戻り値
{"a", "p", "p", "l", "e"}	{"a", "p", "p", "l", "e"}	1
{"a", "p", "p", "l", "e"}	{"a", "p", "r", "i", "l"}	0.4
{"a", "p", "p", "l", "e"}	{"m", "e", "l", "o", "n"}	0
{"a", "p", "p", "l", "e"}	{"p", "e", "n"}	-1

〔プログラム〕

```
1  ○実数型: simRatio(文字型の配列: s1, 文字型の配列: s2)
2  整数型: i, cnt ← 0
3  if (s1の要素数 ≠ s2の要素数)
4      return -1
5  endif
6  for (iを1からs1の要素数まで1ずつ増やす)
7      if ( )
8          cnt ← cnt + 1
9      endif
10 endfor
11 return cnt ÷ s1の要素数  // 実数として計算する
```

解答群

- ア s1[i] ≠ s2[cnt]
- イ s1[i] ≠ s2[i]
- ウ s1[i] = s2[cnt]
- エ s1[i] = s2[i]

問9 次の記述中の□に入る正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数 search は、引数 data で指定された配列に、引数 target で指定された値が含まれていればその要素番号を返し、含まれていなければ -1 を返す。data は昇順に整列されており、値に重複はない。

関数 search には不具合がある。例えば、data の□場合は、無限ループになる。

[プログラム]

```
1  ○整数型: search(整数型の配列: data, 整数型: target)
2      整数型: low, high, middle
3
4      low ← 1
5      high ← dataの要素数
6
7      while (low ≤ high)
8          middle ← (low + high) ÷ 2の商
9          if (data[middle] < target)
10             low ← middle
11          elseif (data[middle] > target)
12             high ← middle
13          else
14             return middle
15          endif
16      endwhile
17
18      return -1
```

解答群

- ア 要素数が1で、targetがその要素の値と等しい
- イ 要素数が2で、targetがdataの先頭要素の値と等しい
- ウ 要素数が2で、targetがdataの末尾要素の値と等しい
- エ 要素に-1が含まれている

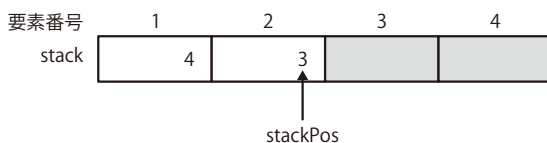
問10 次のプログラム中の a と b に入れる正しい答えの組合せを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数pushは、引数で与えられた整数をスタックに格納する。格納できた場合はtrueを返し、格納できなかった場合はfalseを返す。

関数popは、スタックから値を取り出して返す。スタックが空のときは未定義の値を返す。

スタックを、要素数が4である大域の整数型の配列stack、及び次に値を格納する位置を示す大域の変数stackPosで表現する。スタックの初期状態を図に示す。

プログラムでは、配列の領域外を参照してはならないものとする。



注記 網掛けはその要素が未定義であることを示す。

図 スタックの初期状態

〔プログラム〕

```
1  大域: 整数型: stackPos ← 3
2  大域: 整数型の配列: stack ← {4, 3, 未定義の値, 未定義の値}
3
4  ○論理型: push(整数型: inputData)
5      if (stackPos ≤ stackの要素数)
6          stack[ a ] ← inputData
7          stackPos ← stackPos + 1
8          return true
9      else
10         return false
11     endif
12
13  ○整数型: pop()
14     整数型: popData ← 未定義の値
```

```
15  if (stackPos > 1)
16      stackPos ← 
17      popData ← stack[stackPos]
18      stack[stackPos] ← 未定義の値
19  endif
20  return popData
```

解答群

	a	b
ア	stackPos	stackPos + 1
イ	stackPos	stackPos - 1
ウ	stackPos - 1	stackPos + 1
エ	stackPos - 1	stackPos - 1

問11 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数 search は、二つの文字型の配列を、それぞれ引数 data 及び key で受け取り、data から、key の要素の並びと同じ並びを全て探し、その先頭の要素番号を全て格納した配列を返す。見つからなければ、要素数0の配列を返す。

関数 search = search({"a", "b", "a", "b", "c", "a", "b", "c"}, {"a", "b", "c"}) として呼び出すと、`/** β **/` の行の条件式が真となる回数は 回である。

[プログラム]

```
1  /* keyは、要素数1以上の配列である */
2  ○整数型の配列: search(文字型の配列: data, 文字型の配列: key)
3  整数型: i, j, lenData, lenKey
4  整数型の配列: result ← {} // 要素数0の配列
5
6  lenData ← dataの要素数
7  lenKey ← keyの要素数
8
9  /* (lenData - lenKey + 1) が0以下のときは
10  繰返し処理を実行しない */
11  for (iを1から(lenData - lenKey + 1)まで1ずつ増やす)
12      for (jを1からlenKeyまで1ずつ増やす) // a
13          if (data[i + j - 1]がkey[j]と等しい) /** β **/
14              if (jがlenKeyと等しい)
15                  resultの末尾にiの値を追加する
16              endif
17          else
18              aの行から始まる繰返し処理を終了する
19          endif
20      endfor
21  endfor
22  return result
```

解答群

ア 1 イ 2 ウ 3 エ 4 オ 5
 カ 6 キ 7 ク 8 ケ 9 コ 10

問12 次の記述中の と に入る正しい答えの組合せを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

予防接種の病気Xに対する予防効果を調査するために集めたデータの集計結果を基に、病気Xにかかるかどうか、予防接種の有無に影響されないと仮定した場合の人数を計算する。この人数を理論度数という。表1に集計結果の例を示し、表2に表1を基に計算した理論度数を示す。

関数fは、引数dataで受け取った集計結果を基に計算した理論度数を返す。引数と戻り値は二次元配列で、その行が表の行、その列が表の列に対応する。

表1 集計結果の例

単位 人

	病気Xにかからなかった	病気Xにかかった
予防接種を受けた	82	6
予防接種を受けていない	58	8

表2 表1を基にした理論度数

単位 人

	病気Xにかからなかった	病気Xにかかった
予防接種を受けた	a	
予防接種を受けていない		b

注記 網掛けの部分は表示していない。

〔プログラム〕

```
1  ○実数型の二次元配列: f(実数型の二次元配列: data)
2    実数型: t ← dataの要素の和
3    整数型: row ← dataの行数
4    整数型: col ← dataの列数
5    実数型の二次元配列: result ← {row行col列の未定義の値}
6    整数型: r, c
7    for (rを1からrowまで1ずつ増やす)
8      for (cを1からcolまで1ずつ増やす)
9        result[r, c] ← (dataの行番号rの要素の和) ×
10                       (dataの列番号cの要素の和) ÷ t
11      endfor
12    endfor
13    return result
```

解答群

	a	b
ア	44	33
イ	58	8
ウ	70	7
エ	75	2
オ	80	6
カ	80	8
キ	82	6

問13 次のプログラム中の ～ に入れる正しい答えの組合せを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

一度の注文で購入された商品のリストを、注文ごとに記録した注文データがある。表に、注文データの例を示す。

表 注文データの例

注文番号	購入された商品のリスト
1	A, B, D
2	A, D
3	A
4	A, B, E
5	B
6	C, E

注文データから、商品 x と商品 y とが同一の注文で購入されやすい傾向を示す関連度 L_{xy} を、次の式で計算する。

$$L_{xy} = \frac{M_{xy} \times \text{全注文数}}{K_x \times K_y}$$

ここで、 M_{xy} は商品 x と商品 y とが同一の注文で購入された注文数、 K_x は商品 x が購入された注文数、 K_y は商品 y が購入された注文数を表す。表の例では、 M_{AB} が2、全注文数が6、 K_A が4、 K_B が3であるので、商品 A と商品 B の関連度 L_{AB} は、 $(2 \times 6) / (4 \times 3) = 1.0$ である。

手続 `putRelatedItem` は、大域変数 `orders` に格納された注文データを基に、引数で与えられた商品との関連度が最も大きい商品のうちのひとつと、その関連度を出力する。プログラムでは、商品は文字列で表し、注文は購入された商品の配列、注文データは注文の配列で表している。注文データには2種類以上の商品が含まれるものとする。また、注文データにある商品以外の商品が、引数として与えられることはないものとする。

〔プログラム〕

```
1 // 注文データ（ここでは表の例を与えている）
2 大域：文字列型配列の配列：orders ← {"A", "B", "D"}, {"A", "D"},
3                                     {"A"}, {"A", "B", "E"},
4                                     {"B"}, {"C", "E"}}
5
6 OutputRelatedItem(文字列型：item)
7   文字列型の配列：allItems ← ordersに含まれる文字列を
8                               重複なく辞書順に格納した配列
9   // 表の例では {"A", "B", "C", "D", "E"}
10
11   文字列型の配列：otherItems ← allItemsの複製から値がitemである
12                               要素を除いた配列
13
14   整数型：i, itemCount ← 0
15   整数型の配列：arrayK ← {otherItemsの要素数個の0}
16   整数型の配列：arrayM ← {otherItemsの要素数個の0}
17   実数型：valueL, maxL ←  $-\infty$ 
18   文字列型の配列：order
19   文字列型：relatedItem
20
21   for (orderにordersの要素を順に代入する)
22     if (orderのいずれかの要素の値がitemの値と等しい)
23       itemCount ← itemCount + 1
24     endif
25     for (iを1からotherItemsの要素数まで1ずつ増やす)
26       if (orderのいずれかの要素の値がotherItems[i]の値と等しい)
27         if (orderのいずれかの要素の値がitemの値と等しい)
28           a の値を1増やす
29         endif
30         b の値を1増やす
31       endif
32     endfor
33   endfor
34   for (iを1からotherItemsの要素数まで1ずつ増やす)
```



```

35     valueL ← (arrayM[i] × c) ÷ (itemCount × arrayK[i])
36                                     /* 実数として計算する */
37     if (valueLがmaxLより大きい)
38         maxL ← valueL
39         relatedItem ← otherItems[i]
40     endif
41 endfor
42 relatedItemの値とmaxLの値をこの順にコンマ区切りで出力する

```

解答群

	a	b	c
ア	arrayK[i]	arrayM[i]	allItemsの要素数
イ	arrayK[i]	arrayM[i]	ordersの要素数
ウ	arrayK[i]	arrayM[i]	otherItemsの要素数
エ	arrayM[i]	arrayK[i]	allItemsの要素数
オ	arrayM[i]	arrayK[i]	ordersの要素数
カ	arrayM[i]	arrayK[i]	otherItemsの要素数

問14 次のプログラム中の□□□□に入れる正しい答えを、解答群の中から選べ。

関数 reverseByGroup は 8 ビット型の引数 byte と整数型の引数 n を受け取り、byte の値を n ビット (n は 8 の約数) ごとに区切り、各グループのビット列を逆にした値を返す。例えば、reverseByGroup(10110011, 2) として呼び出すと、戻り値は 01110011 となる。

なお、演算子 $\wedge$ はビット単位の論理積、演算子 $\vee$ はビット単位の論理和、演算子 $\gg$ は論理右シフト、演算子 $\ll$ は論理左シフトを表す。例えば、value $\gg$ n は value の値を n ビットだけ右に論理シフトし、value $\ll$ n は value の値を n ビットだけ左に論理シフトする。

〔プログラム〕

```
1  ○8ビット型: reverseByGroup(8ビット型: byte, 整数型: n)
2    if (byteのビット長 ≠ 8)
3      プログラムを終了
4    endif
5    8ビット型: r ← 00000000
6    整数型: numGroups ← 8 ÷ n
7    整数型: i, j
8    整数型: group, reversed
9    for (iを0からnumGroups - 1まで1ずつ増やす)
10      group ← (byte  $\gg$  (i × n))  $\wedge$  ((1  $\ll$  n) - 1)
11      reversed ← 0
12      for (jを0からn - 1まで1ずつ増やす)
13        □□□□
14      endfor
15      r ← r  $\vee$  (reversed  $\ll$  (i × n)) //元の位置に戻す
16    endfor
17    return r
```

解答群

- ア reversed $\leftarrow$ (group $\ll$ 1) $\vee$ ((reversed $\gg$ j) $\wedge$ 1)
- イ reversed $\leftarrow$ reversed $\wedge$ ((group $\gg$ j) $\vee$ 1)
- ウ reversed $\leftarrow$ (reversed $\ll$ 1) $\vee$ ((group $\gg$ j) $\wedge$ 1)
- エ reversed $\leftarrow$ (reversed $\gg$ 1) $\vee$ ((group $\ll$ j) $\wedge$ 1)

問15 次の記述中の と に入れる正しい答えの組合せを、解答群の中から選べ。

三目並べにおいて自分が勝利する可能性が最も高い手を決定する。次の手順で、ゲームの状態遷移を木構造として表現し、根以外の各節の評価値を求める。その結果、根の子の中で最も評価値が高い手を、最も勝利する可能性が高い手とする。自分が選択した手を○で表し、相手が選択した手を×で表す。

〔手順〕

(1) 現在の盤面の状態を根とし、勝敗がつくか、引き分けとなるまでの考えられる全ての手を木構造で表現する。

(2) 葉の状態を次のように評価する。

①自分が勝ちの場合は10

②自分が負けの場合は-10

③引き分けの場合は0

(3) 葉以外の節の評価値は、その節の全ての子の子の評価値を基に決定する。

①自分の手番の節である場合、子の子の評価値で最大の評価値を節の評価値とする。

②相手の手番の節である場合、子の子の評価値で最小の評価値を節の評価値とする。

ゲームが図の最上部にある根の状態のとき、自分が選択できる手は三つある。そのうちAが指す子の子の評価値は であり、Bが指す子の子の評価値は である。

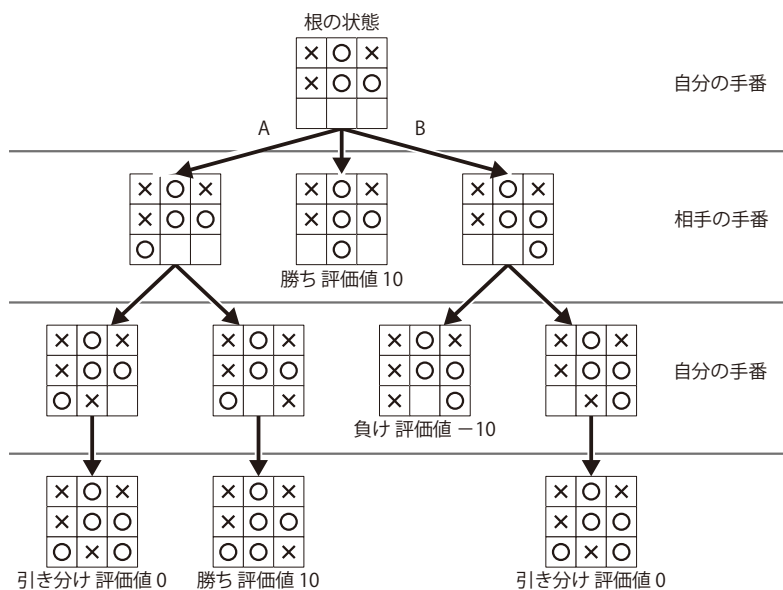


図 三目並べの状態遷移

解答群

	a	b
ア	0	-10
イ	0	0
ウ	10	-10
エ	10	0

問16 次のプログラムの に入れる正しい答えを、解答群の中から選べ。二つの には、同じ答えが入る。ここでは、配列の要素番号は1から始まる。

Unicodeの符号位置を、UTF-8の符号に変換するプログラムである。本間で数値の後ろに"(16)"と記載した場合は、その数値が16進数であることを表す。

Unicodeの各文字には、符号位置と呼ばれる整数値が与えられている。UTF-8は、Unicodeの文字を符号化する方式の一つであり、符号位置が800(16)以上FFFF(16)以下の文字は、次のように3バイトの値に符号化する。

3バイトの長さのビットパターンを1110xxxx 10xxxxxx 10xxxxxxとする。ビットパターンの下線の付いた"x"の箇所に、符号位置を2進数で表した値を右詰めで格納し、余った"x"の箇所に0を格納する。この3バイトの値がUTF-8の符号である。

例えば、ひらがなの"あ"の符号位置である3042(16)を2進数で表すと11000001000010である。これを、上に示したビットパターンの"x"の箇所に右詰めで格納すると、1110xx11 10000001 10000010となる。余った二つの"x"の箇所に0を格納すると、"あ"のUTF-8の符号11100011 10000001 10000010が得られる。

関数encodeは、引数で渡されたUnicodeの符号位置をUTF-8の符号に変換し、先頭から順に1バイトずつ要素に格納した整数型の配列を返す。encodeには、引数として800(16)以上FFFF(16)以下の整数値だけが渡されるものとする。

[プログラム]

```

1  ○整数型の配列: encode(整数型: codePoint)
2      /* utf8Bytesの初期値は、ビットパターンの"x"をすべて0に置き換え、
3         8桁ごとに区切って、それぞれを2進数と見なしたときの値 */
4      整数型の配列: utf8Bytes ← {224, 128, 128}
5      整数型: cp ← codePoint
6      整数型: i
7      for (iをutf8Bytesの要素数から1まで1ずつ減らす)
8          utf8Bytes[i] ← utf8Bytes[i] + (cp ÷ の余り)
9          cp ← cp ÷ の商
10     endfor
11     return utf8Bytes

```

解答群

- | | | |
|-----------------|-----------------|-----------|
| ア ((4 - i) × 2) | イ 2 の (4 - i) 乗 | ウ 2 の i 乗 |
| エ (i × 2) | オ 2 | カ 6 |
| キ 16 | ク 64 | ケ 256 |

問17 製造業のA社では、ECサイト（以下、A社のECサイトをAサイトという）を使用し、個人向けの製品販売を行っている。Aサイトは、A社の製品やサービスが検索可能で、ログイン機能を有しており、あらかじめAサイトに利用登録した個人（以下、会員という）の氏名やメールアドレスといった情報（以下、会員情報という）を管理している。Aサイトは、B社のPaaSで稼働しており、PaaS上のDBMSとアプリケーションサーバを利用している。

A社は、Aサイトの開発、運用をC社に委託している。A社とC社との間の委託契約では、Webアプリケーションプログラムの脆弱性対策は、C社が実施するとしている。

最近、A社の同業他社が運営しているWebサイトで脆弱性が悪用され、個人情報漏えいするという事件が発生した。そこでA社は、セキュリティ診断サービスを行っているD社に、Aサイトの脆弱性診断を依頼した。脆弱性診断の結果、対策が必要なセキュリティ上の脆弱性が複数指摘された。図1にD社からの指摘事項を示す。

- | |
|--|
| <p>項番1 Aサイトで利用しているアプリケーションサーバのOSに既知の脆弱性があり、脆弱性を悪用した攻撃を受けるおそれがある。</p> <p>項番2 Aサイトにクロスサイトスクリプティングの脆弱性があり、会員情報を不正に取得されるおそれがある。</p> <p>項番3 Aサイトで利用しているDBMSに既知の脆弱性があり、脆弱性を悪用した攻撃を受けるおそれがある。</p> |
|--|

図1 D社からの指摘事項

設問

図1中の各項番それぞれに対処する組織の適切な組合せを、解答群の中から選べ。

解答群

	項番1	項番2	項番3
ア	A社	A社	A社
イ	A社	A社	C社
ウ	A社	B社	B社
エ	B社	B社	B社
オ	B社	B社	C社
カ	B社	C社	B社
キ	B社	C社	C社
ク	C社	B社	B社
ケ	C社	B社	C社
コ	C社	C社	B社

問18 A社は従業員200名の電子機器メーカーである。東京に本社があり、新潟に工場がある。

A社では、ファイルサーバを次のように運用している。

- ・ファイルサーバは本社と工場のサーバールームに設置している。
- ・ファイルサーバは、磁気テープを使用してファイルのバックアップを取得している。
- ・土曜日の午前2時からフルバックアップを取得し、翌週の火曜日と木曜日の午前2時から増分バックアップを取得している。
- ・情報システム部の担当者は毎週月曜日、火曜日、木曜日の朝10時頃、バックアップジョブの実行結果の確認と磁気テープの交換を行い、磁気テープは耐火金庫に保管している。
- ・磁気テープには上書きモードでバックアップを取得し、1か月分の磁気テープを世代管理している。
- ・これまで、フルバックアップからのリストアには平均して4時間、1回の増分バックアップからは平均して0.25時間掛かっている。
- ・A社はICT継続のための試験を実施しており、ファイルサーバも試験対象である。

注記 事業影響度分析の結果に基づき、ファイルサーバには72時間の目標復旧時点（RPO）と120時間の目標復旧時間（RTO）が要求事項として定められている。

図1 A社のファイルサーバの運用

A社はISMS認証を取得しており、最高情報セキュリティ責任者（CISO）を中心に情報セキュリティに取り組み、定期的に、情報セキュリティリスクアセスメントを行っている。今般、ISMS認証基準がJIS Q 27001:2023に改正されたことを受け、情報セキュリティリーダーのBさんは、新基準への移行審査に向けて準備することになった。改正によって新たに追加された情報セキュリティ管理策について、Bさんは情報システム部の担当者に取組状況を確認し、その評価結果を表1にまとめた。

表1 Bさんの評価結果（抜粋）

項番	情報セキュリティ管理策	評価結果
1	（事業継続のためのICTの備え）事業継続の目的及びICT継続の要求事項に基づいて、ICTの備えを計画し、実施し、維持し、試験しなければならない。	実施している

Bさんは、移行審査前の内部監査で、内部監査室から表1の項番1に関するファイルサーバの運用について何点か質問を受け、表2のとおり回答した。

表2 内部監査室へのBさんの回答（抜粋）

項番	Bさんの回答
1	例えば、金曜日の正午に障害が発生した場合、少なくとも a1 の時点のデータは復元しなければならない。
2	例えば、木曜日の正午に障害が発生し、ファイルサーバの全データが消失したとすると、バックアップからのリストアには a2 時間掛かると予想される。
3	ICT継続の計画書は、 a3 が承認している。

設問

表2中の ～ に入れる字句の適切な組合せを、aに関する解答群の中から選べ。

aに関する解答群

	a1	a2	a3
ア	月曜日の正午	4.25	CISO
イ	月曜日の正午	4.25	情報システム部の担当者
ウ	月曜日の正午	4.25	内部監査室長
エ	月曜日の正午	4.50	CISO
オ	月曜日の正午	4.50	情報システム部の担当者
カ	火曜日の正午	4.25	情報システム部の担当者
キ	火曜日の正午	4.25	内部監査室長
ク	火曜日の正午	4.50	CISO
ケ	火曜日の正午	4.50	情報システム部の担当者
コ	火曜日の正午	4.50	内部監査室長

問19 A社は、放送会社や運輸会社向けに広告制作ビジネスを展開している。A社は、人事業務の効率化を図るべく、人事業務の委託を検討することにした。A社が委託する業務（以下、B業務という）を図1に示す。

- ・採用予定者から郵送されてくる入社時の誓約書、前職の源泉徴収票などの書類をPDFファイルに変換し、ファイルサーバに格納する。

図1 B業務

委託先候補のC社は、B業務について次のようにA社に提案した。

- ・B業務だけに従事する専任の従業員を割り当てる。
- ・B業務では、図2の複合機のスキャン機能を使用する。

- ・スキャン機能を使用する際は、従業員ごとに付与した利用者IDとパスワードをパネルに入力する。
- ・スキャンしたデータをPDFファイルに変換する。
- ・PDFファイルを従業員ごとに異なる鍵で暗号化して、電子メールに添付する。
- ・スキャンを実行した本人宛てに電子メールを送信する。
- ・PDFファイルが大きい場合は、PDFファイルを添付する代わりに、自社の社内ネットワーク上に設置したサーバ（以下、Bサーバという）¹⁾に自動的に保存し、保存先のURLを電子メールの本文に記載して送信する。

注1) Bサーバにアクセスする際は、従業員ごとの利用者IDとパスワードが必要になる。

図2 複合機のスキャン機能（抜粋）

A社は、C社と業務委託契約を締結する前に、秘密保持契約を締結した。その後、C社に質問表を送付し、回答を受けて、業務委託での情報セキュリティリスクの評価を実施した。その結果、図3の発見があった。

- ・複合機のスキャン機能では、電子メールの差出人アドレス、件名、本文および添付ファイル名を初期設定¹⁾の状態で使用しており、誰がスキャンを実行しても同じである。
- ・複合機のスキャン機能の初期設定情報はベンダーのWebサイトで公開されており、誰でも閲覧できる。

注1) 複合機の初期設定はC社の情報システム部だけが変更可能である。

図3 発見事項

そこでA社では、初期設定の状態のままではA社にとって情報セキュリティリスクがあり、初期設定から変更するという対策が必要であると評価した。

設問

対策が必要であるとA社が評価した情報セキュリティリスクはどれか。解答群のうち、最も適切なものを選び。

解答群

- ア B業務に従事する従業員が、攻撃者からの電子メールを複合機からのものと信じて本文中のURLをクリックし、フィッシングサイトに誘導される。その結果、A社の採用予定者の個人情報が漏えいする。
- イ B業務に従事する従業員が、複合機から送信される電子メールをスパムメールと誤認し、電子メールを削除する。その結果、再スキャンが必要となり、B業務が遅延する。
- ウ 攻撃者が、複合機から送信される電子メールを盗聴し、添付ファイルを暗号化して身代金を要求する。その結果、A社が復号鍵を受け取るために多額の身代金を支払うことになる。
- エ 攻撃者が、複合機から送信される電子メールを盗聴し、本文に記載されているURLを使ってBサーバにアクセスする。その結果、A社の採用予定者の個人情報が漏えいする。

問20 A社は、スマートフォン用ヘルスケア関連アプリケーションソフトウェア（以下、Aアプリという）の開発、販売を行う会社である。Aアプリは、ヘルスケアデータ収集機能によって、利用者の体重や歩数のデータを収集するとともに、アンケート機能によって、勤務先の業種、年収などのデータを収集している。

営業部は、保険会社であるB社と共同のビジネスを検討している。その中で、Aアプリで収集したデータ及び今後収集するデータから匿名加工情報を作成した上で、その匿名加工情報をB社に第三者提供することを検討している。B社では、提供を受けた匿名加工情報を分析して、B社の保険商品のマーケティングに活用しようとしている。分析の対象は、年代、性別、年収区分、住んでいる地域区分及び勤務先の業種と体重及び歩数との関係である。ここで、地域区分とは、北海道、東北、南関東などの区分のことをいう。

加工対象となる"個人情報データベース等"（以下、加工対象情報という）は、表1の顧客属性データと表2のヘルスケアデータの2種類からなり、利用者番号によって関連付けられている。

表1 顧客属性データ

利用者番号	氏名	性別	生年月日	電子メールアドレス	住所	勤務先の業種	年収
114567	情報一郎	男	1984年4月4日	ichiro@●●●.ne.jp	愛知県名古屋市中区上前津 X-X-X	金融業	1,850万
114568	積招花子	女	2003年3月31日	sekimane@▲▲.ne.jp	東京都文京区本駒込 X-X-X	小売業	380万
114573	試験五郎	男	1951年12月9日	shiken@■●■.ne.jp	東京都中央区月島 X-X-X	製造業	630万

表2 ヘルスケアデータ

利用者番号	記録日	体重	歩数
114567	202N年02月11日	121.3	5,321
114568	202N年02月11日	43.5	33,012
114568	202N年02月12日	43.0	12,007
114573	202N年02月12日	63.0	7,604

営業部の情報セキュリティリーダーであるC課長は、加工対象情報に含まれる各情報の項目について、適用する匿名加工情報の作成に係る手法（以下、匿名加工情報の作成に係る手法を匿名加工手法という）を表3にまとめた。

表3 匿名加工手法

項番	手法名	解説
(一)	項目削除	加工対象情報に含まれる個人情報の項目を削除すること。例えば、年齢データを加工対象情報から削除すること。
(二)	一般化	加工対象情報に含まれる記述などについて、上位概念に置き換えること。例えば、購買履歴のデータで"きゅうり"を"野菜"に置き換えること。
(三)	トップ（ボトム）コーディング	加工対象情報に含まれる数値について、特に大きい又は小さい数値をまとめること。例えば、年齢に関するデータで、"98歳"、"115歳"といった数値データを"80歳以上"というデータにまとめること。
(四)	丸め	加工対象情報に含まれる数値について、四捨五入などして丸めること。例えば、生年月日から"34歳"とされる場合について、"30代"のように年代に置き換えること。

注記 本表は、個人情報保護委員会事務局"仮名加工情報・匿名加工情報 信頼ある人物情報の利活用に向けて－制度編－（第2版）"を基にC課長が作成した。

C課長は、加工対象情報である表1及び表2に示す各データに対して、表3の匿名加工手法を必要に応じて適用し匿名加工情報を作成する案を用意した。その案を法務部のD課長に報告したところ、D課長からは、想定するマーケティング用途にも有効であり、匿名加工手法の適用も適切であるとの回答を得た。

設問

解答群に示した3項目それぞれについて、C課長が適用した匿名加工手法はどれか。表3の手法のうち、適用したものの組合せを、解答群の中から選べ。なお、各項目に対して複数の匿名加工手法を適用しても構わないし、一つも適用せずに"加工なし"としても構わない。

解答群

	電子メールアドレス	勤務先の業種	体重
ア	加工なし	加工なし	(三)
イ	加工なし	加工なし	(三), (四)
ウ	加工なし	(一)	(一)
エ	加工なし	(一)	(三)
オ	加工なし	(一)	(三), (四)
カ	(一)	加工なし	(三)
キ	(一)	加工なし	(三), (四)
ク	(一)	(一)	(一)
ケ	(一)	(一)	(三)
コ	(一)	(一)	(三), (四)

問1 エ

→ 01-01参照(オリジナル問題)

プログラムを追うことに慣れていない場合は、次のようなトレース表を作成しましょう。



解説動画

プログラム	a	b	c	tmp
1: 整数型: a ← 5	5			
2: 整数型: b ← 7		7		
3: 整数型: c ← 9			9	
4: 整数型: tmp ← a				5
5: a ← c	9			
6: c ← b			7	
7: b ← tmp		5		

もし慣れているのであれば、わざわざトレース表を作成する必要はありません。変数の変化をメモするだけで十分な場合がほとんどです。

4行目までを実行すると、変数の値は次のような状態になります。

a	b	c	tmp
5	7	9	5

5行目でaの値がcの値と同じになり、6行目cの値がbの値と同じになります。さらにbの値がtmpの値と同じになります。

a	b	c	tmp
5	7	9	5
9	5	7	

このように、変数が変化するたびに二重線などで消して更新していくほうが、圧倒的に早く答えを得ることができる場合があります。

a、b、cの値をコンマ区切りで出力すると「9,5,7」となります。

まずfunction1で何をしようとしているかを確認します。確認すると、nからmの範囲にある4の倍数を数えているようです。順番にすべての数を調べ、「4の倍数ならcountを1増やす」というシンプルな処理です。これは、変数に適当な値を代入して、トレースすることで判明します。



解説動画

次にfunction2を見ていきます。

● n = 100で考えてみる

n = 100、m = 110で考えてみます。100から110の間には100、104、108 という3つの「4の倍数」があります。15行目のfor文では、1つ目の繰り返し処理で「nからmの範囲で最初の4の倍数を見つける」必要があります。100~110の間であれば、100が最初の「4の倍数」です。ここで **a** に、候補の1つである「iを1から2まで1ずつ増やす」を入れて n = 100、m = 110でトレースしてみましょう。13行目でtempNに100 (n) が代入され、100は4の倍数なので1回目で繰り返しが止まります。

プログラム	n	tempN	i
13: 整数型: tempN ← n		100	
14: 整数型: i, j			
15: for (iを1から2まで1ずつ増やす)			1
16: if ((tempN mod 4)が0と等しい)	100は4の倍数であるため 条件を満たす		
17: 繰り返し処理を終了する			

b の繰り返し条件には必ずtempNが関わるはずですが。そうでなければ、15行目からの繰り返し処理でtempNをカウントアップしていく意义がありません。**今後どこかで使うからこそ、カウントアップしたわけです。**

上のトレース表において、tempNは100です。これは「範囲内の最初の"4の倍数"」であり、そこから範囲の上限(例では110)まで繰り返しを行う必要があります。そのため、ある変数をtempNから開始してカウントアップするはずですが。この条件に合致する候補は次の2つです。

① jをtempNからmまで1ずつ増やす

② jをtempNから始めてmを超えない範囲で4ずつ増やす

①のように1ずつ確認していくのは非効率です。すでに最初の「4の倍数」がわかっているわけですから、そこから4つ置きに確認すれば、function1のようにすべての数値を確認しなくても4の倍数を数えることができます。したがって、**b**は「jをtempNから始めてmを超えない範囲で4ずつ増やす」です。

n = 100の場合、**a**を試しに「iを1から2まで1ずつ増やす」としました。これでnからmの範囲にある4の倍数が数えられました。しかし、これはnがすでに4の倍数だったから成功したに過ぎません。

●n = 101で考えてみる

次にn = 101の場合で考えてみます。この場合、**a**に先ほどと同じ「iを1から2まで1ずつ増やす」を入れていると、範囲内の最初の4の倍数である104を見つける前に繰り返しが終わってしまい、正しい結果になりません。

プログラム	n	tempN	i
13: 整数型: tempN ← n		101	
14: 整数型: i, j			
15: for (iを1から2まで1ずつ増やす)			1
16: if ((tempN mod 4) が0と等しい)	条件を満たさない		
19: tempN ← tempN + 1		102	
15: for (iを1から2まで1ずつ増やす)			2
16: if ((tempN mod 4) が0と等しい)	条件を満たさない		
19: tempN ← tempN + 1		103	
			4の倍数ではない 103になってしまう

この問題を解決するためには、**a**に「iを1から3まで1ずつ増やす」を入れます。これで繰り返し処理が1回追加されるため、tempNが104となります。

したがって、正解は**力**です。

問3 カ

→ 02-01参照(R4/12月サンプル問題セット-問3)

単純に値が変化するような問題では、トレース表を作って変化を確認するとよいでしょう。ここでは省略せずに書いていきますが、実際の試験では時間が

限られているため、**できるだけトレース表は省略して書いてください**。なお実

際の繰り返し処理では「endfor」でiの値がカウントアップされます。今回のトレース表では「endfor」を省略していますので、「for」でカウントアップしています。



解説動画

プログラム	in	out	i	tail
1: ○整数型の配列: makeNewArray(整数型の配列: in)	{3,2,1,6,5,4}			
2: 整数型の配列: out ← {} // 要素数0の配列		{}		
3: 整数型: i, tail				
4: outの末尾にin[1]の値を追加する		{3}		
5: for (iを2からinの要素数まで1ずつ増やす)			2	
6: tail ← out[outの要素数] outの末尾の要素をtailに代入				3
7: outの末尾に(tail + in[i])の結果を追加する		{3,5}		
5: for (iを2からinの要素数まで1ずつ増やす) tail + in[2] = 5			3	
6: tail ← out[outの要素数]				5
7: outの末尾に(tail + in[i])の結果を追加する		{3,5,6}		
5: for (iを2からinの要素数まで1ずつ増やす)			4	
6: tail ← out[outの要素数]				6
7: outの末尾に(tail + in[i])の結果を追加する		{3,5,6,12}		
5: for (iを2からinの要素数まで1ずつ増やす)			5	
6: tail ← out[outの要素数]				12
7: outの末尾に(tail + in[i])の結果を追加する		{3,5,6,12,17}		
5: for (iを2からinの要素数まで1ずつ増やす)			6	
6: tail ← out[outの要素数]				17
7: outの末尾に(tail + in[i])の結果を追加する		{3,5,6,12,17,21}		

この問題では、「戻り値の配列の要素番号の値」が問われています。この「戻り値の配列」は、9行目で返されている配列outのことです。トレース表から、配列outの要素番号5の値は「17」であることがわかります。

問 4 ウ

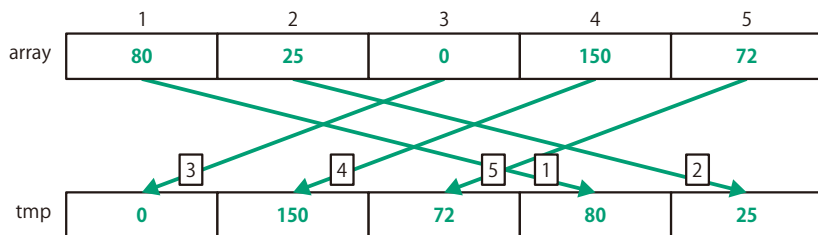
→ 02-01参照(オリジナル)

6行目では、 の式によって計算された値が変数jに代入されます。このjはtmp配列の要素番号として使われ、array[i]の値がtmp[j]に格納される仕組みになっています。つまり、jが示す位置が、並べ替え後のarrayの要素番号(位置)になります。



解説動画

jが3,4,5,1,2の順で変化した場合



そのため、繰り返し処理ごとのjの値を知ることで、配列全体がどのように並べ替えられるかが見えてきます。

shiftが2で、元の配列arrayが{80, 25, 0, 150, 72}の場合を例に、選択肢の式を検証していきます。

●アの場合

まずアの「 $((i + \text{shift}) \bmod n)$ 」です。

- 1周目: $((1 + 2) \bmod 5) = 3$
- 2周目: $((2 + 2) \bmod 5) = 4$
- 3周目: $((3 + 2) \bmod 5) = 0$
- 4周目: $((4 + 2) \bmod 5) = 1$
- 5周目: $((5 + 2) \bmod 5) = 2$

3周目で「0」になってしまいました。この問題では要素番号が1から始まるため、要素番号が0以下になるのは誤りです。

●イの場合

次にイの「 $((i - \text{shift}) \bmod n)$ 」です。

- 1 周目 : $((1 - 2) \bmod 5) + 1 = 0$
- 2 周目 : $((2 - 2) \bmod 5) + 1 = 1$
- 3 周目 : $((3 - 2) \bmod 5) + 1 = 2$
- 4 周目 : $((4 - 2) \bmod 5) + 1 = 3$
- 5 周目 : $((5 - 2) \bmod 5) + 1 = 4$

同様に 1 周目で結果が 0 になったので誤りです。

●ウの場合

続いて、ウの「 $((i + \text{shift} - 1) \bmod n) + 1$ 」です。

- 1 周目 : $((1 + 2 - 1) \bmod 5) + 1 = 3$
- 2 周目 : $((2 + 2 - 1) \bmod 5) + 1 = 4$
- 3 周目 : $((3 + 2 - 1) \bmod 5) + 1 = 5$
- 4 周目 : $((4 + 2 - 1) \bmod 5) + 1 = 1$
- 5 周目 : $((5 + 2 - 1) \bmod 5) + 1 = 2$

この結果により、7 行目で array の各要素が先頭から順に、tmp[3]、tmp[4]、tmp[5]、tmp[1]、tmp[2] に配置されます。そのため、最終的に array は {150, 72, 80, 25, 0} となります。これは元の array が右方向に 2 つシフトされた状態であり、問題文で期待されている動作と一致しています。したがって、ウが正解です。

●エの場合

最後に「エ」の「 $((i - \text{shift} - 1) \bmod n) + 1$ 」も検証していきましょう。

- 1 周目 : $((1 - 2 - 1) \bmod 5) + 1 = -1$
- 2 周目 : $((2 - 2 - 1) \bmod 5) + 1 = 0$
- 3 周目 : $((3 - 2 - 1) \bmod 5) + 1 = 1$
- 4 周目 : $((4 - 2 - 1) \bmod 5) + 1 = 2$
- 5 周目 : $((5 - 2 - 1) \bmod 5) + 1 = 3$

1 周目で結果が -1 に、また 2 周目で結果が 0 になったので誤りです。

「高々」とは「最大でも」という意味です。基本情報技術者試験では時折登場する表現なので、慣れておきましょう。

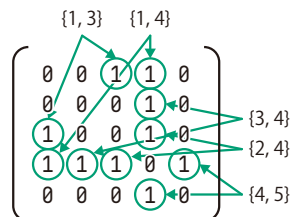
この問題は一見すると難しそうに感じるかもしれませんが、実際にはそれほど難易度が高い問題ではありません。このような一見すると難しそうでも実はそれほどではない問題が、試験ではよく出題されるため、見た瞬間に諦めずに取り組む姿勢が大切です。



解説動画

図1のグラフは人間にとってはわかりやすいものですが、コンピュータには理解できません。そのため、配列で $\{\{1, 3\}, \{1, 4\}, \{3, 4\}, \{2, 4\}, \{4, 5\}\}$ と表現します。図1と図2を比較すると、「1と3が接続され、1と4が接続され……。そうした接続がある箇所を"1"としているのだな」となんとなく、ひらめけるのではないのでしょうか。もしここでひらめけば、残りの説明を細かく読まなくても問題が解けるでしょう。

$\{1, 3\}$ の場合、隣接行列の $[1, 3]$ と $[3, 1]$ に1が入っています。これは図1のグラフが方向性のない無向グラフだからです。方向性がないとは、「 $1 \rightarrow 3$ 」や「 $3 \rightarrow 1$ 」という向きが存在せず、「 $1-3$ 」という接続だけがあることを意味します。そのため、プログラム内で隣接行列を表す配列adjMatrixには、 $[1, 3]$ と $[3, 1]$ に1を入れる必要があります。



プログラム5行目のfor文では、配列edgeListの各要素から先頭の値をuに、2番目の値をvに代入しています。



このままでは、uとvに隣接行列上の位置を意味する値が入っているだけの状態です。配列を隣接行列に変換するためには、adjMatrixにuとvを使って、「1」を代入する必要があります。その処理は次のようになります。

```
adjMatrix[u, v] ← 1
adjMatrix[v, u] ← 1
```

これで $\{1, 3\}$ の場合、adjMatrix $[1, 3]$ とadjMatrix $[3, 1]$ の2か所が「1」になりました。したがって、正解はエです。

マージソートの仕組みを知っている人であれば、この問題を見たときに「マージソートだ」とすぐに気づくはずです。マージソートの基本的なアルゴリズムを理解していれば、それが問題を解く大きなヒントになります。トレース表を作成する場合も、簡略版で済むかもしれません。一方、マージソートを知らない場合は、大きなトレース表を作成しなければならないかもしれません。



解説動画

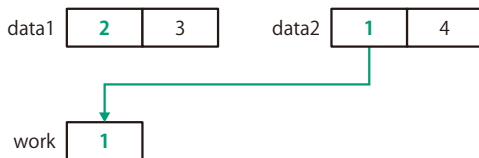
●マージソートとは

マージソートでは、まず各グループの先頭同士を比較します。



問題文にもあるように、各グループはすでに整列されている前提です。つまり、2つのグループのうち、どちらかが必ず最小値です。上記の例では、「2」か「1」のどちらかが必ず最小値であり、「3」や「4」は最小値ではありません。では、各グループの先頭「2」と「1」のどちらが小さいのでしょうか。これは単純な比較だけで判定できます。

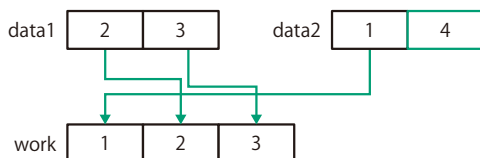
この「比較」は何行目の処理でしょうか。ざっと見たところ比較演算子を含む行は複数ありますが、if文は10行目のみです。おそらくここで比較を行い、条件を満たす場合はdata1の先頭、満たさない場合はdata2の先頭が最小値と判断します。最小値は配列workに配置されます。



上の例のように、data2の先頭が最小値だった場合、その値である「1」は次の比較から外されます。そのため、15行目でjをカウントアップしています。data1の先頭が最小だった場合も同様に、12行目でiをカウントアップしています。

●後処理

マージソートでは多くの場合、後処理があります。仮にdata1のすべての要素がworkに配置されたとしましょう。この場合、data2には要素がまだ残っています（次の例では「4」）。



残っているdata2の要素をすべてまとめてworkに配置する後処理が必要です。それが26行目からの繰り返し処理です。同様に、data1の要素が残っている場合、それをまとめて配置する後処理が20～24行目です。

問題では、27行目で行われる「残ったdata2の要素をworkに配置する処理」の回数が問われています。つまり、9～18行目が終わった時点で、data2に何個の要素が残っているかが答えです。

18行目が終わった時点で、data2には「4」だけが残っています。したがって、正解は「1回」であるイです。

このように、「後処理が必要だ」ということがわかっていれば、わざわざトレースしなくても正解が得られます。

問7 ア

→ 02-01参照 (R4/12 サンプル問題セット - 問11)

整列にはバブルソートやクイックソート、マージソートなどさまざまなアルゴリズムがありますが、今回扱うのは少し変わったアルゴリズムです。



解説動画

ポイントは6行目です。仮にdata[i]の要素の値が「3」だとすると、配列binsの要素番号「3」に「3」代入します。同様に、data[i]の要素の値が「6」であれば、配列binsの要素番号「6」に「6」を代入します。

	1	2	3	4	5	6
bins	1	2	3	4	5	6

このように、要素番号とその要素の値が同じになる配列のアルゴリズムです。ただし、このアルゴリズムには1つ重要な前提条件があります。それは、もとなる配列の値に重複がないことです。もし「3」が2つあると、最初の「3」は上書きされてしまうことになります。どちらも「3」であるため、上書きされたとしても問題ない場合もあります。

アルゴリズムが理解できたところで、問題文を見てみましょう。「戻り値の配列には未定義の要素は含まれず、値は昇順に並んでいる」とあります。これに該当する結果が得られる引数を解答群から選ぶ必要があります。

関数binSortが呼び出されると、2行目で引数である配列dataの要素数を入れ、3行目で配列binsを初期化しています。binsの初期値はn個の未定義の値です。例えば配列dataの要素数が6個の場合、配列binsは6個の未定義の要素を持つ状態からスタートします。

	1	2	3	4	5	6
bins	未定義の値	未定義の値	未定義の値	未定義の値	未定義の値	未定義の値

5～7行目でdataの各要素を順に取り出し、その値を要素番号としてbinsに格納していきます。もしdataの要素に同じ値が複数含まれていた場合はどうなるでしょうか。2度目の配置では、既存の値が上書きされてしまいます。解答群では**イ**、**ウ**、**エ**に同じ値があります。**イ**の場合、戻り値は次のような状態になります。

	1	2	3	4	5	6
bins	1	2	3	4	5	未定義の値

未定義の要素があるため誤りです。**ア**のみ値が重複していないため、6個の値がbinsの6個の要素にちょうど収まります。したがって、正解は**ア**です。

問 8 エ

→ 02-01参照 (R4/12 サンプル問題セット - 問12)

一見すると説明文が長く感じられますが、よく読むと難しい問題ではないことがわかります。配列 $s1$ と配列 $s2$ を比較し、どれだけ似ているかを計算するだけのシンプルな処理です。



解説動画

例えば、 $s1$ と $s2$ が完全に一致していれば、戻り値は1です。これは100%の一致を意味しています。表には戻り値が0.4の例が記載されていますが、なぜ0.4になるのでしょうか。その根拠を表から確認していきましょう。類似度の計算式は次の通りです。

(要素番号が同じ要素の文字同士が一致する要素の組みの個数 ÷ $s1$ の要素数)

$s1$ の要素数は5なので、分母は5になります。次に分子です。表の2行目を見ると1文字目と2文字目の2つだけが同じです。 $s1$ の4文字目と $s2$ の5文字目は同じく「I」ですが、場所が異なるためカウントされていないようです。「文字が一致するだけでなく、位置も一致している」場合にのみ、分子がカウントアップされていきます。

プログラムに目を向けると、カウントアップの処理は8行目にあります。この条件として適切なのは、 $s1[i]$ と $s2[i]$ が同じ場合です。したがって、正解はエです。

この問題は、関数 search に潜むバグを見つけるものです。解答群の選択肢を1つずつ確認し、実際の値に置き換えてプログラムの挙動を追って検証していきます。



解説動画

●アの場合

要素数が1ということは、配列 data に含まれる値は「3」などのたった1つだけの状態です。さらにアは、「target も同じ値である」という条件なので、「target」を「3」としてトレースしてみます。

処理	low	high	middle
4: low ← 1	1		
5: high ← dataの要素数		1	
7: while (low ≤ high)			
8: middle ← (low + high) ÷ 2の商			1
9: if (data[middle] < target)			
11: elseif (data[middle] > target)			
14: return middle			

無限ループにはならず、このプログラムは正常に終了しました。

●イの場合

data に {3, 6} という値が入っており、要素数が2であると仮定します。target は「data の先頭要素と等しい」ため「3」とします。

処理	low	high	middle
4: low ← 1	1		
5: high ← dataの要素数		2	
7: while (low ≤ high)			
8: middle ← (low + high) ÷ 2の商			1
9: if (data[middle] < target)			
11: elseif (data[middle] > target)			
14: return middle			

Ⅰも無限ループにはなりませんでした。

●ウの場合

dataに{3, 6}が入っており、要素数は2、targetは「6」です。

処理	low	high	middle
4: low $\leftarrow$ 1	1		
5: high $\leftarrow$ dataの要素数		2	
7: while (low $\leq$ high)			
8: middle $\leftarrow$ (low + high) $\div$ 2の商			1
9: if (data[middle] < target)			
10: low $\leftarrow$ middle	1		
7: while (low $\leq$ high)			
8: middle $\leftarrow$ (low + high) $\div$ 2の商			1

2回目の繰り返しですが、ここで無限ループになることが確定しました。8行目でmiddleの値が変わらないということは、7行目、9行目、11行目の判定結果も変化がないためです。したがって、正解はウです。

●エの場合

念のため、Ⅰも確認しておきましょう。「要素に"-1"が含まれれば必ず無限ループになるか」が問われています。要素数や、targetと等しい値がある要素番号の指定はされていません。単に"-1"が要素が含まれているだけでは、無限ループは発生しないため、誤りです。

なお、この問題は二分探索を行うアルゴリズムです。本来、**二分探索では調べたい値が真ん中の値より大きい場合、探索範囲をmiddleの次の要素から右側に絞る必要があるため**、式はlow $\leftarrow$ middle + 1とすべきです。また、調べたい値が真ん中の値より小さい場合は、探索範囲をmiddleの前までに絞る必要があるため、式はhigh $\leftarrow$ middle - 1とすべきです。

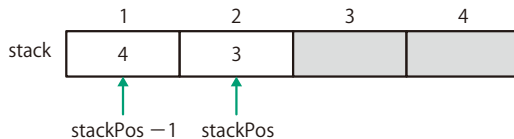
しかし、このプログラムでは式がlow $\leftarrow$ middleおよびhigh $\leftarrow$ middleとなっており、middle自体が再び探索範囲に含まれてしまいます。その結果、探索範囲が狭まらず、同じmiddleが繰り返し計算され続け、無限ループに陥ります。

このプログラムでは、大域変数stackPosが「次に値を格納する位置」を示しています。初期状態ではスタックに「4」と「3」が格納されており、stackPosは「3」になっています。



●関数push

pushの処理では、stackPosが配列の要素数以下である場合、stackの a 番目に値を格納し、次に格納する位置を示すためstackPosを1増やします。このとき、aを「stackPos - 1」にしてしまうと、すでに値が入っている要素を上書きしてしまいます。問題文に書かれている例だと、要素番号2が上書きされます。



stackPosが「3」の場合、値は要素番号「3」の位置に格納すればいいわけですから、stackPosの値そのものを要素番号とすべきです。

●関数pop

popの処理では、まずstackPosが1より大きいかを確認します。stackPosが1の場合は値が存在しないことを意味し、1より大きければ値が存在しているため、値の取り出しが可能であることを意味します。もし取り出し可能であれば、stackPosを1減らしてからstackPosの位置の値を取り出します。

ここで b に「stackPos + 1」を入れてしまうと、「次に値を格納する位置の次」から要素を取り出そうとしてしまいます。図の状態では未定義の値が取り出されます。もしstackPosが4であれば、存在しない要素番号5から取り出そうとしてエラーになる可能性があります。したがって、b には「stackPos - 1」を入れるのが適切です。

取り出した値はpopDataに格納され、stackの該当位置は未定義の値に戻されます。このようにしてスタックは、最後に格納した値から順に取り出される LIFOの動作を実現しています。以上より、a と b の組合せとして正しいのはイです。

まずはトレース表を作成していきます。ただし、トレース表の作成には時間を要するため、いったん途中までにしておきます。その段階でこのアルゴリズムに対する深い知識を得ることができれば、途中からはトレース表を省略できるかもしれません。



解説動画

プログラム	lenData	lenKey	i	j
6 : lenData ← dataの要素数	8			
7 : lenKey ← keyの要素数		3		
11 : for (iを1から(lenData - lenKey + 1)まで1ずつ増やす)			1	
12 : for (jを1からlenKeyまで1ずつ増やす) // a				1
13 : if (data[i + j - 1]がkey[j]と等しい) /** β ***/				
14 : if (jがlenKeyと等しい)				
12 : for (jを1からlenKeyまで1ずつ増やす) // a				2
13 : if (data[i + j - 1]がkey[j]と等しい) /** β ***/				
14 : if (jがlenKeyと等しい)				
12 : for (jを1からlenKeyまで1ずつ増やす) // a				3
13 : if (data[i + j - 1]がkey[j]と等しい) /** β ***/				
18 : aの行から始まる繰返し処理を終了する				

繰返し処理は2つあり、11行目が外側、12行目が内側です。上のトレースでは、外側の繰返し処理1周目までの内容です。外側の繰返し処理1周目の中で、内側の繰返し処理を3回行っています。

では外側の繰返し処理の2周目に入ります。

プログラム	i	j
11 : for (iを1から(lenData - lenKey + 1)まで1ずつ増やす)	2	
12 : for (jを1からlenKeyまで1ずつ増やす) // a		1
13 : if (data[i + j - 1]がkey[j]と等しい) /** β ***/		
18 : aの行から始まる繰返し処理を終了する		

2周目の外側の繰返しでは、条件を満たすものがない状態で終了しました。

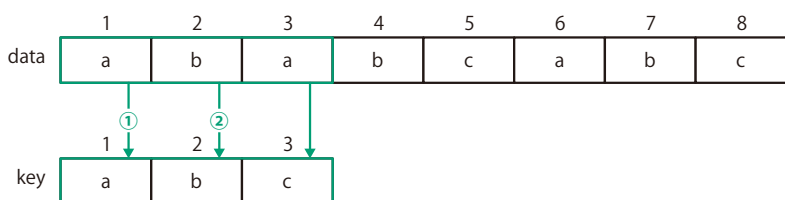
続いて、外側の繰返し処理の3周目に入ります。

処理	i	j
11: for (iを1から(lenData - lenKey + 1)まで1ずつ増やす)	3	
12: for (jを1からlenKeyまで1ずつ増やす) // a		1
13: if (data[i + j - 1]がkey[j]と等しい) /** β /**/		
14: if (jがlenKeyと等しい)		
12: for (jを1からlenKeyまで1ずつ増やす) // a		2
13: if (data[i + j - 1]がkey[j]と等しい) /** β /**/		
14: if (jがlenKeyと等しい)		
12: for (jを1からlenKeyまで1ずつ増やす) // a		3
13: if (data[i + j - 1]がkey[j]と等しい) /** β /**/		
14: if (jがlenKeyと等しい)		
15: resultの末尾にiの値を追加する		

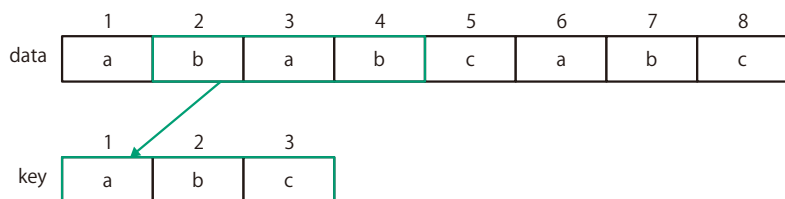
3文字とも合致したため、結果 (result) に「3」が追加されました。

ここまででβは7回実行されています。ただし問われているのは、13行目の「data[i + j - 1]とkey[j]が等しいか」の結果が「真」だった回数です。ここまでわかれば、トレースは省略することができます。

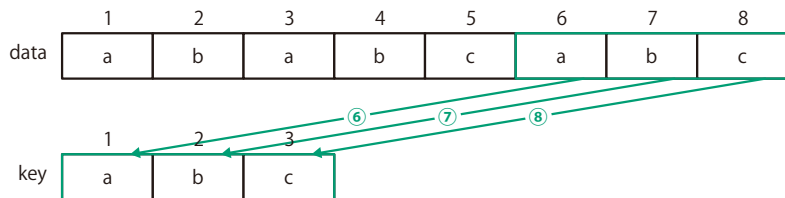
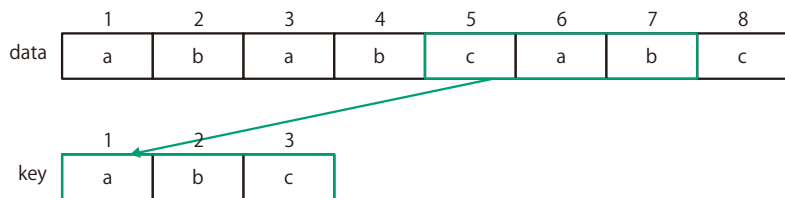
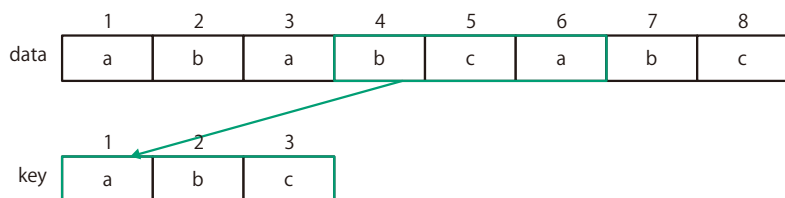
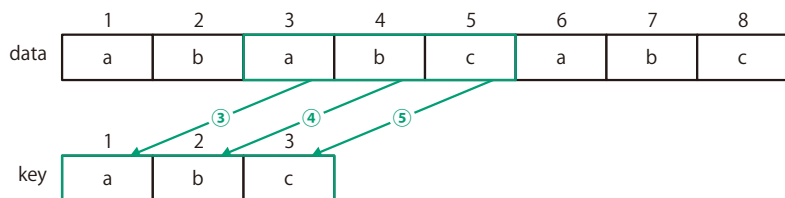
最初から見ていきましょう。まず次のように「dataの先頭からの3文字」と「keyの3文字」をそれぞれ比較していきます。3文字目の比較では文字が合致していないため（「a」と「c」）、カウントアップしませんでした。



次に、dataの2文字目からの3文字分（2～4文字）と、keyをそれぞれ比較します。今回は最初から異なるため、比較は1度だけ行って繰り返し処理は終了です。一度も一致していないため、カウントアップはありません。



同様に処理を続けます。



比較の結果が「真」となった回数は8回なので、正解はクです。

問12 オ

→ 02-02 参照 (R7 公開問題 - 問5)

この問題はプログラムを見れば、どんな計算を行っているのかが見えてくると思います。もしわからなければ、トレース表を作成するとよいでしょう。



解説動画

二次元配列 result は、表 2 を表現しています。result[1, 1] が a で、result[2, 2] が b です。result[1, 1] において、r と c はともに「1」であることから、9 行目のプログラムは次のようになります。

9	result[1, 1] ← (dataの行番号1の要素の和) ×
10	(dataの列番号1の要素の和) ÷ t

「dataの行番号1の要素の和」は「88」で、「dataの列番号1の要素の和」は「140」です。また、変数 t は 2 行目に書かれている通り、すべての値の和なので「154」です。そのため、次のように解釈できます。

9	result[1, 1] ← 88 × 140 ÷ 154
---	-------------------------------

結果は「80」です。同様に b は次のような計算になります。

9	result[2, 2] ← 14 × 66 ÷ 154
---	------------------------------

結果は「6」なので、正解はオです。

このように、**プログラムだけを見て計算式がわかるようであれば、時間をかけずに答えに導くことができます**。もしそれが難しければトレース表を途中まで作成してから、改めて考えてみましょう。それでもわからなければトレース表を完成させるしかありません。

問13 オ

→ 02-02参照 (R6 公開問題 - 問5)

難しそうな問題だと感じるのではないのでしょうか。仮にトレース表を作成すると、かなり大変な作業になります。また1行ずつプログラムを理解すると、これもまたかなり大変です。



解説動画

しかし問題文からヒントを得ることで、実は比較的簡単に答えを導くことが可能です。

● c

まず c から考えます。35行目には次のようなプログラムがあります。

```
35 valueL ← (arrayM[i] × c) ÷ (itemCount × arrayK[i])
```

これと似た式が問題文にありました。

M_{AB} が2, 全注文数が6, K_A が4, K_B が3であるので, 商品Aと商品Bの関連度 L_{AB} は,
 $(2 \times 6) / (4 \times 3) = 1.0$ である。

割り算はここにしかないため、35行目のプログラムがこの式に該当すると思われます。

$$(2 \times 6) / (4 \times 3) = 1.0$$

$\swarrow \quad \searrow \quad \swarrow \quad \searrow$
 $\text{valueL} \leftarrow \underline{\text{arrayM}[i]} \times \underline{\text{c}} \div (\underline{\text{itemCount}} \times \underline{\text{arrayK}[i]})$

c は問題文の「6」に該当します。ではなぜ「6」なのかをさかのぼります。すると「全注文数が6」という説明があります。つまり c には「全注文数」が入ることになります。

全注文数を示す変数はどれでしょうか。 c の候補は「allItemsの要素数」「ordersの要素数」「otherItemsの要素数」の3つで、このうち1つだけが「全注文数」を表します。変数名からすると「orders」が注文データだと思われますが、プログラムを見て確認しましょう。ordersは大域変数です。

```
1 // 注文データ（ここでは表の例を与えている）
2 大域：文字列型配列の配列：orders ← {"A", "B", "D"}, {"A", "D"},
3                                     {"A"}, {"A", "B", "E"},
4                                     {"B"}, {"C", "E"}}
```

1行目のコメントで「注文データ」とありますので、配列ordersは注文を表しているようです。また問題文にあった表を見てもわかるとおり、注文数は「6」です。やはり **c** は「ordersの要素数」で間違いなさそうです。これで正解は**イ**と**オ**のどちらかになりました。

● **a** と **b**

a は28行目、**b** は30行目にあります。

```
25     for (iを1からotherItemsの要素数まで1ずつ増やす)
26         if (orderのいずれかの要素の値がotherItems[i]の値と等しい)
27             if (orderのいずれかの要素の値がitemの値と等しい)
28                 a の値を1増やす
29             endif
30         b の値を1増やす
31     endif
32 endfor
```

- ・27行目の条件に合致した場合：28行目と30行目の両方を実行します
- ・27行目の条件に合致しない場合：30行目だけを実行します

この場合、必ず **b** の値より **a** の値のほうが小さくなります。例えば、**b** の値が100だとすると、**a** は60などになるはずです。解答群を見ると、**a** と **b** は「配列arrayM」「配列arrayK」のいずれかになります。これらの配列は先ほど見た計算式でも使われています。

```
35     valueL ← (arrayM[i] × c) ÷ (itemCount × arrayK[i])
```

対応する説明文は以下でした。

M_{AB} が2、全注文数が6、 K_A が4、 K_B が3であるので、商品Aと商品Bの関連度 L_{AB} は、 $(2 \times 6) \div (4 \times 3) = 1.0$ である。

この例では「2」がarrayMで、「3」がarrayKに該当します。arrayKのほうが大きい
ため、**b**が「arrayK」ということになります。したがって、正解は**オ**です。

変則的な解法だと思ったかもしれませんが、このような解き方でまったく問題ありません。

問 14 ウ

→ 01-05 参照(オリジナル問題)

ビット演算に関わる問題は、一見すると非常に難しく感じるかもしれません。しかし、答えを導き出すための基本的なアプローチは、これまでと同じです。



解説動画

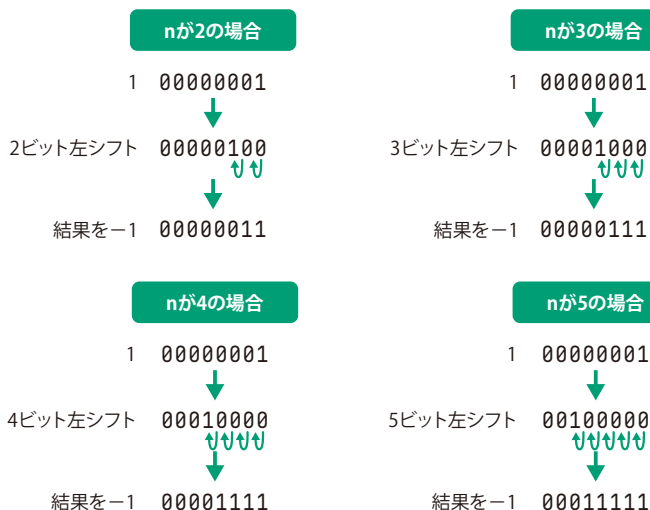
まずはトレースを少しだけ行い、このプログラムが何をしようとしているのかを見極めることが重要です。最後までトレースを行う必要はありません。処理の流れがつかめた段階でいったん止めて、改めてプログラム全体の意図を整理するという流れを意識しましょう。

処理	byte	n	numGroups	i	group
1: 〇8ビット型: reverseByGroup(8ビット型: byte, 整数型: n)	10110011	2			
5: 8ビット型: r ← 00000000					
6: 整数型: numGroups ← 8 ÷ n			4		
9: for (iを0からnumGroups - 1まで1ずつ増やす)				0	
10: group ← (byte >> (i × n)) ∧ ((1 << n) - 1)					

10行目はやや複雑な演算なので、何をしようとしているのかわかりにくいかもしれません。そこで、実際にこの演算を試してみるのが効果的です。

●ビットマスクの作成

10行目の式には論理積が含まれているため、これはビットマスクを用いているのではないかと推測できます。ビットマスクにあたる「 $(1 \ll n) - 1$ 」が、nの値によりいろいろと変化するようです。



ここで作られるビットマスクは、下位nビット分が1になり、それ以外は0になるようです。nが2の場合は下位2ビット分が1、nが5の場合は下位5ビット分が1となります。

●iが0の場合の「byte >> (i × n)」の動き

では次に「byte >> (i × n)」です。繰り返しの1周目は「i」が0です。この場合「(i × n)」は0になるためシフトは行われず、「byte >> (i × n)」の結果はbyteの値そのものです。その結果に対して、先ほどの流れで得たビットマスクがかけられます。これにより、byteの右からnビット分だけが取り出されます。

nが2の場合	nが3の場合	nが4の場合	nが5の場合
10110011	10110011	10110011	10110011
AND 00000011	AND 00000111	AND 00001111	AND 00011111
00000011	00000111	00001111	00010011

ビットマスクを使って、byteの下位nビットを取得し、groupに代入しました。

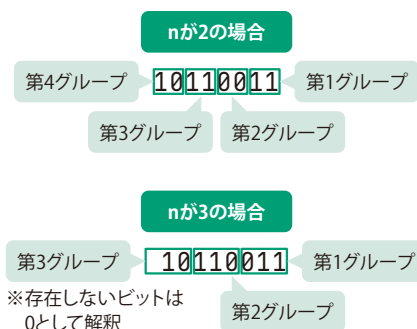
●iが1の場合の「byte >> (i × n)」の動き

11行目以降の処理も気になるのですが、ひとまず9行目からの繰り返し処理の2周目を考えてみます。2周目ではiが1になるため「(i × n)」の結果はnとなります。nビットだけ右シフトしたあと、ビットマスクとの論理積を行うことになります。

nが2の場合	nが3の場合	nが4の場合	nが5の場合
0010110011	00010110011	000010110011	0000010110011
AND 00000011	AND 00000111	AND 00001111	AND 00011111
00000000	00000110	00001011	00000101

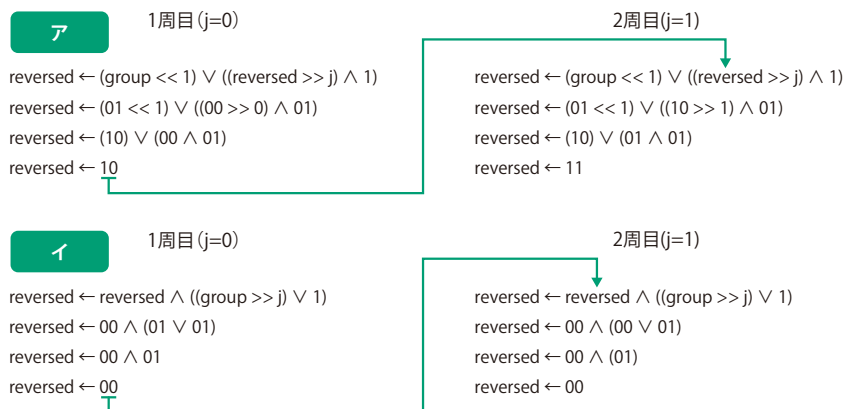
- ・グループごとのビット数が2 (nが2) の場合には、2ビット右シフトしてから下位2ビットを取り出す
- ・グループごとのビット数が3 (nが3) の場合には、3ビット右シフトしてから下位3ビットを取り出す
- ・グループごとのビット数が4 (nが4) の場合には、4ビット右シフトしてから下位4ビットを取り出す
- ・グループごとのビット数が5 (nが5) の場合には、5ビット右シフトしてから下位5ビットを取り出す

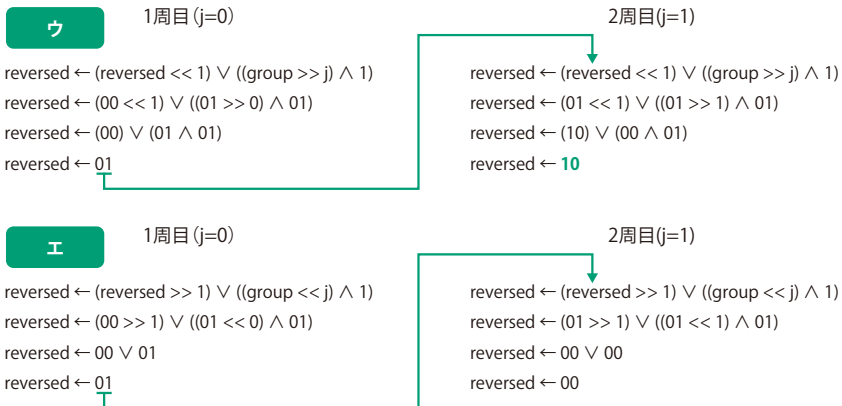
このことから、ビット列を n ビットごとにグループ分けし、1周目は第1グループ、2周目は第2グループを取り出してgroupに代入していることがわかります。同様に、3周目は第3グループ、4周目は第4グループを取り出して、groupに代入します。



こうして得た各グループのビット列を、どこかで逆順にする必要があります。それが13行目の□であると推測できます。15行目にも複雑そうな演算がありますが「//元の位置に戻す」とコメントであるため、ここではなさそうです。

では解答群のうち、グループ内のビット列がたとえば「01」の場合に「10（逆順）」となるのはどれかを考えていきましょう。ここではグループごとのビット数が2の前提で考えていきます。またわかりやすくするため、すべてのビットを2ビットで表現しています。例えば「1」であれば「01」、「0」であれば「00」としています。





ウだけが「10」になるため、正解はウです。もし他にも「10」となったものがあれば、今度は別のビット列で試すことになります。

問 15 ア

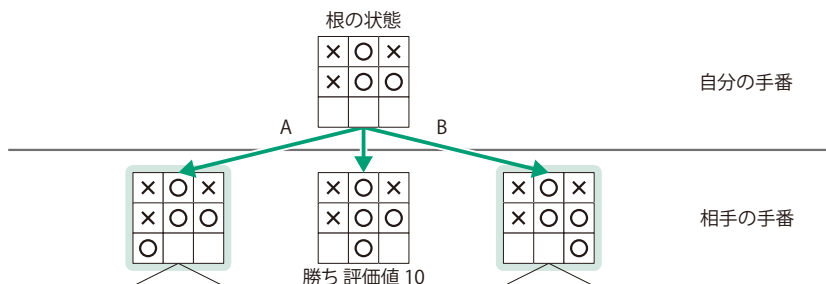
→ 04-03 参照 (R4/12 月 サンプル 問題 セット - 問 15)

この問題は、珍しくプログラムが出てきません。アルゴリズムそのものに関する問題です。問題文が少し長めなので難しそうに感じるかもしれませんが、実際の内容はそれほど複雑ではありません。



解説動画

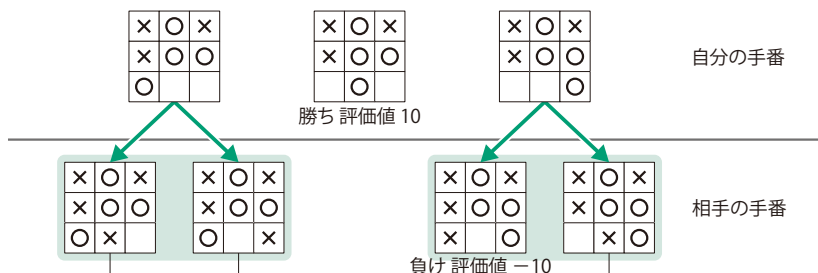
A が指す子の評価値と、B が指す子の評価値が問われています。



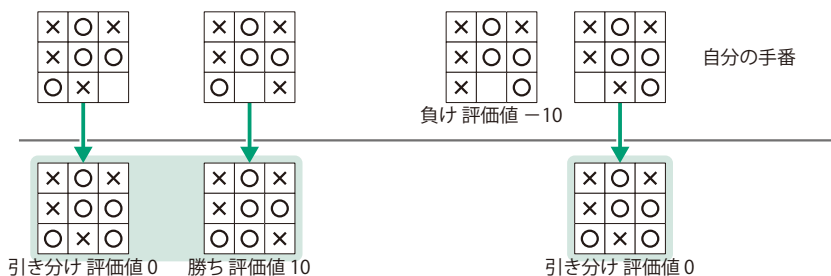
真ん中の評価値は「10」となっています。これは勝敗が決まり、子がない（葉である）ため、評価値が記載されています。AとBは子がある（葉ではない）ので、[手順](3)のルールに従って求めます。

(3) 葉以外の節の評価値は、その節の全ての子の評価値を基に決定する。

では、それぞれの子の評価値を見てみましょう。



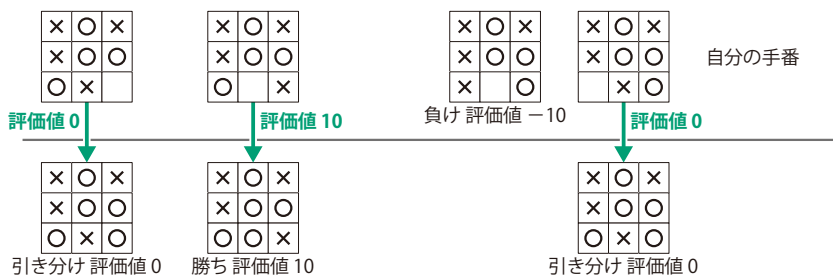
それぞれ子は2つずつあります。右の節の子には1つだけ葉があるため、評価値 -10 が記載されています。それ以外は評価値がないため、さらに下の階層を見ましょう。



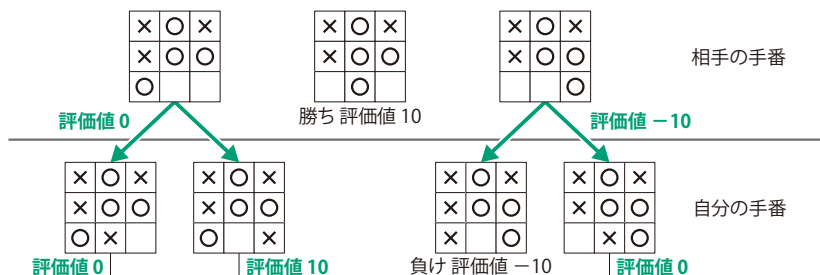
ようやく葉にたどり着きました。子の評価値によって、どのように節の評価値が決まるのでしょうか。そのルールが問題文にありました。

- ①自分の手番の節である場合、子の評価値で最大の評価値を節の評価値とする。
- ②相手の手番の節である場合、子の評価値で最小の評価値を節の評価値とする。

「自分の手番の節」の場合には、「子の評価値のうち最大のもの」が自分の評価値とのことです。それぞれの節に子は1つしかありませんから、そのまま自分の評価値とします。



これでさらに親の節の評価値も求められます。ここは「相手の手番」ですから、子の評価値のうち最小値を採用します。



Aが指す子の評価値は0で、Bが指す子の評価値は-10です。したがって、正解は**A**です。

問16 ク

→ 01-05 参照 (R4/12 サンプル問題 - 問16)

長文でかつ2進数に関わる問題であるため、かなり難しいと感じるはずです。実際、2進数に慣れていないと難しい問題です。しかし、1つずつ理解していけば大丈夫です。**仮に完全に理解できなくても、解答群から候補を2つ程度に絞れただけでもよしとしましょう。**



解説動画

まず冒頭に「符号位置が800(16)以上、FFFF(16)以下」とあります。また最後にも「引数として800(16)以上FFFF(16)以下の整数値だけが渡される」と記載されています。つまりこのプログラムに渡される引数は、この範囲内であることが前提とされています。この範囲外の値は一切今回の問題では考える必要はありません。

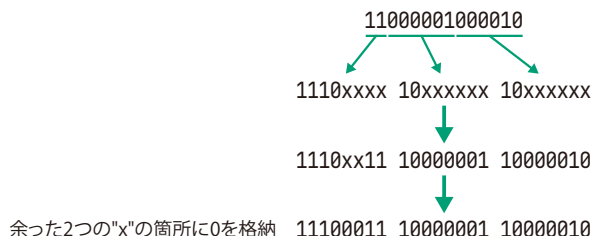
この問題のプログラムは、次のアルゴリズムを実現したものです。

3バイトの長さのビットパターンを1110xxxx 10xxxxxx 10xxxxxxとする。ビットパターンの下線の付いた"x"の箇所に、符号位置を2進数で表した値を右詰めで格納し、余った"x"の箇所に0を格納する。この3バイトの値がUTF-8の符号である。

アルゴリズムの説明では、引数である「符号位置」をまず2進数に変換しています。その結果をビットパターンの「x」に埋めていきます。「例えば……」以降の説明で使用している「3042」を例に考えてみましょう。3042(16)は2進数で「11000001000010」です。またビットパターンは次の通りです。

```
1110xxxx 10xxxxxx 10xxxxxx
```

このビットパターンに対して、3042(16)を2進数にした「11000001000010」を右詰めで格納します。したがって、次のように格納されることになります。



上の図と同じ処理をプログラムで実現します。問題文にある「3042(16)」の例で考えていきます。

● 1 行目の引数

引数codePointは、16進数では「3042」です。codePointのデータ型は「整数型」とあります。整数型とは10進数であるため、codePointは実際には12354となります。ただし、問題を解くにあたってはほとんど影響はありません。

● 4 行目のutf8Bytes

utf8Bytesには3つの要素があります。これら「224」「128」「128」はどこからきた値でしょうか。「3つ」で思い出すものがあると思います。それは次の説明です。

3バイトの長さのビットパターンを1110xxxx 10xxxxxx 10xxxxxxとする。

他に3つに関する説明はありませんから、おそらくこのことでしょう。

- 1110xxxxは224のこと
- 10xxxxxxは128のこと
- 10xxxxxxは128のこと

ということになります。では、「1110xxxx」はなぜ「224」なのでしょう。プログラムには、次のようにコメントがあります。

```
2  /* utf8Bytesの初期値は、ビットパターンの"x"をすべて0に置き換え、
3     8桁ごとに区切って、それぞれを2進数と見なしたときの値 */
4     整数型の配列: utf8Bytes ← {224, 128, 128}
```

このルール通りに、「1110xxxx」の「x」を「0」にすると「11100000」となります。この2進数を10進数にすると224になります。やはりコメントの通り、xを0にしたときの値（10進数）を各要素の値にしているようです。

● 5 行目のcp

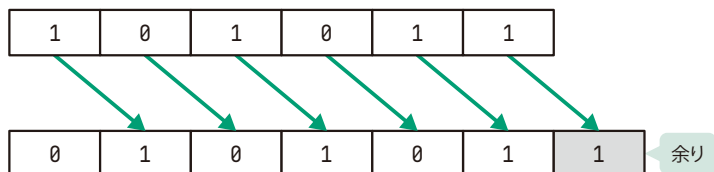
変数cpには、引数codePointをそのまま代入しています。変数cpは7行目からの繰り返し処理で頻繁に変化します。

● 7 行目からの繰り返し処理

繰り返しは「utf8Bytesの要素数である3」からカウントダウンしていきます。最初の繰り返し処理ではiが3なので、utf8Bytesの3つ目の要素を対象に処理を行っています。

```
8     utf8Bytes[3] ← utf8Bytes[3] + (cp ÷  の余り)
```

cpを□で割った余りを求めています。2倍すれば左に1ビットシフト、4倍すれば左に2ビットシフトされることはすでに学習しました。また、割り算を行うことで右にシフトすることも学んだはず。さらに、割り算(右シフト)をしてあふれたビットは「余り」として得られます。



まず1周目の繰り返し処理について考えます。「10xxxxxx」の「xxxxxx」部分に、引数3042(16)を2進数にした「11000001000010」の、右から6ビットを埋めます。そのためには「11000001000010」の右から6ビットと、「128を2進数にした11000000」とのOR演算を行います。

```

11000001000010 変数cpを右に6ビットシフトしてあふれたビット列
      ↓
    000010 ... 変数cpの右6ビット
OR 10000000 ... 128の2進数
-----
    10000010 ... ビットパターンに変数cpの右6ビットを格納した状態
  
```

枠で示した箇所のように「6ビット右シフトした際にあふれたビット」を得るにはどうすればいいでしょうか。仮に1ビット右にシフトするには2で割ります。2ビット右にシフトするには4で割ります。同様に、2のn乗で割ることで、nビットで右シフトすることができます。

- 3ビット右にシフトするには8で割る
- 4ビット右にシフトするには16で割る
- 5ビット右にシフトするには32で割る
- 6ビット右にシフトするには64で割る

つまり、変数cpを64で割ってその余りを得ることで、変数cpの右から6ビットを得ることができます。□には「64」が入るため、正解はクです。

A 社が委託先の C 社を通じて構築・運用している EC サイトに対して脆弱性診断を行いました。その結果、3 つの脆弱性が指摘され、それぞれに対処すべき組織を問うものです。契約上の責任範囲や各組織の役割に基づいて判断していきます。

A 社は自社の EC サイトを B 社の PaaS 環境で稼働させており、OS や DBMS などのミドルウェアは PaaS の提供範囲に含まれています。一方で、Web アプリケーションは C 社が開発・運用を担当しています。したがって、次のように考えます。

- ・ PaaS が提供する OS・DBMS の管理責任 → B 社
- ・ Web アプリケーションの脆弱性対応責任 → C 社
- ・ 契約上委託されていない範囲や A 社独自の問題 → A 社

項番別に見ていきましょう。

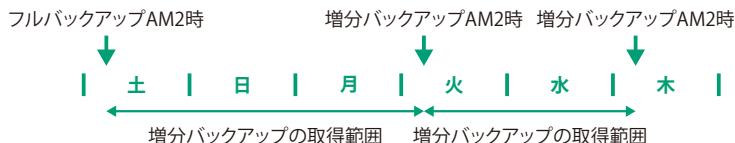
- 項番 1 A サイトで利用しているアプリケーションサーバの OS に既知の脆弱性があり、脆弱性を悪用した攻撃を受けるおそれがある。
- 項番 2 A サイトにクロスサイトスクリプティングの脆弱性があり、会員情報を不正に取得されるおそれがある。
- 項番 3 A サイトで利用している DBMS に既知の脆弱性があり、脆弱性を悪用した攻撃を受けるおそれがある。

項番 1 は OS の脆弱性なので B 社、項番 2 は Web アプリケーションの脆弱性なので C 社、項番 3 はミドルウェアの脆弱性なので B 社が対処すべき項目です。したがって、正解は **カ** です。

問 18 ク

→ 06-03 参照 (R7 公開問題 - 問 6)

状況を整理しましょう。フルバックアップは土曜日の午前2時に取得しています。増分バックアップは火曜日の午前2時と木曜日の午前2時に取得しています。**増分バックアップは、直前に取得したバックアップ（フルまたは増分）以降に変更があったデータのみをバックアップします。**



次に目標復旧時点 (RPO) について確認します。**目標復旧時点は「いつの時点までデータを戻したいか」を示すもの**であり、A社では72時間と定められています。金曜日の正午に障害が発生した場合、その72時間前は火曜日の正午です。したがって、**a1** には「火曜日の正午」が入ります。

次に **a2** を考えます。木曜日の正午に障害が発生した場合、ファイルサーバの全データを復旧するためには、土曜日のフルバックアップと火曜日および木曜日の増分バックアップを順にリストアする必要があります。フルバックアップのリストアには平均4時間、増分バックアップのリストアには1回あたり0.25時間かかるため、合計は4時間 + (0.25時間 × 2回) = 4.5時間です。よって **a2** には「4.5」が入ります。なお、火曜日に取得しているのが差分バックアップだった場合には、土曜日のフルバックアップと木曜日の差分バックアップだけをリストアすればよいので、4.25時間で復旧することができます。

最後に **a3** の承認者について見ていきます。問題文には「A社はISMS認証を取得しており、最高情報セキュリティ責任者 (CISO) を中心に情報セキュリティに取り組んでいる」とありました。ここからもわかるように、ICT継続計画の承認は組織全体のリスクマネジメントと深く関わる重要なものです。

まず候補として「CISO」があります。**CISO (Chief Information Security Officer) は情報セキュリティの最高責任者であり、情報セキュリティ方針の策定や計画の承認にも関わる立場**です。「情報システム部の担当者」は現場レベルでバックアップやシステム運用を担う役割であり、組織全体の計画を承認する立場ではありません。「内部監査室長」は監査業務を統括し、内部統制の評価を行う重要なポジションですが、監査する側の立場です。こちらも承認者には適していません。このように整理すると、ICT継続計画の承認者としてもっとも適切なのはCISOであり、**a3** には「CISO」が入ります。

したがって、正解はクです。

この問題では、複合機のスキャン機能が初期設定のまま使用されており、電子メールの差出人アドレスや件名が固定されています。さらに、この初期設定情報はベンダーのWebサイトで公開されているため、外部の攻撃者も容易に差出人アドレスを知ることができます。この状況では、攻撃者がその差出人アドレスを偽装してフィッシングメールを送信することが容易に行えます。

B業務に従事する従業員は、複合機から送られてくるメールの差出人がいつも同じであることに慣れているため、偽装されたメールを正規のものと誤認して、本文中のURLをクリックしてしまう可能性があります。その結果、フィッシングサイトに誘導され、A社の採用予定者の個人情報が漏えいするリスクがあります。したがって、正解は**ア**です。

- イ いつも同じ差出人から届くメールであり、スパムメールだと思う可能性は低いため、適切ではありません。
- ウ ランサムウェアのシナリオです。たしかに平文であれば盗聴される可能性はあります。しかし、従業員ごとに異なる鍵でPDFファイルを暗号化しているため、内容を知られることはありません。また、暗号化されたPDFファイルがさらに暗号化される可能性もありますが、スキャンした元データはあるため、身代金を支払う必要はありません。
- エ ウと同様に盗聴のリスクはあります。しかし、BサーバにはIDとパスワードによる認証があるため、URLだけの不正アクセスは困難です。

問 20 キ

→ 06-04参照(R7 情報セキュリティマネジメント 公開問題-問15)

A社が開発したアプリでは、多くの個人情報を扱います。それらのデータをB社に提供するには、個人を特定できるような情報を含めてはいけません。そのため、個人が特定できないように匿名加工情報を作成し、それをB社に提供します。B社はこれらの情報を用いて、マーケティングに活用しようとしています。なお、B社が分析に使用するのは、次の項目です。

- ・ 年代
- ・ 性別
- ・ 年収区分
- ・ 住んでいる地域区分
- ・ 勤務先の業種
- ・ 体重
- ・ 歩数

B社はメールアドレスを一切使用しませんので提供しなくてもよいはずです。加工情報からメールアドレスを削除して提供するべきです。また、B社にとって「勤務先の業種」は必要な情報であり、これは加工せずにそのまま提供します。体重についても個人を特定することはできませんが、数値データであるため(3) トップ(ボトム) コーディングと(4) 丸めを適用します。体重を加工してもB社のマーケティングには影響が少ないため、この方法で問題ありません。よって、正解はキです。

索引

記号

-	23
.	135
()	23
[]	96
{}	96
//	10
←	22
+	23
÷	25
×	23
<	30
<<	61
=	30
≠	30
>	30
>>	61
^	61
v	61
≦	30
≧	30
○	71

数字

10進数	53, 58
2進数	53, 58
2要素認証	285
8ビット型	60

A

abs関数	146
and	36
AND回路	59
Availability	297

C

CIA	298
CISO	299
Confidentiality	297
CSIRT	299
CVSS	303

D、E

do	47
----	----

EDR	312
else	32
elseif	33
endfor	48
endif	31
endwhile	45

F

false	19
for	48
FTPS	298

H、I

HTTPS	298
IDS	292
if	30
IMAPS	298
Integrity	297
IPS	292
IPsec	298

J、M

JVN	302
mod	23

O、R

or	36
OR回路	59
return	74

S、T

S/MIME	298
SFTP	298
SMTPS	298
SQLインジェクション	303
TLS	298
true	19

U、V

UPS	281
VPN	298

W、X

while	44
XOR回路	59
XSS	303

あ行

後入先出法	170
暗号化	282
行きがけ順	184
一次元配列	114
入れ子構造	35
インスタンス	129, 133
インスタンスの配列	140
インスタンス変数	134
インデント	32
ウイルス	286
ウイルス定義ファイル	282
運用管理	292
運用的セキュリティ	283
枝	182
エッジ	182
演算	23
オーバーフロー	62
オブジェクト指向	128
親	182
親子関係	182

か行

階乗	80
帰りがけ順	184
加算器	59
型	19
可用性	297
環境のセキュリティ	281
関数	71
完全性	297
完全二分木	185
偽	31
キーロガー	286
機械語	253
木構造	182
擬似言語	9
技術的セキュリティ	282
機密性	297
キュー	170
行	115
共通脆弱性評価システム	303
クライアント証明書	301
クラス	128, 131
繰り返し	44
クロスサイトスクリプティング	303
子	182
後行順	184

降順	107
後置表記法	253
個人情報保護法	299
コメント	10
コンストラクタ	140

さ行

再帰	80
最小権限の原則	282, 287
先入先出法	170
差分バックアップ	290
シグネチャファイル	282
字下げ	32
実数型	19
シノニム	224
シフト演算	57
主記憶装置	18
条件分岐	30
昇順	107
衝突	224
情報セキュリティ	280
情報セキュリティ委員会	299
情報セキュリティポリシ	283
真	31
侵入検知システム	292
侵入防止システム	292
スケアウェア	286
スタック	172
スパイウェア	286
脆弱性	302
整数型	19
生体認証	280
セキュリティ管理	292
節	182
節点	182
先行順	184
挿入ソート	246
増分バックアップ	290
双方向リスト	157
添え字	95

た行

大域変数	91
代入	22
多層防御	282
多要素認証	282
単方向リスト	155
中間順	184

中置表記法	253
データ型	19
デジタル証明書	298
手続	71
通りがけ順	184
匿名加工情報	299
トレース表	11, 26
トロイの木馬	286
ドロッパー	287

な行

流れ図	31
二次元配列	114
二次元配列の宣言	115
二分木	185
二分探索	359
二分探索木	186
根	182
ネスト構造	35
ノード	182

は行

葉	182
バイオメトリクス認証	280
配列	94
配列宣言	96
バックアップ	290
バックドア	286
パッチ	287
幅優先探索	183
バブルソート	105
比較演算子	30
引数	73
左シフト	58, 61
ビット	18, 57
ビット演算	57
ビットマスク	62
ビット列	18
ファイアウォール	282
フィッシングメール	287
深さ優先探索	183
負荷分散装置	299
物理的セキュリティ	280
ブルートフォース攻撃	296
フルバックアップ	290
プログラミング言語	9
ペネトレーションテスト	305
変数	18

変数宣言	19
変数名	21
ポインタ	155
ボット	286

ま行

マージソート	270
マクロウイルス	287
マルウェア	286
右シフト	59, 61
未定義	20
無限ループ	44
無停電電源装置	281
メソッド	132
メンバ変数	131
文字型	19
文字列型	19
戻り値	74

や行

要素	95
要素番号	95

ら行

ランサムウェア	286
ルート	182
ルートキット	286
列	115
連結リスト	155
ロードバランサー	299
ログ	292
ロジックボム	286
論理演算	60
論理演算子	36
論理回路	59
論理型	19
論理積	61
論理和	61

わ行

ワーム	286
ワンタイムパスワード	282

著者 石田 宏実(いしだ・ひろみ)

デジタルプランニング株式会社代表取締役社長。札幌在住。プログラミングとマーケティングに強みを持ち、インターネットマーケティングに有用なソフトウェアを自社開発。

オンライン講座Udemyにて情報処理技術者試験対策コースを多く公開しており、開講から3年半で受講者11万人超。月刊誌『Software Design』にて1年間の連載。著書に『ゲーミフィケーション』(大和出版 共著)、『この1冊で合格! 石田宏実の応用情報技術者 テキスト&問題集』(KADOKAWA)、『データベーススペシャリスト完全攻略の教科書+過去問題』(技術評論社)がある。

STAFF

カバーデザイン	山之口 正和 (OKIKATA)
本文デザイン・DTP	リプロワークス・デザイン室
本文イラスト	神林 美生
カバー制作	鈴木 薫
校正協力	株式会社トップスタジオ
編集	リプロワークス
	畑中 二四
編集長	玉巻 秀雄

本書のご感想をぜひお寄せください

<https://book.impress.co.jp/books/1125101034>

読者登録サービス



アンケート回答者の中から、抽選で図書カード(1,000円分)などを毎月プレゼント。
当選者の発表は賞品の発送をもって代えさせていただきます。
※プレゼントの賞品は変更になる場合があります。



■商品に関する問い合わせ先

このたびは弊社商品をご購入いただきありがとうございます。本書の内容などに関するお問い合わせは、下記のURLまたは二次元コードにある問い合わせフォームからお送りください。

<https://book.impress.co.jp/info/>

上記フォームがご利用いただけない場合のメールでの問い合わせ先
info@impress.co.jp



※お問い合わせの際は、書名、ISBN、お名前、お電話番号、メールアドレスに加えて、「該当するページ」と「具体的な質問内容」「お使いの動作環境」を必ずご明記ください。なお、本書の範囲を超えるご質問にはお答えできないのでご了承ください。

- 電話やFAX でのご質問には対応しておりません。また、封書でのお問い合わせは回答までに日数をいただく場合があります。あらかじめご了承ください。
- インプレスブックスの本書情報ページ <https://book.impress.co.jp/books/1125101034> では、本書のサポート情報や正誤表・訂正情報などを提供しています。あわせてご確認ください。
- 本書の奥付に記載されている初版発行日から3年が経過した場合、もしくは本書で紹介している製品やサービスについて提供会社によるサポートが終了した場合はご質問にお答えできない場合があります。

■落丁・乱丁本などの問い合わせ先

FAX 03-6837-5023

電子メール service@impress.co.jp

※古書店で購入された商品はお取り替えできません

基本情報技術者 科目B 超効率の教科書+よく出る問題集

2025年10月21日 初版発行

著者 石田 宏美

発行人 高橋 隆志

編集人 藤井 貴志

発行所 株式会社インプレス

〒101-0051 東京都千代田区神田神保町一丁目105番地

ホームページ <https://book.impress.co.jp/>

本書は著作権法上の保護を受けています。本書の一部あるいは全部について（ソフトウェア及びプログラムを含む）、株式会社インプレスから文書による許諾を得ずに、いかなる方法においても無断で複写、複製することは禁じられています。

Copyright©2025 Hiromi Ishida. All rights reserved.

印刷所 日経印刷株式会社

ISBN978-4-295-02303-6 C3055

Printed in Japan